Data Analyst and Data Scientist Interview Questions: Core

Python, SQL, Power BI, Excel, Math and Statistics, Machine Learning fundamentals and business cases, with answers.

Total questions: 546. Part 1 lists every question; Part 2 (the answer key, at the end) gives a full answer for each, using the same question number.

Contents

Section	Questions	Numbers
Python	90	Q1 to Q90
SQL	100	Q91 to Q190
Power BI	65	Q191 to Q255
Excel	55	Q256 to Q310
Math and Statistics	90	Q311 to Q400
Machine Learning Fundamentals (up to Linear and Logistic Regression)	75	Q401 to Q475
Business Cases and Product Analytics	71	Q476 to Q546

Tags: Concept = theory; Coding = write code/query/formula; Scenario = a company presents a case and asks you to solve it; Behavioural = soft-skill. Level: Easy / Medium / Hard.

Part 1: Questions

Python

Q1. *[Concept | Easy]*

> What are the differences between a list, tuple, set and dictionary in Python, and when would you use each in data analysis work?

Q2. *[Concept | Easy]*

> Explain mutable versus immutable objects in Python. Why does it matter, and what is the classic bug with mutable default arguments?

Q3. *[Concept | Easy]*

> What is the difference between "is" and "==" in Python? How does this relate to checking for None and NaN?

Q4. *[Concept | Medium]*

> What is the difference between a list comprehension and a generator expression? When should you prefer a generator?

Q5. *[Concept | Medium]*

> Explain *args and **kwargs. Give an example of how they are used in data code.

Q6. *[Concept | Medium]*

> What is the difference between a shallow copy and a deep copy? How does it affect lists of lists and pandas DataFrames?

Q7. *[Concept | Medium]*

> What is a decorator in Python and how would you use one in a data pipeline?

Q8. *[Concept | Medium]*

> What is the GIL and how does it affect data processing in Python?

Q9. *[Concept | Easy]*

> Explain lambda, map, filter and reduce. How do they compare with list comprehensions and pandas apply?

Q10. *[Concept | Easy]*

> What is the difference between a pandas Series and a DataFrame, and what is the role of the index?

Q11. *[Concept | Easy]*

Explain the difference between loc and iloc in pandas, including how slicing behaves.

Q12. *[Concept | Medium]*

Compare pandas merge, join and concat. What are the join types and what does the validate argument do?

Q13. *[Concept | Medium]*

Explain groupby in pandas. What is the difference between agg, transform and apply?

Q14. *[Concept | Easy]*

How do you detect and handle missing values in pandas? What are the pros and cons of dropping versus imputing?

Q15. *[Concept | Medium]*

Why is vectorization faster than loops or apply in pandas and NumPy? Give examples of vectorized alternatives.

Q16. *[Concept | Medium]*

What is SettingWithCopyWarning, why does it appear, and how do you avoid it?

Q17. *[Concept | Medium]*

Explain NumPy broadcasting with examples. What are the rules?

Q18. *[Concept | Medium]*

What is the difference between a view and a copy in NumPy? Why does it matter?

Q19. *[Concept | Medium]*

Explain the differences between pivot, pivot_table, melt, stack and unstack in pandas.

Q20. *[Concept | Medium]*

How can you reduce the memory usage of a large pandas DataFrame?

Q21. *[Concept | Medium]*

What methods can you use to detect outliers, and how do you decide whether to remove them?

Q22. *[Concept | Medium]*

Explain Pearson versus Spearman correlation and the statement "correlation does not imply causation". How do you compute them in pandas?

Q23. *[Concept | Easy]*

> What is the difference between matplotlib and seaborn, and what are Figure and Axes in matplotlib?

Q24. *[Concept | Easy]*

> How do you choose the right chart type for a given analytical question?

Q25. *[Concept | Medium]*

> How do you read a boxplot, and when is a histogram better than a bar chart?

Q26. *[Concept | Hard]*

> What is data leakage and how can preprocessing steps like scaling or imputation cause it? How do you prevent it with scikit-learn?

Q27. *[Concept | Easy]*

> What is the difference between normalization (min-max scaling) and standardization? When do you use each?

Q28. *[Concept | Medium]*

> How do you encode categorical variables? Compare label encoding, one-hot encoding, ordinal encoding and target encoding.

Q29. *[Concept | Medium]*

> How does pandas handle dates and times? Explain to_datetime, the dt accessor, resample and timezone handling.

Q30. *[Concept | Easy]*

> What are Python context managers and why is "with open(...)" preferred?

Q31. *[Concept | Easy]*

> How does exception handling work in Python (try, except, else, finally)? Give a data-loading example.

Q32. *[Concept | Medium]*

> How would you process a CSV file that is larger than available memory using pandas?

Q33. *[Concept | Easy]*

> How do you find and handle duplicate records in pandas? What does keep do?

Q34. *[Concept | Medium]*

How do hash tables back Python dictionaries and sets, and what are the time complexities of common operations?

Q35. *[Concept | Medium]*

Explain how to handle class imbalance in a classification dataset (for example, 1 percent fraud). Which metrics and techniques do you use?

Q36. *[Concept | Medium]*

Explain rolling, expanding, shift and diff in pandas and what each is used for.

Q37. *[Concept | Medium]*

How do you split data into train and test sets and perform cross-validation? Why use stratification or random_state?

Q38. *[Concept | Medium]*

What is a MultiIndex in pandas and when is it useful? How do you work with it?

Q39. *[Concept | Hard]*

Explain how pandas categorical dtype works and what the benefits are. What traps exist when using it?

Q40. *[Concept | Hard]*

Explain the difference between Python's multiprocessing, threading and asyncio, and which you would use to download 1,000 API pages versus to compute features on a huge numeric dataset.

Q41. *[Concept | Medium]*

How do you test whether a difference between two groups is statistically significant in Python? Describe t-test, chi-square and the assumptions.

Q42. *[Coding | Easy]*

Write a function that checks whether a string is a palindrome, ignoring case, spaces and punctuation.

Q43. *[Coding | Easy]*

Given a list of integers and a target, return the indices of the two numbers that add up to the target (Two Sum).

Q44. *[Coding | Easy]*

Count the frequency of each word in a text, ignoring case and punctuation, and return the top 3 most common words.

Q45. *[Coding | Easy]*

Write code to find all duplicate elements in a list, and also remove duplicates while preserving the original order.

Q46. *[Coding | Medium]*

Group a list of words into anagram groups, for example ["eat","tea","tan","ate","nat","bat"].

Q47. *[Coding | Medium]*

Find the length of the longest substring without repeating characters.

Q48. *[Coding | Medium]*

Merge overlapping intervals, e.g. [[1,3],[2,6],[8,10],[15,18]] becomes [[1,6],[8,10],[15,18]]. Relate it to a data problem such as merging booking periods.

Q49. *[Coding | Easy]*

Write a function to check whether a string of brackets "()[]{}" is balanced.

Q50. *[Coding | Easy]*

Given a DataFrame of employees (name, dept, salary), find the second highest salary overall and the second highest salary within each department.

Q51. *[Coding | Medium]*

From a sales DataFrame (region, product, revenue), return the top 2 products by revenue in each region.

Q52. *[Coding | Medium]*

A customers table has multiple rows per customer_id with an updated_at timestamp. Keep only the latest record per customer.

Q53. *[Coding | Medium]*

Fill missing values in a numeric column with the median of its group (for example, missing salary filled by department median), and fall back to the global median when a whole group is missing.

Q54. *[Coding | Medium]*

Given daily revenue, compute month-over-month growth percentage and a 7-day rolling average.

Q55. *[Coding | Medium]*

Create a pivot table of total sales by region (rows) and month (columns) with totals, then convert it back to long format.

Q56. *[Coding | Medium]*

Find customers who have never placed an order (an anti-join) using pandas.

Q57. *[Coding | Medium]*

Compute a running (cumulative) total of sales per customer ordered by date, and the percentage each order contributes to that customer's total.

Q58. *[Coding | Easy]*

Clean a messy text column: trim whitespace, normalise case, remove special characters from names, and standardise phone numbers to digits only.

Q59. *[Coding | Medium]*

A date column contains mixed formats (2024-01-05, 05/01/2024, Jan 5 2024, and junk like "N/A"). Parse it robustly and report the rows that failed.

Q60. *[Coding | Medium]*

Remove outliers from a numeric column using the IQR rule, and show an alternative that caps (winsorizes) instead of dropping.

Q61. *[Coding | Medium]*

Using NumPy only (no loops), standardize every column of a 2D array and find the pair of rows that are closest in Euclidean distance.

Q62. *[Coding | Medium]*

Compute a moving average over a NumPy array efficiently, without a Python loop.

Q63. *[Coding | Medium]*

Create a chart with monthly revenue as bars and conversion rate as a line on a secondary y-axis using matplotlib. Add labels and a legend.

Q64. *[Coding | Easy]*

Draw a correlation heatmap of numeric columns with seaborn, masking the upper triangle and annotating values.

Q65. *[Coding | Easy]*

Bin a continuous age column into groups (0-17, 18-34, 35-54, 55+) and also create quartile-based income groups. Explain cut versus qcut.

Q66. *[Coding | Medium]*

A column "tags" contains comma-separated values like "a,b,c". Split it into one row per tag and count the tags.

Q67. *[Coding | Hard]*

Build a monthly cohort retention table: for each customer's first-purchase month, the percentage of customers who purchase again in each subsequent month.

Q68. *[Coding | Hard]*

Given user event timestamps, assign session IDs where a new session starts after 30 minutes of inactivity, and compute the number of sessions and average session length per user.

Q69. *[Coding | Medium]*

Given a table of login dates per user, find each user's longest streak of consecutive days.

Q70. *[Coding | Medium]*

Flatten a nested JSON API response (orders with a list of items and a customer object) into a tidy DataFrame.

Q71. *[Coding | Hard]*

Implement RFM (recency, frequency, monetary) scoring for customers from a transactions table and label top customers.

Q72. *[Coding | Easy]*

Given a DataFrame of orders, calculate total revenue, average order value, and number of unique customers for each month using named aggregation.

Q73. *[Scenario | Medium]*

Company context: You join a retail chain as a data analyst. The sales team sends a 2-million-row CSV export with columns order_id, order_date, store, product, qty, price, customer_email. Dates come in two formats, store names have inconsistent spelling ("Mumbai", "mumbai ", "MUMBAI"), qty has negatives and nulls, price sometimes has a currency symbol, and some orders appear twice. Leadership wants a trustworthy monthly revenue report by store. How do you clean and deliver this?

Q74. *[Scenario | Medium]*

Company context: A telecom operator sees 18 percent annual customer churn. You are given a customer table (tenure, plan, monthly charges, support calls, contract type, churned flag). The head of retention asks: who is churning, why, and which customers should the team call this month? Outline your analysis in Python.

Q75. *[Scenario | Medium]*

> Company context: An e-commerce company tested a new checkout page. Control had 10,000 visitors with 820 purchases; the variant had 10,200 visitors with 910 purchases. The product manager asks: "Did the new page win? Should we ship it?" Analyze in Python.

Q76. *[Scenario | Hard]*

> Company context: At a marketplace company, finance says monthly revenue is 1.8 million but your merged analytics table shows 2.6 million. You built the table by joining orders to order_items and to payments in pandas. What went wrong and how do you fix and prevent it?

Q77. *[Scenario | Medium]*

> Company context: A logistics company has 40 GB of daily GPS and delivery event logs in CSV. A data analyst's laptop has 16 GB of RAM. You need the average delivery time and late-delivery rate by city and hour. How do you process this?

Q78. *[Scenario | Medium]*

> Company context: A global survey company collected a "country" field as free text from users: "USA", "U.S.A", "United States", "india", "Bharat", "UK", "Untied Kingdom", "Deutschland". You must produce country-level response counts. How do you standardize the values?

Q79. *[Scenario | Hard]*

> Company context: A streaming service's daily active users (DAU) dropped 12 percent last Tuesday and the dashboard owner asks you to find out why. You have an events table (user_id, event_time, platform, country, app_version, event_type). Describe the investigation.

Q80. *[Scenario | Hard]*

> Company context: A grocery chain wants a weekly demand forecast for 500 products to plan inventory. You have 3 years of weekly sales per store and product. Which approach do you take in Python and how do you evaluate it?

Q81. *[Scenario | Medium]*

> Company context: A ride-hailing company notices some trips have a fare of 0.01 or 25,000, distance of 0 km or 4,000 km, and durations of negative minutes. The pricing team wants the average fare per km by city. How do you handle these records?

Q82. *[Scenario | Hard]*

> Company context: A bank wants a model to flag fraudulent card transactions. Only 0.2 percent of 1 million transactions are fraud. Your first logistic regression has 99.8 percent accuracy and the fraud team says it is useless. Explain and fix.

Q83. *[Scenario | Medium]*

Company context: A hospital group receives lab results where 30 percent of cholesterol values and 8 percent of blood pressure values are missing, with different patterns across clinics. A research analyst needs a dataset to compare treatment outcomes. How do you deal with the missing data?

Q84. *[Scenario | Hard]*

Company context: A marketing team runs ads on four channels. Customers often touch several channels before buying, so the same purchase appears in several channel logs and the dashboard attributes 1.4 times actual revenue. You must produce a single, deduplicated attribution table with first-touch, last-touch and linear attribution. How do you do it?

Q85. *[Scenario | Hard]*

Company context: After a merger, two insurance companies must combine customer lists. System A has customer IDs, names and addresses; system B has different IDs and slightly different spellings ("Jon Smith, 12 Baker St" vs "John Smith, 12 Baker Street"). Marketing needs a single unique customer count. How do you match records in Python?

Q86. *[Scenario | Medium]*

Company context: Your team's weekly sales report script, written with pandas, now takes 40 minutes and uses iterrows and apply on 5 million rows. The business needs it in under 5 minutes. How do you optimise it?

Q87. *[Scenario | Medium]*

Company context: A SaaS company stores event timestamps from users in India, the US and Germany. Some are in UTC, others in local time without timezone information, and daily active user counts do not match the finance report. How do you fix the time handling?

Q88. *[Scenario | Medium]*

Company context: A retail store manager reports that products keep running out of stock. You have daily inventory and sales data for 2,000 SKUs. Identify which products stock out most often and estimate lost sales.

Q89. *[Scenario | Medium]*

Company context: A credit-card company wants to segment 100,000 customers into groups for tailored offers. Features: age, income, monthly spend, number of transactions, credit utilization, tenure. Describe how you use clustering in Python and how you ensure the clusters are useful.

Q90. *[Scenario | Medium]*

Company context: A food-delivery company's finance team needs to reconcile the orders database (what customers were charged) against the payment processor's settlement file (what was actually received). Totals differ by about 0.7 percent. Find the unmatched and mismatched transactions with pandas.

SQL

Q91. *[Concept | Easy]*

> What is the difference between WHERE and HAVING?

Q92. *[Concept | Easy]*

> Explain INNER, LEFT, RIGHT, FULL OUTER and CROSS joins.

Q93. *[Concept | Easy]*

> What is the difference between UNION and UNION ALL?

Q94. *[Concept | Easy]*

> What is the difference between COUNT(*), COUNT(column) and COUNT(DISTINCT column)?

Q95. *[Concept | Easy]*

> How does SQL treat NULL values? What pitfalls should an analyst know?

Q96. *[Concept | Medium]*

> Explain the logical order of execution of a SQL query and why it matters.

Q97. *[Concept | Medium]*

> What are window functions and how do they differ from GROUP BY?

Q98. *[Concept | Medium]*

> Differentiate ROW_NUMBER, RANK and DENSE_RANK.

Q99. *[Concept | Medium]*

> Explain the window frame clause: ROWS vs RANGE, and the default frame.

Q100. *[Concept | Medium]*

> What is a CTE? How does it differ from a subquery, a temp table and a view?

Q101. *[Concept | Hard]*

> What is a recursive CTE and when would you use it?

Q102. *[Concept | Easy]*

> What is the difference between a correlated and a non-correlated subquery?

Q103. *[Concept | Medium]*

IN vs EXISTS vs JOIN: when do you use each, and how do they handle NULLs?

Q104. *[Concept | Easy]*

What are primary keys, foreign keys, unique constraints and candidate keys?

Q105. *[Concept | Medium]*

Explain database normalization and the first three normal forms.

Q106. *[Concept | Medium]*

What is denormalization and when is it appropriate? Describe star vs snowflake schema.

Q107. *[Concept | Medium]*

What is an index? How does a B-tree index work and what are the trade-offs?

Q108. *[Concept | Medium]*

Clustered vs non-clustered index, composite index, and covering index: explain.

Q109. *[Concept | Medium]*

How would you read and use an execution plan (EXPLAIN) to optimize a slow query?

Q110. *[Concept | Medium]*

What does 'sargable' mean? Give examples of non-sargable predicates and their fixes.

Q111. *[Concept | Medium]*

Explain ACID properties and transaction isolation levels.

Q112. *[Concept | Easy]*

What is the difference between DELETE, TRUNCATE and DROP?

Q113. *[Concept | Easy]*

Categorize SQL commands: DDL, DML, DQL, DCL, TCL.

Q114. *[Concept | Medium]*

What are the differences between a view, a materialized view and a table?

Q115. *[Concept | Medium]*

Compare stored procedures, functions and triggers.

Q116. *[Concept | Medium]*

What is the difference between a self join and other joins, and when is a self join used?

Q117. *[Concept | Medium]*

What is the difference between DISTINCT and GROUP BY? When do they give the same result?

Q118. *[Concept | Hard]*

What is the fan-out (join explosion) problem and how do you prevent wrong aggregates because of it?

Q119. *[Concept | Medium]*

Explain the CASE expression and conditional aggregation.

Q120. *[Concept | Medium]*

How can you pivot and unpivot data in SQL?

Q121. *[Concept | Medium]*

What is the difference between LAG/LEAD and FIRST_VALUE/LAST_VALUE? What is the LAST_VALUE trap?

Q122. *[Concept | Medium]*

What are the main ways to remove duplicate rows from a table, and how do you decide which to keep?

Q123. *[Concept | Medium]*

How do WHERE and ON conditions differ in LEFT JOIN vs INNER JOIN?

Q124. *[Concept | Hard]*

Explain partitioning vs sharding vs indexing and when each helps.

Q125. *[Concept | Medium]*

Describe the difference between OLTP and OLAP databases and between row-store and column-store.

Q126. *[Concept | Medium]*

What is a LATERAL join (or CROSS APPLY) and when is it useful?

Q127. *[Concept | Hard]*

How do MERGE / UPSERT statements work, and what are idempotent loads?

Q128. *[Concept | Hard]*

Explain what a deadlock is, and how locking affects analytical queries.

Q129. *[Concept | Medium]*

What is the difference between RANK-based top-N and using LIMIT, and how do ties affect 'top N' questions?

Q130. *[Concept | Medium]*

How do you handle date and time operations in SQL (truncation, differences, time zones)? Give common patterns.

Q131. *[Concept | Hard]*

Compare the join algorithms (nested loop, hash join, merge join) and when the optimizer picks each.

Q132. *[Coding | Easy]*

Second highest salary. Table employees(emp_id INT, name VARCHAR, dept_id INT, salary INT). Sample: (1,'A',10,100), (2,'B',10,200), (3,'C',20,200), (4,'D',20,150). Return the second highest distinct salary, or NULL if it does not exist.

Q133. *[Coding | Medium]*

Top 3 salaries per department. Tables: employees(emp_id, name, dept_id, salary), departments(dept_id, dept_name). Return dept_name, name, salary for employees who are among the top 3 distinct salaries in their department.

Q134. *[Coding | Easy]*

Employees earning more than their manager. Table employees(emp_id, name, salary, manager_id) where manager_id references emp_id. Sample: (1,'Joe',70,3), (2,'Henry',80,4), (3,'Sam',60,NULL), (4,'Max',90,NULL). Return names of such employees.

Q135. *[Coding | Easy]*

Find duplicate emails. Table users(id INT, email VARCHAR). Sample: (1,'a@x.com'), (2,'b@x.com'), (3,'a@x.com'). Return each email that appears more than once and how many times.

Q136. *[Coding | Easy]*

Customers who never ordered. Tables customers(customer_id, name) and orders(order_id, customer_id, amount). Return names of customers with no orders. Show two ways.

Q137. *[Coding | Easy]*

Running total of sales. Table sales(sale_date DATE, store_id INT, amount NUMERIC). Show each store's daily amount, and a running total per store ordered by date. Sample: store 1: 1-Jan 100, 2-Jan 50, 3-Jan 25.

Q138. *[Coding | Medium]*

7-day moving average of revenue. Table daily_revenue(dt DATE, revenue NUMERIC), one row per day, but some days may be missing. Compute a trailing 7-calendar-day average, only reporting where a full 7 days of window exists.

Q139. *[Coding | Medium]*

Month-over-month revenue growth. Table orders(order_id, order_date DATE, amount NUMERIC). Return month, revenue, previous month revenue and percent growth.

Q140. *[Coding | Hard]*

Longest login streak (gaps and islands). Table logins(user_id INT, login_date DATE), possibly duplicates per day. Return each user's longest streak of consecutive login days, with start and end dates.

Q141. *[Coding | Medium]*

Users who logged in on 3 or more consecutive days. Table logins(user_id, login_date). Return distinct user_ids. Show a method without the date-minus-row-number trick.

Q142. *[Coding | Medium]*

Median salary. Table employees(emp_id, dept_id, salary). Compute the median salary per department without relying on a built-in median function.

Q143. *[Coding | Medium]*

Latest record per user. Table profile_changes(change_id, user_id, plan, changed_at TIMESTAMP). Return the current plan of each user (their most recent row).

Q144. *[Coding | Medium]*

Pivot monthly sales. Table sales(product VARCHAR, sale_month INT, amount NUMERIC) with months 1 to 3. Produce one row per product with columns m1, m2, m3 and a total.

Q145. *[Coding | Easy]*

Percent of total. Table orders(order_id, category VARCHAR, amount NUMERIC). Return each category's revenue and its share of total revenue as a percentage with 1 decimal.

Q146. *[Coding | Medium]*

Customers who bought every product (relational division). Tables products(product_id), purchases(customer_id, product_id). Return customers who purchased all products.

Q147. *[Coding | Medium]*

Find missing IDs in a sequence. Table invoices(invoice_no INT), values like 1,2,3,5,6,9. Return the gaps as ranges (4-4, 7-8).

Q148. *[Coding | Hard]*

Merge overlapping intervals. Table bookings(room_id INT, start_ts TIMESTAMP, end_ts TIMESTAMP). Collapse overlapping or touching bookings per room into merged blocks and compute total occupied time.

Q149. *[Coding | Hard]*

Sessionization. Table events(user_id, event_ts TIMESTAMP, event_name). Define a session as a series of events with no gap greater than 30 minutes. Assign a session number to each event and compute events and duration per session.

Q150. *[Coding | Easy]*

Rising temperature. Table weather(record_date DATE, temperature INT) with one row per day. Return the dates whose temperature is higher than the previous day's.

Q151. *[Coding | Easy]*

Employees in the same department with the same salary. Table employees(emp_id, name, dept_id, salary). Return pairs of employees (without mirror duplicates) sharing department and salary.

Q152. *[Coding | Medium]*

Top 2 products per category by revenue. Tables products(product_id, category), order_items(order_id, product_id, qty, unit_price). Return category, product_id, revenue for the top 2 products in each category.

Q153. *[Coding | Medium]*

Pareto analysis. Table orders(customer_id, amount). Find the smallest set of top customers that account for 80 percent of total revenue.

Q154. *[Coding | Easy]*

Orders per customer including zero. Tables customers(customer_id, name), orders(order_id, customer_id). Return every customer with their number of orders, zero for none.

Q155. *[Coding | Medium]*

Fill missing dates with zero. Table daily_signups(dt DATE, signups INT) lacks rows for days with no signups. Produce a continuous series for January 2024 with 0 for missing days.

Q156. *[Coding | Medium]*

Employees earning above their department average. Table employees(emp_id, name, dept_id, salary). Return them with the department average and the gap.

Q157. *[Coding | Medium]*

Conversion between two events. Table events(user_id, event_name, event_ts) with names 'view' and 'purchase'. Compute, per day, the number of users who viewed and the percentage of those who purchased the same day.

Q158. *[Coding | Medium]*

Customers who ordered in two consecutive months. Table orders(order_id, customer_id, order_date). Return customers having orders in at least two consecutive calendar months.

Q159. *[Coding | Medium]*

Consecutive identical values. Table logs(id INT, num INT) where id is sequential. Find numbers that appear at least three times consecutively. Sample: ids 1..7 with nums 1,1,1,2,1,2,2.

Q160. *[Coding | Medium]*

Highest paid employee in each department including ties. Tables employees(emp_id, name, dept_id, salary) and departments(dept_id, dept_name). Return department, name and salary of everyone who has the maximum salary in their department.

Q161. *[Coding | Hard]*

Cumulative distinct users. Table logins(user_id, login_date). For each date, report the cumulative number of distinct users who have ever logged in up to that date.

Q162. *[Coding | Hard]*

Rolling 30-day active users. Table events(user_id, event_date DATE). For each date D, count distinct users with at least one event in the window D-29 to D.

Q163. *[Coding | Hard]*

Nth highest salary function and handling of fewer than N distinct values. Table employees(emp_id, salary). Write a query for N = 5 that returns NULL if fewer than 5 distinct salaries exist, and explain why LIMIT/OFFSET needs care.

Q164. *[Scenario | Medium]*

Company: an e-commerce marketplace. Management wants monthly customer retention: of customers whose first purchase was in a given month, what percent purchase again in each following month? Tables: orders(order_id, customer_id, order_date, amount). Walk through the analysis and write the SQL.

Q165. *[Scenario | Medium]*

Company: a ride-hailing platform. Operations asks for the daily cancellation rate of requests made by unbanned riders with unbanned drivers, for 1 to 3 October 2023. Tables: trips(trip_id, rider_id, driver_id, status ('completed','cancelled_by_driver','cancelled_by_rider'), request_date DATE), users(user_id, banned ('Yes'/'No'), role). Write the query and explain the steps.

Q166. *[Scenario | Medium]*

Company: a video streaming service. Product wants a signup-to-paid funnel for users who signed up in March 2024: signup, started free trial, watched at least one video, converted to paid. Tables: users(user_id, signup_ts), trials(user_id, trial_start_ts), views(user_id, view_ts), subscriptions(user_id, paid_start_ts). Produce funnel counts and step conversion rates.

Q167. *[Scenario | Hard]*

Company: a retail bank. The fraud team wants to flag accounts that made 3 or more card transactions within any 10-minute window, and transactions that look like duplicate charges (same account, merchant and amount within 2 minutes). Table: transactions(txn_id, account_id, merchant_id, amount, txn_ts TIMESTAMP). Provide queries and reasoning.

Q168. *[Scenario | Medium]*

Company: an online fashion retailer. Marketing wants an RFM (recency, frequency, monetary) segmentation of customers as of 2024-06-30 to target campaigns. Table: orders(order_id, customer_id, order_date, amount). Build scores from 1 to 5 for each dimension and label top customers as 'Champions'.

Q169. *[Scenario | Medium]*

Company: a music and video streaming app. The content team wants the top 3 shows by total watch minutes in each genre for last month, excluding shows with fewer than 100 distinct viewers. Tables: shows(show_id, title, genre), watch_events(user_id, show_id, minutes_watched, watch_date). Write the query.

Q170. *[Scenario | Hard]*

Company: a retail bank. Each account has transactions with signed amounts (deposits positive, withdrawals negative). Finance needs, for every account, the end-of-month balance for every month from the account's opening to December 2024, including months with no transactions, and a flag for months where the balance was negative at any point. Tables: accounts(account_id, opened_date), transactions(account_id, txn_ts, amount). Describe the approach and give SQL.

Q171. *[Scenario | Medium]*

Company: a mobile game studio. Compute Day-1, Day-7 and Day-30 retention for each install-date cohort. Tables: installs(user_id, install_date), sessions(user_id, session_date). Retention Dn = share of the cohort that has a session exactly n days after install. Explain choices and write the SQL.

Q172. *[Scenario | Medium]*

Company: a SaaS subscription business. Compute monthly logo churn: for each month, the percentage of customers active at the start of the month who are not active at the end. Table: subscriptions(customer_id, start_date, end_date) where end_date is NULL for active subscriptions. Explain your definitions and write SQL.

Q173. *[Scenario | Medium]*

Company: an e-commerce site running an A/B test on a new checkout page. Table assignments(user_id, variant ('A','B'), assigned_at), orders(order_id, user_id, order_ts, amount). Compute conversion rate, average order value and revenue per user for each variant, counting only orders after assignment, and say what else you would check.

Q174. *[Scenario | Medium]*

Company: a supermarket chain. The merchandising team wants to know which pairs of products are most often bought together in the same basket. Table: order_items(order_id, product_id). Return the top 10 pairs with the number of baskets, support (percent of all baskets) and lift is a stretch goal.

Q175. *[Scenario | Medium]*

Company: a social video app. Compute daily active users (DAU), monthly active users (MAU) over a trailing 30 days, and the stickiness ratio DAU/MAU for each day. Table: activity(user_id, activity_date). Show your approach and the limits of the query.

Q176. *[Scenario | Hard]*

Company: a lender. Risk wants a delinquency report: for each monthly loan origination cohort, the percentage of loans that were ever 30+ days past due within their first 12 months on book. Tables: loans(loan_id, origination_date, amount), payments_due(loan_id, due_date, paid_date NULL if unpaid). Describe the logic and give SQL.

Q177. *[Scenario | Medium]*

Company: a food delivery platform. Operations wants delivery-time statistics per city for the last 30 days: average, median, 90th percentile, and percent of deliveries over 45 minutes, excluding cancelled orders. Table: deliveries(delivery_id, city, ordered_ts, delivered_ts, status). Provide the SQL and explain why the average alone is misleading.

Q178. *[Scenario | Medium]*

Company: a ride-hailing app. Growth wants to know what share of newly registered riders take their first ride within 7 days of signup, and the median time to first ride, by signup week. Tables: riders(rider_id, signup_ts), rides(ride_id, rider_id, ride_ts, status). Walk through the solution.

Q179. *[Scenario | Hard]*

Company: an e-commerce firm. A key dashboard query takes 4 minutes: it computes monthly revenue by category using orders (500M rows), order_items (1.5B rows) and products (200K rows) with a filter on the last 12 months. As the analytics engineer, explain how you would diagnose and speed it up.

Q180. *[Scenario | Medium]*

Company: a healthcare analytics firm receiving daily patient-visit data loads. Before building reports, you must run a data quality audit on visits(visit_id, patient_id, visit_date, doctor_id, charge) and patients(patient_id, dob). Write the checks you would run in SQL and what each reveals.

Q181. *[Scenario | Hard]*

Company: a B2B subscription company. Finance wants a monthly MRR bridge: starting MRR, new, expansion, contraction, churn, and ending MRR. Table: mrr_snapshot(customer_id, month DATE (first of month), mrr NUMERIC), one row per customer per month in which they were paying. Write the SQL.

Q182. *[Scenario | Medium]*

Company: a hotel booking platform. Detect double bookings: any two reservations for the same room whose date ranges overlap. Table: reservations(res_id, room_id, check_in DATE, check_out DATE) where check_out is the departure day (the room is free that night). Return the conflicting pairs and explain the overlap logic.

Q183. *[Scenario | Medium]*

Company: an online electronics store. The CEO asks about customer lifetime value: for each customer's first-purchase month cohort, what is the average cumulative revenue per customer after 1, 3, 6 and 12 months, and what is the repeat purchase rate? Tables: orders(order_id, customer_id, order_date, amount). Provide the approach and SQL.

Q184. *[Scenario | Hard]*

Company: a streaming service with subscription plans whose prices changed over time. Table plan_prices(plan_id, price, valid_from DATE, valid_to DATE (NULL for current)) and invoices(invoice_id, customer_id, plan_id, invoice_date, billed_amount). Finance suspects some invoices were billed at the wrong price. Write a query to find invoices whose billed amount differs from the price valid on the invoice date, and quantify the total over- and under-billing.

Q185. *[Scenario | Medium]*

Company: a marketplace with a CRM containing duplicate customer records. customers(customer_id, email, phone, full_name, created_at). The team wants to identify duplicate customers (same normalized email or same phone), pick a master record for each group (oldest), and produce a mapping table old_id to master_id. Explain and write SQL.

Q186. *[Scenario | Medium]*

Company: an e-commerce firm using last-touch marketing attribution. Tables: sessions(session_id, user_id, session_ts, channel) and purchases(purchase_id, user_id, purchase_ts, amount). Attribute each purchase's revenue to the channel of the user's most recent session before the purchase (within 7 days). Provide the SQL and compare with first-touch.

Q187. *[Scenario | Hard]*

Company: a ride-hailing company. Marketplace analytics wants, for each city and hour of day, the supply-demand ratio (available drivers per ride request) and to identify the 5 worst city-hours in the last 7 days. Tables: ride_requests(request_id, city_id, request_ts), driver_status_log(driver_id, city_id, status ('online','offline'), status_ts). Explain how to derive hourly driver availability from the status log and write SQL.

Q188. *[Scenario | Easy]*

Company: a retail chain. A regional manager asks for the sales report 'total sales per store for each quarter of 2024, with the store's rank in its region for each quarter'. Tables: stores(store_id, region), sales(sale_id, store_id, sale_date, amount). Walk through the logic and give SQL.

Q189. *[Scenario | Hard]*

Company: a streaming platform. Analysts notice that a viewing-hours report shows totals 3 times higher than finance's number after someone added a join to a subscriptions table. Tables: viewing(user_id, title_id, minutes, view_date), subscriptions(user_id, plan, start_date, end_date) where users can have several subscription rows (plan changes). Diagnose and fix the query: SELECT SUM(v.minutes) FROM viewing v JOIN subscriptions s ON s.user_id = v.user_id.

Q190. *[Scenario | Medium]*

Company: a payments fintech. The finance team needs a daily reconciliation between internal ledger entries and bank settlement files. Tables: ledger(txn_id, amount, txn_date), bank_settlements(bank_ref, txn_id, amount, settled_date). Produce a report of: matched, in ledger but missing from bank, in bank but missing from ledger, and amount mismatches.

Power BI

Q191. *[Concept | Easy]*

What are the main components of Power BI and how do they fit together in a typical workflow?

Q192. *[Concept | Medium]*

Explain Import, DirectQuery, Live Connection and Composite models. When would you choose each?

Q193. *[Concept | Easy]*

What is a star schema and why is it the recommended modeling approach in Power BI?

Q194. *[Concept | Medium]*

Describe relationship cardinality and cross-filter direction in Power BI. What problems does bidirectional filtering cause?

Q195. *[Concept | Easy]*

What is the difference between a calculated column and a measure? When do you use each?

Q196. *[Concept | Medium]*

Explain row context and filter context in DAX and how they interact.

Q197. *[Concept | Hard]*

What is context transition in DAX? Give an example where it happens and a pitfall.

Q198. *[Concept | Easy]*

When should you do a transformation in Power Query versus in DAX?

Q199. *[Concept | Hard]*

What is query folding in Power Query, how can you check whether it is happening, and what breaks it?

Q200. *[Concept | Easy]*

What is the difference between Merge and Append in Power Query?

Q201. *[Concept | Medium]*

Explain Row-Level Security in Power BI: static vs dynamic RLS, how roles are defined, and how they are tested and assigned.

Q202. *[Concept | Medium]*

What is an on-premises data gateway? Explain the types, when you need one, and common refresh problems.

Q203. *[Concept | Hard]*

Explain incremental refresh: how it works, prerequisites and limitations.

Q204. *[Concept | Medium]*

What are the main techniques to optimize the performance of a Power BI data model and report?

Q205. *[Concept | Hard]*

How do you diagnose a slow Power BI report using Performance Analyzer, DAX Studio and VertiPaq Analyzer? What do the numbers tell you?

Q206. *[Concept | Medium]*

What are role-playing dimensions and how do you handle a Date table related to Sales by both Order Date and Ship Date?

Q207. *[Concept | Hard]*

How do you model a many-to-many relationship in Power BI? Describe the bridge table pattern and the native many-to-many option.

Q208. *[Concept | Easy]*

What types of filters exist in Power BI reports, and how do they differ from slicers?

Q209. *[Concept | Easy]*

Explain drill-down, drill-through, bookmarks and tooltip pages. What does each do for dashboard usability?

Q210. *[Concept | Medium]*

Describe Power BI Service workspaces, roles, apps and sharing options. What is the recommended way to distribute reports to a large audience?

Q211. *[Concept | Medium]*

What are dataflows, and how do they differ from semantic models (datasets)? When would you use them?

Q212. *[Concept | Easy]*

Why do you need a dedicated Date table, and how do you create and configure one for time intelligence?

Q213. *[Concept | Hard]*

Explain table storage modes (Import, DirectQuery, Dual) and how aggregation tables improve performance in a composite model.

Q214. *[Concept | Medium]*

What are Power Query parameters and how are they used (for example for environment switching or filtering)?

Q215. *[Concept | Easy]*

What is the difference between a Power BI report and a dashboard?

Q216. *[Concept | Medium]*

What are best practices for designing an effective, user-friendly Power BI dashboard or report?

Q217. *[Concept | Medium]*

How do you handle errors, nulls, and inconsistent data types in Power Query?

Q218. *[Concept | Easy]*

Describe Power BI licensing and the differences between Free, Pro, Premium Per User and capacity (Premium/Fabric) in terms of what an analyst needs to know.

Q219. *[Concept | Hard]*

What does the VertiPaq engine do and why does cardinality matter for model size and speed?

Q220. *[Concept | Medium]*

What are the main visuals in Power BI and when would you choose a matrix over a table, a waterfall, a scatter chart or a decomposition tree?

Q221. *[Concept | Medium]*

What are field parameters and what-if parameters? How do they differ and what are the use cases?

Q222. *[Concept | Hard]*

How do you manage development, testing and production of Power BI content using deployment pipelines and version control?

Q223. *[Coding | Easy]*

Write DAX measures for Total Sales, Total Cost, Profit, and Profit Margin % given a Sales table with Quantity, UnitPrice and UnitCost columns. Explain why you use SUMX.

Q224. *[Coding | Medium]*

Explain CALCULATE with examples: override a filter, add a filter on a related dimension, and the difference between boolean filter arguments and FILTER().

Q225. *[Coding | Medium]*

Explain ALL, ALLEXCEPT, ALLSELECTED and REMOVEFILTERS with a measure for each use case.

Q226. *[Coding | Medium]*

Write measures for Year-to-Date sales, Prior-Year sales and Year-over-Year growth %, using a marked Date table.

Q227. *[Coding | Medium]*

Write a rolling (moving) 3-month average of sales and a 12-month trailing total.

Q228. *[Coding | Medium]*

Write a running (cumulative) total measure that works over dates, and one that resets each year.

Q229. *[Coding | Medium]*

How do you rank products by sales and show only the Top 5 with RANKX and TOPN? Write the DAX and explain handling of ties and filter context.

Q230. *[Coding | Hard]*

Write DAX to count new customers versus returning customers in a selected period, given a Sales table with CustomerKey and OrderDate.

Q231. *[Coding | Medium]*

Show how to use RELATED, RELATEDTABLE and SUMX to compute values across tables, for example total discounted sales where the discount rate lives on the Product table and quantity and price on Sales.

Q232. *[Coding | Easy]*

Write a calculated column that categorizes customers by age (under 18, 18-34, 35-54, 55+) using SWITCH, and explain why SWITCH(TRUE(), ...) is used.

Q233. *[Coding | Easy]*

How do you handle blanks and division by zero in DAX? Show DIVIDE, IF with ISBLANK, COALESCE, and HASONEVALUE/SELECTEDVALUE.

Q234. *[Coding | Medium]*

Write the dynamic RLS setup: tables, relationship, DAX role filter, and a variant where a manager sees all their regions, using USERPRINCIPALNAME.

Q235. *[Coding | Medium]*

Write Power Query M code that loads a CSV with a header row, promotes headers, sets types, trims text, removes duplicates and filters out rows with a null OrderID. Then show how to unpivot month columns.

Q236. *[Coding | Hard]*

Write a measure for closing inventory balance (semi-additive) where the stock snapshot table has one row per product per day, and the total over time must be the last available value rather than a sum.

Q237. *[Scenario | Medium]*

Company context: A national retail chain (200 stores, 50,000 SKUs) wants a Sales Performance dashboard for executives, regional managers and store managers. Data lives in a SQL Server warehouse with a Sales fact

(about 300 million rows), and Product, Store, Customer and Date dimensions. How would you design the dashboard and data model end to end?

Q238. *[Scenario | Medium]*

Company context: A logistics company's Power BI report takes about 40 seconds to load a page, and users complain. The model is Import mode (about 80 million rows), a single flat table of 70 columns, several bidirectional relationships, and the page has 25 visuals plus many slicers. Walk through how you would find and fix the slowness.

Q239. *[Scenario | Medium]*

Company context: A SaaS company wants an executive KPI scorecard showing Monthly Recurring Revenue (MRR), churn rate, customer acquisition cost, and net revenue retention, each against a target. How would you design the KPI measures and the visual layout in Power BI?

Q240. *[Scenario | Medium]*

Company context: A national insurance company has 12 regional managers, and each should only see policies and claims for their own region in the same sales report; the national head sees everything, and some managers cover two regions. The report is shared through the Power BI Service. Design the row-level security solution.

Q241. *[Scenario | Medium]*

Company context: A finance team receives 36 monthly Excel files (one per month, same layout) in a SharePoint folder with inconsistent headers and extra title rows. They manually copy them into a master workbook each month. How would you automate this in Power BI?

Q242. *[Scenario | Medium]*

Company context: A manufacturing firm's scheduled refresh in the Power BI Service started failing overnight. The model pulls from an on-premises SQL Server through a gateway and from an Excel file on a network share. Describe how you troubleshoot and resolve it.

Q243. *[Scenario | Hard]*

Company context: A telecom operator has a call-detail fact table with 2 billion rows and needs interactive dashboards. Analysts also need drill-down to individual call records, and the data is refreshed in the warehouse every hour. Import will not fit. What architecture would you propose?

Q244. *[Scenario | Medium]*

Company context: A CEO asks for a "real-time" operations dashboard showing orders as they happen on an e-commerce site, refreshed within seconds on a wall display. Power BI Import refresh is limited. What options do you have and which would you recommend?

Q245. *[Scenario | Hard]*

Company context: A consumer goods company has sales facts at Day, Product and Store grain, but the budget is stored at Month and Product Category grain by Region. Management wants a single visual showing Actual versus Budget by category and month, and they see wrong totals when they try to relate the tables. How do you model and calculate this?

Q246. *[Scenario | Hard]*

Company context: A subscription streaming service wants a customer retention analysis: for each monthly signup cohort, what percentage of customers are still active 1, 2, 3 ... 12 months later? The data has a Customers table (SignupDate) and an Activity table (CustomerKey, ActivityDate). How do you build the cohort retention matrix?

Q247. *[Scenario | Medium]*

Company context: Five departments (Sales, Marketing, Finance, Supply Chain, HR) each built their own Power BI reports with their own copies of the "Customer" and "Revenue" logic, and the numbers don't match in executive meetings. How would you fix this in Power BI?

Q248. *[Scenario | Medium]*

Company context: A retailer wants to know which products drive 80 percent of revenue (ABC / Pareto analysis) and wants a Pareto chart in Power BI that updates with slicers. How would you build it?

Q249. *[Scenario | Medium]*

Company context: In a matrix showing Margin % by product, the Total row displays 18 percent, but when analysts average the row values they get 24 percent, and one custom measure "Avg Price" total looks wrong. Management says the totals are "wrong". How do you explain and fix this?

Q250. *[Scenario | Medium]*

Company context: A finance department runs a recurring monthly Excel pack of profit and loss statements with pivot tables and wants to move it to Power BI. How do you plan and execute the migration so users accept it?

Q251. *[Scenario | Medium]*

Company context: A multinational shows revenue in USD, but regional managers want to see sales in their local currency and the CFO wants to simulate the impact of a 5 percent currency swing. Exchange rates are in a daily rates table. How do you build this in Power BI?

Q252. *[Scenario | Medium]*

Company context: A call center wants an operations dashboard with Average Handle Time (AHT), Service Level (percent of calls answered within 20 seconds), Abandon Rate and calls by hour of day, filterable by team and date, with color-coded status. Describe the measures and layout.

Q253. *[Scenario | Hard]*

Company context: An HR analytics team tracks employees whose manager and department change over time (slowly changing dimension). Sales reps also move territories during the year. The business wants sales credited to the territory the rep belonged to on the date of each sale. How do you model and calculate this in Power BI?

Q254. *[Scenario | Medium]*

Company context: Executives want a single chart where they can choose which measure (Revenue, Profit, Units) and which breakdown (Region, Category, Channel) to show, instead of having 9 separate visuals cluttering the page. How would you implement this?

Q255. *[Scenario | Hard]*

Company context: A fast-growing retailer has an Import-mode Power BI model of 4 GB that refreshes fully each night in 3 hours and now hits the memory limit on its Premium capacity during peak hours when many users are querying and refreshing at once. Propose a plan to reduce refresh time and memory, and explain how you would verify improvements.

Excel

Q256. *[Concept | Easy]*

Compare VLOOKUP, INDEX-MATCH and XLOOKUP. When would you choose each one?

Q257. *[Concept | Easy]*

What is the difference between exact match and approximate match in VLOOKUP, and when is approximate match appropriate?

Q258. *[Concept | Easy]*

Explain relative, absolute and mixed cell references and give an example of when mixed references are useful.

Q259. *[Concept | Easy]*

What is a PivotTable, and how do you create one from raw data? What are its main areas?

Q260. *[Concept | Medium]*

What is a pivot cache? Why does a PivotTable not update automatically, and how do you handle a source range that grows?

Q261. *[Concept | Medium]*

What is the difference between a calculated field and a calculated item in a PivotTable? What are their limitations?

Q262. *[Concept | Easy]*

What are slicers and timelines, and how do they differ from regular PivotTable filters?

Q263. *[Concept | Medium]*

What are dynamic array functions in Excel? Explain spilling, the # operator and name a few key functions.

Q264. *[Concept | Medium]*

What are volatile functions in Excel? Give examples and explain why they matter for performance.

Q265. *[Concept | Easy]*

What are named ranges and Excel Tables, and what advantages do structured references give over normal cell references?

Q266. *[Concept | Medium]*

Explain Power Query vs Power Pivot in Excel. When would you use each?

Q267. *[Concept | Easy]*

What types of conditional formatting are available in Excel, and how do formula-based rules work?

Q268. *[Concept | Easy]*

What is data validation, and what kinds of restrictions can you apply? How would you create a drop-down list?

Q269. *[Concept | Medium]*

Describe Excel's What-If Analysis tools: Scenario Manager, Goal Seek and Data Tables. When would you use each?

Q270. *[Concept | Medium]*

What is Excel Solver and what are its three components? Which solving methods does it offer?

Q271. *[Concept | Easy]*

What is a macro, and how does it relate to VBA? How do you record and run one, and what are the risks?

Q272. *[Concept | Easy]*

How do you choose the right chart type in Excel? Give guidance for common analytical goals.

Q273. *[Concept | Medium]*

What is an array formula? Explain the difference between legacy Ctrl+Shift+Enter formulas and dynamic array formulas, with an example.

Q274. *[Concept | Medium]*

What are the best practices for designing an Excel dashboard for business stakeholders?

Q275. *[Concept | Easy]*

List the common Excel error values (#N/A, #REF!, #VALUE!, #DIV/0!, #NAME?, #NUM!, #NULL!, #SPILL!) and how you handle them.

Q276. *[Concept | Easy]*

What Excel features and functions do you use for data cleaning before analysis?

Q277. *[Concept | Hard]*

A large workbook with many formulas is slow. How do Excel's calculation modes work, and what would you do to improve performance?

Q278. *[Concept | Medium]*

What is the difference between SUMIF, SUMIFS and SUMPRODUCT? When would you prefer each?

Q279. *[Concept | Easy]*

What are the benefits of converting a range into an Excel Table?

Q280. *[Concept | Hard]*

Explain LET and LAMBDA in Excel. How do they improve formulas, and what is a practical example?

Q281. *[Coding | Easy]*

A reference table in A1:D500 has Names in column A and Employee IDs in column D (the ID column is to the right of the Name column). Write a formula to return the Name for the ID typed in G2, i.e. a lookup that returns a value to the left of the key. Provide both INDEX-MATCH and XLOOKUP versions.

Q282. *[Coding | Medium]*

A sales sheet has Region in column A, Product in column B and Price in column C (rows 2 to 1000). Write formulas to return the price for the combination Region = G1 and Product = G2 using (a) INDEX-MATCH, (b) XLOOKUP and (c) a non-array approach with SUMIFS.

Q283. *[Coding | Medium]*

Write a formula that assigns a customer discount based on order value using a lookup table: 0 for under 1,000; 5% for 1,000 to 4,999; 10% for 5,000 to 19,999; 15% for 20,000 and above. Show an approach using a lookup table and a nested IFS approach.

Q284. *[Coding | Easy]*

Dates are in column A (A2:A1000) and amounts in column C. Write formulas to get total sales for March 2025 (a) using SUMIFS with date boundaries from a start-date cell, and (b) using a SUMPRODUCT with YEAR and MONTH.

Q285. *[Coding | Medium]*

Customer IDs are in A2:A1000 with duplicates. Write formulas to (a) count distinct customers, (b) flag the first occurrence of each customer, and (c) count how many customers purchased more than once.

Q286. *[Coding | Medium]*

Given a sales table in A1:D500 (Rep, Region, Month, Revenue), write dynamic-array formulas to list the top 5 reps by revenue for Region "West", showing rep and revenue.

Q287. *[Coding | Easy]*

Names are in column A like " john   SMITH " with extra spaces and inconsistent case. Write formulas to clean and standardise names, and to split into first and last names.

Q288. *[Coding | Easy]*

Monthly revenue is in B2:B13. Write formulas for the running (cumulative) total, the percentage share of the annual total, and the month-over-month growth.

Q289. *[Coding | Medium]*

Compute a sales rep's commission with tiered marginal rates: 2% on the first 50,000, 4% on the portion from 50,000 to 100,000, and 6% on anything above 100,000. Revenue is in B2. Provide a formula.

Q290. *[Coding | Medium]*

Write formulas for the following date tasks: employee tenure in years and months from a hire date in B2, the last day of the month for an invoice date in C2, a due date 30 days after the invoice date skipping weekends and holidays listed in H2:H20, and the number of working days between two dates.

Q291. *[Coding | Medium]*

Quantities are in B2:B20 and unit prices in C2:C20. Write formulas for the total revenue without a helper column, the weighted average price, and the average price excluding zero-quantity rows.

Q292. *[Coding | Medium]*

Write a conditional formatting formula to (a) highlight duplicate emails in A2:A1000, case-insensitively and ignoring trailing spaces, and (b) shade a whole row in A2:F500 when the due date in column E is past today and the status in column F is not "Paid".

Q293. *[Coding | Hard]*

Rank salespeople by revenue in a sheet where ties exist, producing unique ranks with ties broken by the order in the list, and a dense rank. Also return the name of the second highest seller. Revenue is in B2:B50 and names in A2:A50.

Q294. *[Coding | Medium]*

Write a VBA macro that loops through the rows of the active sheet's used range, highlights in red every row where the value in column D is below 0, and deletes completely blank rows. Explain how to make it efficient.

Q295. *[Scenario | Medium]*

Company context: You work in the finance team of a retail chain. At month end you must reconcile the sales ledger (system export, 12,000 rows: Invoice No, Date, Amount) against the bank/payment gateway settlement report (11,850 rows: Reference, Date, Amount). Totals differ by 18,420. How would you find the differences in Excel?

Q296. *[Scenario | Medium]*

Company context: A subscription software company's CRM has 40,000 customer records with duplicates caused by multiple sign-ups ("Jon Smith, jon.smith@gmail.com" and "JON SMITH , Jon.Smith@Gmail.com"). Marketing wants a clean, deduplicated customer list and a count of unique customers. How would you do it in Excel without losing data?

Q297. *[Scenario | Medium]*

Company context: A pharmaceutical distributor pays sales reps monthly commission. The plan: 3% of revenue for reps in Region North and South, 4% in East and West; an extra 1% bonus on revenue above the rep's quota if the rep achieved at least 100% of quota; a cap of 15,000 per month; and no commission for reps on probation. Data: Rep, Region, Revenue, Quota, Status in a table. Build the commission calculation.

Q298. *[Scenario | Medium]*

Company context: A regional bank branch network sends you 12 monthly Excel files (same columns, one sheet each) in a shared folder, plus one file each month is added. The analyst currently copies and pastes them into one sheet each time. How would you automate consolidation in Excel?

Q299. *[Scenario | Medium]*

Company context: A coffee-shop chain is planning to open a new store. The finance lead has a profit model with price, units sold, variable cost and fixed costs, and wants to know how monthly profit changes with price (from 3 to 6) and units sold (2,000 to 6,000), plus the break-even units at a price of 4.50. How would you do this in Excel?

Q300. *[Scenario | Hard]*

Company context: A furniture manufacturer makes tables, chairs and desks. Each needs wood hours, labour hours and has a profit: table 70 profit (4 wood, 3 labour), chair 30 profit (1 wood, 2 labour), desk 90 profit (5

wood, 4 labour). Weekly capacity is 400 wood hours and 300 labour hours, and at least 20 chairs must be made. Use Excel to determine the production mix that maximizes profit.

Q301. *[Scenario | Medium]*

Company context: An e-commerce company's leadership wants a one-page weekly sales dashboard from a transaction export (Order ID, Date, Category, Region, Customer ID, Quantity, Revenue, Cost). KPIs requested: total revenue, profit margin %, orders, average order value, revenue by region and category, and a monthly trend. Outline how you would build it.

Q302. *[Scenario | Medium]*

Company context: A wholesaler tracks inventory in a sheet with Item, Current Stock, Reorder Level, Lead Time (days) and Last Restock Date. The warehouse manager wants visual alerts for items needing reorder and for stock that has not moved for 90 days, plus a list of items to order each morning. How would you set this up?

Q303. *[Scenario | Medium]*

Company context: A market research firm receives a messy CSV export from a survey tool: dates stored as text ("2025/03/07", "07-Mar-25"), numbers with commas and currency symbols as text ("$1,200"), a blank-looking column with spaces, Yes/yes/Y/Y . variants, and some totals rows in the middle. Describe how you would clean it for analysis.

Q304. *[Scenario | Medium]*

Company context: An HR department at a 3,000-person company has an attendance export with Employee ID, Date, Clock-in, Clock-out. Management wants (1) hours worked per day, (2) overtime beyond 8 hours, (3) a monthly summary per employee, and (4) a macro that formats and emails nothing but prepares the monthly report sheet. Describe the solution.

Q305. *[Scenario | Medium]*

Company context: A mid-sized manufacturer has a loan of 500,000 at 8% annual interest. The CFO asks: what is the monthly payment for 5 years, how much interest will be paid in total, and what annual interest rate would make the payment 9,500 per month? Show how to do it in Excel.

Q306. *[Scenario | Medium]*

Company context: The accounts receivable team of a B2B distributor has a list of open invoices (Customer, Invoice No, Invoice Date, Due Date, Amount). The CFO wants an aging report with buckets: Current (not yet due), 1-30, 31-60, 61-90, 90+ days overdue, totals per customer, as of a report date. How would you build it?

Q307. *[Scenario | Hard]*

Company context: A consumer-goods company has 36 months of monthly sales and wants a 6-month forecast with a confidence range, and to understand whether the trend is seasonal. Using Excel only, how would you build and present the forecast?

Q308. *[Scenario | Medium]*

Company context: An insurance company's claims team needs a data-entry sheet where staff choose Country, then State/Province (only valid states for that country), then City. They also must enter claim amount (between 100 and 1,000,000) and a unique claim ID. Entries that break the rules must be rejected. How would you build it?

Q309. *[Scenario | Hard]*

Company context: A telecom operator has two files: a Customers table (Customer ID, Name, Plan, Signup Date) and a Usage table (Customer ID, Month, Minutes, Data GB) with 5 million usage rows. The analyst's VLOOKUP-based workbook crashes. Management wants average usage per plan per month. What approach would you take in Excel?

Q310. *[Coding | Easy]*

Write formulas to count (a) orders in column C with amount above 500 in region "East" (column A), (b) orders where the status in column B is blank, and (c) the average amount for East orders, ignoring errors in the amount column.

Math and Statistics

Q311. *[Concept | Easy]*

Explain mean, median and mode. When would you prefer the median over the mean?

Q312. *[Concept | Easy]*

What is the difference between variance and standard deviation, and why does the sample variance divide by n-1?

Q313. *[Concept | Easy]*

How do you detect outliers, and how should you handle them?

Q314. *[Concept | Easy]*

What is the difference between correlation and causation? Give an example and explain how you would establish causation.

Q315. *[Concept | Medium]*

Compare Pearson and Spearman correlation. When does each fail or mislead?

Q316. *[Concept | Easy]*

State the Central Limit Theorem and explain why it matters for data analysts.

Q317. *[Concept | Medium]*

What is the Law of Large Numbers and how does it differ from the Central Limit Theorem? Does it support the gambler's fallacy?

Q318. *[Concept | Easy]*

What is the difference between standard deviation and standard error?

Q319. *[Concept | Easy]*

What is a p-value? What does it not mean?

Q320. *[Concept | Easy]*

Explain Type I error, Type II error, significance level and statistical power, and how they trade off.

Q321. *[Concept | Medium]*

How should you interpret a 95% confidence interval? What are common misinterpretations?

Q322. *[Concept | Medium]*

How are confidence intervals and hypothesis tests related? If a 95% CI for a difference is [-0.5, 3.0], what can you conclude?

Q323. *[Concept | Easy]*

What are the different types of t-tests and when do you use each?

Q324. *[Concept | Medium]*

Explain the chi-square goodness-of-fit and chi-square test of independence. What are their assumptions?

Q325. *[Concept | Medium]*

What is ANOVA, what does the F-test tell you, and what do you do after a significant result?

Q326. *[Concept | Medium]*

What is the multiple comparisons problem and how can it affect A/B testing with many metrics or segments?

Q327. *[Concept | Hard]*

Compare Bonferroni correction with Benjamini-Hochberg (FDR). When would you choose each?

Q328. *[Concept | Easy]*

Describe the normal distribution and its key properties, including the empirical rule.

Q329. *[Concept | Medium]*

When would you model data with a binomial versus a Poisson distribution? How are they related?

Q330. *[Concept | Medium]*

What is the exponential distribution and what does the memoryless property mean?

Q331. *[Concept | Easy]*

State Bayes' theorem and explain prior, likelihood and posterior with an example.

Q332. *[Concept | Medium]*

State the key properties of expectation and variance, including linearity and variance of a sum.

Q333. *[Concept | Easy]*

What are common types of sampling bias and how do you mitigate them?

Q334. *[Concept | Medium]*

What is Simpson's paradox? Why does it happen and how do you decide whether to use the aggregated or the segmented data?

Q335. *[Concept | Medium]*

What is survivorship bias? Give an example of how it can distort an analysis.

Q336. *[Concept | Medium]*

What factors determine statistical power and sample size in a test? Give the sample size formula for comparing two means.

Q337. *[Coding | Medium]*

What is the bootstrap, when is it useful, and how would you compute a 95% bootstrap confidence interval for the median in Python?

Q338. *[Concept | Medium]*

Contrast the frequentist and Bayesian approaches to inference. What are the pros and cons of Bayesian A/B testing?

Q339. *[Concept | Hard]*

Explain maximum likelihood estimation. Derive the MLE for the success probability of a Bernoulli sample.

Q340. *[Concept | Medium]*

List the assumptions of linear regression and how you would diagnose violations.

Q341. *[Concept | Medium]*

What is multicollinearity, how do you detect it and what are its consequences?

Q342. *[Concept | Medium]*

What are eigenvalues and eigenvectors and how are they used in PCA?

Q343. *[Concept | Easy]*

What is the dot product and cosine similarity? When is cosine similarity preferred over Euclidean distance?

Q344. *[Concept | Medium]*

Explain gradient descent and the role of derivatives and the learning rate.

Q345. *[Concept | Medium]*

Derive the gradient of the mean squared error for simple linear regression and explain how it relates to the closed-form solution.

Q346. *[Concept | Easy]*

What is the difference between covariance and correlation?

Q347. *[Concept | Medium]*

What are non-parametric tests such as Mann-Whitney U, and when should you use them instead of a t-test?

Q348. *[Concept | Hard]*

Explain the bias-variance trade-off and how it relates to estimator quality (MSE).

Q349. *[Concept | Medium]*

What are the types of missing data (MCAR, MAR, MNAR) and how do they affect your handling choices?

Q350. *[Concept | Hard]*

What is the difference between a confidence interval, a prediction interval and a tolerance interval?

Q351. *[Concept | Hard]*

What is regression to the mean, and how can it mislead analysts?

Q352. *[Concept | Hard]*

What is the delta method and why is it needed for ratio metrics (for example revenue per session) in A/B tests?

Q353. *[Concept | Hard]*

Explain the sampling distribution of the difference of two proportions and write the z-test statistic, including when you use the pooled variance.

Q354. *[Concept | Easy]*

Probability: given a disease affecting 1% of a population, a test has 95% sensitivity and 90% specificity. A random person tests positive. What is the probability they have the disease?

Q355. *[Concept | Easy]*

Two fair dice are rolled. (a) What is the probability that the sum is 7? (b) Given that at least one die shows a 6, what is the probability that both dice show 6?

Q356. *[Coding | Medium]*

Birthday problem: how many people are needed in a room for the probability of a shared birthday to exceed 50%? Compute the exact probability for 23 people and show how to verify by simulation.

Q357. *[Concept | Medium]*

You flip a fair coin repeatedly. What is the expected number of flips until you first see two heads in a row (HH)? Is it the same as the expected number of flips until you see H followed by T (HT)?

Q358. *[Concept | Medium]*

Sample size calculation: a landing page converts at 10%. You want to detect an absolute lift to 11% with 5% two-sided significance and 80% power. How many visitors per variant do you need?

Q359. *[Concept | Easy]*

A sample of 64 delivery times has a mean of 50 minutes and a standard deviation of 16 minutes. Construct a 95% confidence interval for the true mean and interpret it.

Q360. *[Concept | Medium]*

Control converts 200 out of 4,000 visitors; treatment converts 240 out of 4,000. Run a two-proportion z-test and state your conclusion.

Q361. *[Concept | Medium]*

Group A (n = 25) has mean 52 and SD 10; group B (n = 25) has mean 47 and SD 10. Test whether the means differ at the 5% level.

Q362. *[Concept | Medium]*

A test of whether a customer converts, for two page layouts: Layout A: 30 converted, 70 did not; Layout B: 50 converted, 50 did not. Perform a chi-square test of independence.

Q363. *[Concept | Easy]*

A support center receives on average 3 calls per hour (Poisson). What is the probability of (a) no calls in an hour and (b) at least 2 calls in an hour?

Q364. *[Concept | Easy]*

You flip a fair coin 10 times. What is the probability of (a) exactly 7 heads and (b) at least 7 heads?

Q365. *[Concept | Easy]*

Exam scores are approximately normal with mean 70 and SD 10. (a) What fraction score above 85? (b) What score is needed to be in the top 10%?

Q366. *[Concept | Easy]*

Compute the expected value, variance and standard deviation of one roll of a fair six-sided die, and the variance of the sum of two independent rolls.

Q367. *[Coding | Medium]*

Monty Hall: you pick one of three doors; the host, who knows where the car is, opens a different door with a goat and offers you a switch. Should you switch? Prove it and show a simulation.

Q368. *[Concept | Medium]*

Given x = (1, 2, 3, 4, 5) and y = (2, 4, 5, 4, 5), compute the Pearson correlation and the least-squares regression line.

Q369. *[Concept | Easy]*

Perform two steps of gradient descent on f(w) = (w - 3)^2 starting at w = 0 with learning rate 0.1. What happens with learning rate 1.1?

Q370. *[Concept | Medium]*

Find the eigenvalues and eigenvectors of A = [[2, 1], [1, 2]] and explain what they mean geometrically.

Q371. *[Concept | Medium]*

What is the expected number of rolls of a fair die needed to see every face at least once? (coupon collector)

Q372. *[Concept | Easy]*

What is the probability of getting at least one six in four rolls of a fair die? Explain the complement trick.

Q373. *[Concept | Hard]*

Brain teaser: a family has two children. (a) At least one is a boy; what is the probability both are boys? (b) The older child is a boy; what is the probability both are boys? Why do they differ?

Q374. *[Scenario | Medium]*

An e-commerce company redesigned its checkout page. In a two-week A/B test, the new checkout converted 5.4% of 50,000 visitors against 5.0% of 50,000 for the old checkout. The product manager wants to launch. Walk through how you would analyse the result and make a recommendation.

Q375. *[Scenario | Hard]*

A pharmaceutical company runs a drug trial. The primary endpoint (blood pressure reduction) shows p = 0.12. However, among 20 pre-specified secondary subgroup analyses, one (women over 60) shows p = 0.03. The team wants to claim the drug works for women over 60. How do you respond?

Q376. *[Scenario | Medium]*

A retailer emailed a discount coupon to its 100,000 loyalty customers. Compared with customers who did not open the email, those who opened it spent 40% more in the next month. Marketing claims the campaign increased spending by 40%. Evaluate the claim and propose a better measurement.

Q377. *[Scenario | Medium]*

A streaming service's daily active users dropped 10% overnight on Tuesday. The CEO asks you to find out why. How do you approach it?

Q378. *[Scenario | Medium]*

A hospital compares two treatments. Treatment A: small stones 81 of 87 successes, large stones 192 of 263 successes. Treatment B: small stones 234 of 270, large stones 55 of 80. Management concludes B is better because its overall rate is higher. Do you agree?

Q379. *[Scenario | Medium]*

An A/B test at a travel site was designed for a 50/50 traffic split. The final counts are 50,600 users in control and 49,400 in treatment. The treatment shows a significant lift in bookings. A teammate says the small imbalance (1.2%) is harmless. What do you say?

Q380. *[Scenario | Medium]*

A product manager checks the A/B test dashboard every day and plans to stop the test as soon as the p-value dips under 0.05. On day 6 of a planned 14-day test it does. Is it safe to launch?

Q381. *[Scenario | Medium]*

A social app tested a new feed layout. In week 1 engagement was up 12%, but by week 4 the lift had shrunk to 2% and was no longer significant. What could explain this and what should you recommend?

Q382. *[Scenario | Hard]*

A ride-sharing marketplace wants to test a new driver incentive. Randomizing riders or drivers to treatment and control means they share the same supply-demand pool. What is the issue with a simple user-level A/B test and what alternatives would you use?

Q383. *[Scenario | Hard]*

An online store's A/B test measures revenue per user. A single user in the treatment group made a $50,000 purchase (a B2B order) and the treatment now appears to win with p = 0.01. Typical order size is $60. How do you handle it?

Q384. *[Scenario | Medium]*

A marketing team wants to compare five variants of an ad creative against the control, at alpha = 0.05 each. Variant C has p = 0.03 and others are above 0.2. They want to declare C the winner. What do you advise?

Q385. *[Scenario | Medium]*

An A/B test of a new recommendation widget returned a 1.5% relative lift in revenue with p = 0.40 and a 95% CI of [-2%, +5%]. The stakeholder says "so it has no effect, kill the project." How do you respond?

Q386. *[Scenario | Hard]*

A news app's click-through rate (CTR) fell from 4.0% to 3.6% month over month, but CTR rose within every device segment (mobile, desktop, tablet). How can this be, and what do you report?

Q387. *[Scenario | Medium]*

A company ran a customer satisfaction survey by email. Only 5% responded, and the average Net Promoter Score (NPS) was 62. Leadership wants to report NPS 62 as the company-wide score. What are the concerns and what would you do?

Q388. *[Scenario | Medium]*

A bank's fraud model flags transactions. Fraud occurs in 0.1% of transactions; the model catches 99% of fraud (sensitivity) and has a 1% false positive rate. An analyst says "the model is 99% accurate, so flagged transactions are almost surely fraud". Evaluate.

Q389. *[Scenario | Hard]*

A retail chain launched a loyalty program in 100 of its 200 stores (chosen by regional managers, not randomly). Average weekly sales per store in program stores went from 100 to 120 (thousand dollars), while non-program stores went from 100 to 108. How do you estimate the program's effect?

Q390. *[Scenario | Medium]*

A regression for a retailer reports: weekly sales = 50 + 3.2 * ad_spend (in thousands), R^2 = 0.64, p < 0.001 for ad_spend. The marketing VP concludes that every extra $1,000 of advertising will produce $3,200 of additional sales. Critique this.

Q391. *[Scenario | Medium]*

A telecom analyst finds that customers who use the mobile app have 30% lower churn than those who do not, and suggests forcing everyone to install the app to cut churn. Evaluate.

Q392. *[Scenario | Medium]*

A hospital uses a screening test for a condition. As a data scientist you can choose the decision threshold on the model's risk score. Explain how sensitivity, specificity, ROC and the choice of threshold depend on the costs of errors.

Q393. *[Scenario | Hard]*

A video platform tested a new recommendation algorithm. Click-through rate rose 3% (significant), but average watch time per user fell 2% (significant) and 30-day retention is unchanged so far. How do you decide whether to ship?

Q394. *[Scenario | Medium]*

Your team plans an A/B test with a required sample of 14,750 users per arm. The site gets 3,000 eligible new visitors per day, split 50/50. How long should the test run, and what else should you consider in setting the duration?

Q395. *[Scenario | Easy]*

A city's data show that months with higher ice cream sales also have more drowning incidents (correlation 0.85). A councillor proposes banning ice cream near beaches. What is wrong, and how would you explain it?

Q396. *[Scenario | Hard]*

A growth team wants to reduce variance in A/B tests and has heard about CUPED. Explain how it works with an example, and when it is helpful.

Q397. *[Scenario | Medium]*

A company A/B tests a new onboarding flow. 40% of users in the treatment group never complete onboarding and drop out; the analyst removes them and compares only completers in treatment with all users in control, finding a big win. Is this valid?

Q398. *[Scenario | Hard]*

A food-delivery company wants to know whether a new fast-delivery feature increases order frequency. They cannot randomize, because the feature rolled out in one city only on March 1. Describe how you would estimate the effect using other cities.

Q399. *[Scenario | Medium]*

A survey for a mobile game finds that 30% of its 2,000 sampled players say they spend money in the game. The company wants a 95% confidence interval for the true share of paying players, and wants to know how many players it must survey for a margin of error of +/- 1 percentage point.

Q400. *[Scenario | Hard]*

A fintech app tested three onboarding variations (A, B, C). Average first-week transaction counts per user are: A 4.1, B 4.4, C 4.9, with n = 1,000 per group and a common within-group SD of 3. The analyst ran three pairwise t-tests. Describe a better analysis and carry out the ANOVA.

Machine Learning Fundamentals (up to Linear and Logistic Regression)

Q401. *[Concept | Easy]*

Walk me through the end-to-end machine learning workflow for a typical analytics project.

Q402. *[Concept | Easy]*

What is the difference between supervised and unsupervised learning? Give examples of each.

Q403. *[Concept | Easy]*

Why do we split data into training, validation and test sets? What is each used for?

Q404. *[Concept | Easy]*

What is k-fold cross-validation and why is it preferred over a single train/test split?

Q405. *[Concept | Medium]*

When would standard k-fold cross-validation be the wrong choice, and what would you use instead?

Q406. *[Concept | Medium]*

Explain the bias-variance trade-off.

Q407. *[Concept | Easy]*

What are overfitting and underfitting? How do you detect and fix each?

Q408. *[Concept | Medium]*

What is data leakage? Give examples and explain how to prevent it.

Q409. *[Concept | Medium]*

Explain L1 (Lasso) and L2 (Ridge) regularization. How do they differ?

Q410. *[Concept | Hard]*

Why does L1 regularization produce sparse solutions while L2 does not? Explain intuitively.

Q411. *[Concept | Medium]*

How do you choose the regularization strength (lambda) and why must features be scaled before regularization?

Q412. *[Concept | Easy]*

What are the assumptions of linear regression?

Q413. *[Concept | Medium]*

Explain how Ordinary Least Squares (OLS) estimates the coefficients, including the normal equation.

Q414. *[Concept | Easy]*

What is R-squared and how do you interpret it? What are its limitations?

Q415. *[Concept | Medium]*

What is adjusted R-squared and when does it differ materially from R-squared?

Q416. *[Concept | Medium]*

What is multicollinearity, what problems does it cause and how do you handle it?

Q417. *[Concept | Medium]*

What is the Variance Inflation Factor (VIF)? How is it computed and what threshold is used?

Q418. *[Concept | Medium]*

What is heteroscedasticity? How do you detect it and what can you do about it?

Q419. *[Concept | Medium]*

How do you perform residual analysis for a linear regression model and what patterns should you look for?

Q420. *[Concept | Easy]*

How do you interpret the coefficients, p-values and confidence intervals in a multiple linear regression output?

Q421. *[Concept | Easy]*

Why can't we use linear regression for a binary classification problem? What does logistic regression do differently?

Q422. *[Concept | Medium]*

Explain odds, log-odds (logit) and odds ratio. How do they relate to logistic regression coefficients?

Q423. *[Concept | Easy]*

What is the sigmoid function and what are its key properties?

Q424. *[Concept | Medium]*

What is the cost function of logistic regression and why is mean squared error not used?

Q425. *[Concept | Medium]*

What are the assumptions of logistic regression, and what happens with perfect separation?

Q426. *[Concept | Medium]*

How should the classification threshold be chosen in logistic regression? Why is 0.5 not always right?

Q427. *[Concept | Easy]*

Explain the confusion matrix and the metrics derived from it.

Q428. *[Concept | Easy]*

What is the difference between precision and recall? When do you prioritise one over the other?

Q429. *[Concept | Medium]*

What is the F1 score and the F-beta score? Why is the harmonic mean used and what are the limitations?

Q430. *[Concept | Medium]*

Explain the ROC curve and AUC. How do you interpret an AUC of 0.5, 0.8 and 1.0?

Q431. *[Concept | Hard]*

When would you use a precision-recall curve (PR-AUC) rather than ROC-AUC?

Q432. *[Concept | Easy]*

Compare MAE, MSE and RMSE. When would you use each?

Q433. *[Concept | Easy]*

What is the difference between normalization (min-max scaling) and standardization? When is scaling needed?

Q434. *[Concept | Medium]*

How do you encode categorical variables for linear and logistic regression? Compare the main approaches.

Q435. *[Concept | Medium]*

What types of missing data exist and how do you handle missing values?

Q436. *[Concept | Medium]*

What problems does class imbalance create and what are the basic techniques to deal with it?

Q437. *[Concept | Easy]*

How does the K-Nearest Neighbours (KNN) algorithm work? What are its strengths, weaknesses and how do you choose k?

Q438. *[Concept | Medium]*

Explain the Naive Bayes classifier. Why is it called naive and what are its pros and cons?

Q439. *[Concept | Medium]*

Explain gradient descent. What are the differences between batch, stochastic and mini-batch gradient descent, and how does the learning rate matter?

Q440. *[Concept | Medium]*

What is feature engineering? Give common techniques for tabular data and explain how you decide which features to keep.

Q441. *[Coding | Easy]*

Write Python code using scikit-learn to fit a linear regression model with a train/test split, scale the features in a leak-free way, and report MAE, RMSE and R2.

Q442. *[Coding | Medium]*

Fit an OLS regression with statsmodels and explain how to read the key parts of the summary output.

Q443. *[Coding | Medium]*

Write code to compute the VIF for every feature of a dataframe and iteratively remove features with VIF above 5.

Q444. *[Coding | Medium]*

Write code to produce residual diagnostics for a fitted statsmodels OLS model: residual vs fitted plot, Q-Q plot, and the Breusch-Pagan test for heteroscedasticity.

Q445. *[Coding | Medium]*

Write code for a logistic regression classifier on an imbalanced dataset using a stratified split, class weights and a classification report with ROC-AUC.

Q446. *[Coding | Medium]*

Write code that evaluates a model with stratified 5-fold cross-validation while avoiding leakage from preprocessing, and reports mean and standard deviation of several metrics.

Q447. *[Coding | Medium]*

Show how to tune Ridge and Lasso regression with cross-validation in scikit-learn and inspect which features Lasso selects.

Q448. *[Coding | Hard]*

Write code to plot the ROC and precision-recall curves for a classifier and select a threshold that achieves at least 80% recall with the highest possible precision.

Q449. *[Coding | Hard]*

Implement linear regression using batch gradient descent from scratch with NumPy and explain the key steps.

Q450. *[Coding | Medium]*

Write a scikit-learn pipeline using ColumnTransformer that imputes, scales numeric columns, one-hot encodes categorical columns, fits a logistic regression, and tunes the regularization strength with GridSearchCV.

Q451. *[Scenario | Medium]*

A retail bank wants to predict which credit card customers will churn (close their account) in the next 90 days so the retention team can call them. Only 8% of customers churn. How would you build and evaluate a model? Go step by step.

Q452. *[Scenario | Hard]*

A lender is building a loan default model (3% default rate). Regulators require the model to be explainable, and the business needs predicted probability of default (PD) to price loans. Which model and metrics would you choose and how would you validate it?

Q453. *[Scenario | Medium]*

A real estate company wants a house price prediction model from 20,000 past sales (features: size, bedrooms, location, age, condition, sale date). Price is right-skewed. Describe how you would build a linear regression model and which pitfalls you would check.

Q454. *[Scenario | Hard]*

A hospital wants to predict which patients will be readmitted within 30 days of discharge so care managers can arrange follow-up. About 15% are readmitted. Describe your modelling approach, metric choice, and key pitfalls.

Q455. *[Scenario | Hard]*

A payments company has 1 million card transactions per day with 0.1% fraud. A data scientist reports 99.9% accuracy and ROC-AUC 0.97 for a logistic regression model. Is this good? How would you evaluate and improve it?

Q456. *[Scenario | Medium]*

An email service wants a quick spam filter using the text of emails. A colleague proposes Naive Bayes while another proposes logistic regression. Which would you pick, how would you build it, and how would you evaluate it?

Q457. *[Scenario | Medium]*

A telecom company wants to predict subscriber churn from raw tables of customers, monthly bills, call records and support tickets. You are asked to design the features. What features would you engineer and how would you check them?

Q458. *[Scenario | Medium]*

An insurer wants to predict the cost of each claim. The claim amount is heavily right-skewed with a few very large claims; a linear regression gives an R2 of 0.25 and poor residual plots. What do you do?

Q459. *[Scenario | Medium]*

A marketing team will email a discount offer to customers, and each email costs 0.10 dollars. A purchase from a responder yields 20 dollars profit. Historically 2% respond. You build a logistic regression response model. How do you decide who to target and how do you report the model's value?

Q460. *[Scenario | Medium]*

A health-tech startup builds a screening model for a serious disease with 1% prevalence. The logistic regression model has 98% accuracy, 40% recall and 50% precision. The medical director is unhappy. How do you respond and improve the model?

Q461. *[Scenario | Hard]*

A retailer builds a weekly sales forecasting regression model with random 5-fold cross-validation and gets R2 = 0.95, but after launch the error is three times higher. What likely went wrong and how would you redo the validation?

Q462. *[Scenario | Medium]*

An e-commerce company's data scientist predicts customer lifetime value (CLV) using linear regression. A handful of whales spend 100 times the typical customer and the model's coefficients change dramatically when they are removed. How do you handle this?

Q463. *[Scenario | Medium]*

A bank wants to find customers similar to its best credit-card customers with a K-Nearest Neighbours model using age, income, number of products, account balance and region. A first attempt gives odd neighbours. What would you check and fix?

Q464. *[Scenario | Medium]*

An online store fits a logistic regression to predict whether a visitor will purchase. The product manager asks: 'What do the coefficients tell us? If we cut page load time by one second, how many more purchases will we get?' How do you answer?

Q465. *[Scenario | Easy]*

At a delivery company, a regression model to predict parcel delivery time gives R2 = 0.99 on training data but 0.45 on test data. What is happening and what steps would you take?

Q466. *[Scenario | Medium]*

A real estate analyst fits a multiple regression to predict rent. R2 is 0.85, but the coefficient for 'number of bedrooms' is negative and insignificant, while 'square footage' and 'number of rooms' also have unexpected signs. What is the likely cause and how would you fix it?

Q467. *[Scenario | Medium]*

A bank's loan dataset has 25% missing values in the 'annual income' column, and missingness is higher for self-employed applicants and for applicants who were later rejected. How do you treat this feature in a default model?

Q468. *[Scenario | Hard]*

You are predicting house prices across 3,000 zip codes using linear regression. One-hot encoding the zip code creates 3,000 columns and many zip codes have fewer than 5 sales. How do you handle this high-cardinality feature?

Q469. *[Scenario | Medium]*

An HR department wants to predict employee attrition and present the main drivers to leadership. Attrition is 12%. They have 1,500 employees and 30 features. How would you build the model and ensure the results are trustworthy and interpretable?

Q470. *[Scenario | Medium]*

When fitting a regression of employee salary on years of experience, education and job level, the residual plot shows a funnel shape: variance increases with predicted salary. The manager wants confidence intervals for the effect of experience. What do you do?

Q471. *[Scenario | Hard]*

A ride-hailing company predicts whether a rider will cancel a booking after a driver is assigned (12% cancel). Each wrongly flagged rider triggers a 1 dollar incentive; each missed cancellation costs 4 dollars in wasted driver time. The team wants one number to choose between two logistic regression variants and an operating threshold. How do you proceed?

Q472. *[Scenario | Hard]*

A consumer-lending logistic regression model performed well at launch (AUC 0.78), but after 9 months its AUC is 0.66 and the approval rate has shifted. What could be causing this and how do you diagnose and respond?

Q473. *[Scenario | Hard]*

A genomics lab has 150 patient samples and 5,000 gene-expression features and wants to predict whether a patient responds to a drug. The first analyst fits logistic regression and gets 100% training accuracy. How would you approach this properly?

Q474. *[Scenario | Hard]*

A hospital group builds a model predicting 30-day readmission, and one feature, 'discharge summary completed', has by far the largest coefficient and the model has AUC 0.96. The clinicians are suspicious. What do you do?

Q475. *[Scenario | Medium]*

A food-delivery company predicts delivery time in minutes. Operations says late deliveries (very long times) hurt customers far more than slightly early ones, and a few extreme delays occur due to weather. Which error metric(s) and loss would you recommend and how would you report results?

Business Cases and Product Analytics

Q476. *[Scenario | Medium]*

You are a data analyst at Swiggy. The VP of Operations says: "Daily orders dropped 20% yesterday versus the same weekday last week. Find out why." You have tables orders(order_id, user_id, restaurant_id, city, platform, created_at, status, cancel_reason, delivery_minutes), sessions(session_id, user_id, platform, app_version, started_at), restaurants(restaurant_id, is_active, city) and a delivery_partner_log table. How do you root-cause this?

Q477. *[Scenario | Easy]*

Netflix wants to define a north-star metric for its streaming product. Data available: viewing_events(user_id, title_id, device, start_ts, end_ts, minutes_watched), subscriptions(user_id, plan, start_date, cancel_date), titles(title_id, genre, release_date). What north-star metric would you propose, and what guardrail metrics would you pair with it?

Q478. *[Scenario | Medium]*

Uber's rider app saw "completed trips per day" fall 15% in Bengaluru last Monday while the rest of India was flat. You have trips(trip_id, rider_id, driver_id, request_ts, status, city, surge_multiplier, eta_minutes, fare) and driver_status(driver_id, ts, online_flag). How do you diagnose this?

Q479. *[Scenario | Medium]*

Amazon.in's "Add to Cart" rate on mobile web fell from 6.0% to 5.1% over two weeks, but desktop is stable. Available: events(user_id, session_id, event_name, device, browser, ts, product_id), product_catalog, and release notes. What is your approach?

Q480. *[Scenario | Easy]*

Flipkart is launching "Flipkart Minutes" (10-minute delivery). Define the success metrics: a primary metric, supporting metrics, and guardrails. Data available includes order, delivery timestamps, dark-store inventory and customer feedback.

Q481. *[Scenario | Hard]*

A bank's credit card division reports that "average monthly spend per active card" rose 12% quarter over quarter, and leadership is celebrating. You suspect the metric is misleading. Data: card_master(card_id, open_date, segment, status), transactions(txn_id, card_id, amount, mcc, ts). How do you check?

Q482. *[Scenario | Medium]*

Spotify (or a music streaming app) sees DAU stay flat while MAU keeps rising for six months. Data: user_activity(user_id, date, minutes_listened, platform), signup table. What does the DAU/MAU ratio tell you and what would you investigate?

Q483. *[Scenario | Medium]*

A telecom company (Airtel/Jio style) tracks ARPU and sees it fell 5% in a quarter even though revenue grew 3%. The CFO asks you to explain. Data: subscribers(sub_id, plan, activation_date, status, circle), billing(sub_id, month, revenue), plan_catalog.

Q484. *[Scenario | Medium]*

Zomato's average delivery time rose from 29 to 34 minutes over two weeks. Operations wants to know whether it is an average-versus-tail problem and where to act. Data: orders with timestamps (placed, accepted, prepared, picked_up, delivered), restaurant_id, rider_id, distance_km.

Q485. *[Scenario | Hard]*

Instagram wants to measure the "health" of Stories. Product says time spent is up 8%, but a researcher worries about quality. Propose a metric framework that includes a north-star, input metrics, guardrails and counter-metrics, and explain how you would evaluate trade-offs.

Q486. *[Scenario | Easy]*

An e-commerce company (Myntra-style) has this checkout funnel for last week: Product view 1,000,000; Add to cart 120,000; Checkout start 70,000; Payment page 45,000; Order success 36,000. How do you analyze the funnel and decide where to focus? Show the SQL.

Q487. *[Scenario | Medium]*

You are given a table logins(user_id, login_date) for a fitness app. Product wants a weekly cohort retention table: users grouped by their signup week, showing the percent active in weeks 0 to 8 after signup. Describe the method and write the SQL.

Q488. *[Scenario | Medium]*

A SaaS company's monthly customer churn rose from 2% to 3.5%. You have subscriptions(customer_id, plan, start_date, end_date, mrr, industry, acquisition_channel), support_tickets, and product_usage(customer_id, date, feature, events). How do you analyze churn and what do you recommend?

Q489. *[Scenario | Hard]*

A telecom operator wants to predict which prepaid customers will churn in the next 30 days to target retention offers. Data: 24 months of usage, recharge history, complaints, tower quality, demographics. Offers cost Rs 50 per customer; average customer margin is Rs 400/month. Describe the end-to-end approach including how to choose the threshold.

Q490. *[Scenario | Medium]*

A new user onboarding flow for a fintech app (UPI wallet) has these steps: download, signup, KYC upload, bank link, first transaction. Only 18% of downloads make a first transaction within 7 days. Describe how you would find the biggest opportunities.

Q491. *[Scenario | Medium]*

Subscription app: you have a retention curve showing 100% in month 0, 60% in month 1, 45% in month 2, 38% in month 3, 35% in month 6, 34% in month 12. The CEO asks "is that good?" and "what is customer lifetime value?" Monthly revenue per subscriber is Rs 300 with 70% gross margin. Explain.

Q492. *[Scenario | Medium]*

Walmart / BigBasket wants to know why customers stop buying groceries. Define "churn" for a non-subscription retailer where users can go weeks between orders, and describe how to detect it early.

Q493. *[Scenario | Hard]*

A ride-sharing company observes that 7-day retention of new riders improved from 30% to 36% after a signup-bonus launch, but 60-day retention is unchanged. What happened and what do you advise?

Q494. *[Scenario | Medium]*

YouTube (or an edtech app) wants to understand which early user behaviours predict long-term retention to design onboarding. You have event logs for the first 7 days and a flag for active at day 60. How do you find the 'aha moment' without confusing correlation with causation?

Q495. *[Scenario | Medium]*

A bank's mobile app shows 4.5 million MAU but only 1.2 million log in more than once a week. The head of digital asks you to define engagement tiers and tie them to business value. How do you do it?

Q496. *[Scenario | Medium]*

A retailer (Reliance Retail / Big Basket style) has 2 million customers with transaction data: customer_id, order_date, order_value, category, channel. Marketing wants customer segments for targeted campaigns. Walk through RFM segmentation and an alternative clustering approach.

Q497. *[Scenario | Hard]*

An airline wants to test raising ancillary fees (checked baggage) from Rs 600 to Rs 750. Data: bookings with fare class, baggage purchased (yes/no) at booking, route, customer segment, historical price changes by route. How do you estimate price elasticity and the revenue impact?

Q498. *[Scenario | Medium]*

Ola/Uber is considering dynamic "surge" pricing changes. How would you evaluate whether a new surge algorithm improves the marketplace without just raising prices? What metrics and experiment design would you use?

Q499. *[Scenario | Medium]*

A D2C brand spends Rs 5 crore a month across Google Ads, Meta, influencers, and email. Last-click attribution in the dashboard says Google gets 70% of conversions. The CMO doubts it. How would you assess true channel contribution?

Q500. *[Scenario | Medium]*

A marketing team ran a 20% discount email campaign to 500,000 customers; 8% of recipients purchased in the following week, generating Rs 4 crore. Leadership says the campaign was a big success. How do you evaluate it properly? You did not keep a holdout.

Q501. *[Scenario | Medium]*

Your company's CAC rose from Rs 800 to Rs 1,300 within two quarters while LTV is Rs 3,000. Marketing asks for more budget; finance is worried. How do you analyze CAC and recommend allocation?

Q502. *[Scenario | Hard]*

A bank wants to decide cross-sell targeting for personal loans. Data: 3 million customers with demographics, balances, transactions, product holdings; a past campaign where 300,000 customers were contacted (not random) and 4% converted. How would you build a targeting strategy and avoid selection bias?

Q503. *[Scenario | Medium]*

A hospital group wants to reduce readmissions within 30 days. Data: admissions(patient_id, admit_date, discharge_date, diagnosis, ward, doctor, insurance), readmissions flag, follow-up call logs. How would you analyze the problem and which metrics would you report?

Q504. *[Scenario | Medium]*

An insurer notices rising claims cost per policy in motor insurance (Rs 11,000 to Rs 13,500 over a year). Data: policies(policy_id, vehicle_age, city, age_band, premium, start_date), claims(claim_id, policy_id, amount, cause, date). Break down the drivers and propose actions.

Q505. *[Scenario | Medium]*

Zomato Gold-style membership: members order 2.5x more than non-members. The product manager says "membership drives orders, let's push it everywhere." What would you check?

Q506. *[Scenario | Medium]*

Amazon wants to test a new checkout button color/design on the mobile app. Baseline checkout conversion is 10%. The team wants to detect a 3% relative lift. Daily eligible traffic is 200,000 sessions. Design the experiment: hypothesis, randomization unit, sample size, duration, and decision criteria.

Q507. *[Scenario | Medium]*

> After a 14-day A/B test of a new recommendation carousel on Flipkart, results are: control conversion 5.00% (n = 400,000), treatment 5.12% (n = 400,000), p = 0.04. Revenue per user is up 0.5% but not significant, and page load time is 80 ms slower. Product wants to ship. What do you recommend?

Q508. *[Scenario | Hard]*

> Your manager checks the A/B test dashboard every day and wants to stop the test as soon as $p < 0.05$. On day 4 of a planned 14-day test, $p = 0.03$. What is the problem and how can the team monitor experiments safely?

Q509. *[Scenario | Hard]*

> You cannot randomise users for a feature because the pricing change applies to an entire city (for example, a new delivery fee in Hyderabad). How would you estimate the causal impact on orders?

Q510. *[Scenario | Hard]*

> You run an A/B test on a social network feature where users are connected (for example a new sharing button). Treatment users are sharing more, and their friends in control are also seeing more content. The measured lift is smaller than expected. What is going on and how can you fix the design?

Q511. *[Scenario | Medium]*

> Your A/B test on a ride-hailing app shows overall treatment effect on revenue per rider is +1% (not significant), but the data science team spots a +6% effect for new riders and -2% for veteran riders. How should you treat subgroup findings, and what is variance reduction (CUPED)?

Q512. *[Scenario | Medium]*

> Test results for a new onboarding flow: the treatment shows 12% sign-up completion vs 11.5% control, but you find that treatment has 52.5% of users and control has 47.5% when the plan was 50/50. What is this and what should you do?

Q513. *[Scenario | Medium]*

> Swiggy wants to run an experiment on a new "free delivery above Rs 199" threshold. Design the experiment end-to-end including metrics, unit, risks, and how you will decide.

Q514. *[Scenario | Hard]*

> A streaming company tests a new homepage ranking. Short-term engagement (hours watched in week 1) is up 4%, but the data science lead suspects long-term subscriber churn could rise because the model favours addictive but low-satisfaction titles. How would you handle long-term outcomes in experiments?

Q515. *[Scenario | Easy]*

An analyst at a food delivery company reports that a test improved average order value from Rs 420 to Rs 440 and p = 0.001. A skeptical stakeholder asks "is it really meaningful?" and also notes order values have a long right tail. How do you respond?

Q516. *[Scenario | Medium]*

A bank wants to know if a new email nudge increased the number of customers setting up automatic bill pay. They sent the email to 100,000 customers chosen as 'most engaged' and compared to the remaining 900,000. Adoption was 9% vs 3%. Is the nudge effective?

Q517. *[Scenario | Medium]*

The COO of a logistics company (Delhivery-style) asks you to build an executive dashboard for daily operations. Data: shipments(awb, origin_hub, dest_hub, pickup_ts, out_for_delivery_ts, delivered_ts, status, attempt_count), hub_capacity, rider_attendance. What KPIs and design do you propose?

Q518. *[Scenario | Medium]*

Your company's two dashboards show different monthly active users: the product dashboard says 8.2M and the finance dashboard says 7.4M. Executives are confused. How do you resolve it?

Q519. *[Scenario | Medium]*

While preparing a weekly report you notice that yesterday's revenue is double the usual amount in the orders table. What do you do before sending the report to leadership?

Q520. *[Scenario | Medium]*

You get a customer dataset for a churn analysis with: 12% missing income values, duplicate customer records, dates stored in mixed formats, and an 'age' column with values up to 250. Describe how you would clean and validate it before analysis, and what you tell stakeholders.

Q521. *[Scenario | Medium]*

An analyst discovers that the tracking event "purchase" fires twice for iOS users after a recent app update, inflating conversion rate and revenue by roughly 8% on iOS. The metric has been reported to the board for three weeks. How do you handle it?

Q522. *[Scenario | Medium]*

A telecom's billing data warehouse has customers, accounts, and invoices tables. A report of "revenue by customer segment" gives a total that is 15% higher than finance's number. SQL: SELECT segment, SUM(i.amount) FROM customers c JOIN accounts a ON c.cust_id = a.cust_id JOIN invoices i ON a.account_id = i.account_id GROUP BY segment. How do you debug?

Q523. *[Scenario | Medium]*

> You are building a KPI tree for a consumer lending app (for example loans via a mobile app). The CEO says "grow profitable disbursals." Construct the KPI tree and name the key trade-off metrics.

Q524. *[Scenario | Easy]*

> The head of a hospital asks for a dashboard on emergency room (ER) performance. List metrics, show how you would compute wait-time metrics in SQL, and mention pitfalls in presenting them.

Q525. *[Scenario | Medium]*

> A retail chain wants a "store performance" scorecard to compare 400 stores. Stores differ in size, location, and age. How do you create a fair comparison and what pitfalls do you avoid?

Q526. *[Scenario | Medium]*

> Design a metrics and monitoring plan for a customer support operation (chat + call) at an e-commerce company. Leadership wants lower cost but higher satisfaction. Data: tickets(ticket_id, channel, created_ts, first_response_ts, resolved_ts, agent_id, category, csat, reopened_flag).

Q527. *[Scenario | Hard]*

> A data platform team keeps getting complaints that the "conversion rate" dashboard is wrong. You are asked to put data quality monitoring in place for the key tables feeding it. What would you implement?

Q528. *[Scenario | Medium]*

> A senior product manager at a marketplace company asks you to "just confirm" that the new feature caused the 5% GMV rise last month. Your analysis shows GMV rose mainly because of a festival season and a concurrent marketing campaign; the feature's effect is statistically indistinguishable from zero. How do you communicate this?

Q529. *[Scenario | Easy]*

> You must present a quarterly churn analysis to a non-technical leadership audience (CEO, CMO, CFO) in 10 minutes. The analysis includes a gradient boosting model, SHAP values, cohort curves, and a segment breakdown. How do you structure the presentation?

Q530. *[Scenario | Medium]*

> Three stakeholders each ask for an urgent analysis in the same week: Marketing wants campaign ROI, Finance wants a revenue forecast for the board, and the Product team wants a funnel analysis before a sprint planning. You can only deliver two comfortably. How do you prioritise?

Q531. *[Behavioural | Medium]*

> Tell me about a time you used data to influence a major business decision.

Q532. *[Behavioural | Medium]*

Describe a time your analysis was wrong or you found an error after sharing results. What did you do?

Q533. *[Behavioural | Medium]*

Tell me about a time you had to work with messy or incomplete data and still deliver a result on a deadline.

Q534. *[Behavioural | Medium]*

Tell me about a time you disagreed with a stakeholder or manager about an analytical approach or conclusion. How was it resolved?

Q535. *[Behavioural | Easy]*

Tell me about a time you explained a complex analysis to a non-technical audience.

Q536. *[Behavioural | Medium]*

Describe a project where you took initiative beyond your assigned tasks and created measurable impact.

Q537. *[Behavioural | Medium]*

Tell me about a time when the data did not support what leadership wanted to hear, or when you had to deliver bad news.

Q538. *[Behavioural | Easy]*

Why do you want to be a data analyst, and tell me about the project you are proudest of.

Q539. *[Scenario | Medium]*

A stakeholder sends this request: "Can you pull the number of active users?" It is ambiguous. Walk through how you clarify a vague analytics request before building anything, using an example for an online food delivery business.

Q540. *[Scenario | Medium]*

Estimation: Swiggy is considering launching in a new Tier-2 city with a population of 2 million. Estimate the number of daily food delivery orders the city could generate at maturity and the number of delivery partners needed.

Q541. *[Scenario | Medium]*

Estimation: Estimate the total number of UPI-style digital payments processed per day by a mid-sized fintech app with 20 million registered users and how much infrastructure capacity (peak transactions per second) it would need.

Q542. *[Scenario | Medium]*

Forecasting: A retailer wants a demand forecast for the next 8 weeks for 500 SKUs to plan inventory. Data: 3 years of weekly sales per SKU, promotions calendar, price, holidays. Describe your approach, accuracy metrics, and how you use the forecast for inventory decisions.

Q543. *[Scenario | Hard]*

Paradox: A hospital reports that overall treatment B has a higher success rate than treatment A (A 78%, B 83%). But within both mild and severe cases, treatment A is better (mild: A 93% vs B 87%; severe: A 73% vs B 69%). A manager asks which treatment to recommend. Explain.

Q544. *[Scenario | Medium]*

A video game company (or any freemium app) has 3% of users paying, with a heavy-tailed revenue distribution: top 1% of payers generate 50% of revenue. Product wants to optimise "average revenue per user (ARPU)". What issues arise, and how would you analyze and experiment on it?

Q545. *[Scenario | Hard]*

A credit card company launched a fraud detection rule that blocks 2% of transactions; fraud losses dropped from 0.12% to 0.07% of spend, but complaints rose and spend on affected cards fell. How do you evaluate the trade-off and tune the rule?

Q546. *[Scenario | Medium]*

A ride-hailing company sees that the average rating of drivers has stayed at 4.8 for two years, yet rider complaints are rising. Data: trips(trip_id, driver_id, rider_id, rating, tags), complaints(ticket_id, trip_id, category). Is the rating metric still useful, and what would you build instead?

Part 2: Answer Key

Python

A1. What are the differences between a list, tuple, set and dictionary in Python, and when would you use each in data analysis work?

List: ordered, mutable, allows duplicates, O(1) append and index access, O(n) membership test. Use it for sequences of records you will modify, such as a collection of rows being built up.

Tuple: ordered, immutable, hashable if its items are hashable. Use it for fixed records, return values, and as dictionary keys (for example (region, year)).

Set: unordered collection of unique hashable items with O(1) average membership test. Use it for de-duplication, fast "is this ID in the list" checks, and set algebra (union, intersection, difference) such as finding customers present in one table but not another.

Dictionary: key-value mapping with O(1) average lookup; insertion ordered since Python 3.7. Use it for lookups, counting, mapping codes to labels, and building column-rename maps for pandas.

Rule of thumb: if you do many "x in collection" checks on a large collection, convert the list to a set first; it turns an O(n*m) loop into roughly O(n+m).

A2. Explain mutable versus immutable objects in Python. Why does it matter, and what is the classic bug with mutable default arguments?

Immutable objects (int, float, str, tuple, frozenset, bytes) cannot be changed after creation; any "modification" creates a new object. Mutable objects (list, dict, set, most user classes, NumPy arrays, DataFrames) can be changed in place, so every name pointing to the same object sees the change.

It matters because assignment copies references, not data. If b = a and a is a list, b.append(1) also changes a. It also matters for dictionary keys and set members, which must be hashable (effectively immutable).

The default-argument bug: default values are evaluated once, at function definition time.

```
def add_item(x, bucket=[]):
    bucket.append(x)
    return bucket

add_item(1)  # [1]
add_item(2)  # [1, 2]  <- surprising, same list reused
```

Fix: use None as the sentinel.

```
def add_item(x, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(x)
    return bucket
```

A3. What is the difference between "is" and "==" in Python? How does this relate to checking for None and NaN?

"==" compares values (calls __eq__). "is" compares identity, meaning whether two names point to the very same object in memory (compare id()).

Use "is" for singletons: "if x is None". Do not use "is" to compare numbers or strings; small-integer and string interning make it appear to work sometimes but it is not guaranteed.

NaN is a special case: float('nan') == float('nan') is False, because NaN is defined as not equal to anything including itself. So "x == np.nan" never works. Use np.isnan(x), math.isnan(x), pd.isna(x) or df['col'].isna(). Also None and NaN differ: None is a Python object (NoneType); NaN is a float. Pandas treats both as missing in object columns via isna(), and recent pandas uses pd.NA for nullable dtypes, where comparisons propagate NA rather than returning True or False.

A4. What is the difference between a list comprehension and a generator expression? When should you prefer a generator?

A list comprehension [f(x) for x in data] builds the entire list in memory immediately. A generator expression (f(x) for x in data) produces values lazily, one at a time, and holds only the current state, so memory use is O(1) regardless of input size.

Prefer a generator when you iterate once, the data is large or unbounded (reading a multi-GB file line by line), or you feed it into a consuming function such as sum(), max(), any(), all() or ''.join().

Prefer a list when you need to index, slice, take len(), or iterate multiple times; a generator is exhausted after one pass.

```
total = sum(len(line) for line in open('big.txt'))   # never loads whole file
squares = [x*x for x in range(10)]                    # need a reusable list
```

Functions using yield are generator functions and are the right tool for streaming data pipelines (for example yielding cleaned chunks).

A5. Explain *args and **kwargs. Give an example of how they are used in data code.

*args collects extra positional arguments into a tuple; **kwargs collects extra keyword arguments into a dictionary. The names are convention; the stars matter. At a call site, *seq unpacks a sequence into positional arguments and **mapping unpacks a dict into keyword arguments.

```
def summarize(df, *cols, **agg_kwargs):
    return df[list(cols)].agg(**agg_kwargs) if agg_kwargs else df[list(cols)].describe()

def log_call(func):
    def wrapper(*args, **kwargs):
        print('calling', func.__name__)
        return func(*args, **kwargs)
    return wrapper

params = {'sep': ';', 'encoding': 'latin-1'}
df = pd.read_csv('file.csv', **params)
```

Typical uses: decorators and wrappers that forward arguments unchanged, flexible helper functions, and passing a configuration dictionary into pandas or plotting calls. Order in a signature: positional, *args, keyword-only, **kwargs.

A6. What is the difference between a shallow copy and a deep copy? How does it affect lists of lists and pandas DataFrames?

A shallow copy creates a new outer container but the elements inside are still references to the same objects. A deep copy recursively copies everything, so the new structure is fully independent.

```
import copy
a = [[1, 2], [3, 4]]
b = a.copy()          # or list(a), a[:]
c = copy.deepcopy(a)
a[0].append(99)
# b[0] is also [1, 2, 99]; c[0] is still [1, 2]
```

For pandas, df.copy() defaults to deep=True, which copies the data buffers and index, so changes to the copy do not alter the original. df.copy(deep=False) shares data. Note that deep copy of a DataFrame holding Python objects (such as lists inside cells) does not recursively copy those objects; the cells still reference the same lists. Plain assignment df2 = df is not a copy at all, it is just another name for the same object. Under pandas Copy-on-Write (default in pandas 3), derived objects behave as independent copies lazily.

A7. What is a decorator in Python and how would you use one in a data pipeline?

A decorator is a callable that takes a function and returns a new function, usually adding behavior before or after the original call without modifying its code. The @name syntax is sugar for func = name(func).

```
import time, functools

def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        result = func(*args, **kwargs)
        print(f'{func.__name__} took {time.perf_counter()-t0:.2f}s')
        return result
    return wrapper

@timer
def load_and_clean(path):
    return pd.read_csv(path).dropna()
```

Use functools.wraps to preserve the function name and docstring. Common data-pipeline uses: timing and logging steps, retrying flaky database or API calls, caching expensive computations (functools.lru_cache), validating that a returned DataFrame has expected columns, and DataFrame.pipe style chaining.

A8. What is the GIL and how does it affect data processing in Python?

The Global Interpreter Lock in CPython is a mutex that lets only one thread execute Python bytecode at a time. Consequences:

- CPU-bound pure Python code does not speed up with threads; use multiprocessing or concurrent.futures.ProcessPoolExecutor, which runs separate processes with their own GIL.

- I/O-bound work (HTTP calls, database queries, file reads) does benefit from threads or asyncio because the GIL is released while waiting.

- NumPy, pandas internals, and many C extensions release the GIL during heavy numeric operations, so vectorized code can use multiple cores in some cases.

Practical guidance for analysts: vectorize first, then use Dask, Polars, or multiprocessing (joblib) for parallelism. Newer Python versions include an experimental free-threaded build that removes the GIL, but most libraries still assume the traditional build.

A9. Explain lambda, map, filter and reduce. How do they compare with list comprehensions and pandas apply?

lambda creates a small anonymous function: lambda x: x*2. map(f, it) applies f to each element, filter(f, it) keeps elements where f is truthy, and functools.reduce(f, it) folds a sequence to a single value (for example reduce(lambda a, b: a + b, nums)). map and filter return lazy iterators in Python 3.

List comprehensions are generally preferred for readability: [x*2 for x in nums if x > 0] replaces map plus filter.

In pandas, Series.map maps values via function, dict or Series (great for code-to-label lookups), Series.apply runs a Python function per element (slow, row-by-row), and DataFrame.apply works on rows or columns. Prefer vectorized operations (df['a'] * 2, np.where, str accessor methods) over apply because they run in compiled code and are often 10 to 100 times faster. Use lambda with sort keys: sorted(items, key=lambda t: t[1]).

A10. What is the difference between a pandas Series and a DataFrame, and what is the role of the index?

A Series is a one-dimensional labelled array with a single dtype and an index. A DataFrame is a two-dimensional table: a dictionary-like collection of Series (columns) sharing a common row index, where each column can have its own dtype.

The index is the row label. Operations between pandas objects align on index labels, not position: s1 + s2 adds values with matching labels and gives NaN where a label exists in only one. The index enables fast label lookups (.loc), joins, resampling with a DatetimeIndex, and groupby results. Index does not have to be unique, but non-unique indexes can make .loc return multiple rows and slow some operations. Use reset_index() to turn the index back into a column and set_index('col') to promote one. Beware that after filtering, the index keeps the original labels, so .iloc and .loc can give different results.

A11. Explain the difference between loc and iloc in pandas, including how slicing behaves.

loc is label-based; iloc is integer position-based.

```
df.loc[2:5, 'name':'age']   # rows with LABELS 2..5 inclusive, columns name..age inclusive
df.iloc[2:5, 0:3]           # rows at positions 2,3,4 (end exclusive), columns 0,1,2
df.loc[df['age'] > 30, 'salary']   # boolean mask with loc, returns a column
```

Key points: loc slices are inclusive of both ends; iloc slices follow normal Python end-exclusive rules. loc accepts boolean arrays and callables; iloc accepts integer lists/arrays and boolean arrays (not a boolean Series with an index). If the index is integers that are not in order (after sorting or filtering), df.loc[0] and df.iloc[0]

return different rows. Use loc for assignment to avoid chained-assignment problems: df.loc[mask, 'col'] = value. Use .at and .iat for fast scalar access.

A12. Compare pandas merge, join and concat. What are the join types and what does the validate argument do?

merge is the SQL-style join on columns (or index) using on, left_on, right_on, with how = inner, left, right, outer or cross. join is a convenience for joining on the index (df1.join(df2, how='left')). concat stacks objects along an axis: axis=0 appends rows (union of columns, missing filled with NaN), axis=1 aligns columns side by side on the index.

Join semantics: inner keeps only matching keys; left keeps all left rows; right keeps all right rows; outer keeps all keys from both; cross gives the Cartesian product.

Pitfalls: duplicate keys on both sides multiply rows (many-to-many explosion). Use validate='one_to_one', 'one_to_many', 'many_to_one' or 'many_to_many' to raise an error if the key relationship is not as expected, and indicator=True to add a _merge column telling whether a row came from left_only, right_only or both. Overlapping non-key column names get suffixes (_x, _y by default; use suffixes=). Always check row counts before and after a merge.

A13. Explain groupby in pandas. What is the difference between agg, transform and apply?

groupby follows split-apply-combine: split rows by key, apply a function per group, combine the results.

- agg (aggregate) reduces each group to one row: df.groupby('dept')['salary'].agg(['mean','max']) or named aggregation .agg(avg_sal=('salary','mean')).

- transform returns a result with the same length and index as the input, so you can attach it back to the original rows: df['dept_mean'] = df.groupby('dept')['salary'].transform('mean'). It is used for group-wise normalisation, filling NaN with group means and computing share of group total.

- apply passes each whole group (a DataFrame) to an arbitrary function and combines whatever comes back (scalar, Series, or DataFrame). It is the most flexible but slowest; use it when agg and transform cannot express the logic, e.g. top-N per group.

Other useful items: filter keeps or drops whole groups (g.filter(lambda x: len(x) > 5)); as_index=False keeps keys as columns; dropna=False keeps NaN keys; observed=True avoids empty categorical groups; size counts rows including NaN whereas count counts non-null values.

A14. How do you detect and handle missing values in pandas? What are the pros and cons of dropping versus imputing?

Detect: df.isna().sum() gives counts per column, df.isna().mean() gives the fraction, and df.info() shows non-null counts. Visualise the pattern with a heatmap of df.isna() or the missingno library.

Handle:

- dropna(subset=[...], thresh=n) removes rows or columns; simple, but loses data and biases results if missingness is not random.

- fillna with constant, mean/median (median is robust to skew), mode for categoricals, or forward/back fill (ffill/bfill) for time series; interpolate() for numeric trends.

- Group-wise imputation: df['x'] = df['x'].fillna(df.groupby('g')['x'].transform('median')).

- Model-based imputation (KNNImputer, IterativeImputer) for more accuracy.

- Add an indicator column (is_missing) when missingness itself is informative.

First understand why data is missing: MCAR (completely at random), MAR (depends on observed variables), MNAR (depends on the missing value itself). Dropping is safe mainly for MCAR and small fractions. Impute inside a train-only pipeline to avoid leakage. Never silently fill; document the choice.

A15. Why is vectorization faster than loops or apply in pandas and NumPy? Give examples of vectorized alternatives.

Vectorized operations run in compiled C loops over contiguous memory and avoid per-element Python interpreter overhead (type checks, function call, object boxing). Python-level loops, iterrows and apply with a Python function pay that overhead for every row.

Alternatives:

- Arithmetic: df['total'] = df['price'] * df['qty'] instead of apply.

- Conditionals: np.where(df['x'] > 0, 'pos', 'neg') or np.select([c1, c2], [v1, v2], default=v3) instead of if/else in apply.

- Strings: df['name'].str.strip().str.lower().

- Mapping: df['code'].map(mapping_dict).

- Dates: df['d'].dt.month.

- Group operations: groupby().transform instead of looping over groups.

If you truly need row logic, itertuples is much faster than iterrows (which creates a Series per row); numba or Cython can compile numeric loops. Measure with %timeit rather than guessing.

A16. What is SettingWithCopyWarning, why does it appear, and how do you avoid it?

It warns that you may be assigning to a copy of a slice rather than to the original DataFrame, so the change might not propagate (or might unexpectedly do so). It arises from chained indexing:

```
df[df['age'] > 30]['bonus'] = 100        # two operations: boolean filter creates a copy,
then assignment on the copy
```

Fixes:

```
df.loc[df['age'] > 30, 'bonus'] = 100    # single operation, assigns on the original
sub = df[df['age'] > 30].copy()          # explicit copy if you want an independent subset
sub['bonus'] = 100
```

Also prefer assign() for method chains: df = df.assign(bonus=lambda d: np.where(d.age > 30, 100, 0)). In pandas 2.x you can enable Copy-on-Write with pd.options.mode.copy_on_write = True, which makes chained assignment never modify the parent and gives consistent behaviour; it is the default in pandas 3.

A17. Explain NumPy broadcasting with examples. What are the rules?

Broadcasting lets NumPy do arithmetic on arrays of different shapes without copying data. Rules: compare shapes from the trailing dimension backwards; two dimensions are compatible if they are equal or one of them is 1; missing leading dimensions are treated as 1. A dimension of size 1 is stretched to match the other.

```
a = np.arange(6).reshape(3, 2)   # shape (3,2)
```

```
b = np.array([10, 20])           # shape (2,)    -> broadcast to (3,2)
a + b
c = np.array([[1],[2],[3]])      # shape (3,1)   -> broadcast to (3,2)
a * c
X = np.random.rand(100, 4)
Z = (X - X.mean(axis=0)) / X.std(axis=0)   # (100,4) minus (4,) works
```

Shapes (3,2) and (3,) are not compatible (2 vs 3); fix with b[:, None] to make (3,1). Broadcasting is why standardisation, centering and distance matrices can be computed without loops.

A18. What is the difference between a view and a copy in NumPy? Why does it matter?

A view shares the same memory buffer as the original array; a copy owns new memory. Basic slicing (a[1:5], a[:, 0]), reshape (when possible), transpose and ravel (when contiguous) return views. Fancy indexing with an integer list or a boolean mask always returns a copy.

```
a = np.arange(10)
v = a[2:5]; v[0] = 99      # modifies a too
f = a[[2, 3, 4]]; f[0] = -1  # a unchanged
a.copy()                      # explicit copy
np.shares_memory(a, v)        # True
```

It matters for correctness (accidental mutation of source data) and performance (views are free; copies cost memory and time). Use .base to check whether an array is a view and .copy() when you want independence. flatten() always copies while ravel() returns a view if it can.

A19. Explain the differences between pivot, pivot_table, melt, stack and unstack in pandas.

pivot reshapes long data to wide using unique index/column pairs and no aggregation; it raises an error if duplicates exist. pivot_table does the same but aggregates duplicates (aggfunc='sum', 'mean', etc.), supports fill_value, margins=True for totals and multiple values/aggregations.

melt does the reverse: wide to long. id_vars stay as identifiers, value_vars become rows with var_name and value_name.

stack moves the innermost column level into the row index (wide to long on a MultiIndex); unstack moves an index level into columns.

```
wide = df.pivot_table(index='region', columns='month', values='sales', aggfunc='sum',
fill_value=0)
long = wide.reset_index().melt(id_vars='region', var_name='month', value_name='sales')
```

Long (tidy) format is best for seaborn, groupby and modelling; wide format is better for reports and heatmaps.

A20. How can you reduce the memory usage of a large pandas DataFrame?

- Check usage with df.info(memory_usage='deep') or df.memory_usage(deep=True).

- Downcast numerics: pd.to_numeric(col, downcast='integer'/'float') or specify int32, int16, float32 in read_csv dtype.

- Convert low-cardinality strings to category dtype; memory falls dramatically (a column with 10 distinct values over millions of rows).
- Use nullable or pyarrow-backed string dtypes instead of Python object strings.
- Load only needed columns (usecols) and rows (nrows, skiprows), parse dates with parse_dates.
- Read in chunks (chunksize) and aggregate incrementally.
- Use sparse dtypes for mostly-zero data, boolean instead of int for flags.
- Store in Parquet (columnar, compressed, typed) rather than CSV.
- Delete intermediate DataFrames and call gc.collect(); avoid unnecessary copies.
- When it still does not fit, move to Dask, Polars, DuckDB or a database.

A21. What methods can you use to detect outliers, and how do you decide whether to remove them?

Methods:

- IQR rule: values below Q1 - 1.5*IQR or above Q3 + 1.5*IQR (robust, used by boxplots).
- Z-score: |z| > 3, assumes roughly normal data and is itself sensitive to outliers; the modified z-score using median and MAD is more robust.
- Percentile capping (winsorizing) at 1st/99th.
- Visual: boxplots, histograms, scatter plots.
- Model based: Isolation Forest, Local Outlier Factor, DBSCAN, Mahalanobis distance for multivariate cases.
- Domain rules: negative ages, impossible dates, price of zero.

Deciding: determine whether the value is an error (typo, unit mismatch, sensor fault) or a genuine rare event (a large enterprise customer, a fraud case). Errors should be corrected or removed; genuine extremes should usually be kept, or handled with robust statistics (median), log transforms, or robust models. Always report the number removed, compare results with and without them, and never remove outliers in a fraud or anomaly project where they are the target.

A22. Explain Pearson versus Spearman correlation and the statement "correlation does not imply causation". How do you compute them in pandas?

Pearson measures the strength of a linear relationship between two continuous variables, ranges from -1 to 1, and is sensitive to outliers. Spearman is Pearson computed on ranks; it captures any monotonic relationship, is robust to outliers and works with ordinal data. Kendall tau is another rank option for small samples.

```
df[['a','b']].corr()                    # Pearson
df[['a','b']].corr(method='spearman')
from scipy.stats import pearsonr
r, p = pearsonr(df.a, df.b)
```

Correlation does not imply causation: a third confounding variable can drive both (ice cream sales and drownings both rise with temperature), the direction can be reversed, or the link can be coincidence. Also a correlation near zero does not mean independence, since Pearson misses non-linear relations such as y = x^2. To argue causation you need randomized experiments (A/B tests) or careful causal methods (difference-in-differences, instrumental variables, matching).

A23. What is the difference between matplotlib and seaborn, and what are Figure and Axes in matplotlib?

matplotlib is the low-level, highly customisable plotting library. seaborn is built on top of it and provides a high-level interface for statistical plots (histplot, kdeplot, boxplot, violinplot, heatmap, pairplot, lmplot) with better default styling, direct DataFrame support through data=, x=, y=, hue=, and automatic aggregation and confidence intervals.

In matplotlib, the Figure is the overall canvas/window; an Axes is one plot area within it (with its own x and y axis, title, labels). The recommended object-oriented style is:

```
fig, ax = plt.subplots(figsize=(8, 4))
sns.lineplot(data=df, x='date', y='sales', hue='region', ax=ax)
ax.set_title('Sales by region'); ax.set_xlabel('Date'); ax.set_ylabel('Sales')
fig.tight_layout(); fig.savefig('sales.png', dpi=150)
```

Seaborn axes-level functions accept ax=; figure-level functions (relplot, catplot, displot, pairplot) create their own figure and facet grid. Use plt.subplots(nrows, ncols) for multi-panel charts.

A24. How do you choose the right chart type for a given analytical question?

Start from the question:

- Comparison across categories: bar chart (horizontal if labels are long), sorted by value.

- Trend over time: line chart; use area for cumulative composition.

- Distribution of one variable: histogram, KDE, boxplot or violin plot.

- Relationship between two numeric variables: scatter plot (add hue or size for a third variable; use hexbin or alpha for dense data).

- Correlation matrix: heatmap.

- Part-to-whole: stacked bar or a simple bar of shares; pie charts only for very few categories.

- Many-variable overview: pairplot or small multiples (facets).

- Geographical: choropleth or map.

Good practice: label axes and units, start bar charts at zero, avoid 3D and dual axes that mislead, use consistent and colour-blind-safe palettes, limit categories, give the chart a title that states the insight, and annotate key points.

A25. How do you read a boxplot, and when is a histogram better than a bar chart?

A boxplot shows the median (line in the box), the first and third quartiles (box edges, whose difference is the IQR), whiskers extending to the most extreme points within 1.5*IQR of the box, and individual dots for points beyond that (potential outliers). Box position and asymmetry show skew; a median closer to one edge indicates skew. It compares distributions across groups compactly but hides multimodality, which a violin plot or histogram reveals.

A histogram bins a continuous numeric variable and shows frequency per bin, so bars touch and order matters; bin width changes the shape, so try several bin counts. A bar chart compares discrete categories with gaps between bars and an arbitrary order. Use histograms for distributions (age, price), bar charts for category summaries (sales per region).

```
sns.boxplot(data=df, x='segment', y='revenue')
```

```
sns.histplot(df['revenue'], bins=30, kde=True)
```

A26. What is data leakage and how can preprocessing steps like scaling or imputation cause it? How do you prevent it with scikit-learn?

Data leakage is when information from outside the training set (test data or future data) influences model training, giving over-optimistic evaluation that collapses in production. Examples: fitting a StandardScaler or imputer on the full dataset before splitting (test statistics leak into training); target encoding computed with the whole data; using features that are only known after the outcome (a "refund_issued" column when predicting churn); random splitting of time series data; duplicate records across train and test.

Prevention: split first, then fit transformers on the training set only and apply them to the test set. In scikit-learn use Pipeline and ColumnTransformer so cross-validation refits preprocessing within each fold.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
pre = ColumnTransformer([
  ('num', Pipeline([('imp', SimpleImputer(strategy='median')), ('sc', StandardScaler())]),
num_cols),
  ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)])
model = Pipeline([('pre', pre), ('clf', LogisticRegression(max_iter=1000))])
cross_val_score(model, X, y, cv=5)
```

For time data use TimeSeriesSplit and group-aware splits (GroupKFold) when rows from the same entity repeat.

A27. What is the difference between normalization (min-max scaling) and standardization? When do you use each?

Min-max normalization rescales to a fixed range, usually [0, 1]: x' = (x - min) / (max - min). Standardization (z-score) centers on zero mean and unit standard deviation: z = (x - mean) / std.

Use standardization for algorithms that assume or benefit from centered, roughly Gaussian features, or are sensitive to variance: linear and logistic regression with regularisation, SVM, PCA, k-means (distance based), neural networks. Use min-max when you need a bounded range (image pixels, some neural nets) and the data has no extreme outliers, since a single outlier compresses all other values. RobustScaler (median and IQR) is better when outliers exist.

Tree-based models (decision trees, random forests, gradient boosting) do not need scaling. Always fit the scaler on training data only. For skewed variables consider a log or Box-Cox transform before scaling.

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler
```

A28. How do you encode categorical variables? Compare label encoding, one-hot encoding, ordinal encoding and target encoding.

- One-hot encoding: one binary column per category (pd.get_dummies or OneHotEncoder). No false ordering; best for nominal variables with modest cardinality. Drop one level (drop_first or drop='first') for linear models to avoid the dummy-variable trap (perfect multicollinearity).

- Ordinal encoding: integers that respect a real order (low < medium < high); use OrdinalEncoder with explicit categories.

- Label encoding: arbitrary integers per category; fine for the target variable or for tree models, but introduces a false order for linear or distance-based models.

- Target (mean) encoding: replace the category by the average target value; handles high cardinality but leaks if computed on all data, so compute within cross-validation folds with smoothing.

- Frequency or count encoding and hashing: alternatives for high-cardinality features.

- Native support: LightGBM, CatBoost accept categorical columns directly.

Handle unseen categories at prediction time with handle_unknown='ignore'.

A29. How does pandas handle dates and times? Explain to_datetime, the dt accessor, resample and timezone handling.

pd.to_datetime converts strings or numbers to datetime64[ns]. Use format= for speed and exactness, errors='coerce' to turn unparseable values into NaT, dayfirst=True for DD/MM data, and unit='s' for epoch seconds.

The .dt accessor extracts components: .dt.year, .month, .day_name(), .dayofweek, .quarter, .date, and supports floor/ceil/strftime. Differences of datetimes give Timedelta; use .dt.days or .dt.total_seconds().

With a DatetimeIndex you can slice by strings (df.loc['2024-01':'2024-03']) and resample (downsample or upsample) like a time-based groupby: df.resample('W').sum(), df.resample('MS')['sales'].mean(). Also shift, diff, pct_change, rolling('7D').

Timezones: dt.tz_localize('UTC') attaches a zone to naive timestamps, dt.tz_convert('Asia/Kolkata') converts between zones. Store as UTC and convert for display; mixing naive and aware datetimes raises errors. pd.Grouper(key='date', freq='M') allows time grouping combined with other keys.

A30. What are Python context managers and why is "with open(...)" preferred?

A context manager defines setup and teardown around a block via __enter__ and __exit__ (or the contextlib.contextmanager decorator). The "with" statement guarantees cleanup runs even if an exception occurs.

```
with open('data.csv') as f:
    header = f.readline()
# file is closed here, even on error
```

Without it you must remember f.close() in a try/finally. Other examples: database connections and transactions (with sqlite3.connect(...) as conn), locks, temporary directories, and timing blocks. Custom one:

```
from contextlib import contextmanager
import time
@contextmanager
```

```
def timer(label):
    t0 = time.perf_counter()
    yield
    print(label, time.perf_counter() - t0)
```

This avoids leaked file handles and half-written resources in pipelines.

A31. How does exception handling work in Python (try, except, else, finally)? Give a data-loading example.

try holds code that may fail; except catches specific exceptions; else runs only if no exception occurred; finally always runs (cleanup). Catch specific exceptions, not a bare except, so real bugs are not hidden; raise to signal errors and use custom exceptions for domain problems.

```
import pandas as pd
def load(path):
    try:
        df = pd.read_csv(path)
    except FileNotFoundError:
        print('File missing:', path)
        return None
    except pd.errors.ParserError as e:
        print('Bad format:', e)
        return None
    else:
        print('Loaded', len(df), 'rows')
        return df
    finally:
        print('load() finished')
```

Use logging instead of print in real pipelines, validate input early (assert or explicit checks), and re-raise when the caller should decide what to do.

A32. How would you process a CSV file that is larger than available memory using pandas?

Use chunked reading and incremental aggregation so only a slice is in memory:

```
import pandas as pd
totals = {}
for chunk in pd.read_csv('big.csv', chunksize=500_000,
                         usecols=['region', 'sales'],
                         dtype={'region': 'category', 'sales': 'float32'}):
    s = chunk.groupby('region')['sales'].sum()
    for k, v in s.items():
        totals[k] = totals.get(k, 0) + v
result = pd.Series(totals)
```

Tips: select only needed columns, set compact dtypes, filter early, and combine partial aggregates correctly (sums and counts combine; means need sum and count; medians do not combine and need approximation). Better long-term options: convert to Parquet once, use Dask or Polars (lazy scan with streaming), DuckDB (SQL directly over CSV or Parquet files), or load into a database and query there.

A33. How do you find and handle duplicate records in pandas? What does keep do?

df.duplicated(subset=None, keep='first') returns a boolean mask marking duplicates; df.drop_duplicates(subset=[...], keep=...) removes them. keep='first' marks all but the first occurrence as duplicate, keep='last' marks all but the last, keep=False marks every row that has a duplicate (useful to inspect all conflicting copies).

```
df.duplicated().sum()                              # count exact duplicate rows
df[df.duplicated(['email'], keep=False)].sort_values('email')   # inspect
df = df.sort_values('updated_at').drop_duplicates('customer_id', keep='last')  # keep latest
```

Before dropping, decide the business key: exact row duplicates are usually load errors, but duplicates on an ID with different values need a rule (latest record, most complete record, aggregate). Normalise text first (strip, lower, remove punctuation) so near duplicates match, and log how many rows were removed.

A34. How do hash tables back Python dictionaries and sets, and what are the time complexities of common operations?

A dict stores keys in a hash table: the key's hash() chooses a slot; collisions are resolved by open addressing with probing. Because lookup goes straight to a slot, get, set, delete and membership are O(1) on average and O(n) worst case with pathological collisions. Keys must be hashable and immutable-ish (str, int, tuple of hashables), and their hash must be consistent with equality. Lists and dicts are unhashable.

Complexities: list index O(1), append O(1) amortised, insert or remove at front O(n), "x in list" O(n); set/dict "x in" O(1); sorting O(n log n); list.pop(0) is O(n) so use collections.deque for queues. Dictionaries keep insertion order (3.7+). Counter, defaultdict and OrderedDict from collections extend it. Choosing the right structure often decides whether a solution passes a time limit.

A35. Explain how to handle class imbalance in a classification dataset (for example, 1 percent fraud). Which metrics and techniques do you use?

Accuracy is misleading (99 percent by predicting all negative). Use precision, recall, F1, PR-AUC (preferred over ROC-AUC under heavy imbalance), confusion matrix, and choose a threshold by business cost.

Techniques:

- Stratified train/test split and stratified cross-validation so each fold keeps the ratio.

- Class weights: class_weight='balanced' in logistic regression, random forest, SVM; scale_pos_weight in XGBoost.

- Resampling on the training set only: random undersampling of the majority, oversampling of the minority, or SMOTE (imblearn). Never resample before the split.

- Threshold moving using predict_proba and a precision-recall curve.

- Anomaly detection approaches or ensemble methods like BalancedRandomForest.

- Collect more minority data or engineer better features.

```
from sklearn.model_selection import train_test_split
X_tr, X_te, y_tr, y_te = train_test_split(X, y, stratify=y, test_size=0.2, random_state=42)
```

A36. Explain rolling, expanding, shift and diff in pandas and what each is used for.

- rolling(window).agg: fixed-size moving window, e.g. df['sales'].rolling(7).mean() for a 7-period moving average; min_periods controls how many observations are needed; time-based windows like rolling('7D') work with a DatetimeIndex; center=True centres the window.

- expanding(): window grows from the start, giving cumulative statistics (running mean, running max).

- ewm(span=n).mean(): exponentially weighted moving average that gives recent points more weight.

- shift(n): moves values down by n rows, used to create lag features or compare to previous period; shift(-1) looks ahead.

- diff(n): value minus the value n rows earlier; pct_change(n) gives the percentage version.

Combine with groupby for per-entity calculations: df.groupby('store')['sales'].shift(1). Be careful to sort by time first and to avoid look-ahead leakage when making features.

A37. How do you split data into train and test sets and perform cross-validation? Why use stratification or random_state?

train_test_split holds out a fraction (typically 20 to 30 percent) to estimate generalisation. random_state fixes the shuffling seed so results are reproducible. stratify=y keeps class proportions the same in both parts, important for classification with imbalance.

K-fold cross-validation splits training data into k folds, trains on k-1 and validates on the remaining one, rotating; the mean and standard deviation of scores give a more stable estimate than one split. Variants: StratifiedKFold for classification, GroupKFold when rows belong to groups (customers), TimeSeriesSplit for temporal data (train on the past, validate on the future).

```
from sklearn.model_selection import cross_val_score, StratifiedKFold
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
scores = cross_val_score(model, X, y, cv=cv, scoring='f1')
```

Keep a final untouched test set for the last evaluation; use CV (or a validation split) for tuning hyperparameters.

A38. What is a MultiIndex in pandas and when is it useful? How do you work with it?

A MultiIndex is a hierarchical index with two or more levels on rows or columns. It is produced by groupby on multiple keys, set_index(['a','b']), pivot_table, or unstack/stack.

```
s = df.groupby(['region', 'product'])['sales'].sum()     # MultiIndex Series
s.loc['West']                  # all products in West
s.loc[('West', 'Pen')]         # single entry
s.xs('Pen', level='product')   # cross-section on inner level
s.unstack('product')           # inner level to columns
s.reset_index()                # back to flat columns
df.columns = ['_'.join(c) for c in df.columns]   # flatten column MultiIndex
```

Useful for panel data (entity and time), compact summaries and fast hierarchical slicing (sort the index with sort_index() for performance). The downside is complexity: many analysts prefer a flat table with reset_index() for plotting and merging.

A39. Explain how pandas categorical dtype works and what the benefits are. What traps exist when using it?

A categorical column stores a small array of unique categories plus integer codes per row. Benefits: much lower memory for repetitive strings, faster groupby/sort/merge on keys, and the ability to define an order (ordered=True) so that sorting and comparisons follow a logical sequence such as low < medium < high.

```
df['size'] = pd.Categorical(df['size'], categories=['S','M','L'], ordered=True)
df['city'] = df['city'].astype('category')
df['size'].cat.codes; df['size'].cat.categories
```

Traps: groupby on categoricals includes unobserved categories unless observed=True, producing empty groups; assigning a value not in categories gives NaN or an error; concatenating categoricals with different category sets falls back to object unless unioned (pd.api.types.union_categoricals); arithmetic is not supported; merging on categoricals with different categories can be slow or change dtype; and rare categories must be handled before modelling. Use cat.add_categories, cat.remove_unused_categories and cat.reorder_categories to manage them.

A40. Explain the difference between Python's multiprocessing, threading and asyncio, and which you would use to download 1,000 API pages versus t...

threading: multiple threads in one process; limited by the GIL for CPU-bound code but effective for I/O waits. asyncio: single-threaded cooperative concurrency using event loop and await; scales to thousands of concurrent network connections with low overhead, but needs async-compatible libraries (aiohttp, httpx). multiprocessing: separate processes with separate memory and GILs; true parallelism for CPU-bound work, at the cost of process start-up and data serialization (pickling) between processes.

For 1,000 API pages (I/O bound): ThreadPoolExecutor or asyncio with a semaphore to respect rate limits, plus retries and backoff.

```
from concurrent.futures import ThreadPoolExecutor
import requests
def fetch(p): return requests.get(url, params={'page': p}, timeout=30).json()
with ThreadPoolExecutor(max_workers=8) as ex:
    pages = list(ex.map(fetch, range(1, 1001)))
```

For heavy numeric computation: vectorize with NumPy first, then ProcessPoolExecutor, joblib, Dask, or numba; split data into chunks to limit pickling overhead.

A41. How do you test whether a difference between two groups is statistically significant in Python? Describe t-test, chi-square and the assumpti...

Choose by data type. Comparing means of a continuous metric between two independent groups: Welch's t-test (does not assume equal variances), or Mann-Whitney U if the data is heavily skewed or ordinal. Paired measurements: paired t-test or Wilcoxon signed-rank. Three or more groups: one-way ANOVA or Kruskal-Wallis. Comparing proportions or association between categorical variables (conversion rate by variant): chi-square test of independence or two-proportion z-test; Fisher's exact test for small counts.

```
from scipy import stats
t, p = stats.ttest_ind(a, b, equal_var=False)
chi2, p, dof, exp = stats.chi2_contingency(pd.crosstab(df.variant, df.converted))
```

Assumptions: independent observations, approximately normal sampling distribution (fine for large n via central limit theorem), no big outliers. Interpret p-value as the probability of results at least this extreme if the null is true, not the probability the null is true. Report effect size and confidence intervals, and correct for multiple comparisons (Bonferroni, Benjamini-Hochberg). Decide sample size beforehand with a power analysis.

A42. Write a function that checks whether a string is a palindrome, ignoring case, spaces and punctuation.

Clean the string to alphanumeric lowercase characters and compare it with its reverse, or use two pointers for O(1) extra space.

```
def is_palindrome(s: str) -> bool:
    cleaned = [ch.lower() for ch in s if ch.isalnum()]
    return cleaned == cleaned[::-1]

def is_palindrome_two_pointers(s):
    i, j = 0, len(s) - 1
    while i < j:
        while i < j and not s[i].isalnum(): i += 1
        while i < j and not s[j].isalnum(): j -= 1
        if s[i].lower() != s[j].lower(): return False
        i += 1; j -= 1
    return True

print(is_palindrome('A man, a plan, a canal: Panama'))   # True
```

Time O(n). The first version uses O(n) extra memory.

A43. Given a list of integers and a target, return the indices of the two numbers that add up to the target (Two Sum).

Brute force checks all pairs in O(n^2). Use a dictionary of seen value to index for O(n) time.

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        need = target - x
        if need in seen:
            return [seen[need], i]
        seen[x] = i
    return []

print(two_sum([2, 7, 11, 15], 9))   # [0, 1]
```

Explanation: for each number, we look up whether its complement has appeared earlier. Insert after checking, so the same element is not used twice. Space O(n). If the array is sorted, a two-pointer approach works with O(1) space.

A44. Count the frequency of each word in a text, ignoring case and punctuation, and return the top 3 most common words.

```
import re
from collections import Counter

def top_words(text, n=3):
    words = re.findall(r"[a-z']+", text.lower())
    return Counter(words).most_common(n)

text = "The cat and the hat. The cat sat; the hat did not."
print(top_words(text))   # [('the', 4), ('cat', 2), ('hat', 2)]
```

Counter is a dict subclass for counting hashable items; most_common(n) returns the n highest in descending order (ties keep first-seen order). Time O(n) for counting plus O(k log n) for the top. Without Counter:

```
freq = {}
for w in words:
    freq[w] = freq.get(w, 0) + 1
sorted(freq.items(), key=lambda kv: -kv[1])[:3]
```

In pandas: pd.Series(words).value_counts().head(3).

A45. Write code to find all duplicate elements in a list, and also remove duplicates while preserving the original order.

```
from collections import Counter

def find_duplicates(lst):
    return [k for k, c in Counter(lst).items() if c > 1]

def dedupe_keep_order(lst):
    seen = set()
    out = []
    for x in lst:
        if x not in seen:
            seen.add(x)
            out.append(x)
    return out

data = [3, 1, 3, 2, 1, 5]
print(find_duplicates(data))        # [3, 1]
print(dedupe_keep_order(data))      # [3, 1, 2, 5]
print(list(dict.fromkeys(data)))    # one-liner that preserves order
```

set(lst) removes duplicates but loses order. Both approaches are O(n) because set and dict membership are O(1) on average. Elements must be hashable; for lists of lists convert to tuples first.

A46. Group a list of words into anagram groups, for example ["eat","tea","tan","ate","nat","bat"].

Words that are anagrams share the same sorted-letter signature, so use it as a dictionary key.

```
from collections import defaultdict

def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        groups[''.join(sorted(w))].append(w)
    return list(groups.values())

print(group_anagrams(["eat","tea","tan","ate","nat","bat"]))
# [['eat','tea','ate'], ['tan','nat'], ['bat']]
```

Time O(n * k log k) where k is word length. To get O(n*k), use a tuple of 26 letter counts as the key instead of sorting. defaultdict avoids the explicit "if key not in dict" check.

A47. Find the length of the longest substring without repeating characters.

Use a sliding window with a dict that stores the last index of each character.

```
def longest_unique_substring(s):
    last = {}
    start = 0
    best = 0
    for i, ch in enumerate(s):
        if ch in last and last[ch] >= start:
            start = last[ch] + 1
        last[ch] = i
        best = max(best, i - start + 1)
    return best

print(longest_unique_substring("abcabcbb"))  # 3
print(longest_unique_substring("pwwkew"))    # 3
```

When we meet a character already inside the current window, we move the window start just past its previous position. Each character is processed once, so time is O(n) and space is O(min(n, alphabet)).

A48. Merge overlapping intervals, e.g. [[1,3],[2,6],[8,10],[15,18]] becomes [[1,6],[8,10],[15,18]]. Relate it to a data problem such as merging b...

Sort by start, then sweep: if the current interval starts before or at the end of the last merged one, extend it; otherwise start a new one.

```
def merge_intervals(intervals):
    intervals = sorted(intervals, key=lambda x: x[0])
    merged = []
    for s, e in intervals:
        if merged and s <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], e)
        else:
            merged.append([s, e])
```

```
    return merged

print(merge_intervals([[1,3],[2,6],[8,10],[15,18]]))
```

Time O(n log n) from sorting. Data use cases: combining overlapping hotel bookings to compute occupied days, merging overlapping sessions or subscription periods, finding total downtime from outage logs.

A49. Write a function to check whether a string of brackets "()[]{}" is balanced.

Use a stack: push opening brackets; for a closing bracket, the top of the stack must be the matching opener.

```
def is_balanced(s):
    pairs = {')': '(', ']': '[', '}': '{'}
    stack = []
    for ch in s:
        if ch in pairs.values():
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack

print(is_balanced("{[()]}"))   # True
print(is_balanced("([)]"))     # False
print(is_balanced("(("))       # False
```

Time and space O(n). The final "not stack" catches unclosed openers. Lists work as stacks since append and pop at the end are O(1).

A50. Given a DataFrame of employees (name, dept, salary), find the second highest salary overall and the second highest salary within each depart...

```
import pandas as pd
df = pd.DataFrame({'name': list('ABCDEF'),
                   'dept': ['IT','IT','IT','HR','HR','HR'],
                   'salary': [100, 90, 90, 70, 80, 60]})

# overall second highest distinct salary
second = df['salary'].drop_duplicates().nlargest(2).iloc[-1]   # 90

# per department using dense rank so ties are handled
df['rnk'] = df.groupby('dept')['salary'].rank(method='dense', ascending=False)
per_dept = df[df['rnk'] == 2]
print(per_dept[['dept', 'name', 'salary']])
```

Using method='dense' mirrors SQL DENSE_RANK so equal salaries share a rank and the next distinct salary is rank 2. Guard against departments with fewer than two distinct salaries (they simply have no rank-2 row; nlargest(2).iloc[-1] would return the only value, so check len first). Equivalent SQL: DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC).

A51. From a sales DataFrame (region, product, revenue), return the top 2 products by revenue in each region.

```
import pandas as pd
sales = pd.DataFrame({
  'region': ['N','N','N','S','S','S'],
  'product': ['a','b','c','a','b','c'],
  'revenue': [100, 300, 200, 50, 20, 70]})

# aggregate first if multiple rows per product
agg = sales.groupby(['region', 'product'], as_index=False)['revenue'].sum()

top2 = (agg.sort_values(['region', 'revenue'], ascending=[True, False])
           .groupby('region')
           .head(2))
# alternative: agg.groupby('region', group_keys=False).apply(lambda g: g.nlargest(2,
'revenue'))
# alternative with rank:
agg['rk'] = agg.groupby('region')['revenue'].rank(method='first', ascending=False)
top2_rank = agg[agg['rk'] <= 2]
print(top2)
```

sort_values plus groupby().head(n) is fast and readable. Use rank with method='dense' or 'min' if ties should all be included.

A52. A customers table has multiple rows per customer_id with an updated_at timestamp. Keep only the latest record per customer.

```
import pandas as pd
df = pd.DataFrame({
  'customer_id': [1, 1, 2, 2, 3],
  'email': ['a@x', 'a@y', 'b@x', 'b@x', 'c@x'],
  'updated_at': pd.to_datetime(['2024-01-01','2024-03-01','2024-02-01','2024-02-15','2024-01-
10'])})

# Option 1: sort then drop duplicates
latest = (df.sort_values('updated_at')
            .drop_duplicates('customer_id', keep='last'))

# Option 2: index of the max timestamp per group
latest2 = df.loc[df.groupby('customer_id')['updated_at'].idxmax()]

# Option 3: rank/transform
latest3 = df[df['updated_at'] == df.groupby('customer_id')['updated_at'].transform('max')]
```

idxmax returns exactly one row per group (first max if tied), while the transform approach returns all tied rows. Make sure updated_at is a real datetime and not a string, otherwise sorting is lexical.

A53. Fill missing values in a numeric column with the median of its group (for example, missing salary filled by department median), and fall bac...

```
import pandas as pd, numpy as np
```

```
df = pd.DataFrame({'dept': ['A','A','A','B','B','C'],
                   'salary': [10, np.nan, 30, 50, np.nan, np.nan]})

group_med = df.groupby('dept')['salary'].transform('median')
df['salary_filled'] = df['salary'].fillna(group_med).fillna(df['salary'].median())
print(df)
```

transform('median') returns a Series aligned with df, so fillna matches by index. Dept C has no observed values, so its group median is NaN and the second fillna applies the global median. In a modelling workflow compute the medians on the training set only and store them in a dictionary (or use a scikit-learn imputer inside a pipeline) so the same values are applied to test data.

A54. Given daily revenue, compute month-over-month growth percentage and a 7-day rolling average.

```
import pandas as pd, numpy as np
rng = pd.date_range('2024-01-01', periods=120, freq='D')
df = pd.DataFrame({'date': rng, 'revenue': np.random.randint(100, 500, len(rng))})

# monthly totals and MoM %
monthly = (df.set_index('date')['revenue'].resample('MS').sum().to_frame('revenue'))
monthly['mom_pct'] = monthly['revenue'].pct_change() * 100
# equivalently: (revenue - revenue.shift(1)) / revenue.shift(1) * 100

# 7-day rolling mean
df = df.sort_values('date')
df['rolling_7d'] = df['revenue'].rolling(window=7, min_periods=1).mean()
# time-aware: df.set_index('date')['revenue'].rolling('7D').mean()
print(monthly.round(1))
```

Notes: the first month has NaN growth. If a month has no data, resample gives 0 for sum, which creates division problems in the next pct_change; handle with replace(0, np.nan) when needed. Partial months (the current month) will look like a decline, so flag or exclude them.

A55. Create a pivot table of total sales by region (rows) and month (columns) with totals, then convert it back to long format.

```
import pandas as pd
df = pd.DataFrame({
  'region': ['N','N','S','S','N','S'],
  'order_date': pd.to_datetime(['2024-01-05','2024-02-10','2024-01-15','2024-02-20','2024-01-
25','2024-02-28']),
  'sales': [100, 150, 200, 50, 75, 125]})
df['month'] = df['order_date'].dt.to_period('M').astype(str)

pt = pd.pivot_table(df, index='region', columns='month', values='sales',
                    aggfunc='sum', fill_value=0, margins=True, margins_name='Total')
print(pt)

long = (pt.drop(index='Total', columns='Total')
          .reset_index()
          .melt(id_vars='region', var_name='month', value_name='sales'))
```

```
print(long)
```

margins=True adds a "Total" row and column. Dropping them before melting prevents double counting. Use observed=True with categorical keys and aggfunc as a dictionary for several measures.

A56. Find customers who have never placed an order (an anti-join) using pandas.

```
import pandas as pd
customers = pd.DataFrame({'customer_id': [1,2,3,4], 'name': list('ABCD')})
orders = pd.DataFrame({'order_id': [10,11,12], 'customer_id': [1,1,3]})

# Option 1: isin
no_orders = customers[~customers['customer_id'].isin(orders['customer_id'])]

# Option 2: merge with indicator
m = customers.merge(orders[['customer_id']].drop_duplicates(), on='customer_id',
                    how='left', indicator=True)
no_orders2 = m[m['_merge'] == 'left_only'].drop(columns='_merge')

# Option 3: left join and check null on right column
m2 = customers.merge(orders, on='customer_id', how='left')
no_orders3 = m2[m2['order_id'].isna()]
print(no_orders)
```

The isin approach is simplest and fast. In merge, drop duplicate keys on the right first to avoid row multiplication. Equivalent SQL: LEFT JOIN ... WHERE o.customer_id IS NULL, or NOT EXISTS.

A57. Compute a running (cumulative) total of sales per customer ordered by date, and the percentage each order contributes to that customer's tot...

```
import pandas as pd
df = pd.DataFrame({
  'customer': ['a','a','a','b','b'],
  'date': pd.to_datetime(['2024-01-01','2024-01-05','2024-02-01','2024-01-02','2024-01-09']),
  'amount': [10, 20, 30, 5, 15]})

df = df.sort_values(['customer', 'date'])
df['running_total'] = df.groupby('customer')['amount'].cumsum()
df['customer_total'] = df.groupby('customer')['amount'].transform('sum')
df['pct_of_customer'] = (df['amount'] / df['customer_total'] * 100).round(1)
df['order_number'] = df.groupby('customer').cumcount() + 1
print(df)
```

cumsum, cummax, cumcount and rank all work per group. Always sort first, because cumulative operations follow row order. SQL equivalent: SUM(amount) OVER (PARTITION BY customer ORDER BY date).

A58. Clean a messy text column: trim whitespace, normalise case, remove special characters from names, and standardise phone numbers to digits on...

```
import pandas as pd
```

```
df = pd.DataFrame({
  'name': ['  john SMITH ', 'MARY  o\'neil!!', None],
  'phone': ['(555) 123-4567', '555.123.4568', '+1 555 123 4569']})

df['name_clean'] = (df['name'].str.strip()
                              .str.replace(r'[^A-Za-z\' ]', '', regex=True)
                              .str.replace(r'\s+', ' ', regex=True)
                              .str.title())

digits = df['phone'].str.replace(r'\D', '', regex=True)
df['phone_clean'] = digits.str[-10:]            # keep last 10 digits, drop country code
df.loc[digits.str.len() < 10, 'phone_clean'] = pd.NA
print(df)
```

The .str accessor is vectorised and propagates NaN for missing values. Always set regex=True explicitly. Validate afterwards (length checks, value_counts of lengths) and keep the raw column for audit.

A59. A date column contains mixed formats (2024-01-05, 05/01/2024, Jan 5 2024, and junk like "N/A"). Parse it robustly and report the rows that f...

```
import pandas as pd
s = pd.Series(['2024-01-05', '05/01/2024', 'Jan 5 2024', 'N/A', '2024/13/45'])

# Pandas 2.x: format='mixed' parses each element individually
parsed = pd.to_datetime(s, errors='coerce', format='mixed', dayfirst=True)

failed = s[parsed.isna()]
print(parsed)
print('Unparsed:', failed.tolist())

# Explicit multi-format approach (more controlled)
formats = ['%Y-%m-%d', '%d/%m/%Y', '%b %d %Y']
out = pd.Series(pd.NaT, index=s.index, dtype='datetime64[ns]')
for f in formats:
    mask = out.isna()
    out[mask] = pd.to_datetime(s[mask], format=f, errors='coerce')
```

Key points: errors='coerce' converts bad values to NaT instead of raising. Ambiguous values like 05/01/2024 depend on dayfirst; you must confirm the convention with the data owner. Trying known formats in order is faster and more deterministic than letting the parser guess. Always count NaT before and after to quantify loss.

A60. Remove outliers from a numeric column using the IQR rule, and show an alternative that caps (winsorizes) instead of dropping.

```
import pandas as pd
def iqr_bounds(s, k=1.5):
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    return q1 - k * iqr, q3 + k * iqr
```

```
df = pd.DataFrame({'price': [10, 12, 11, 13, 12, 300, 9, -50]})
lo, hi = iqr_bounds(df['price'])

removed = df[(df['price'] < lo) | (df['price'] > hi)]
clean = df[df['price'].between(lo, hi)]

capped = df.copy()
capped['price'] = capped['price'].clip(lower=lo, upper=hi)
print(lo, hi); print(removed)
```

Compute bounds per group (groupby + transform of quantile) when groups have different scales. Report how many rows were affected. Capping preserves sample size, dropping removes information; choose based on whether extremes are errors or real.

A61. Using NumPy only (no loops), standardize every column of a 2D array and find the pair of rows that are closest in Euclidean distance.

```
import numpy as np
rng = np.random.default_rng(0)
X = rng.normal(loc=[5, 100, 0], scale=[1, 20, 0.1], size=(6, 3))

Z = (X - X.mean(axis=0)) / X.std(axis=0, ddof=0)     # column-wise z-score via broadcasting

# pairwise distances via broadcasting: (n,1,d) - (1,n,d) -> (n,n,d)
D = np.sqrt(((Z[:, None, :] - Z[None, :, :]) ** 2).sum(axis=2))
np.fill_diagonal(D, np.inf)                           # ignore self-distance
i, j = np.unravel_index(np.argmin(D), D.shape)
print('closest rows:', i, j, 'distance:', D[i, j])
```

axis=0 aggregates down rows giving one value per column. The distance matrix uses O(n^2 * d) memory in this form; for large n use scipy.spatial.distance.cdist or sklearn pairwise_distances, or a KD-tree (scipy.spatial.cKDTree) for nearest neighbours.

A62. Compute a moving average over a NumPy array efficiently, without a Python loop.

```
import numpy as np

def moving_average(a, w):
    c = np.cumsum(np.insert(a, 0, 0.0))
    return (c[w:] - c[:-w]) / w

x = np.array([1, 2, 3, 4, 5, 6], dtype=float)
print(moving_average(x, 3))                # [2. 3. 4. 5.]

# Alternative: convolution
print(np.convolve(x, np.ones(3) / 3, mode='valid'))

# Alternative: sliding_window_view
from numpy.lib.stride_tricks import sliding_window_view
print(sliding_window_view(x, 3).mean(axis=1))
```

The cumulative-sum trick is O(n) independent of window size. It returns n - w + 1 values (valid positions only). Cumsum can accumulate floating-point error on very long float arrays. In pandas the same result is pd.Series(x).rolling(3).mean(), which pads the first w-1 positions with NaN.

A63. Create a chart with monthly revenue as bars and conversion rate as a line on a secondary y-axis using matplotlib. Add labels and a legend.

```
import matplotlib.pyplot as plt
import pandas as pd
df = pd.DataFrame({'month': ['Jan','Feb','Mar','Apr','May','Jun'],
                   'revenue': [120, 135, 150, 160, 155, 180],
                   'conv': [2.1, 2.4, 2.3, 2.8, 3.0, 3.2]})

fig, ax1 = plt.subplots(figsize=(8, 4))
bars = ax1.bar(df['month'], df['revenue'], color='#4C78A8', label='Revenue (k$)')
ax1.set_ylabel('Revenue (k$)'); ax1.set_xlabel('Month')
ax1.bar_label(bars, padding=2, fontsize=8)

ax2 = ax1.twinx()
line, = ax2.plot(df['month'], df['conv'], color='#F58518', marker='o', label='Conversion
(%)')
ax2.set_ylabel('Conversion rate (%)')
ax2.set_ylim(0, 4)

ax1.legend([bars, line], ['Revenue (k$)', 'Conversion (%)'], loc='upper left')
ax1.set_title('Revenue and conversion rate, H1')
fig.tight_layout()
plt.show()
```

twinx creates a second y-axis sharing the x-axis. Dual-axis charts can mislead if scales are chosen to exaggerate correlation; label both axes clearly or use two stacked subplots instead.

A64. Draw a correlation heatmap of numeric columns with seaborn, masking the upper triangle and annotating values.

```
import numpy as np, pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.DataFrame(np.random.randn(200, 5), columns=list('ABCDE'))
df['F'] = df['A'] * 0.8 + np.random.randn(200) * 0.3

corr = df.corr(numeric_only=True)
mask = np.triu(np.ones_like(corr, dtype=bool))

plt.figure(figsize=(7, 5))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='coolwarm',
            vmin=-1, vmax=1, center=0, square=True, linewidths=0.5,
            cbar_kws={'label': 'Pearson r'})
plt.title('Correlation matrix')
```

```
plt.tight_layout(); plt.show()
```

Use a diverging colormap centred at zero so sign is clear. The mask hides the redundant upper triangle. For highly correlated feature pairs (|r| above 0.9) consider dropping one to reduce multicollinearity before regression.

A65. Bin a continuous age column into groups (0-17, 18-34, 35-54, 55+) and also create quartile-based income groups. Explain cut versus qcut.

```
import pandas as pd
df = pd.DataFrame({'age': [5, 22, 40, 67, 30, 17, 55],
                   'income': [0, 30, 70, 50, 90, 10, 120]})

df['age_group'] = pd.cut(df['age'], bins=[0, 17, 34, 54, 120],
                         labels=['0-17', '18-34', '35-54', '55+'], right=True)
df['income_q'] = pd.qcut(df['income'], q=4, labels=['Q1','Q2','Q3','Q4'], duplicates='drop')
print(df)
print(df['age_group'].value_counts().sort_index())
```

pd.cut makes bins of chosen edges (equal width if you pass an integer), so group sizes can be uneven. pd.qcut makes bins with equal numbers of observations by quantile, so edges depend on the data. With right=True intervals are (a, b]; set include_lowest=True to include the minimum edge. qcut can fail with many ties unless duplicates='drop'.

A66. A column "tags" contains comma-separated values like "a,b,c". Split it into one row per tag and count the tags.

```
import pandas as pd
df = pd.DataFrame({'id': [1, 2, 3],
                   'tags': ['python,sql', 'sql', 'python, excel ,sql']})

long = (df.assign(tag=df['tags'].str.split(','))
          .explode('tag')
          .assign(tag=lambda d: d['tag'].str.strip().str.lower()))

counts = long['tag'].value_counts()
print(long)
print(counts)

# one-hot (wide) alternative
dummies = df['tags'].str.replace(' ', '').str.get_dummies(sep=',')
```

explode turns each list element into its own row, repeating other columns. Strip whitespace after splitting to avoid " sql" versus "sql". Rows with NaN stay as NaN; use dropna first if needed. Use get_dummies(sep=',') for a multi-hot matrix per record.

A67. Build a monthly cohort retention table: for each customer's first-purchase month, the percentage of customers who purchase again in each sub...

```
import pandas as pd
orders = pd.DataFrame({
  'customer_id': [1,1,1,2,2,3,3,4],
  'order_date': pd.to_datetime(['2024-01-10','2024-02-12','2024-04-03','2024-01-20','2024-03-
15',
                                '2024-02-02','2024-02-20','2024-03-05'])})

orders['order_month'] = orders['order_date'].dt.to_period('M')
orders['cohort'] = orders.groupby('customer_id')['order_month'].transform('min')
orders['period'] = (orders['order_month'] - orders['cohort']).apply(lambda x: x.n)

cohort_counts = (orders.groupby(['cohort', 'period'])['customer_id']
                       .nunique().unstack(fill_value=0))
cohort_size = cohort_counts[0]
retention = cohort_counts.divide(cohort_size, axis=0).round(3)
print(retention)

# heatmap
import seaborn as sns; sns.heatmap(retention, annot=True, fmt='.0%', cmap='Blues')
```

Period 0 is always 100 percent. Count distinct customers (nunique) so several orders in one month are not double counted. Later periods for recent cohorts are unobserved, so they appear as 0; mask those cells (periods beyond data end) to avoid misreading them as churn.

A68. Given user event timestamps, assign session IDs where a new session starts after 30 minutes of inactivity, and compute the number of session...

```
import pandas as pd
ev = pd.DataFrame({
  'user': ['u1']*5 + ['u2']*3,
  'ts': pd.to_datetime(['2024-05-01 10:00','2024-05-01 10:10','2024-05-01 10:50',
                        '2024-05-01 10:55','2024-05-02 09:00',
                        '2024-05-01 12:00','2024-05-01 12:20','2024-05-01 14:00'])})

ev = ev.sort_values(['user', 'ts'])
gap = ev.groupby('user')['ts'].diff()
new_session = gap.isna() | (gap > pd.Timedelta(minutes=30))
ev['session_id'] = new_session.groupby(ev['user']).cumsum()

sess = (ev.groupby(['user', 'session_id'])['ts']
          .agg(start='min', end='max', events='size'))
sess['length_min'] = (sess['end'] - sess['start']).dt.total_seconds() / 60

summary = sess.groupby('user').agg(sessions=('events', 'size'),
                                   avg_len_min=('length_min', 'mean'))
print(summary)
```

Logic: diff gives the gap to the previous event for the same user; a flag marks the first event or a long gap; cumulative sum of the flags per user labels each session. Single-event sessions have length 0, which you may want to treat separately. SQL equivalent uses LAG and a running SUM.

A69. Given a table of login dates per user, find each user's longest streak of consecutive days.

```
import pandas as pd
df = pd.DataFrame({
  'user': ['a']*6 + ['b']*3,
  'date': pd.to_datetime(['2024-01-01','2024-01-02','2024-01-03','2024-01-05','2024-01-
06','2024-01-06',
                          '2024-02-01','2024-02-03','2024-02-04'])})

df = df.drop_duplicates(['user', 'date']).sort_values(['user', 'date'])
# consecutive days share the same (date - rank) value
df['rn'] = df.groupby('user').cumcount()
df['grp'] = df['date'] - pd.to_timedelta(df['rn'], unit='D')

streaks = df.groupby(['user', 'grp']).size().rename('streak').reset_index()
longest = streaks.groupby('user')['streak'].max()
print(longest)     # a: 3, b: 2
```

The trick: within a consecutive run the date increases by one day as the row number increases by one, so date minus row number is constant for the run. De-duplicate dates first. This is the classic "gaps and islands" pattern, also solved in SQL with ROW_NUMBER.

A70. Flatten a nested JSON API response (orders with a list of items and a customer object) into a tidy DataFrame.

```
import pandas as pd
data = [
  {'order_id': 1, 'customer': {'id': 10, 'name': 'Asha', 'city': 'Pune'},
   'items': [{'sku': 'A1', 'qty': 2, 'price': 5.0}, {'sku': 'B2', 'qty': 1, 'price': 9.5}]},
  {'order_id': 2, 'customer': {'id': 11, 'name': 'Ravi', 'city': 'Delhi'},
   'items': [{'sku': 'A1', 'qty': 5, 'price': 5.0}]}]

df = pd.json_normalize(
    data,
    record_path='items',                        # one row per item
    meta=['order_id', ['customer', 'id'], ['customer', 'name'], ['customer', 'city']],
    sep='_')
df['line_total'] = df['qty'] * df['price']
print(df)
```

json_normalize flattens nested dicts into columns with a separator, and record_path expands a list of dicts into rows while meta carries parent fields. For irregular JSON, use errors='ignore' for missing meta keys, or explode a list column and then normalize the dict column. Validate row counts: total items in the JSON should equal rows in the result.

A71. Implement RFM (recency, frequency, monetary) scoring for customers from a transactions table and label top customers.

```
import pandas as pd, numpy as np
tx = pd.DataFrame({
  'customer_id': np.random.randint(1, 201, 2000),
  'order_date': pd.to_datetime('2024-01-01') + pd.to_timedelta(np.random.randint(0, 365,
2000), unit='D'),
  'amount': np.random.gamma(2, 30, 2000)})

snapshot = tx['order_date'].max() + pd.Timedelta(days=1)
rfm = tx.groupby('customer_id').agg(
    recency=('order_date', lambda s: (snapshot - s.max()).days),
    frequency=('order_date', 'count'),
    monetary=('amount', 'sum'))

rfm['R'] = pd.qcut(rfm['recency'], 5, labels=[5,4,3,2,1])            # recent is better
rfm['F'] = pd.qcut(rfm['frequency'].rank(method='first'), 5, labels=[1,2,3,4,5])
rfm['M'] = pd.qcut(rfm['monetary'], 5, labels=[1,2,3,4,5])
rfm['score'] = rfm[['R','F','M']].astype(int).sum(axis=1)
rfm['segment'] = pd.cut(rfm['score'], [0, 6, 9, 12, 15], labels=['At
risk','Average','Loyal','Champions'])
print(rfm['segment'].value_counts())
```

Notes: lower recency is better, so labels are reversed. rank(method='first') breaks ties so qcut can create equal-sized bins. Use a fixed snapshot date for reproducibility. Validate segments with business owners and track their sizes over time.

A72. Given a DataFrame of orders, calculate total revenue, average order value, and number of unique customers for each month using named aggrega...

```
import pandas as pd
orders = pd.DataFrame({
  'order_id': [1,2,3,4,5],
  'customer_id': [10,11,10,12,11],
  'order_date': pd.to_datetime(['2024-01-03','2024-01-15','2024-02-01','2024-02-10','2024-02-
25']),
  'amount': [100.0, 50.0, 75.0, 120.0, 30.0]})

summary = (orders
    .groupby(orders['order_date'].dt.to_period('M'))
    .agg(revenue=('amount', 'sum'),
         aov=('amount', 'mean'),
         customers=('customer_id', 'nunique'),
         orders=('order_id', 'count'))
    .reset_index(names='month'))
print(summary)
```

Named aggregation (output_name=(column, function)) gives clean column names and avoids a MultiIndex on columns. AOV is revenue divided by number of orders, which equals the mean of the amount only if each row is one order; if rows are line items, aggregate to order level first.

A73. Company context: You join a retail chain as a data analyst. The sales team sends a 2-million-row CSV export with columns order_id, order_dat...

Step 1, load safely and profile: read with dtype=str (or explicit dtypes) so nothing is silently coerced, then df.info(), df.describe(include='all'), isna().sum(), and value_counts() on store.

Step 2, standardise text: df['store'] = df['store'].str.strip().str.title(); inspect remaining variants with value_counts() and fix typos with a mapping dictionary agreed with the business.

Step 3, parse numbers and dates:

```
df['price'] = pd.to_numeric(df['price'].str.replace(r'[^\d.\-]', '', regex=True),
errors='coerce')
df['qty'] = pd.to_numeric(df['qty'], errors='coerce')
df['order_date'] = pd.to_datetime(df['order_date'], format='mixed', dayfirst=False,
errors='coerce')
```

Count the rows turned to NaN/NaT and review a sample of failures.

Step 4, handle invalid values: negative qty may be returns; check with the business whether to keep them as a separate returns flag rather than delete. Null qty rows cannot be priced; quarantine them in a separate file with a count rather than guess.

Step 5, deduplicate: exact duplicate rows are removed with drop_duplicates(); same order_id with differing values is investigated and resolved by a rule (latest wins).

Step 6, validate: row counts before and after at every step, revenue = qty*price, no future dates, store list matches master list.

Step 7, aggregate: df.groupby(['store', df.order_date.dt.to_period('M')])['revenue'].sum().unstack(). Save a cleaned Parquet and a log of removed rows. Present the report with caveats (rows excluded, assumptions) and make the script re-runnable.

A74. Company context: A telecom operator sees 18 percent annual customer churn. You are given a customer table (tenure, plan, monthly charges, su...

1) Define the target and population: churned = 1 within the observation window; ensure the features are measured before the churn event to avoid leakage (for example, do not use "cancellation_reason").

2) Explore: churn rate overall and by segment: df.groupby('contract')['churned'].mean(); plot churn versus tenure buckets, monthly charges and support calls with seaborn barplot/boxplot. Typical finding: month-to-month contracts, short tenure, high charges and many support calls have higher churn.

3) Clean: handle missing total_charges (blank strings for new customers), encode categoricals, check class imbalance (18 percent is moderate).

4) Model: stratified split, pipeline with ColumnTransformer, start with logistic regression for interpretability, then gradient boosting.

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
Xtr, Xte, ytr, yte = train_test_split(X, y, stratify=y, test_size=0.2, random_state=1)
model.fit(Xtr, ytr)
proba = model.predict_proba(Xte)[:, 1]
```

5) Evaluate with ROC-AUC, PR-AUC, and lift at top decile instead of accuracy.

6) Explain: coefficients or permutation importance/SHAP to answer "why".

7) Act: score all active customers, rank by churn probability times customer value, and give the retention team the top N sized to their call capacity. Propose an A/B test of the retention offer to prove it works, and monitor model drift monthly.

A75. Company context: An e-commerce company tested a new checkout page. Control had 10,000 visitors with 820 purchases; the variant had 10,200 vi...

Step 1, state hypotheses: H0 conversion rates are equal; H1 they differ (two-sided). Pre-register the metric, alpha 0.05 and that the test ran its planned duration, so we are not peeking.

Step 2, compute rates: control 8.20 percent, variant 910/10200 = 8.92 percent, absolute lift 0.72 points, relative lift about 8.8 percent.

Step 3, test:

```
from statsmodels.stats.proportion import proportions_ztest, proportion_confint
count = [910, 820]; nobs = [10200, 10000]
z, p = proportions_ztest(count, nobs)
lo, hi = proportion_confint(910, 10200, method='wilson')
# also a CI for the difference
import numpy as np
p1, p2 = 910/10200, 820/10000
se = np.sqrt(p1*(1-p1)/10200 + p2*(1-p2)/10000)
diff_ci = (p1 - p2 - 1.96*se, p1 - p2 + 1.96*se)
```

The p-value comes out around 0.07, so the result is not significant at 5 percent, and the CI for the difference includes zero (roughly -0.06 to +1.5 points).

Step 4, check validity: sample ratio mismatch (chi-square on visitor split), randomisation, novelty effects, segment consistency (but avoid fishing without correction), and guardrail metrics like revenue per visitor and refund rate.

Step 5, recommend: not statistically conclusive. Either extend the test to the sample size from a power analysis for the minimum detectable effect, or ship if the cost of being wrong is low and the CI supports the business case. Communicate the uncertainty and not just the p-value.

A76. Company context: At a marketplace company, finance says monthly revenue is 1.8 million but your merged analytics table shows 2.6 million. Yo...

This is a join fan-out (row multiplication). orders is one row per order, order_items is many per order, and payments can also be many per order (split payments). Joining orders to both children produces items x payments rows per order, and summing an order-level amount across those rows overcounts.

Diagnose:

```
print(len(orders), len(merged))
merged.groupby('order_id').size().describe()
orders['order_id'].is_unique, payments.groupby('order_id').size().max()
```

Compare sum of orders.total versus sum over merged; spot-check one order with several items and two payments.

Fix: aggregate each child table to the order grain before joining.

```
items_agg = items.groupby('order_id', as_index=False).agg(item_count=('qty','sum'),
items_value=('line_total','sum'))
pay_agg = payments.groupby('order_id', as_index=False).agg(paid=('amount','sum'))
fact = (orders.merge(items_agg, on='order_id', how='left', validate='one_to_one')
              .merge(pay_agg, on='order_id', how='left', validate='one_to_one'))
```

Prevent: always state the grain of each table, use validate= in merge, assert row counts after joins, reconcile totals to finance on every run, and keep measures at the correct grain (never sum an order-level column on a line-level table).

A77. Company context: A logistics company has 40 GB of daily GPS and delivery event logs in CSV. A data analyst's laptop has 16 GB of RAM. You ne...

Do not load everything. Options in order of preference:

1) Convert once to Parquet with typed columns, then read only needed columns; columnar storage makes the later queries fast. DuckDB can do both directly:

```
import duckdb
res = duckdb.sql("""
  SELECT city, EXTRACT(hour FROM delivered_at) AS hr,
         AVG(date_diff('minute', picked_at, delivered_at)) AS avg_min,
         AVG(CASE WHEN delivered_at > promised_at THEN 1 ELSE 0 END) AS late_rate
  FROM read_csv_auto('logs/*.csv')
  GROUP BY 1, 2
""").df()
```

2) Pandas chunking: read_csv with usecols, parse_dates and chunksize, compute partial sums and counts per (city, hour) for each chunk, add them across chunks, then divide. Averages must be built from sums and counts, never an average of chunk averages.

3) Dask or Polars lazy scan_csv with streaming collect for parallel, out-of-core processing.

4) Sampling for exploration only; use the full data for the final numbers.

Also set dtypes (category for city, float32 for numbers), filter bad rows early, validate row counts against the source, and schedule the job to process each day's file incrementally and append results.

A78. Company context: A global survey company collected a "country" field as free text from users: "USA", "U.S.A", "United States", "india", "Bha...

Approach in layers, from cheap and certain to fuzzy:

1) Normalise: lower-case, strip, remove punctuation and double spaces.

```
s = df['country'].str.lower().str.strip().str.replace(r'[^\w\s]', '', regex=True)
```

2) Explicit alias dictionary for common variants (usa, us, united states, america -> United States; uk, great britain, england -> United Kingdom; bharat -> India; deutschland -> Germany). Apply with s.map(alias).fillna(...).

3) Match against an official list (pycountry or the ISO table) exactly, then fuzzy-match the leftovers:

```
from rapidfuzz import process, fuzz
valid = [c.name for c in pycountry.countries]
```

```
def fix(x):
    m = process.extractOne(x, valid, scorer=fuzz.WRatio, score_cutoff=88)
    return m[0] if m else None
```

4) Review: sort the unmatched or low-confidence values by frequency and resolve the top ones manually, adding them to the alias dictionary so the process becomes repeatable.

5) Validate: report the percentage matched, the unmatched count as "Unknown", and spot-check a sample of fuzzy matches (avoid false matches such as "Niger" to "Nigeria").

6) Count results: df['country_std'].value_counts(dropna=False). Prevent the problem at the source with a dropdown in the form.

A79. Company context: A streaming service's daily active users (DAU) dropped 12 percent last Tuesday and the dashboard owner asks you to find out...

1) Verify the drop is real before explaining it: check for data pipeline problems (missing partitions, delayed loads, a change in logging), compare event volume per hour, null rates and distinct user counts, and ask engineering about deployments. Confirm seasonality by comparing with the same weekday over previous weeks.

2) Quantify and localise by decomposing DAU with slices:

```
dau = (events.assign(day=events.event_time.dt.date)
             .groupby(['day','platform'])['user_id'].nunique().unstack())
change = (dau.loc['2024-06-11'] - dau.loc['2024-06-04']) / dau.loc['2024-06-04']
```

Repeat for country, app_version, acquisition channel, new versus returning users. Rank segments by absolute contribution to the drop, not percent alone.

3) Check the funnel: did app opens, logins or playback starts fall first? Use a timestamp-level view to find the exact hour when the metric broke.

4) Form hypotheses: a faulty app release on Android, outage in one region, authentication bug, tracking SDK change, holiday or competitor event, marketing campaign ending.

5) Test them: compare version adoption curves, crash logs, error rates, and reactivation curves in the following days.

6) Report: root cause, magnitude, confidence, whether users were truly lost or only unlogged, and a recommended fix plus monitoring (automated anomaly alert on DAU by segment).

A80. Company context: A grocery chain wants a weekly demand forecast for 500 products to plan inventory. You have 3 years of weekly sales per sto...

1) Set a baseline first: seasonal naive (same week last year) and a moving average. Any fancier model must beat these.

2) Prepare data: complete the calendar (fill weeks with no sales as zero, not missing), correct for stock-outs (zero sales caused by no stock is not zero demand), flag promotions and holidays, aggregate sensibly (store x product may be sparse).

3) Features for a global model (one gradient boosting model across all series): lags (1, 2, 4, 52), rolling means, week of year, price, promotion flag, holidays, store attributes, category.

```
df = df.sort_values(['store','sku','week'])
g = df.groupby(['store','sku'])['units']
df['lag_1'] = g.shift(1); df['lag_52'] = g.shift(52)
df['roll_4'] = g.shift(1).rolling(4).mean().reset_index(level=[0,1], drop=True)
```

Alternatives: statsmodels ETS/SARIMA, Prophet for few series with strong seasonality; LightGBM for many series.

4) Validate with time-based splits only (train on the past, test on the final 12 weeks, or rolling-origin cross-validation); never shuffle.

5) Metrics: MAE, RMSE, WAPE and bias; compare against baseline per product class. Use prediction intervals or quantile forecasts because inventory needs a service level, not just a mean.

6) Deploy: retrain periodically, monitor error drift, and give planners override capability.

A81. Company context: A ride-hailing company notices some trips have a fare of 0.01 or 25,000, distance of 0 km or 4,000 km, and durations of neg...

1) Profile: describe() with percentiles, histograms on log scale, and counts of rule violations.

2) Define business-rule validity first: duration > 0 and < 8 hours, distance between 0.2 and 200 km, fare above the minimum fare, speed = distance / hours below 150 km/h, pickup and drop-off coordinates within the city boundary.

```
df['speed'] = df['distance_km'] / (df['duration_min'] / 60)
valid = (df.duration_min.between(1, 480) & df.distance_km.between(0.2, 200)
         & df.fare.between(df.min_fare, 5000) & (df.speed < 150))
bad = df[~valid]; good = df[valid]
```

3) Investigate the invalid group before deleting: is it concentrated in one app version, city or driver (GPS bug, test accounts, cancelled trips)? Report the share removed (for example 0.8 percent).

4) Statistical outlier handling on the remaining data: compute fare per km and cap or exclude beyond per-city IQR bounds; use the median alongside the mean.

5) Compute: good.groupby('city').apply(lambda g: g.fare.sum() / g.distance_km.sum()), a ratio of sums rather than a mean of ratios, which is less affected by short trips.

6) Share assumptions, thresholds and excluded counts, and send bad-data findings to engineering to fix at the source.

A82. Company context: A bank wants a model to flag fraudulent card transactions. Only 0.2 percent of 1 million transactions are fraud. Your first...

Diagnosis: predicting "not fraud" for everything gets 99.8 percent accuracy, so the metric is meaningless here. Recall on fraud is probably near zero. Switch to precision, recall, F1, and PR-AUC, and think in terms of cost: a missed fraud costs the transaction amount; a false alarm costs analyst time and customer annoyance.

Fix steps:

1) Time-based split (train on earlier months, test on the latest), because fraud patterns evolve and random splits leak.

2) Features: transaction amount relative to the customer's average, velocity (number of transactions in last 1 hour / 24 hours), distance from the previous transaction location, merchant category risk, device or time-of-day anomalies, using only information available at transaction time.

3) Imbalance handling on training data only: class_weight='balanced', or scale_pos_weight in XGBoost, or undersampling the majority class; SMOTE with care.

```
from xgboost import XGBClassifier
clf = XGBClassifier(scale_pos_weight=(y_tr==0).sum()/(y_tr==1).sum(), n_estimators=400,
max_depth=5)
```

4) Evaluate with precision-recall curve; choose the threshold that maximises expected savings: gain = fraud_caught_amount - cost_per_review * alerts.

5) Monitor in production: drift, alert volume, feedback loop from investigator labels, regular retraining.

6) Explain: SHAP values for each flagged transaction so investigators understand the reason.

A83. Company context: A hospital group receives lab results where 30 percent of cholesterol values and 8 percent of blood pressure values are mis...

1) Quantify and visualise: missing percentage per column and per clinic, heatmap or missingno matrix, and test whether missingness relates to other variables (df.groupby('clinic')['chol'].apply(lambda s: s.isna().mean())).

2) Understand the mechanism with clinicians: cholesterol may be missing because the test was not ordered for low-risk patients (MAR or even MNAR), so naive dropping or mean filling biases the study.

3) Strategy by column:

- Blood pressure at 8 percent: impute using model-based imputation (IterativeImputer or multiple imputation by chained equations) with age, BMI, sex, clinic and diagnosis as predictors.

- Cholesterol at 30 percent: add a "was_tested" indicator, impute with the same method, and run a sensitivity analysis (complete-case versus imputed) to see whether conclusions change.

4) Do not impute the outcome variable; drop only rows missing the outcome.

5) Prevent leakage: fit imputers within train folds if building a predictive model.

6) Document everything: methods, percent imputed, and uncertainty. Multiple imputation (several datasets pooled by Rubin's rules) is preferred to single imputation for inference because it reflects the added uncertainty.

A84. Company context: A marketing team runs ads on four channels. Customers often touch several channels before buying, so the same purchase appe...

1) Build a clean touchpoint table: customer_id, channel, touch_time. Normalise IDs across logs (same customer key), union the logs with pd.concat, and drop exact duplicate touches.

2) Connect touches to conversions: for each conversion, keep only touches within a lookback window (for example 30 days) before the purchase time.

```
t = touches.merge(conv[['customer_id','conv_id','conv_time','revenue']], on='customer_id')
t = t[(t.touch_time <= t.conv_time) & (t.touch_time >= t.conv_time - pd.Timedelta(days=30))]
t = t.sort_values(['conv_id','touch_time'])
```

3) Assign credit so each conversion's revenue sums to 100 percent:

```
first = t.groupby('conv_id').head(1).assign(credit=lambda d: d.revenue)
last  = t.groupby('conv_id').tail(1).assign(credit=lambda d: d.revenue)
t['n'] = t.groupby('conv_id')['channel'].transform('size')
linear = t.assign(credit=t.revenue / t['n'])
```

4) Aggregate by channel: linear.groupby('channel')['credit'].sum().

5) Reconcile: total credit per model must equal total revenue (assert np.isclose). That check removes the 1.4x overcount.

6) Explain the trade-offs: last-touch favours bottom-funnel channels, first-touch top-funnel; recommend an incrementality test (holdout) for real causal effect.

A85. Company context: After a merger, two insurance companies must combine customer lists. System A has customer IDs, names and addresses; system...

This is record linkage (entity resolution).

1) Standardise: lower-case, strip punctuation, expand abbreviations (st to street, rd to road), parse addresses, normalise phones and emails, and split names. Create a match key such as postcode plus house number.

2) Blocking: comparing all pairs is O(n^2). Only compare records that share a block key (same postcode, or same first letters of surname plus birth year).

3) Score candidate pairs with several fields using fuzzy similarity:

```
from rapidfuzz import fuzz
def score(a, b):
    return (0.4*fuzz.token_sort_ratio(a.name, b.name)
          + 0.4*fuzz.token_sort_ratio(a.addr, b.addr)
          + 0.2*(100 if a.dob == b.dob else 0))
```

Libraries such as recordlinkage or splink implement this with probabilistic (Fellegi-Sunter) models.

4) Decide thresholds: above 90 auto-match, 75 to 90 manual review, below 75 non-match. Evaluate on a labelled sample (precision and recall of matches).

5) Cluster matches into groups (connected components with networkx) so transitive matches share one master ID; keep a golden record rule (most recent, most complete).

6) Report the unique customer count with a range reflecting uncertainty, and keep a mapping table from old IDs to the master ID for traceability. Be careful with false merges (family members at the same address).

A86. Company context: Your team's weekly sales report script, written with pandas, now takes 40 minutes and uses iterrows and apply on 5 million ...

1) Profile before changing anything: time each step (time.perf_counter, %prun, line_profiler) to find the real bottleneck; usually one or two steps dominate.

2) Replace row loops with vectorized code:

```
# slow
df['tier'] = df.apply(lambda r: 'big' if r.amount > 1000 else 'small', axis=1)
# fast
df['tier'] = np.where(df['amount'] > 1000, 'big', 'small')
```

Use np.select for multiple conditions, str methods for text, map for lookups, and groupby/transform in place of looping over groups.

3) Reduce data: read only needed columns, filter early, use compact dtypes (category, int32/float32), and parse dates with an explicit format.

4) Replace repeated merges inside loops with one merge on a dictionary or index; sort and set the index when joining many times.

5) Use faster storage: Parquet instead of CSV.

6) Avoid repeated concatenation in a loop (build a list, concat once) and avoid chained copies.

7) If still slow, switch the heavy step to Polars, DuckDB or numba, or parallelise by partition with joblib.

8) Test equivalence: compare the new output to the old using pd.testing.assert_frame_equal, then add a timing check to the job.

A87. Company context: A SaaS company stores event timestamps from users in India, the US and Germany. Some are in UTC, others in local time witho...

1) Find out how each source records time: ask the owner or infer from patterns (activity peaks) and check the schema. Create a column source_tz for each source.

2) Localise naive timestamps using that source's zone, convert everything to UTC for storage:

```
import pandas as pd
def to_utc(df, tz):
    t = pd.to_datetime(df['event_time'])
    if t.dt.tz is None:
        t = t.dt.tz_localize(tz, ambiguous='NaT', nonexistent='shift_forward')
    return t.dt.tz_convert('UTC')
```

Daylight saving transitions create ambiguous (clock repeats) and non-existent (clock skips) local times, so handle those arguments explicitly and count affected rows.

3) Define the business day: finance may report DAU by the company's reporting timezone, so convert UTC to that zone before extracting the date: df['event_time_utc'].dt.tz_convert('America/New_York').dt.date.

4) Recompute DAU with nunique per date and compare with finance day by day; the differences should shrink to near zero and should be explainable by the definition of an "active" user.

5) Document the convention (store UTC, report in X) and add validation tests, such as no timestamps in the future and no timezone-naive columns.

A88. Company context: A retail store manager reports that products keep running out of stock. You have daily inventory and sales data for 2,000 S...

1) Define a stock-out: end-of-day inventory equals zero (or sales zero while on-hand is zero) on a day when the product would normally sell.

```
inv['stockout'] = inv['on_hand'] == 0
so = inv.groupby('sku').agg(days=('date','nunique'), so_days=('stockout','sum'))
so['so_rate'] = so['so_days'] / so['days']
```

2) Rank the SKUs by stock-out rate and by revenue impact (weighted by margin); a Pareto chart shows that a small share of SKUs creates most of the loss.

3) Estimate demand: for each SKU use the average sales on in-stock days with the same weekday (and adjust for promotions) as expected demand.

```
base =
(inv[~inv.stockout].groupby(['sku','weekday'])['units_sold'].mean().rename('exp_units'))
lost = inv[inv.stockout].merge(base.reset_index(), on=['sku','weekday'])
lost['lost_rev'] = lost['exp_units'] * lost['price']
```

4) Look for root causes: supplier lead times, reorder points, store, seasonality, delivery delays.

5) Recommend: raise safety stock for the top-loss SKUs, review reorder points, and track service level weekly. State the assumptions behind the lost-sales estimate (no substitution of other products).

A89. Company context: A credit-card company wants to segment 100,000 customers into groups for tailored offers. Features: age, income, monthly sp...

1) Prepare features: choose behavioural variables, handle missing values, log-transform skewed variables (spend, income), cap extreme outliers, and scale (StandardScaler), because k-means is distance based. Remove highly correlated duplicates.

2) Choose k with several signals:

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
for k in range(2, 10):
    km = KMeans(n_clusters=k, n_init=10, random_state=0).fit(Xs)
    print(k, km.inertia_, silhouette_score(Xs, km.labels_, sample_size=10000,
random_state=0))
```

Look for the elbow in inertia and a high silhouette, but also business interpretability (3 to 6 segments).

3) Fit the final model and attach labels to the original data.

4) Profile: df.groupby('cluster')[features].mean() and sizes, then name each segment ("high spenders, high utilization", "dormant", "young low-income").

5) Check stability by re-running with other seeds or on a sample, and consider alternatives (Gaussian mixture, hierarchical, DBSCAN) if clusters are not spherical. Visualise with PCA to 2D.

6) Validate usefulness: do segments differ in response to offers (A/B test), can marketing act on them, and are they stable over time? Clusters have no ground truth, so business value is the final test.

A90. Company context: A food-delivery company's finance team needs to reconcile the orders database (what customers were charged) against the pay...

1) Align keys and formats: strip whitespace and standardise the transaction reference in both tables; convert amounts to Decimal or integer cents to avoid floating-point mismatches; ensure the same currency and timezone for dates.

2) Full outer join with an indicator:

```
rec = orders.merge(settle, on='txn_ref', how='outer', suffixes=('_ord', '_set'),
indicator=True, validate='one_to_one')
```

```
only_orders = rec[rec['_merge'] == 'left_only']       # charged but not settled
only_settle = rec[rec['_merge'] == 'right_only']      # settled but unknown to orders
both = rec[rec['_merge'] == 'both']
mismatch = both[both['amount_cents_ord'] != both['amount_cents_set']]
```

validate='one_to_one' will expose duplicate references, which are a typical cause (double posting). Handle duplicates separately with groupby counts.

3) Explain the gaps by category: fees deducted by the processor, refunds and chargebacks, timing differences at period boundaries (settled next day), partial captures, failed payments.

4) Summarise the reconciliation: matched count and amount, each exception bucket with count and total value so the buckets add up to the 0.7 percent difference.

5) Deliver an exceptions file for finance to investigate, and automate the check daily with alerts when unmatched value exceeds a tolerance.

SQL

A91. What is the difference between WHERE and HAVING?

WHERE filters individual rows before any grouping or aggregation happens. HAVING filters groups after GROUP BY and aggregation, so it can reference aggregate functions such as COUNT, SUM or AVG, which WHERE cannot.

Logical order of evaluation: FROM/JOIN, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, LIMIT.

```
SELECT department, AVG(salary) AS avg_sal
FROM employees
WHERE status = 'active'          -- row filter, runs first
GROUP BY department
HAVING AVG(salary) > 70000;      -- group filter, runs after aggregation
```

Performance tip: put every condition that does not need an aggregate in WHERE, so fewer rows reach the grouping step and indexes can be used.

A92. Explain INNER, LEFT, RIGHT, FULL OUTER and CROSS joins.

INNER JOIN returns only rows with a match in both tables.

LEFT (OUTER) JOIN returns all rows from the left table and matching rows from the right; unmatched right columns are NULL.

RIGHT JOIN is the mirror image: all rows of the right table, matched rows of the left.

FULL OUTER JOIN returns all rows from both sides, with NULLs where there is no match (not supported in MySQL; emulate with LEFT JOIN UNION RIGHT JOIN).

CROSS JOIN returns the Cartesian product: every row of A paired with every row of B (n x m rows). It has no ON clause and is useful for generating combinations, for example a calendar crossed with a product list.

```
SELECT c.name, o.order_id
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id;  -- customers with no orders still
appear
```

A common anti-join pattern uses LEFT JOIN ... WHERE right.key IS NULL to find rows with no match.

A93. What is the difference between UNION and UNION ALL?

Both stack the result sets of two queries vertically; the queries must return the same number of columns with compatible types. UNION removes duplicate rows (it implies a sort or hash distinct), so it is slower. UNION ALL keeps all rows including duplicates and is faster because there is no deduplication step.

Use UNION ALL by default when you know the sets cannot overlap or duplicates are fine, and UNION only when you really need distinct rows. INTERSECT and EXCEPT (MINUS in Oracle) are the related set operators for common rows and rows only in the first set.

A94. What is the difference between COUNT(*), COUNT(column) and COUNT(DISTINCT column)?

COUNT(*) counts all rows in the group, including rows with NULLs. COUNT(column) counts only rows where that column is not NULL. COUNT(DISTINCT column) counts the number of distinct non-NULL values.

Example with values (1, 1, NULL): COUNT(*) = 3, COUNT(col) = 2, COUNT(DISTINCT col) = 1.

Also note COUNT(1) is equivalent to COUNT(*) in practically all engines. All aggregates except COUNT(*) ignore NULLs, which affects AVG: AVG(col) divides by the non-NULL count, not the row count.

A95. How does SQL treat NULL values? What pitfalls should an analyst know?

NULL means unknown or missing, not zero or empty string. Comparisons with NULL (=, <>, <, >) yield UNKNOWN, which WHERE treats as not true, so the row is filtered out. You must use IS NULL / IS NOT NULL.

Pitfalls:

1. WHERE col <> 'X' silently drops rows where col is NULL. Use col IS DISTINCT FROM 'X' (Postgres) or add OR col IS NULL.

2. NOT IN (subquery) returns no rows if the subquery contains any NULL. Use NOT EXISTS instead.

3. Aggregates (SUM, AVG, MIN, MAX, COUNT(col)) ignore NULLs; a SUM over all NULLs is NULL, not 0. Wrap with COALESCE.

4. NULL + 5 = NULL, and concatenation with NULL gives NULL in most dialects.

5. GROUP BY and DISTINCT treat NULLs as equal to each other (one group), and ORDER BY places NULLs first or last depending on the engine (NULLS FIRST / NULLS LAST controls it).

6. Joins never match on NULL = NULL.

Useful functions: COALESCE(a, b, ...), NULLIF(a, b) (returns NULL if a = b, handy to avoid division by zero).

A96. Explain the logical order of execution of a SQL query and why it matters.

Written order is SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT, but logical processing order is:

1. FROM and JOIN (build the row source)

2. WHERE (filter rows)

3. GROUP BY (form groups)

4. HAVING (filter groups)

5. Window functions (computed over the result so far)

6. SELECT (evaluate expressions, assign aliases)

7. DISTINCT

8. ORDER BY

9. LIMIT/OFFSET

Consequences: you cannot use a SELECT alias in WHERE or HAVING (in standard SQL), but you can in ORDER BY. You cannot filter on a window function in WHERE; wrap the query in a CTE/subquery, or use QUALIFY where supported (Snowflake, BigQuery, Teradata). Filters in WHERE on the right table of a LEFT JOIN turn it into an inner join; put such conditions in the ON clause instead.

A97. What are window functions and how do they differ from GROUP BY?

A window function computes a value for each row using a set of related rows (the window) defined by OVER (PARTITION BY ... ORDER BY ... frame). Unlike GROUP BY, it does not collapse rows: every input row remains in the output and gets an extra computed column.

Categories:

- Ranking: ROW_NUMBER, RANK, DENSE_RANK, NTILE, PERCENT_RANK

- Value/offset: LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE

- Aggregate as window: SUM, AVG, COUNT, MIN, MAX OVER (...)

```
SELECT emp_id, dept, salary,
       AVG(salary) OVER (PARTITION BY dept) AS dept_avg,
       salary - AVG(salary) OVER (PARTITION BY dept) AS diff_from_avg
FROM employees;
```

With GROUP BY you would get one row per dept and lose the employee detail. Window functions run after WHERE/GROUP BY/HAVING, so they can only appear in SELECT and ORDER BY.

A98. Differentiate ROW_NUMBER, RANK and DENSE_RANK.

All three number rows within a partition according to ORDER BY. They differ on ties.

- ROW_NUMBER gives unique sequential numbers (1,2,3,4); ties are broken arbitrarily unless the ORDER BY is deterministic.

- RANK gives tied rows the same rank and skips the following numbers (1,2,2,4).

- DENSE_RANK gives tied rows the same rank and does not skip (1,2,2,3).

For salaries 100, 90, 90, 80: ROW_NUMBER = 1,2,3,4; RANK = 1,2,2,4; DENSE_RANK = 1,2,2,3.

Use DENSE_RANK for "Nth highest value" questions, ROW_NUMBER for deduplication or picking exactly one row per group, and RANK when gaps after ties are meaningful (sports style ranking). NTILE(n) splits ordered rows into n roughly equal buckets.

A99. Explain the window frame clause: ROWS vs RANGE, and the default frame.

The frame defines which rows relative to the current row are included in an aggregate window function: ROWS|RANGE BETWEEN <start> AND <end>, where bounds are UNBOUNDED PRECEDING, n PRECEDING, CURRENT ROW, n FOLLOWING, UNBOUNDED FOLLOWING.

ROWS counts physical rows. RANGE works on values of the ORDER BY column: all peers (rows with equal ORDER BY value) of the current row are included.

Default: if ORDER BY is present and no frame is given, the default is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. This is why a running SUM with duplicate order values gives the same total for tied rows, and why LAST_VALUE with the default frame returns the current row's value rather than the partition's last. Without ORDER BY, the frame is the whole partition.

Best practice: state the frame explicitly with ROWS for running totals and moving averages:

```
SUM(amount) OVER (ORDER BY order_date, order_id
                  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
AVG(amount) OVER (ORDER BY order_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)  -- 7-row
moving avg
```

A100. What is a CTE? How does it differ from a subquery, a temp table and a view?

A Common Table Expression is a named temporary result set defined with WITH name AS (query) that exists only for the duration of one statement. It improves readability, lets you reference the same intermediate result several times, and supports recursion.

Versus subquery: a CTE is named, can be reused in the same statement and reads top-down; a subquery is inline. Performance is generally the same, though older Postgres (before 12) always materialized CTEs as an optimization fence; newer versions inline them unless MATERIALIZED is specified.

Versus temp table: a temp table is physically stored for the session, can be indexed and analyzed, and persists across statements; a CTE cannot be indexed and vanishes after the statement. Use temp tables for large intermediate results reused many times.

Versus view: a view is a stored, named query persisted in the database catalog and reusable across queries and users; a CTE is local to one query.

```
WITH monthly AS (
  SELECT DATE_TRUNC('month', order_date) AS m, SUM(amount) AS rev
  FROM orders GROUP BY 1
)
SELECT m, rev, rev - LAG(rev) OVER (ORDER BY m) AS change FROM monthly;
```

A101. What is a recursive CTE and when would you use it?

A recursive CTE has an anchor member (the base rows) UNION ALL a recursive member that references the CTE itself. The engine repeatedly runs the recursive member against the rows produced in the previous iteration until no new rows are returned.

Typical uses: traversing hierarchies (org charts, category trees, bill of materials), generating number or date series, and graph path finding.

```
WITH RECURSIVE org AS (
  SELECT emp_id, manager_id, name, 1 AS lvl
  FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.emp_id, e.manager_id, e.name, o.lvl + 1
  FROM employees e JOIN org o ON e.manager_id = o.emp_id
)
SELECT * FROM org ORDER BY lvl;
```

Cautions: cycles in the data cause infinite loops, so add a depth limit (WHERE lvl < 20) or track a path array; some engines limit recursion depth (SQL Server MAXRECURSION default 100). SQL Server and Oracle omit the RECURSIVE keyword.

A102. What is the difference between a correlated and a non-correlated subquery?

A non-correlated subquery is independent of the outer query; it can run once and its result is reused. A correlated subquery references columns from the outer query, so conceptually it is re-evaluated for each outer row.

```
-- non-correlated
SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);

-- correlated: employees earning more than their own department average
SELECT * FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id);
```

Correlated subqueries can be slow on large tables, although optimizers often decorrelate them into joins. A window function (AVG(salary) OVER (PARTITION BY dept_id)) or a join to a pre-aggregated CTE is usually faster and clearer. EXISTS/NOT EXISTS are the classic correlated forms.

A103. IN vs EXISTS vs JOIN: when do you use each, and how do they handle NULLs?

IN compares a value against a list or subquery result. EXISTS tests whether a correlated subquery returns at least one row and short-circuits on the first match. JOIN combines columns and can multiply rows if the match is one-to-many.

Semi-join intent ("customers who have at least one order"): EXISTS or IN avoid row duplication; a JOIN would need DISTINCT. Modern optimizers often plan IN and EXISTS the same way, but EXISTS tends to be safer on large subquery results.

NULL behavior: x NOT IN (subquery) yields no rows if the subquery returns any NULL, because x <> NULL is UNKNOWN. NOT EXISTS is not affected. So for anti-joins prefer NOT EXISTS or LEFT JOIN ... IS NULL.

```
SELECT c.* FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

A104. What are primary keys, foreign keys, unique constraints and candidate keys?

Primary key: a column or set of columns that uniquely identifies each row; it cannot be NULL and there is only one per table. A candidate key is any minimal set of columns that could serve as the primary key; the one chosen is the primary key and the others are alternate keys.

Unique constraint: guarantees unique values but permits NULLs (how many NULLs depends on the engine), and a table can have many.

Foreign key: a column that references the primary (or unique) key of another table, enforcing referential integrity. Options like ON DELETE CASCADE / SET NULL / RESTRICT define behavior when the parent row is deleted.

Surrogate key: an artificial key (auto-increment or UUID) with no business meaning; natural key: a real-world attribute such as email or ISBN. Composite key: a key made of multiple columns.

Primary and unique constraints are backed by indexes, which is why lookups by key are fast.

A105. Explain database normalization and the first three normal forms.

Normalization organizes tables to reduce redundancy and avoid update, insert and delete anomalies.

1NF: each column holds atomic (indivisible) values, no repeating groups or arrays, and rows are identifiable by a key. Example: do not store 'red,blue,green' in one cell.

2NF: 1NF plus no partial dependency: every non-key attribute depends on the whole of a composite key. Example: in order_items(order_id, product_id, product_name, qty), product_name depends only on product_id, so move it to a products table.

3NF: 2NF plus no transitive dependency: non-key attributes depend only on the key, not on other non-key attributes. Example: employees(emp_id, dept_id, dept_name) violates 3NF since dept_name depends on dept_id; move it to departments.

BCNF is a stricter 3NF: every determinant must be a candidate key.

Trade-off: more normalization means more joins. Transactional (OLTP) systems favor 3NF; analytical (OLAP) warehouses often denormalize into star schemas for read speed.

A106. What is denormalization and when is it appropriate? Describe star vs snowflake schema.

Denormalization deliberately introduces redundancy (pre-joined or duplicated columns, summary tables) to speed reads and simplify queries at the cost of storage and more complex writes and consistency maintenance. It suits read-heavy analytics, dashboards and data warehouses.

Star schema: a central fact table (measures such as sales_amount, quantity plus foreign keys) surrounded by denormalized dimension tables (date, product, customer, store). Few joins, simple queries, fast aggregation.

Snowflake schema: dimensions are further normalized into sub-dimensions (product to category to department). Saves storage and reduces redundancy but needs more joins.

Fact table grain (one row per what?) must be defined first. Slowly changing dimensions (Type 1 overwrite, Type 2 add new row with validity dates) handle attribute changes over time.

A107. What is an index? How does a B-tree index work and what are the trade-offs?

An index is a separate data structure that lets the database find rows without scanning the whole table. The most common is the B-tree (balanced tree): sorted keys in a tree with leaf nodes holding row locations (or the row itself in a clustered index). Lookups, range scans and ORDER BY on the indexed columns cost O(log n).

Benefits: faster equality and range filters, joins, sorting, and uniqueness enforcement.

Costs: extra storage; every INSERT, UPDATE of indexed columns and DELETE must also maintain the index, slowing writes; too many indexes hurt.

When indexes are not used: functions on the column (WHERE LOWER(email) = ...) unless a functional index exists; leading wildcard LIKE '%abc'; implicit type conversion; low selectivity columns (e.g. boolean); optimizer judging a full scan cheaper because a large fraction of rows is returned.

Other types: hash (equality only), bitmap (low cardinality, warehouses), GiST/GIN (full text, JSON, arrays), columnstore.

A108. Clustered vs non-clustered index, composite index, and covering index: explain.

Clustered index: determines the physical order of the table rows; the leaf level is the data itself. A table can have only one (in SQL Server, InnoDB's primary key is clustered). Non-clustered (secondary) index: a separate structure whose leaves point to the row (or to the clustered key); a table can have many.

Composite index: an index on multiple columns, e.g. (customer_id, order_date). It follows the leftmost-prefix rule: it can serve filters on customer_id, or customer_id and order_date, but not efficiently on order_date alone. Column order should put equality-filtered and more selective columns first, then range columns.

Covering index: contains all columns a query needs (via key columns or INCLUDE columns), so the engine answers from the index alone (index-only scan) without touching the table.

```
CREATE INDEX ix_orders_cust_date ON orders (customer_id, order_date) INCLUDE (amount);
```

A109. How would you read and use an execution plan (EXPLAIN) to optimize a slow query?

EXPLAIN shows the plan the optimizer chose; EXPLAIN ANALYZE (Postgres, MySQL 8) also executes the query and shows actual times and row counts.

What to look for:

1. Scan type: sequential/full table scan on a large table with a selective filter suggests a missing index; index seek/scan is good.

2. Row estimates vs actuals: big differences mean stale statistics (run ANALYZE / update statistics).

3. Join algorithm: nested loop (good for small outer set with indexed inner), hash join (large unsorted sets), merge join (pre-sorted inputs). A nested loop over millions of rows is a red flag.

4. Expensive sorts, spills to disk, or large intermediate row counts.

5. The costliest node in the tree.

Typical fixes: add or adjust indexes, rewrite non-sargable predicates, select only needed columns, filter early, replace correlated subqueries, avoid SELECT DISTINCT used to mask join duplication, partition large tables, pre-aggregate, and refresh statistics.

A110. What does 'sargable' mean? Give examples of non-sargable predicates and their fixes.

SARGable (Search ARGument able) means a predicate can use an index seek. Wrapping an indexed column in a function or expression usually prevents it.

Non-sargable -> sargable:

```
WHERE YEAR(order_date) = 2024
-> WHERE order_date >= '2024-01-01' AND order_date < '2025-01-01'

WHERE order_date + 7 > CURRENT_DATE
-> WHERE order_date > CURRENT_DATE - 7

WHERE LOWER(email) = 'a@b.com'
-> store email normalized, or create a functional index on LOWER(email)

WHERE name LIKE '%son'     -- leading wildcard
-> LIKE 'son%' or use full-text / trigram index
```

```
WHERE col_varchar = 123    -- implicit cast on the column
-> WHERE col_varchar = '123'
```

Use half-open date ranges (>= start AND < next_start) rather than BETWEEN for timestamps to avoid missing or double counting boundary values.

A111. Explain ACID properties and transaction isolation levels.

ACID: Atomicity (a transaction is all-or-nothing), Consistency (it moves the database from one valid state to another, honoring constraints), Isolation (concurrent transactions do not see each other's partial work), Durability (committed data survives crashes, via write-ahead logging).

Isolation levels and the anomalies they allow:

- READ UNCOMMITTED: dirty reads possible.

- READ COMMITTED: no dirty reads; non-repeatable reads and phantoms possible (default in Postgres, SQL Server, Oracle).

- REPEATABLE READ: no dirty or non-repeatable reads; phantoms possible in the standard (MySQL InnoDB default; Postgres snapshot isolation prevents phantoms too).

- SERIALIZABLE: transactions behave as if executed one after another; safest but most blocking or most retries.

Dirty read = reading uncommitted data. Non-repeatable read = same row read twice gives different values. Phantom read = repeated query returns different set of rows. Higher isolation costs concurrency.

A112. What is the difference between DELETE, TRUNCATE and DROP?

DELETE is DML: removes rows (optionally with WHERE), logs each row, fires row triggers, can be rolled back, and does not reset identity counters.

TRUNCATE is DDL (in most engines): removes all rows quickly by deallocating pages, minimal logging, no WHERE, usually resets identity, does not fire row triggers. It can be rolled back in Postgres and SQL Server inside a transaction but implicitly commits in MySQL and Oracle.

DROP removes the whole table object (structure, data, indexes, constraints) from the database.

Use DELETE with WHERE for selective removal, TRUNCATE to empty a staging table, DROP to remove the table permanently.

A113. Categorize SQL commands: DDL, DML, DQL, DCL, TCL.

DDL (Data Definition): CREATE, ALTER, DROP, TRUNCATE, RENAME; they define structure.

DML (Data Manipulation): INSERT, UPDATE, DELETE, MERGE; they change data.

DQL (Data Query): SELECT.

DCL (Data Control): GRANT, REVOKE; they manage permissions.

TCL (Transaction Control): COMMIT, ROLLBACK, SAVEPOINT, SET TRANSACTION.

Some people include SELECT under DML. Many engines (MySQL, Oracle) auto-commit DDL, while Postgres and SQL Server allow transactional DDL.

A114. What are the differences between a view, a materialized view and a table?

A table stores data physically. A view is a saved SELECT that stores no data; each query on it runs the underlying query, so results are always current, and it is useful for abstraction, security (exposing only some columns) and reuse. A materialized view stores the query result physically and must be refreshed (manually, on schedule, or incrementally depending on the engine). It is much faster to read for expensive aggregations but can be stale.

Choose a view for freshness and simplicity, a materialized view for heavy repeated aggregations such as dashboards, and an ordinary table (ETL-loaded) when you need indexes, partitioning and full control. Simple views can sometimes be updated through; complex ones (with joins, aggregates, DISTINCT) generally cannot.

A115. Compare stored procedures, functions and triggers.

Stored procedure: a saved routine invoked with CALL/EXEC; can take input/output parameters, perform DML, manage transactions, and returns zero or more result sets. Not usable inside a SELECT.

Function (UDF): returns a value (scalar) or a table, and can be used inside queries (SELECT, WHERE). Usually restricted from making persistent data changes and cannot manage transactions in many engines. Scalar UDFs applied per row can hurt performance.

Trigger: code that fires automatically on INSERT/UPDATE/DELETE (BEFORE/AFTER/INSTEAD OF), used for auditing, enforcing complex rules or maintaining derived data. Hidden side effects make them hard to debug and can slow writes, so use sparingly.

For analysts the key point: pushing logic into set-based SQL is generally faster than row-by-row procedural logic or cursors.

A116. What is the difference between a self join and other joins, and when is a self join used?

A self join is not a separate join type: it is any join (usually INNER or LEFT) of a table to itself, using two different aliases. It is used for hierarchical data (employee to manager), comparing rows within the same table (pairs, consecutive events, rows with a larger value), and finding duplicates.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

Use LEFT JOIN so top-level employees with NULL manager are kept. For consecutive-row comparison, window functions (LAG/LEAD) are usually simpler and faster than self joins.

A117. What is the difference between DISTINCT and GROUP BY? When do they give the same result?

For simply deduplicating selected columns, SELECT DISTINCT a, b and SELECT a, b ... GROUP BY a, b return the same rows, and optimizers often generate the same plan. GROUP BY is needed when you want aggregates (COUNT, SUM) per group or want to filter groups with HAVING. DISTINCT applies to the entire select list, not to a single column, so SELECT DISTINCT a, b is not distinct on a alone.

DISTINCT is also used inside aggregates: COUNT(DISTINCT user_id). Be careful using DISTINCT to hide duplicates caused by a bad join; fix the join grain instead, since it masks logic errors and costs a sort or hash.

A118. What is the fan-out (join explosion) problem and how do you prevent wrong aggregates because of it?

Fan-out occurs when you join a table to another on a one-to-many (or many-to-many) relationship and then aggregate a measure from the 'one' side: each row is repeated once per matching child row, inflating SUM and COUNT.

Example: orders joined to order_items, then SUM(orders.shipping_fee) counts the fee once per item.

Prevention:

1. Know the grain of each table before joining.

2. Aggregate each many-side table to the join key's grain first (in a CTE), then join the pre-aggregated results one-to-one.

3. Use COUNT(DISTINCT key) for counts, though DISTINCT does not fix SUMs.

4. Validate: compare row counts and totals before and after each join.

5. Avoid joining two independent one-to-many tables together (items and payments) before aggregating; aggregate each separately and join on order_id.

```
WITH i AS (SELECT order_id, SUM(qty*price) item_total FROM order_items GROUP BY order_id),
     p AS (SELECT order_id, SUM(paid) paid_total FROM payments GROUP BY order_id)
SELECT o.order_id, i.item_total, p.paid_total
FROM orders o LEFT JOIN i USING (order_id) LEFT JOIN p USING (order_id);
```

A119. Explain the CASE expression and conditional aggregation.

CASE evaluates conditions in order and returns the first matching result; ELSE defaults to NULL if omitted. It has a simple form (CASE col WHEN 1 THEN ...) and a searched form (CASE WHEN cond THEN ...).

Conditional aggregation puts CASE inside an aggregate to compute several filtered metrics in one pass, which is also the standard way to pivot rows into columns.

```
SELECT customer_id,
       COUNT(*) AS orders,
       SUM(CASE WHEN status = 'returned' THEN 1 ELSE 0 END) AS returns,
       AVG(CASE WHEN status = 'completed' THEN amount END) AS avg_completed
FROM orders GROUP BY customer_id;
```

Remember that AVG ignores the NULLs produced by a CASE without ELSE, which is often desirable. Some engines also offer COUNT(*) FILTER (WHERE ...) (Postgres, SQLite).

A120. How can you pivot and unpivot data in SQL?

Pivot turns row values into columns. The portable method is conditional aggregation:

```
SELECT product,
       SUM(CASE WHEN month = 'Jan' THEN sales END) AS jan,
       SUM(CASE WHEN month = 'Feb' THEN sales END) AS feb
FROM monthly_sales GROUP BY product;
```

SQL Server and Oracle have a PIVOT operator; Snowflake and BigQuery have PIVOT too; Postgres has crosstab() in the tablefunc extension. Pivot columns must be known at query time; fully dynamic pivots require dynamic SQL.

Unpivot turns columns into rows. Portable method: UNION ALL of one SELECT per column; or CROSS JOIN LATERAL / VALUES (Postgres), UNPIVOT (SQL Server, Oracle, Snowflake), UNNEST (BigQuery).

```
SELECT product, 'Jan' AS month, jan AS sales FROM wide
UNION ALL
SELECT product, 'Feb', feb FROM wide;
```

A121. What is the difference between LAG/LEAD and FIRST_VALUE/LAST_VALUE? What is the LAST_VALUE trap?

LAG(col, n, default) reads the value from n rows before the current row in the partition ordering; LEAD reads n rows after. FIRST_VALUE and LAST_VALUE return the value at the first or last row of the window frame; NTH_VALUE returns the nth.

The trap: with ORDER BY and no explicit frame, the default frame ends at CURRENT ROW (and includes peers), so LAST_VALUE returns the current row's value, not the last in the partition. Fix by specifying the frame:

```
LAST_VALUE(price) OVER (PARTITION BY product ORDER BY dt
                        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
```

LAG and LEAD ignore the frame; they depend only on ordering. The first row's LAG is NULL unless a default is given.

A122. What are the main ways to remove duplicate rows from a table, and how do you decide which to keep?

First define what 'duplicate' means (which columns) and which row survives (latest, earliest, lowest id).

Methods:

1. ROW_NUMBER: number rows per duplicate key ordered by the survivor rule, delete or filter rn > 1.
2. Self join or correlated delete on id: delete rows where a smaller id exists with the same key.
3. GROUP BY with MIN(id): keep MIN(id) per key, delete others.
4. Create a new table with SELECT DISTINCT or DISTINCT ON (Postgres) and swap.
5. Prevent in future with a UNIQUE constraint or upsert (INSERT ... ON CONFLICT / MERGE).

```
DELETE FROM users WHERE id IN (
  SELECT id FROM (
    SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY created_at DESC, id) rn
    FROM users) t
  WHERE rn > 1);
```

Always preview with a SELECT, back up first, and run inside a transaction.

A123. How do WHERE and ON conditions differ in LEFT JOIN vs INNER JOIN?

For an INNER JOIN, a condition in ON or in WHERE gives the same result. For an outer join they differ.

In a LEFT JOIN, ON conditions decide which right-table rows match; non-matching left rows are still preserved with NULLs. A WHERE condition on the right table's columns is applied after the join and removes the NULL-extended rows, effectively converting the LEFT JOIN into an INNER JOIN.

```
-- keeps all customers; counts only 2024 orders
SELECT c.id, COUNT(o.id)
FROM customers c
LEFT JOIN orders o ON o.cust_id = c.id AND o.order_date >= '2024-01-01'
GROUP BY c.id;

-- drops customers without 2024 orders (acts as inner join)
... LEFT JOIN orders o ON o.cust_id = c.id WHERE o.order_date >= '2024-01-01'
```

Use WHERE o.id IS NULL only for anti-joins.

A124. Explain partitioning vs sharding vs indexing and when each helps.

Indexing: an auxiliary structure to speed lookups on a table within one database.

Partitioning: splitting a large table into smaller physical pieces (by range such as date, list, or hash) inside one database. Queries filtering on the partition key benefit from partition pruning, maintenance is easier (drop an old partition instead of deleting millions of rows), and loads can be parallelized. The filter must use the partition key for pruning.

Sharding: horizontally distributing data across multiple database servers (for example by customer_id hash) to scale storage and throughput beyond one machine. It adds complexity: cross-shard joins and transactions, rebalancing, routing.

Rule of thumb: first tune queries and indexes, then partition big tables (time series logs, events), then shard only when one node cannot handle the load. In warehouses such as BigQuery or Snowflake, partitioning/clustering keys serve the same purpose of reducing scanned data.

A125. Describe the difference between OLTP and OLAP databases and between row-store and column-store.

OLTP (online transaction processing): many small, concurrent reads and writes (orders, payments); highly normalized, row-oriented, strong ACID, indexes for point lookups. Examples: PostgreSQL, MySQL, SQL Server as application backends.

OLAP (online analytical processing): fewer but heavy read queries scanning many rows and aggregating a few columns; denormalized star schemas, often columnar, batch-loaded. Examples: Snowflake, BigQuery, Redshift, ClickHouse.

Row stores keep all columns of a row together (efficient for fetching whole records). Column stores keep each column together, which compresses well and lets a query that touches 3 of 100 columns read only those 3, making aggregations much faster. Analysts should avoid SELECT * in column stores because cost is proportional to columns scanned.

A126. What is a LATERAL join (or CROSS APPLY) and when is it useful?

A LATERAL join (Postgres, MySQL 8, Snowflake; CROSS/OUTER APPLY in SQL Server) lets the subquery on the right reference columns from tables to its left, evaluated per left row. It is like a correlated subquery in the FROM clause that can return multiple rows and columns.

Typical uses: top-N rows per group, table-valued functions per row, unnesting arrays or JSON.

```
-- latest 3 orders for each customer
SELECT c.customer_id, o.order_id, o.order_date
FROM customers c
CROSS JOIN LATERAL (
  SELECT order_id, order_date FROM orders
  WHERE customer_id = c.customer_id
  ORDER BY order_date DESC LIMIT 3) o;
```

With an index on (customer_id, order_date) this is very efficient compared with ranking every order using ROW_NUMBER. Use LEFT JOIN LATERAL ... ON TRUE to keep customers with no orders.

A127. How do MERGE / UPSERT statements work, and what are idempotent loads?

MERGE (SQL standard; SQL Server, Oracle, Snowflake, BigQuery) compares a source to a target on a key and applies WHEN MATCHED (UPDATE/DELETE) or WHEN NOT MATCHED (INSERT) actions in one atomic statement. Postgres uses INSERT ... ON CONFLICT (key) DO UPDATE and MySQL uses INSERT ... ON DUPLICATE KEY UPDATE.

```
MERGE INTO dim_customer t
USING stg_customer s ON t.customer_id = s.customer_id
WHEN MATCHED THEN UPDATE SET t.email = s.email, t.updated_at = s.updated_at
WHEN NOT MATCHED THEN INSERT (customer_id, email, updated_at)
                      VALUES (s.customer_id, s.email, s.updated_at);
```

An idempotent load can be run repeatedly with the same input and yield the same final state, which makes pipeline retries safe. Upserts on a unique key, or delete-then-insert of a date partition inside one transaction, achieve this. The source must be deduplicated on the key, otherwise MERGE can fail or produce nondeterministic results.

A128. Explain what a deadlock is, and how locking affects analytical queries.

A deadlock occurs when two transactions each hold a lock the other needs, so neither can proceed. The database detects the cycle and aborts one transaction (the victim), which must be retried. Prevention: access tables and rows in a consistent order, keep transactions short, use appropriate isolation levels and indexes (to lock fewer rows).

Locking basics: shared locks for reads, exclusive locks for writes; row-level vs table-level granularity. With MVCC (Postgres, Oracle, InnoDB), readers do not block writers and writers do not block readers because readers see a snapshot. In lock-based engines (SQL Server default read committed without snapshot) long analytical reads can block writers, which is why read replicas, snapshot isolation (READ_COMMITTED_SNAPSHOT) or a warehouse are used for heavy analysis. Never run unbounded updates during peak hours without batching.

A129. What is the difference between RANK-based top-N and using LIMIT, and how do ties affect 'top N' questions?

LIMIT N (TOP N, FETCH FIRST N ROWS ONLY) returns exactly N rows after ordering, cutting arbitrarily through ties. If the 3rd and 4th highest values are tied, LIMIT 3 returns only one of them and the choice is nondeterministic unless the ORDER BY includes a unique tiebreaker.

Alternatives:

- DENSE_RANK() <= N returns all rows whose value is among the top N distinct values (may return more than N rows).

- RANK() <= N returns all rows tied within the top N positions.

- FETCH FIRST N ROWS WITH TIES (standard SQL, Oracle, Postgres 13+) returns N rows plus any ties with the Nth.

- ROW_NUMBER() <= N gives exactly N rows per group, with an explicit tiebreaker.

Always clarify the requirement: 'top 3 salaries' (distinct values) versus 'top 3 employees' (rows), and per group or overall.

A130. How do you handle date and time operations in SQL (truncation, differences, time zones)? Give common patterns.

Common building blocks differ by dialect but the patterns are the same:

- Truncate to period: DATE_TRUNC('month', ts) (Postgres, Snowflake, BigQuery has DATE_TRUNC(date, MONTH)), DATEFROMPARTS or EOMONTH in SQL Server, DATE_FORMAT in MySQL.

- Differences: DATEDIFF(day, a, b) in SQL Server/Snowflake; b - a in Postgres for dates returns days; AGE() or EXTRACT(EPOCH FROM b - a)/86400 for precision.

- Add intervals: ts + INTERVAL '7 days' (Postgres), DATE_ADD (MySQL, BigQuery), DATEADD (SQL Server).

- Extract parts: EXTRACT(dow FROM ts), EXTRACT(week FROM ts).

- Time zones: store in UTC and convert for reporting with AT TIME ZONE 'America/New_York'. Business day definitions must use the local zone or daily counts shift.

- Filtering ranges: use half-open intervals ts >= '2024-03-01' AND ts < '2024-04-01' instead of BETWEEN with an end-of-day time.

- Calendar tables or generate_series help include dates with zero activity.

Be aware of week start conventions (Sunday vs Monday) and ISO week-year edge cases at year end.

A131. Compare the join algorithms (nested loop, hash join, merge join) and when the optimizer picks each.

Nested loop join: for each row of the outer input, probe the inner input (ideally via an index). Best when the outer set is small and the inner has an index on the join key; cost grows as outer rows times lookup cost. Works for any join condition including non-equi joins.

Hash join: build an in-memory hash table on the smaller input, then probe it with the larger. Best for large, unsorted, unindexed inputs with equality conditions; needs memory and may spill to disk.

Merge (sort-merge) join: both inputs sorted on the join key, then scanned together once. Best when inputs are already sorted (from indexes or ORDER BY) or very large; equality and range-ish conditions.

The optimizer chooses based on cardinality estimates, available indexes and memory. Wrong estimates (stale statistics, skewed data, correlated predicates) lead to bad choices, which is why up-to-date statistics matter. Distributed warehouses add broadcast vs shuffle joins: broadcast copies a small table to all nodes, shuffle redistributes both by join key.

A132. Second highest salary. Table employees(emp_id INT, name VARCHAR, dept_id INT, salary INT). Sample: (1,'A',10,100), (2,'B',10,200), (3,'C',20...

Approach 1, subquery (returns NULL automatically because an aggregate over zero rows is NULL):

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

For the sample, the max is 200, and the max below it is 150.

Approach 2, DENSE_RANK (generalizes to Nth):

```
SELECT MAX(salary) AS second_highest_salary
FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
      FROM employees) t
WHERE rnk = 2;
```

The outer MAX wraps the result so a missing second salary produces NULL instead of an empty set.

Approach 3: SELECT DISTINCT salary ... ORDER BY salary DESC LIMIT 1 OFFSET 1 (wrap in a subquery to return NULL when empty). Use DISTINCT or the ties would return the same salary twice.

A133. Top 3 salaries per department. Tables: employees(emp_id, name, dept_id, salary), departments(dept_id, dept_name). Return dept_name, name, sa...

```
WITH ranked AS (
  SELECT d.dept_name, e.name, e.salary,
         DENSE_RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) AS rnk
  FROM employees e
  JOIN departments d ON d.dept_id = e.dept_id
)
SELECT dept_name, name, salary
FROM ranked
WHERE rnk <= 3
ORDER BY dept_name, salary DESC;
```

DENSE_RANK is used so that people tied on a salary share a rank and no top-3 salary tier is skipped. If the requirement was exactly 3 employees per department, use ROW_NUMBER with a deterministic tiebreaker (ORDER BY salary DESC, emp_id). Use LEFT JOIN to departments if employees may have a missing department and should be retained.

A134. Employees earning more than their manager. Table employees(emp_id, name, salary, manager_id) where manager_id references emp_id. Sample: (1,...

Self join employees to its manager:

```
SELECT e.name AS employee
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Joe (70) reports to Sam (60), so Joe qualifies. Henry (80) reports to Max (90), so no. Employees without a manager drop out of the inner join, which is correct here since there is nothing to compare against.

A135. Find duplicate emails. Table users(id INT, email VARCHAR). Sample: (1,'a@x.com'), (2,'b@x.com'), (3,'a@x.com'). Return each email that appea...

```
SELECT email, COUNT(*) AS cnt
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

Result: a@x.com, 2. To return the full duplicate rows, use a window count:

```
SELECT * FROM (
  SELECT u.*, COUNT(*) OVER (PARTITION BY LOWER(TRIM(email))) AS c FROM users u) t
WHERE c > 1;
```

Normalizing case and whitespace catches near-duplicates; decide with the business whether 'A@x.com' equals 'a@x.com'.

A136. Customers who never ordered. Tables customers(customer_id, name) and orders(order_id, customer_id, amount). Return names of customers with n...

Anti-join with LEFT JOIN:

```
SELECT c.name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

NOT EXISTS:

```
SELECT c.name
FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

Avoid NOT IN (SELECT customer_id FROM orders) unless customer_id is NOT NULL in orders, because a single NULL makes NOT IN return nothing. Test the IS NULL on a column that is non-nullable in orders (the primary key).

A137. Running total of sales. Table sales(sale_date DATE, store_id INT, amount NUMERIC). Show each store's daily amount, and a running total per s...

```
SELECT store_id, sale_date, amount,
       SUM(amount) OVER (PARTITION BY store_id
```

```
                        ORDER BY sale_date
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM sales
ORDER BY store_id, sale_date;
```

Output for store 1: 100, 150, 175. If there can be several rows per store per day, first aggregate to daily level in a CTE, or the explicit ROWS frame would produce a different value per row for the same date (RANGE would give the same value for peers). Add a tiebreaker column to ORDER BY for determinism.

A138. 7-day moving average of revenue. Table daily_revenue(dt DATE, revenue NUMERIC), one row per day, but some days may be missing. Compute a tra...

If every day is present, ROWS is fine:

```
SELECT dt, revenue,
       AVG(revenue) OVER (ORDER BY dt ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS ma7
FROM daily_revenue;
```

Missing dates would make 6 PRECEDING span more than 7 calendar days. Robust approach: build a date spine and fill zeros, or use a calendar RANGE frame (Postgres 11+):

```
WITH spine AS (
  SELECT d::date AS dt FROM generate_series(
    (SELECT MIN(dt) FROM daily_revenue), (SELECT MAX(dt) FROM daily_revenue), '1 day') d
), filled AS (
  SELECT s.dt, COALESCE(r.revenue, 0) AS revenue
  FROM spine s LEFT JOIN daily_revenue r USING (dt)
)
SELECT dt, revenue,
       CASE WHEN ROW_NUMBER() OVER (ORDER BY dt) >= 7
            THEN AVG(revenue) OVER (ORDER BY dt ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) END
AS ma7
FROM filled;
```

The CASE restricts output to rows with a complete 7-day window.

A139. Month-over-month revenue growth. Table orders(order_id, order_date DATE, amount NUMERIC). Return month, revenue, previous month revenue and ...

```
WITH monthly AS (
  SELECT DATE_TRUNC('month', order_date)::date AS month, SUM(amount) AS revenue
  FROM orders
  GROUP BY 1
)
SELECT month, revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev_revenue,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
             / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 2) AS growth_pct
FROM monthly
ORDER BY month;
```

The first month has NULL growth. NULLIF avoids division by zero. If months with zero orders may be absent, LAG compares to the previous existing month rather than the previous calendar month; join to a month spine first to be strict. For year over year use LAG(revenue, 12) on a complete monthly series.

A140. Longest login streak (gaps and islands). Table logins(user_id INT, login_date DATE), possibly duplicates per day. Return each user's longest...

Idea: after deduplicating to one row per user per day, subtract an increasing row number from the date. Consecutive days produce the same constant, identifying an island.

```
WITH d AS (
  SELECT DISTINCT user_id, login_date FROM logins
), g AS (
  SELECT user_id, login_date,
         login_date - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date))::int AS
grp
  FROM d
), streaks AS (
  SELECT user_id, MIN(login_date) AS start_date, MAX(login_date) AS end_date,
         COUNT(*) AS streak_len
  FROM g
  GROUP BY user_id, grp
), ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY streak_len DESC, start_date) AS
rn
  FROM streaks
)
SELECT user_id, start_date, end_date, streak_len
FROM ranked WHERE rn = 1;
```

Example: dates 1,2,3,5,6 with row numbers 1,2,3,4,5 give grp values day0 for the first three and day1 for the last two, so two islands of length 3 and 2. In SQL Server use DATEADD(day, -rn, login_date). The alternative is LAG to flag a new streak when the gap is more than 1 and a cumulative SUM of flags.

A141. Users who logged in on 3 or more consecutive days. Table logins(user_id, login_date). Return distinct user_ids. Show a method without the da...

Use LEAD to look two days ahead on the deduplicated dates:

```
WITH d AS (SELECT DISTINCT user_id, login_date FROM logins)
SELECT DISTINCT user_id
FROM (
  SELECT user_id, login_date,
         LEAD(login_date, 2) OVER (PARTITION BY user_id ORDER BY login_date) AS d2
  FROM d) t
WHERE d2 = login_date + 2;
```

Because dates are distinct and ordered, if the date two rows ahead is exactly 2 days later, the middle row must be the day in between. For general N, use LEAD(login_date, N-1) with + (N-1), or the island method with HAVING COUNT(*) >= N. A self-join version (a JOIN b ON b.date = a.date + 1 JOIN c ON c.date = a.date + 2) works for fixed small N.

A142. Median salary. Table employees(emp_id, dept_id, salary). Compute the median salary per department without relying on a built-in median funct...

Rank rows from both ends, then average the middle one or two:

```
WITH r AS (
  SELECT dept_id, salary,
         ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary) AS rn_asc,
         COUNT(*)     OVER (PARTITION BY dept_id) AS cnt
  FROM employees
)
SELECT dept_id, AVG(salary) AS median_salary
FROM r
WHERE rn_asc IN (FLOOR((cnt + 1) / 2.0), CEIL((cnt + 1) / 2.0))
GROUP BY dept_id;
```

For odd cnt both expressions give the same middle position; for even cnt they give the two middle positions, and AVG averages them. Built-in options: PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) (Postgres, Oracle, Snowflake, SQL Server as window function), MEDIAN() in Snowflake/Oracle, PERCENTILE_CONT as window in BigQuery. PERCENTILE_DISC returns an actual value rather than an interpolation.

A143. Latest record per user. Table profile_changes(change_id, user_id, plan, changed_at TIMESTAMP). Return the current plan of each user (their m...

```
SELECT user_id, plan, changed_at
FROM (
  SELECT user_id, plan, changed_at,
         ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY changed_at DESC, change_id DESC) AS
rn
  FROM profile_changes) t
WHERE rn = 1;
```

The change_id tiebreaker makes the result deterministic when two changes share a timestamp. Alternatives: Postgres DISTINCT ON (user_id) ... ORDER BY user_id, changed_at DESC; or join to a subquery of MAX(changed_at) per user (returns multiple rows on ties); or QUALIFY ROW_NUMBER() ... = 1 in Snowflake/BigQuery.

A144. Pivot monthly sales. Table sales(product VARCHAR, sale_month INT, amount NUMERIC) with months 1 to 3. Produce one row per product with colum...

```
SELECT product,
       SUM(CASE WHEN sale_month = 1 THEN amount ELSE 0 END) AS m1,
       SUM(CASE WHEN sale_month = 2 THEN amount ELSE 0 END) AS m2,
       SUM(CASE WHEN sale_month = 3 THEN amount ELSE 0 END) AS m3,
       SUM(amount) AS total
FROM sales
GROUP BY product
ORDER BY product;
```

Conditional aggregation works in every dialect. Use ELSE 0 if empty cells should show zero, or omit ELSE to keep NULL for no data. Postgres alternative: SUM(amount) FILTER (WHERE sale_month = 1).

A145. Percent of total. Table orders(order_id, category VARCHAR, amount NUMERIC). Return each category's revenue and its share of total revenue as...

```
SELECT category,
       SUM(amount) AS revenue,
       ROUND(100.0 * SUM(amount) / SUM(SUM(amount)) OVER (), 1) AS pct_of_total
FROM orders
GROUP BY category
ORDER BY revenue DESC;
```

SUM(SUM(amount)) OVER () is a window function applied on top of the grouped result: the inner SUM is the group aggregate, the outer one totals across all groups. The 100.0 literal forces decimal arithmetic and avoids integer division.

A146. Customers who bought every product (relational division). Tables products(product_id), purchases(customer_id, product_id). Return customers ...

Count distinct products per customer and compare with the total:

```
SELECT customer_id
FROM purchases
GROUP BY customer_id
HAVING COUNT(DISTINCT product_id) = (SELECT COUNT(*) FROM products);
```

This assumes purchases contains only valid product_ids (join to products to be safe). The double NOT EXISTS version expresses 'there is no product that this customer has not bought':

```
SELECT c.customer_id
FROM (SELECT DISTINCT customer_id FROM purchases) c
WHERE NOT EXISTS (
  SELECT 1 FROM products p
  WHERE NOT EXISTS (SELECT 1 FROM purchases x
                    WHERE x.customer_id = c.customer_id AND x.product_id = p.product_id));
```

A147. Find missing IDs in a sequence. Table invoices(invoice_no INT), values like 1,2,3,5,6,9. Return the gaps as ranges (4-4, 7-8).

Use LEAD to compare each number with the next:

```
SELECT invoice_no + 1 AS gap_start,
       next_no - 1   AS gap_end
FROM (
  SELECT invoice_no, LEAD(invoice_no) OVER (ORDER BY invoice_no) AS next_no
  FROM invoices) t
WHERE next_no - invoice_no > 1;
```

Output: (4,4) and (7,8). To list every missing number individually, join generate_series(MIN, MAX) to the table with an anti-join:

```
SELECT s.n FROM generate_series((SELECT MIN(invoice_no) FROM invoices),
                                (SELECT MAX(invoice_no) FROM invoices)) s(n)
LEFT JOIN invoices i ON i.invoice_no = s.n
WHERE i.invoice_no IS NULL;
```

Leading or trailing gaps need an agreed expected start and end.

A148. Merge overlapping intervals. Table bookings(room_id INT, start_ts TIMESTAMP, end_ts TIMESTAMP). Collapse overlapping or touching bookings pe...

Order by start; a new block begins when the current start is after the running maximum end of all earlier rows.

```
WITH x AS (
  SELECT room_id, start_ts, end_ts,
         MAX(end_ts) OVER (PARTITION BY room_id ORDER BY start_ts, end_ts
                           ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING) AS prev_max_end
  FROM bookings
), flagged AS (
  SELECT *, CASE WHEN prev_max_end IS NULL OR start_ts > prev_max_end THEN 1 ELSE 0 END AS
new_block
  FROM x
), grp AS (
  SELECT *, SUM(new_block) OVER (PARTITION BY room_id ORDER BY start_ts, end_ts) AS block_id
  FROM flagged
)
SELECT room_id, block_id, MIN(start_ts) AS block_start, MAX(end_ts) AS block_end
FROM grp
GROUP BY room_id, block_id;
```

Use start_ts >= prev_max_end instead of > if you want touching intervals to stay separate. Total occupied time is the sum of (block_end - block_start) over blocks. Using a running maximum of ends (not just the previous row's end) is essential, because a long early booking can swallow several later ones.

A149. Sessionization. Table events(user_id, event_ts TIMESTAMP, event_name). Define a session as a series of events with no gap greater than 30 mi...

```
WITH lagged AS (
  SELECT *, LAG(event_ts) OVER (PARTITION BY user_id ORDER BY event_ts) AS prev_ts
  FROM events
), flagged AS (
  SELECT *, CASE WHEN prev_ts IS NULL
                 OR event_ts - prev_ts > INTERVAL '30 minutes' THEN 1 ELSE 0 END AS is_new
  FROM lagged
), sessions AS (
  SELECT *, SUM(is_new) OVER (PARTITION BY user_id ORDER BY event_ts
                              ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS session_no
  FROM flagged
)
SELECT user_id, session_no,
       MIN(event_ts) AS session_start, MAX(event_ts) AS session_end,
```

```
       COUNT(*) AS events,
       MAX(event_ts) - MIN(event_ts) AS duration
FROM sessions
GROUP BY user_id, session_no;
```

Pattern: LAG to get the previous event, flag a new session when the gap exceeds the threshold (or no previous), cumulative SUM of flags to number sessions. A global session id is (user_id, session_no).

A150. Rising temperature. Table weather(record_date DATE, temperature INT) with one row per day. Return the dates whose temperature is higher than...

```
SELECT record_date
FROM (
  SELECT record_date, temperature,
         LAG(temperature) OVER (ORDER BY record_date) AS prev_temp,
         LAG(record_date) OVER (ORDER BY record_date) AS prev_date
  FROM weather) t
WHERE temperature > prev_temp
  AND record_date = prev_date + 1;
```

The second condition ensures the previous row is truly the previous calendar day when dates can be missing. Self-join alternative: JOIN weather y ON y.record_date = w.record_date - 1 AND w.temperature > y.temperature (use DATEADD or DATEDIFF in SQL Server/MySQL).

A151. Employees in the same department with the same salary. Table employees(emp_id, name, dept_id, salary). Return pairs of employees (without mi...

```
SELECT a.name AS emp1, b.name AS emp2, a.dept_id, a.salary
FROM employees a
JOIN employees b
  ON a.dept_id = b.dept_id
 AND a.salary = b.salary
 AND a.emp_id < b.emp_id;
```

The condition a.emp_id < b.emp_id removes self-pairs (A,A) and mirrored pairs (B,A). With n employees sharing a salary this produces n(n-1)/2 pairs. A grouping alternative that lists the group: GROUP BY dept_id, salary HAVING COUNT(*) > 1 with STRING_AGG(name, ', ').

A152. Top 2 products per category by revenue. Tables products(product_id, category), order_items(order_id, product_id, qty, unit_price). Return ca...

```
WITH rev AS (
  SELECT p.category, p.product_id, SUM(oi.qty * oi.unit_price) AS revenue
  FROM order_items oi
  JOIN products p ON p.product_id = oi.product_id
  GROUP BY p.category, p.product_id
), ranked AS (
  SELECT *, RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rnk
  FROM rev
)
```

```
SELECT category, product_id, revenue
FROM ranked
WHERE rnk <= 2
ORDER BY category, revenue DESC;
```

The aggregation happens first, then the window function ranks aggregated rows (windows are evaluated after GROUP BY). Switch to ROW_NUMBER for exactly 2 rows per category.

A153. Pareto analysis. Table orders(customer_id, amount). Find the smallest set of top customers that account for 80 percent of total revenue.

```
WITH cust AS (
  SELECT customer_id, SUM(amount) AS rev FROM orders GROUP BY customer_id
), cum AS (
  SELECT customer_id, rev,
         SUM(rev) OVER (ORDER BY rev DESC, customer_id
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_rev,
         SUM(rev) OVER () AS total_rev
  FROM cust
)
SELECT customer_id, rev,
       ROUND(100.0 * running_rev / total_rev, 1) AS cum_pct
FROM cum
WHERE running_rev - rev < 0.8 * total_rev;
```

The condition running_rev - rev < 80% of total keeps every customer whose preceding cumulative share was still under 80 percent, so the customer that crosses the threshold is included. Report count of selected customers divided by total customers to state the '20 percent of customers drive 80 percent of revenue' insight.

A154. Orders per customer including zero. Tables customers(customer_id, name), orders(order_id, customer_id). Return every customer with their num...

```
SELECT c.customer_id, c.name, COUNT(o.order_id) AS order_count
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.name
ORDER BY order_count DESC;
```

COUNT(o.order_id) counts only non-NULL order ids, so unmatched customers get 0. COUNT(*) would wrongly return 1 for them, since the NULL-extended row still exists.

A155. Fill missing dates with zero. Table daily_signups(dt DATE, signups INT) lacks rows for days with no signups. Produce a continuous series for...

```
SELECT d.dt::date AS dt, COALESCE(s.signups, 0) AS signups
FROM generate_series('2024-01-01'::date, '2024-01-31'::date, INTERVAL '1 day') AS d(dt)
LEFT JOIN daily_signups s ON s.dt = d.dt::date
ORDER BY dt;
```

The spine is the left table so all days remain. Without generate_series: recursive CTE (WITH RECURSIVE days AS (SELECT DATE '2024-01-01' AS dt UNION ALL SELECT dt + 1 FROM days WHERE dt < '2024-01-31')) or a permanent calendar/date dimension table. BigQuery uses GENERATE_DATE_ARRAY and UNNEST; SQL Server uses a recursive CTE or numbers table.

A156. Employees earning above their department average. Table employees(emp_id, name, dept_id, salary). Return them with the department average an...

```
SELECT emp_id, name, dept_id, salary, dept_avg, salary - dept_avg AS above_avg_by
FROM (
  SELECT e.*, AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg
  FROM employees e) t
WHERE salary > dept_avg
ORDER BY dept_id, salary DESC;
```

The window function must be computed in a subquery or CTE because it cannot be used directly in WHERE. A join-to-aggregate form also works: JOIN (SELECT dept_id, AVG(salary) a FROM employees GROUP BY dept_id) d USING (dept_id) WHERE salary > d.a.

A157. Conversion between two events. Table events(user_id, event_name, event_ts) with names 'view' and 'purchase'. Compute, per day, the number of...

```
SELECT event_ts::date AS day,
       COUNT(DISTINCT CASE WHEN event_name = 'view' THEN user_id END) AS viewers,
       COUNT(DISTINCT CASE WHEN event_name = 'purchase' THEN user_id END) AS purchasers,
       ROUND(100.0 *
         COUNT(DISTINCT CASE WHEN event_name = 'purchase' THEN user_id END) /
         NULLIF(COUNT(DISTINCT CASE WHEN event_name = 'view' THEN user_id END), 0), 2) AS
conv_pct
FROM events
GROUP BY 1
ORDER BY 1;
```

Caveat: this counts purchasers who may not have viewed that day. To count only viewers who purchased, aggregate per user-day first: compute a flag has_view and has_purchase per (user, day), then SUM(has_purchase) over users with has_view = 1. Always make the denominator population and numerator population the same set of users.

A158. Customers who ordered in two consecutive months. Table orders(order_id, customer_id, order_date). Return customers having orders in at least...

```
WITH m AS (
  SELECT DISTINCT customer_id, DATE_TRUNC('month', order_date)::date AS mth
  FROM orders
)
SELECT DISTINCT a.customer_id
FROM m a
JOIN m b
  ON b.customer_id = a.customer_id
 AND b.mth = (a.mth + INTERVAL '1 month')::date;
```

First reduce to distinct customer-months so duplicates do not matter. The join matches month M with month M+1. A window alternative: LEAD(mth) OVER (PARTITION BY customer_id ORDER BY mth) = mth + interval '1 month'.

A159. Consecutive identical values. Table logs(id INT, num INT) where id is sequential. Find numbers that appear at least three times consecutivel...

```
SELECT DISTINCT num AS consecutive_num
FROM (
  SELECT num,
         LAG(num)  OVER (ORDER BY id) AS p,
         LEAD(num) OVER (ORDER BY id) AS n
  FROM logs) t
WHERE num = p AND num = n;
```

Result: 1 (ids 1 to 3). This relies on ids being gapless; if gaps are possible, use ROW_NUMBER() OVER (ORDER BY id) instead of id. For runs of arbitrary length use the island trick: ROW_NUMBER() OVER (ORDER BY id) - ROW_NUMBER() OVER (PARTITION BY num ORDER BY id) as the group key, then HAVING COUNT(*) >= 3.

A160. Highest paid employee in each department including ties. Tables employees(emp_id, name, dept_id, salary) and departments(dept_id, dept_name)...

```
SELECT d.dept_name, e.name, e.salary
FROM employees e
JOIN departments d ON d.dept_id = e.dept_id
WHERE (e.dept_id, e.salary) IN (
  SELECT dept_id, MAX(salary) FROM employees GROUP BY dept_id);
```

Row-value IN is supported in Postgres, MySQL, Oracle; in SQL Server use a join to the grouped subquery or RANK() = 1:

```
SELECT dept_name, name, salary FROM (
  SELECT d.dept_name, e.name, e.salary,
         RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) r
  FROM employees e JOIN departments d USING (dept_id)) t
WHERE r = 1;
```

Using RANK (or DENSE_RANK) returns all tied top earners.

A161. Cumulative distinct users. Table logins(user_id, login_date). For each date, report the cumulative number of distinct users who have ever lo...

COUNT(DISTINCT) is not allowed in a running window in most engines, so count each user once on their first login date and then take a running sum:

```
WITH first_seen AS (
  SELECT user_id, MIN(login_date) AS first_date FROM logins GROUP BY user_id
), per_day AS (
  SELECT first_date AS dt, COUNT(*) AS new_users FROM first_seen GROUP BY first_date
)
```

```
SELECT dt, new_users,
       SUM(new_users) OVER (ORDER BY dt ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
cumulative_users
FROM per_day
ORDER BY dt;
```

Dates on which no new users appeared are missing; left join from a date spine and COALESCE(new_users,0) if every date is required. The same first-occurrence idea handles 'new vs returning' classification.

A162. Rolling 30-day active users. Table events(user_id, event_date DATE). For each date D, count distinct users with at least one event in the wi...

Window functions cannot do COUNT(DISTINCT) over a frame in most engines, so use a range join to a date spine:

```
WITH days AS (
  SELECT DISTINCT event_date AS dt FROM events
)
SELECT d.dt, COUNT(DISTINCT e.user_id) AS mau_30d
FROM days d
JOIN events e
  ON e.event_date BETWEEN d.dt - 29 AND d.dt
GROUP BY d.dt
ORDER BY d.dt;
```

This is correct but can be expensive (each event joins to up to 30 days). Efficiency options: first reduce events to distinct (user_id, event_date); for large data compute per user the sorted active dates and use gaps between consecutive activity (a user is active in the window [d, next_d + 29] style intervals), or use HyperLogLog sketches (APPROX_COUNT_DISTINCT, HLL_COUNT) for approximate answers. In Postgres date + integer arithmetic works; use INTERVAL in other engines.

A163. Nth highest salary function and handling of fewer than N distinct values. Table employees(emp_id, salary). Write a query for N = 5 that retu...

```
SELECT (
  SELECT DISTINCT salary
  FROM employees
  ORDER BY salary DESC
  LIMIT 1 OFFSET 4
) AS fifth_highest_salary;
```

Wrapping the subquery in a scalar SELECT converts an empty result into NULL. DISTINCT is needed so ties count once; without it, three people with the top salary would occupy positions 1 to 3.

Window alternative that is easier to parameterize:

```
SELECT MAX(salary) AS nth_salary
FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) r FROM employees) t
WHERE r = 5;
```

MAX over an empty set gives NULL as well. In SQL Server use OFFSET 4 ROWS FETCH NEXT 1 ROWS ONLY; in MySQL a stored function would need N-1 computed into a variable before LIMIT.

A164. Company: an e-commerce marketplace. Management wants monthly customer retention: of customers whose first purchase was in a given month, wha...

Step 1, define the cohort: a customer's cohort month is the month of their first order (trigger event = first purchase). Retention in month N = distinct cohort customers who ordered in cohort month + N, divided by the cohort size (month 0 is the denominator).

Step 2, compute cohort month per customer.

Step 3, compute each order's month and the month offset.

Step 4, count distinct customers per cohort and offset, divide by the offset 0 count.

```
WITH first_order AS (
  SELECT customer_id, DATE_TRUNC('month', MIN(order_date))::date AS cohort_month
  FROM orders GROUP BY customer_id
), activity AS (
  SELECT DISTINCT o.customer_id, f.cohort_month,
         (EXTRACT(YEAR FROM AGE(DATE_TRUNC('month', o.order_date), f.cohort_month)) * 12
          + EXTRACT(MONTH FROM AGE(DATE_TRUNC('month', o.order_date), f.cohort_month)))::int
AS month_offset
  FROM orders o JOIN first_order f USING (customer_id)
), counts AS (
  SELECT cohort_month, month_offset, COUNT(*) AS customers
  FROM activity GROUP BY 1, 2
)
SELECT cohort_month, month_offset, customers,
       ROUND(100.0 * customers / FIRST_VALUE(customers) OVER (PARTITION BY cohort_month ORDER
BY month_offset), 1) AS retention_pct
FROM counts
ORDER BY cohort_month, month_offset;
```

Notes for the interviewer: use DISTINCT so several orders in one month count once; month 0 is always 100 percent; the most recent cohorts have incomplete follow-up, so mask offsets that extend past the data end date; pivot offsets into columns with conditional aggregation for a triangle heatmap; decide if 'returned' means any order or an order in exactly that month (the above is exactly that month, i.e. N-month retention rather than rolling retention).

A165. Company: a ride-hailing platform. Operations asks for the daily cancellation rate of requests made by unbanned riders with unbanned drivers,...

Step 1, restrict to trips where both the rider and the driver are not banned. This needs two joins to the users table, once per role.

Step 2, restrict the date range.

Step 3, rate = cancelled trips / all trips that day; compute with conditional aggregation.

Step 4, guard against zero denominators and round.

```
SELECT t.request_date AS day,
       ROUND(1.0 * SUM(CASE WHEN t.status <> 'completed' THEN 1 ELSE 0 END) / COUNT(*), 2) AS
cancellation_rate
```

```
FROM trips t
JOIN users r ON r.user_id = t.rider_id  AND r.banned = 'No'
JOIN users d ON d.user_id = t.driver_id AND d.banned = 'No'
WHERE t.request_date BETWEEN DATE '2023-10-01' AND DATE '2023-10-03'
GROUP BY t.request_date
ORDER BY t.request_date;
```

Because the inner joins already exclude banned users, no extra filter is needed. Days with zero eligible trips will not appear; if the report needs them, left join from a date spine and use NULLIF on COUNT(*). Follow-up analyses: split by who cancelled (driver vs rider), by hour and city, and compare to a baseline week to spot anomalies.

A166. Company: a video streaming service. Product wants a signup-to-paid funnel for users who signed up in March 2024: signup, started free trial,...

Step 1, define the population: users with signup_ts in March 2024; every step is measured within that cohort.

Step 2, produce one flag per user per step, using LEFT JOIN to pre-aggregated or EXISTS checks so there is no fan-out.

Step 3, enforce order (a step counts only if it follows the previous step), otherwise the funnel can contain impossible paths.

Step 4, aggregate flags and divide.

```
WITH cohort AS (
  SELECT user_id, signup_ts FROM users
  WHERE signup_ts >= '2024-03-01' AND signup_ts < '2024-04-01'
), steps AS (
  SELECT c.user_id,
         t.first_trial,
         v.first_view,
         s.first_paid
  FROM cohort c
  LEFT JOIN (SELECT user_id, MIN(trial_start_ts) first_trial FROM trials GROUP BY user_id) t
         ON t.user_id = c.user_id AND t.first_trial >= c.signup_ts
  LEFT JOIN (SELECT user_id, MIN(view_ts) first_view FROM views GROUP BY user_id) v
         ON v.user_id = c.user_id AND v.first_view >= t.first_trial
  LEFT JOIN (SELECT user_id, MIN(paid_start_ts) first_paid FROM subscriptions GROUP BY
user_id) s
         ON s.user_id = c.user_id AND s.first_paid >= v.first_view
)
SELECT COUNT(*) AS signed_up,
       COUNT(first_trial) AS trial,
       COUNT(first_view)  AS watched,
       COUNT(first_paid)  AS paid,
       ROUND(100.0 * COUNT(first_trial) / COUNT(*), 1) AS signup_to_trial_pct,
       ROUND(100.0 * COUNT(first_view) / NULLIF(COUNT(first_trial), 0), 1) AS
trial_to_view_pct,
       ROUND(100.0 * COUNT(first_paid) / NULLIF(COUNT(first_view), 0), 1) AS
view_to_paid_pct,
       ROUND(100.0 * COUNT(first_paid) / COUNT(*), 1) AS overall_pct
FROM steps;
```

COUNT(column) ignores NULLs, so it counts users who reached the step. Since each later join conditions on the earlier step's timestamp, a NULL earlier step produces NULL later steps and the funnel is strictly nested. Add a time limit (for example paid within 14 days of signup) in the ON clause when the business defines a conversion window. Segment by acquisition channel to find where the biggest drop-off is.

A167. Company: a retail bank. The fraud team wants to flag accounts that made 3 or more card transactions within any 10-minute window, and transac...

Part A: burst detection. A sliding window means 'for each transaction, count transactions of the same account in the next 10 minutes (inclusive of itself)'. Using a range frame on time:

```
SELECT DISTINCT account_id
FROM (
  SELECT account_id, txn_ts,
         COUNT(*) OVER (PARTITION BY account_id ORDER BY txn_ts
                        RANGE BETWEEN CURRENT ROW AND INTERVAL '10 minutes' FOLLOWING) AS
n_next_10m
  FROM transactions) t
WHERE n_next_10m >= 3;
```

If the engine lacks interval RANGE frames, use LEAD(txn_ts, 2) per account and test that it is within 10 minutes of the current row:

```
SELECT DISTINCT account_id
FROM (SELECT account_id, txn_ts,
             LEAD(txn_ts, 2) OVER (PARTITION BY account_id ORDER BY txn_ts) AS third_ts
      FROM transactions) t
WHERE third_ts - txn_ts <= INTERVAL '10 minutes';
```

Part B: duplicate charges.

```
SELECT txn_id, account_id, merchant_id, amount, txn_ts
FROM (
  SELECT t.*,
         LAG(txn_ts) OVER (PARTITION BY account_id, merchant_id, amount ORDER BY txn_ts) AS
prev_ts
  FROM transactions t) x
WHERE txn_ts - prev_ts <= INTERVAL '2 minutes';
```

Reasoning: partitioning by the duplicate key and comparing only to the previous row gives each repeat exactly once. Next steps: examine precision of timestamps, exclude legitimately repeating merchants (subscriptions), and build a rule score. Performance: index (account_id, txn_ts) helps the partitioned scans, and filtering to a recent window first limits volume.

A168. Company: an online fashion retailer. Marketing wants an RFM (recency, frequency, monetary) segmentation of customers as of 2024-06-30 to tar...

Step 1, aggregate per customer: recency = days since last order, frequency = number of orders, monetary = total spend, restricted to orders on or before the as-of date (avoid look-ahead).

Step 2, bucket each metric into quintiles with NTILE(5). For recency, lower is better, so order descending so that the most recent customers get score 5.

Step 3, combine scores and label.

```
WITH base AS (
  SELECT customer_id,
         DATE '2024-06-30' - MAX(order_date) AS recency_days,
         COUNT(*) AS frequency,
         SUM(amount) AS monetary
  FROM orders
  WHERE order_date <= DATE '2024-06-30'
  GROUP BY customer_id
), scored AS (
  SELECT *,
         NTILE(5) OVER (ORDER BY recency_days DESC) AS r_score,
         NTILE(5) OVER (ORDER BY frequency ASC)    AS f_score,
         NTILE(5) OVER (ORDER BY monetary ASC)     AS m_score
  FROM base
)
SELECT customer_id, recency_days, frequency, monetary, r_score, f_score, m_score,
       CASE WHEN r_score >= 4 AND f_score >= 4 AND m_score >= 4 THEN 'Champions'
            WHEN r_score >= 4 AND f_score <= 2 THEN 'New / Promising'
            WHEN r_score <= 2 AND f_score >= 4 THEN 'At risk'
            WHEN r_score <= 2 AND f_score <= 2 THEN 'Lost'
            ELSE 'Regular' END AS segment
FROM scored;
```

Caveats: NTILE splits ties arbitrarily across buckets (customers with identical frequency may land in different buckets), so many analysts use fixed business thresholds or PERCENT_RANK for frequency, which has heavy ties. Review segment sizes, and refresh monthly.

A169. Company: a music and video streaming app. The content team wants the top 3 shows by total watch minutes in each genre for last month, exclud...

Step 1, filter watch events to last month using half-open dates.

Step 2, aggregate per show: total minutes and distinct viewers; apply the viewer threshold with HAVING.

Step 3, join to shows for genre; rank within genre.

Step 4, keep rank <= 3.

```
WITH per_show AS (
  SELECT w.show_id,
         SUM(w.minutes_watched) AS total_minutes,
         COUNT(DISTINCT w.user_id) AS viewers
  FROM watch_events w
  WHERE w.watch_date >= DATE_TRUNC('month', CURRENT_DATE) - INTERVAL '1 month'
    AND w.watch_date <  DATE_TRUNC('month', CURRENT_DATE)
  GROUP BY w.show_id
  HAVING COUNT(DISTINCT w.user_id) >= 100
), ranked AS (
  SELECT s.genre, s.title, p.total_minutes, p.viewers,
         DENSE_RANK() OVER (PARTITION BY s.genre ORDER BY p.total_minutes DESC) AS rnk
  FROM per_show p JOIN shows s USING (show_id)
)
SELECT genre, title, total_minutes, viewers, rnk
FROM ranked WHERE rnk <= 3
```

```
ORDER BY genre, rnk;
```

Discussion: the threshold prevents tiny-sample shows from distorting the ranking. Use ROW_NUMBER with a tiebreaker if exactly 3 titles per genre are required. Consider normalizing by catalog age or number of episodes if the ranking is intended to measure engagement rather than raw volume.

A170. Company: a retail bank. Each account has transactions with signed amounts (deposits positive, withdrawals negative). Finance needs, for ever...

Step 1, build a month spine per account: months between opened_date and 2024-12 using generate_series joined to accounts.

Step 2, aggregate net movement per account per month.

Step 3, left join the spine to movements (COALESCE to 0) and take a cumulative sum to get end-of-month balance.

Step 4, for the 'negative at any point' flag, compute the running balance per transaction, then MIN per month, and join that too.

```
WITH spine AS (
  SELECT a.account_id, m::date AS month_start
  FROM accounts a
  CROSS JOIN LATERAL generate_series(DATE_TRUNC('month', a.opened_date),
                                     DATE '2024-12-01', INTERVAL '1 month') m
), txn AS (
  SELECT account_id, txn_ts, amount,
         SUM(amount) OVER (PARTITION BY account_id ORDER BY txn_ts, txn_id
                           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS bal_after
  FROM transactions
), monthly AS (
  SELECT account_id, DATE_TRUNC('month', txn_ts)::date AS month_start,
         SUM(amount) AS net, MIN(bal_after) AS min_balance
  FROM txn GROUP BY 1, 2
)
SELECT s.account_id, s.month_start,
       SUM(COALESCE(m.net, 0)) OVER (PARTITION BY s.account_id ORDER BY s.month_start) AS
end_balance,
       COALESCE(m.min_balance, 0) < 0 AS was_negative
FROM spine s
LEFT JOIN monthly m USING (account_id, month_start)
ORDER BY s.account_id, s.month_start;
```

Check: if transactions has no txn_id, add a tiebreaker or use a row number. For months without activity, was_negative should instead reflect the carried balance; refine with end_balance < 0 OR min_balance < 0. Validate by reconciling the final month balance with SUM(amount) per account, and make sure transactions before the spine start (if any) are included in the opening balance.

A171. Company: a mobile game studio. Compute Day-1, Day-7 and Day-30 retention for each install-date cohort. Tables: installs(user_id, install_dat...

Step 1, cohort = installs grouped by install_date; the denominator is the cohort size.

Step 2, for each user, check whether a session exists on install_date + n for each n.

Step 3, aggregate per cohort.

```
WITH act AS (
  SELECT DISTINCT user_id, session_date FROM sessions
)
SELECT i.install_date,
       COUNT(*) AS installs,
       ROUND(100.0 * COUNT(a1.user_id)  / COUNT(*), 1) AS d1_pct,
       ROUND(100.0 * COUNT(a7.user_id)  / COUNT(*), 1) AS d7_pct,
       ROUND(100.0 * COUNT(a30.user_id) / COUNT(*), 1) AS d30_pct
FROM installs i
LEFT JOIN act a1  ON a1.user_id  = i.user_id AND a1.session_date  = i.install_date + 1
LEFT JOIN act a7  ON a7.user_id  = i.user_id AND a7.session_date  = i.install_date + 7
LEFT JOIN act a30 ON a30.user_id = i.user_id AND a30.session_date = i.install_date + 30
GROUP BY i.install_date
ORDER BY i.install_date;
```

Because act is distinct per (user, date), each LEFT JOIN is at most one match per user, so no fan-out and COUNT works. Exclude cohorts that have not yet reached day n (install_date + n > latest data date) or they will show artificially low retention. Define the metric clearly: 'classic' D7 (exactly day 7) vs 'rolling' D7 (any session on or after day 7). Timezone of install_date and session_date must match.

A172. Company: a SaaS subscription business. Compute monthly logo churn: for each month, the percentage of customers active at the start of the mo...

Definitions: a customer is active on a date if start_date <= date and (end_date IS NULL or end_date > date). Opening customers for month M = active on the first day of M. Churned = opening customers whose end_date falls inside M (before the next month starts).

```
WITH months AS (
  SELECT m::date AS month_start
  FROM generate_series('2024-01-01'::date, '2024-12-01'::date, '1 month') m
), opening AS (
  SELECT m.month_start, s.customer_id, s.end_date
  FROM months m
  JOIN subscriptions s
    ON s.start_date <= m.month_start
   AND (s.end_date IS NULL OR s.end_date > m.month_start)
)
SELECT month_start,
       COUNT(*) AS opening_customers,
       COUNT(*) FILTER (WHERE end_date IS NOT NULL
                        AND end_date < month_start + INTERVAL '1 month') AS churned,
       ROUND(100.0 * COUNT(*) FILTER (WHERE end_date IS NOT NULL
                        AND end_date < month_start + INTERVAL '1 month') / COUNT(*), 2) AS
churn_pct
FROM opening
GROUP BY month_start
ORDER BY month_start;
```

Caveats: customers with several subscription rows should be deduplicated at customer level (a customer who cancels and re-subscribes within the month is not churn); new customers acquired during the month are

excluded from the denominator by design; revenue churn (MRR lost) uses the same population but sums MRR instead of counting. Pitfall: dividing by the closing count or including new customers understates churn.

A173. Company: an e-commerce site running an A/B test on a new checkout page. Table assignments(user_id, variant ('A','B'), assigned_at), orders(o...

Step 1, take each user once (assignment is the denominator, not orders).

Step 2, left join orders occurring after assigned_at (and maybe within the test window) so non-converters stay in the denominator.

Step 3, aggregate by variant.

```
WITH user_outcome AS (
  SELECT a.user_id, a.variant,
         COUNT(o.order_id) AS orders,
         COALESCE(SUM(o.amount), 0) AS revenue
  FROM assignments a
  LEFT JOIN orders o
    ON o.user_id = a.user_id AND o.order_ts >= a.assigned_at
   AND o.order_ts < a.assigned_at + INTERVAL '14 days'
  GROUP BY a.user_id, a.variant
)
SELECT variant,
       COUNT(*) AS users,
       ROUND(100.0 * AVG(CASE WHEN orders > 0 THEN 1.0 ELSE 0 END), 2) AS conversion_pct,
       ROUND(SUM(revenue) / NULLIF(SUM(orders), 0), 2) AS avg_order_value,
       ROUND(AVG(revenue), 2) AS revenue_per_user
FROM user_outcome
GROUP BY variant;
```

Additional checks before concluding: sample ratio mismatch (are group sizes close to the intended split, for example via a chi-square test), users appearing in both variants, outliers in revenue (cap or use medians), novelty effects over time, and statistical significance (a two-proportion z-test for conversion, a t-test or bootstrap for revenue per user). SQL provides the counts, the significance test is run in Python or R.

A174. Company: a supermarket chain. The merchandising team wants to know which pairs of products are most often bought together in the same basket...

Step 1, make sure each (order, product) appears once.

Step 2, self join on order_id with product_a < product_b to produce each unordered pair once.

Step 3, count baskets per pair, divide by total baskets for support.

Stretch: lift = support(A,B) / (support(A) * support(B)).

```
WITH items AS (SELECT DISTINCT order_id, product_id FROM order_items),
totals AS (SELECT COUNT(DISTINCT order_id) AS n_orders FROM items),
single AS (SELECT product_id, COUNT(*) AS n FROM items GROUP BY product_id),
pairs AS (
  SELECT a.product_id AS p1, b.product_id AS p2, COUNT(*) AS together
  FROM items a
  JOIN items b ON a.order_id = b.order_id AND a.product_id < b.product_id
  GROUP BY 1, 2
)
```

```
SELECT p.p1, p.p2, p.together,
       ROUND(100.0 * p.together / t.n_orders, 3) AS support_pct,
       ROUND((p.together::numeric / t.n_orders) /
             ((s1.n::numeric / t.n_orders) * (s2.n::numeric / t.n_orders)), 2) AS lift
FROM pairs p
CROSS JOIN totals t
JOIN single s1 ON s1.product_id = p.p1
JOIN single s2 ON s2.product_id = p.p2
ORDER BY p.together DESC
LIMIT 10;
```

Performance note: pair generation is quadratic in basket size, so for huge data filter to items with minimum support first, and sample or cap giant baskets. Lift above 1 means the pair co-occurs more than chance; very popular items inflate raw counts, which is why lift is more informative than counts.

A175. Company: a social video app. Compute daily active users (DAU), monthly active users (MAU) over a trailing 30 days, and the stickiness ratio ...

Step 1, distinct (user_id, activity_date) pairs.

Step 2, DAU per date is a simple distinct count.

Step 3, MAU for date D is the distinct users active in [D-29, D], requiring a range join (distinct counts cannot be windowed).

Step 4, ratio.

```
WITH a AS (SELECT DISTINCT user_id, activity_date FROM activity),
dau AS (SELECT activity_date AS dt, COUNT(*) AS dau FROM a GROUP BY 1),
mau AS (
  SELECT d.dt, COUNT(DISTINCT a.user_id) AS mau
  FROM dau d
  JOIN a ON a.activity_date BETWEEN d.dt - 29 AND d.dt
  GROUP BY d.dt
)
SELECT d.dt, d.dau, m.mau, ROUND(100.0 * d.dau / m.mau, 1) AS stickiness_pct
FROM dau d JOIN mau m USING (dt)
ORDER BY d.dt;
```

Interpretation: a ratio near 20 percent is typical for many apps; social apps aim for 50 percent or higher. Limits: the early 29 days lack a full window, so exclude them; the range join is heavy on big data, so aggregate to user-day first, restrict date ranges, or use approximate distinct counts (HLL) in a warehouse. Report weekday-adjusted trends since DAU has weekly seasonality.

A176. Company: a lender. Risk wants a delinquency report: for each monthly loan origination cohort, the percentage of loans that were ever 30+ day...

Step 1, define delinquency: an installment is 30+ days late if paid_date is NULL and the as-of date is more than 30 days past due_date, or paid_date - due_date > 30.

Step 2, restrict installments to the first 12 months after origination.

Step 3, flag loans with at least one late installment.

Step 4, cohort the loans, and only include loans that have had 12 months to season (or report observed months).

```
WITH inst AS (
  SELECT l.loan_id,
         DATE_TRUNC('month', l.origination_date)::date AS cohort,
         CASE WHEN COALESCE(p.paid_date, DATE '2025-01-31') - p.due_date > 30 THEN 1 ELSE 0
END AS late30
  FROM loans l
  JOIN payments_due p ON p.loan_id = l.loan_id
  WHERE p.due_date < l.origination_date + INTERVAL '12 months'
    AND l.origination_date <= DATE '2025-01-31' - INTERVAL '12 months'
), per_loan AS (
  SELECT loan_id, cohort, MAX(late30) AS ever_30dpd FROM inst GROUP BY loan_id, cohort
)
SELECT cohort, COUNT(*) AS loans, SUM(ever_30dpd) AS delinquent_loans,
       ROUND(100.0 * SUM(ever_30dpd) / COUNT(*), 2) AS ever_30dpd_pct
FROM per_loan
GROUP BY cohort
ORDER BY cohort;
```

Here 2025-01-31 is the as-of date used to treat unpaid installments. Alternatives: weight by balance (dollar delinquency) rather than count, compute vintage curves by months on book, and pay attention to right-censoring (young cohorts have not had time to default). Joining loans to payments_due before aggregating requires MAX at loan level to avoid counting a loan several times.

A177. Company: a food delivery platform. Operations wants delivery-time statistics per city for the last 30 days: average, median, 90th percentile...

Step 1, compute delivery minutes = delivered_ts - ordered_ts for completed orders in the window.

Step 2, use ordered-set aggregate functions for percentiles.

Step 3, conditional aggregation for the late rate.

```
SELECT city,
       COUNT(*) AS deliveries,
       ROUND(AVG(mins)::numeric, 1) AS avg_min,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY mins) AS median_min,
       PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY mins) AS p90_min,
       ROUND(100.0 * AVG(CASE WHEN mins > 45 THEN 1.0 ELSE 0 END), 1) AS pct_over_45
FROM (
  SELECT city, EXTRACT(EPOCH FROM (delivered_ts - ordered_ts)) / 60.0 AS mins
  FROM deliveries
  WHERE status = 'delivered'
    AND ordered_ts >= CURRENT_DATE - INTERVAL '30 days'
    AND delivered_ts IS NOT NULL
) d
GROUP BY city
ORDER BY p90_min DESC;
```

Why not just the average: delivery times are right-skewed, so a few extreme delays inflate the mean while most customers see something near the median; the 90th percentile captures the tail experience that causes complaints and churn. Also check data quality: negative durations, timestamps in different time zones, and

cancelled orders that carry delivered_ts. In MySQL, which lacks PERCENTILE_CONT, use a ROW_NUMBER/PERCENT_RANK approach.

A178. Company: a ride-hailing app. Growth wants to know what share of newly registered riders take their first ride within 7 days of signup, and t...

Step 1, restrict rides to completed ones (or whichever definition is agreed), take the first ride per rider.

Step 2, left join to riders so non-riders remain in the denominator.

Step 3, compute hours between signup and first ride, flag within 7 days.

Step 4, aggregate by signup week; use a percentile for time to first ride.

```
WITH first_ride AS (
  SELECT rider_id, MIN(ride_ts) AS first_ts
  FROM rides WHERE status = 'completed' GROUP BY rider_id
), j AS (
  SELECT DATE_TRUNC('week', r.signup_ts)::date AS signup_week,
         r.rider_id,
         f.first_ts,
         EXTRACT(EPOCH FROM (f.first_ts - r.signup_ts)) / 3600.0 AS hours_to_first
  FROM riders r
  LEFT JOIN first_ride f ON f.rider_id = r.rider_id AND f.first_ts >= r.signup_ts
)
SELECT signup_week,
       COUNT(*) AS signups,
       ROUND(100.0 * COUNT(*) FILTER (WHERE hours_to_first <= 168) / COUNT(*), 1) AS
pct_ride_in_7d,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY hours_to_first)
         FILTER (WHERE hours_to_first <= 168) AS median_hours_to_first
FROM j
GROUP BY signup_week
ORDER BY signup_week;
```

Only include signup weeks that are at least 7 days old to avoid censoring bias. The median is computed among converters only, so label it clearly; a survival curve (cumulative percent by day) is a fuller view. Segment by acquisition channel and city to guide marketing spend.

A179. Company: an e-commerce firm. A key dashboard query takes 4 minutes: it computes monthly revenue by category using orders (500M rows), order_...

Step 1, reproduce and measure: run EXPLAIN (ANALYZE, BUFFERS) or the warehouse query profile; note scanned rows/bytes, join types, sorts, spills.

Step 2, check obvious query problems:

- Is the date filter sargable (order_date >= X AND order_date < Y, not YEAR(order_date)=...)?

- Is the filter applied before the join, so only the last 12 months of orders reach order_items?

- SELECT only needed columns (critical for columnar stores).

- Avoid DISTINCT or fan-out from joining extra one-to-many tables.

- Aggregate order_items to (order_id) or (product_id, month) before joining wide tables when possible.

Step 3, physical design:

- Index orders(order_date) and order_items(order_id) (or composite covering index) in a row store; in a warehouse partition or cluster orders by order_date and cluster order_items by order_id.

- Partition by month so the optimizer prunes 11/12 of the data.

- Update statistics so the planner picks a hash join with the small products table broadcast.

Step 4, precompute: build a daily summary table or materialized view revenue_by_day_category refreshed incrementally; the dashboard reads thousands of rows instead of billions.

Step 5, verify: compare totals between old and new versions, compare timings, document SLA, and monitor.

```
-- sketch of the summary-table approach
CREATE TABLE agg_revenue_daily AS
SELECT o.order_date, p.category, SUM(oi.qty * oi.unit_price) AS revenue
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id
WHERE o.order_date >= DATE '2023-01-01'
GROUP BY o.order_date, p.category;
```

Then the dashboard does SUM(revenue) grouped by DATE_TRUNC('month', order_date), category.

A180. Company: a healthcare analytics firm receiving daily patient-visit data loads. Before building reports, you must run a data quality audit on...

Step-by-step checklist with queries:

1. Row counts and freshness:

```
SELECT COUNT(*) AS rows, MIN(visit_date) AS first_day, MAX(visit_date) AS last_day FROM
visits;
```

2. Primary key uniqueness:

```
SELECT visit_id, COUNT(*) FROM visits GROUP BY visit_id HAVING COUNT(*) > 1;
```

3. Null rates per column:

```
SELECT 100.0 * COUNT(*) FILTER (WHERE patient_id IS NULL) / COUNT(*) AS pct_null_patient,
       100.0 * COUNT(*) FILTER (WHERE charge IS NULL) / COUNT(*) AS pct_null_charge
FROM visits;
```

4. Orphans (referential integrity):

```
SELECT v.visit_id FROM visits v LEFT JOIN patients p USING (patient_id) WHERE p.patient_id IS
NULL;
```

5. Domain and range violations: negative or absurd charges, visit_date in the future, dob after visit date, patient age over 120.

```
SELECT * FROM visits v JOIN patients p USING (patient_id)
WHERE v.visit_date > CURRENT_DATE OR v.charge < 0 OR p.dob > v.visit_date;
```

6. Volume anomalies: daily counts vs trailing average to catch missing loads (a day with zero rows or a 50 percent drop).

7. Duplicate business records: same patient, doctor and date appearing more than once.

8. Distribution checks: min, max, percentiles of charge for outliers.

Findings are logged with counts; critical failures block downstream reports. Automate in a scheduled job or testing framework (for example dbt tests: unique, not_null, relationships, accepted_values).

A181. Company: a B2B subscription company. Finance wants a monthly MRR bridge: starting MRR, new, expansion, contraction, churn, and ending MRR. T...

Idea: build a customer x month grid so that churned and reactivated customers are explicit, then compare each month with the previous one.

Step 1, grid = every customer crossed with every month from their first month to the last month in the data, left joined to the snapshot with COALESCE(mrr, 0).

Step 2, LAG gives previous month MRR per customer.

Step 3, classify the change:

- new: prev = 0 and current > 0

- churn: prev > 0 and current = 0

- expansion: both > 0 and current > prev

- contraction: both > 0 and current < prev

```
WITH months AS (SELECT DISTINCT month FROM mrr_snapshot),
first_m AS (SELECT customer_id, MIN(month) AS first_month FROM mrr_snapshot GROUP BY
customer_id),
grid AS (
  SELECT f.customer_id, m.month, COALESCE(s.mrr, 0) AS mrr
  FROM first_m f
  JOIN months m ON m.month >= f.first_month
  LEFT JOIN mrr_snapshot s ON s.customer_id = f.customer_id AND s.month = m.month
), chg AS (
  SELECT customer_id, month, mrr AS cur_mrr,
         COALESCE(LAG(mrr) OVER (PARTITION BY customer_id ORDER BY month), 0) AS prev_mrr
  FROM grid
)
SELECT month,
       SUM(prev_mrr) AS starting_mrr,
       SUM(CASE WHEN prev_mrr = 0 AND cur_mrr > 0 THEN cur_mrr ELSE 0 END) AS new_mrr,
       SUM(CASE WHEN prev_mrr > 0 AND cur_mrr > prev_mrr THEN cur_mrr - prev_mrr ELSE 0 END)
AS expansion,
       SUM(CASE WHEN prev_mrr > 0 AND cur_mrr > 0 AND cur_mrr < prev_mrr THEN cur_mrr -
prev_mrr ELSE 0 END) AS contraction,
       SUM(CASE WHEN prev_mrr > 0 AND cur_mrr = 0 THEN -prev_mrr ELSE 0 END) AS churn,
       SUM(cur_mrr) AS ending_mrr
FROM chg
GROUP BY month
ORDER BY month;
```

Reconciliation check: starting + new + expansion + contraction + churn must equal ending for every month. The first month has starting MRR of zero, so everything appears as new; treat it as the opening balance.

Reactivations (a customer who returns after a gap) land in new here; some teams report them as a separate 'reactivation' bucket by checking whether the customer had earlier MRR.

A182. Company: a hotel booking platform. Detect double bookings: any two reservations for the same room whose date ranges overlap. Table: reservat...

Two half-open ranges [a_in, a_out) and [b_in, b_out) overlap if and only if a_in < b_out AND b_in < a_out. Same-day turnover (one checks out the day another checks in) is not an overlap with this half-open convention.

```
SELECT a.res_id AS res_a, b.res_id AS res_b, a.room_id,
       GREATEST(a.check_in, b.check_in) AS overlap_start,
       LEAST(a.check_out, b.check_out)  AS overlap_end
FROM reservations a
JOIN reservations b
  ON a.room_id = b.room_id
 AND a.res_id < b.res_id
 AND a.check_in  < b.check_out
 AND b.check_in  < a.check_out
ORDER BY a.room_id, overlap_start;
```

The condition a.res_id < b.res_id returns each pair once and avoids comparing a booking with itself. Reasoning for correctness: non-overlap happens only when one range ends before or when the other starts, so the negation is the AND of the two inequalities. Performance: index on (room_id, check_in); Postgres can enforce no-overlap proactively with an EXCLUDE constraint on daterange (using gist). Inclusive end dates change the comparisons to <=.

A183. Company: an online electronics store. The CEO asks about customer lifetime value: for each customer's first-purchase month cohort, what is t...

Step 1, cohort month per customer = month of first order, cohort size = distinct customers.

Step 2, for each order compute months since first order.

Step 3, cumulative revenue at offset N = revenue from orders with offset <= N, divided by cohort size (so customers with zero revenue in later months are still counted).

Step 4, repeat purchase rate = share of customers with at least 2 orders.

```
WITH f AS (
  SELECT customer_id, MIN(order_date) AS first_date,
         DATE_TRUNC('month', MIN(order_date))::date AS cohort
  FROM orders GROUP BY customer_id
), o AS (
  SELECT f.cohort, f.customer_id, od.amount,
         (DATE_PART('year', od.order_date) - DATE_PART('year', f.first_date)) * 12
         + (DATE_PART('month', od.order_date) - DATE_PART('month', f.first_date)) AS m
  FROM f JOIN orders od ON od.customer_id = f.customer_id
), size AS (SELECT cohort, COUNT(*) AS n FROM f GROUP BY cohort)
SELECT s.cohort, s.n AS customers,
       ROUND(SUM(CASE WHEN o.m <= 1  THEN o.amount END) / s.n, 2) AS ltv_1m,
       ROUND(SUM(CASE WHEN o.m <= 3  THEN o.amount END) / s.n, 2) AS ltv_3m,
       ROUND(SUM(CASE WHEN o.m <= 6  THEN o.amount END) / s.n, 2) AS ltv_6m,
       ROUND(SUM(CASE WHEN o.m <= 12 THEN o.amount END) / s.n, 2) AS ltv_12m
```

```
FROM size s JOIN o USING (cohort)
GROUP BY s.cohort, s.n
ORDER BY s.cohort;
```

Repeat rate:

```
SELECT ROUND(100.0 * COUNT(*) FILTER (WHERE c >= 2) / COUNT(*), 1)
FROM (SELECT customer_id, COUNT(*) c FROM orders GROUP BY customer_id) t;
```

Caveat: cohorts younger than 12 months have incomplete 12-month values; show NULL for horizons not yet reached. Use gross margin instead of revenue for a true LTV, and compare to customer acquisition cost.

A184. Company: a streaming service with subscription plans whose prices changed over time. Table plan_prices(plan_id, price, valid_from DATE, vali...

This is a point-in-time (as-of) join to a slowly changing dimension (type 2): join on the key and require invoice_date to fall inside [valid_from, valid_to).

```
SELECT i.invoice_id, i.customer_id, i.plan_id, i.invoice_date,
       i.billed_amount, p.price AS expected_price,
       i.billed_amount - p.price AS diff
FROM invoices i
LEFT JOIN plan_prices p
  ON p.plan_id = i.plan_id
 AND i.invoice_date >= p.valid_from
 AND (p.valid_to IS NULL OR i.invoice_date < p.valid_to)
WHERE p.price IS NULL OR i.billed_amount <> p.price;
```

Rows with p.price IS NULL mean no valid price existed (data gap). Summary:

```
SELECT SUM(CASE WHEN diff > 0 THEN diff END) AS total_overbilled,
       SUM(CASE WHEN diff < 0 THEN diff END) AS total_underbilled,
       COUNT(*) AS bad_invoices
FROM (the query above without the WHERE, filtered to diff <> 0) x;
```

Validation of the price table matters: overlapping validity periods for one plan would duplicate invoice rows (fan-out), so first test that no two rows for a plan overlap. Alternative: use LEAD(valid_from) to derive valid_to when the column is not stored. Report results by plan and month, and flag them for the billing team.

A185. Company: a marketplace with a CRM containing duplicate customer records. customers(customer_id, email, phone, full_name, created_at). The te...

Step 1, normalize match keys: LOWER(TRIM(email)), digits-only phone.

Step 2, build duplicate groups by each key.

Step 3, choose the master: earliest created_at (tie by smallest id).

Step 4, produce the mapping; review manually for ambiguous cases.

Email-based version:

```
WITH norm AS (
  SELECT customer_id, created_at,
         LOWER(TRIM(email)) AS email_key
```

```
  FROM customers
  WHERE email IS NOT NULL AND TRIM(email) <> ''
), ranked AS (
  SELECT customer_id, email_key,
         FIRST_VALUE(customer_id) OVER (PARTITION BY email_key
                                        ORDER BY created_at, customer_id) AS master_id,
         COUNT(*) OVER (PARTITION BY email_key) AS group_size
  FROM norm
)
SELECT customer_id AS old_id, master_id
FROM ranked
WHERE group_size > 1 AND customer_id <> master_id;
```

Phone-based duplicates repeat the pattern with REGEXP_REPLACE(phone, '[^0-9]', '', 'g'). Records matching on email in one group and phone in another form chains (A-B by email, B-C by phone); handling transitive groups needs connected components, usually done with a recursive CTE or in Python/graph tools. Keep the mapping table, update dependent tables (orders) via the mapping, and avoid deleting the source until validated. Fuzzy name matching (trigram similarity, edit distance) is a later pass with human review.

A186. Company: an e-commerce firm using last-touch marketing attribution. Tables: sessions(session_id, user_id, session_ts, channel) and purchases...

Step 1, for each purchase, find sessions of the same user that happened before it and within 7 days.

Step 2, pick the latest one per purchase (last touch) with ROW_NUMBER, or the earliest (first touch).

Step 3, sum revenue per channel; purchases with no qualifying session go to 'direct/unknown'.

```
WITH cand AS (
  SELECT p.purchase_id, p.amount, s.channel,
         ROW_NUMBER() OVER (PARTITION BY p.purchase_id ORDER BY s.session_ts DESC) AS
rn_last,
         ROW_NUMBER() OVER (PARTITION BY p.purchase_id ORDER BY s.session_ts ASC)  AS
rn_first
  FROM purchases p
  JOIN sessions s
    ON s.user_id = p.user_id
   AND s.session_ts <= p.purchase_ts
   AND s.session_ts >= p.purchase_ts - INTERVAL '7 days'
)
SELECT COALESCE(c.channel, 'direct/unknown') AS channel,
       SUM(CASE WHEN c.rn_last = 1 THEN c.amount END)  AS last_touch_revenue,
       SUM(CASE WHEN c.rn_first = 1 THEN c.amount END) AS first_touch_revenue
FROM cand c
GROUP BY 1
ORDER BY last_touch_revenue DESC NULLS LAST;
```

Include a LEFT JOIN from purchases (with unmatched ones labelled direct) to avoid losing revenue; reconcile that attributed revenue totals equal total purchase revenue. Compare models: last-touch favors bottom-funnel channels (search, email), first-touch favors discovery (social, display); linear or time-decay models split credit and are better for planning budgets.

A187. Company: a ride-hailing company. Marketplace analytics wants, for each city and hour of day, the supply-demand ratio (available drivers per ...

Step 1, convert the event log (online/offline toggles) into intervals: for each driver, an 'online' event lasts until the next event (LEAD). The online interval is [status_ts, next_ts).

Step 2, for each hour bucket, compute how many driver-minutes of overlap each interval has, or count drivers online at some point in the hour; driver-hours = SUM(overlap minutes)/60.

Step 3, count requests per city-hour.

Step 4, ratio and rank worst.

```
WITH intervals AS (
  SELECT driver_id, city_id, status_ts AS start_ts,
         COALESCE(LEAD(status_ts) OVER (PARTITION BY driver_id ORDER BY status_ts),
                  NOW()) AS end_ts,
         status
  FROM driver_status_log
), online AS (
  SELECT driver_id, city_id, start_ts, end_ts FROM intervals WHERE status = 'online'
), hours AS (
  SELECT h AS hour_start FROM generate_series(DATE_TRUNC('hour', NOW()) - INTERVAL '7 days',
                                              DATE_TRUNC('hour', NOW()), INTERVAL '1 hour') h
), supply AS (
  SELECT o.city_id, hr.hour_start,
         SUM(EXTRACT(EPOCH FROM (LEAST(o.end_ts, hr.hour_start + INTERVAL '1 hour')
                                 - GREATEST(o.start_ts, hr.hour_start))) / 3600.0) AS
driver_hours
  FROM online o
  JOIN hours hr ON o.start_ts < hr.hour_start + INTERVAL '1 hour' AND o.end_ts >
hr.hour_start
  GROUP BY 1, 2
), demand AS (
  SELECT city_id, DATE_TRUNC('hour', request_ts) AS hour_start, COUNT(*) AS requests
  FROM ride_requests
  WHERE request_ts >= NOW() - INTERVAL '7 days'
  GROUP BY 1, 2
)
SELECT d.city_id, d.hour_start, d.requests, COALESCE(s.driver_hours, 0) AS driver_hours,
       COALESCE(s.driver_hours, 0) / d.requests AS drivers_per_request
FROM demand d LEFT JOIN supply s USING (city_id, hour_start)
WHERE d.requests >= 20
ORDER BY drivers_per_request ASC
LIMIT 5;
```

Notes: the interval-overlap formula (least of ends minus greatest of starts, with a positive-overlap join condition) is the key technique; treat hours with tiny request counts carefully (the minimum request filter); consider driver_hours as an average count of online drivers in that hour. Decisions: surge pricing, incentives, driver notifications in under-supplied zones.

A188. Company: a retail chain. A regional manager asks for the sales report 'total sales per store for each quarter of 2024, with the store's rank...

Step 1, aggregate sales to store and quarter.

Step 2, join store to region.

Step 3, rank within region and quarter with a window function over the aggregated rows.

```
WITH q AS (
  SELECT store_id, DATE_TRUNC('quarter', sale_date)::date AS quarter_start,
         SUM(amount) AS total_sales
  FROM sales
  WHERE sale_date >= DATE '2024-01-01' AND sale_date < DATE '2025-01-01'
  GROUP BY store_id, DATE_TRUNC('quarter', sale_date)
)
SELECT s.region, q.quarter_start, q.store_id, q.total_sales,
       RANK() OVER (PARTITION BY s.region, q.quarter_start ORDER BY q.total_sales DESC) AS
region_rank
FROM q JOIN stores s USING (store_id)
ORDER BY s.region, q.quarter_start, region_rank;
```

Stores with no sales in a quarter are missing; to show zero, cross join stores with a quarters list and LEFT JOIN the aggregate. Mention that RANK leaves gaps after ties, which is acceptable for sports-style ranking.

A189. Company: a streaming platform. Analysts notice that a viewing-hours report shows totals 3 times higher than finance's number after someone a...

Diagnosis: the join is on user_id only, so each viewing row matches every subscription row of that user. A user with 3 subscription rows has each viewing row repeated 3 times: classic fan-out from a one-to-many join, so the SUM is inflated.

Verification steps:

1. Compare COUNT(*) of viewing with COUNT(*) of the joined result; they should be equal when the relationship is many-to-one.

2. Check cardinality: SELECT user_id, COUNT(*) FROM subscriptions GROUP BY user_id HAVING COUNT(*) > 1.

Fix, choosing the grain deliberately. Option 1: point-in-time join so a viewing row matches only the plan active on that date:

```
SELECT s.plan, SUM(v.minutes) AS minutes
FROM viewing v
JOIN subscriptions s
  ON s.user_id = v.user_id
 AND v.view_date >= s.start_date
 AND (s.end_date IS NULL OR v.view_date < s.end_date)
GROUP BY s.plan;
```

This is correct only if subscription periods do not overlap for a user (test it). Option 2: if you just need the total, avoid the join; use SELECT SUM(minutes) FROM viewing. Option 3: reduce subscriptions to one row per user (for example the latest plan) before joining.

Guardrails: assert row counts after joins, add data tests for unique keys on the dimension, and reconcile each report's totals against a source-of-truth control number.

A190. Company: a payments fintech. The finance team needs a daily reconciliation between internal ledger entries and bank settlement files. Tables...

Use a FULL OUTER JOIN on the transaction id, aggregating each side first so duplicates in either side do not create wrong matches.

```
WITH l AS (SELECT txn_id, SUM(amount) AS ledger_amt, COUNT(*) AS ledger_rows FROM ledger
GROUP BY txn_id),
     b AS (SELECT txn_id, SUM(amount) AS bank_amt,   COUNT(*) AS bank_rows   FROM
bank_settlements GROUP BY txn_id)
SELECT COALESCE(l.txn_id, b.txn_id) AS txn_id,
       l.ledger_amt, b.bank_amt,
       CASE WHEN b.txn_id IS NULL THEN 'missing_in_bank'
            WHEN l.txn_id IS NULL THEN 'missing_in_ledger'
            WHEN l.ledger_amt <> b.bank_amt THEN 'amount_mismatch'
            WHEN l.ledger_rows > 1 OR b.bank_rows > 1 THEN 'duplicate_check'
            ELSE 'matched' END AS status
FROM l FULL OUTER JOIN b ON l.txn_id = b.txn_id;
```

Summary counts and value:

```
SELECT status, COUNT(*) AS n, SUM(COALESCE(ledger_amt, 0) - COALESCE(bank_amt, 0)) AS
net_difference
FROM (the query above) r GROUP BY status;
```

Notes: compare in minor currency units or numeric types, never floating point; allow for settlement lag (a ledger entry from today may legitimately settle tomorrow, so compare on a lagged date window); fees deducted by the bank cause systematic small mismatches that need a tolerance or fee table. MySQL lacks FULL OUTER JOIN, so UNION a LEFT and RIGHT join.

Power BI

A191. What are the main components of Power BI and how do they fit together in a typical workflow?

Power BI is a suite of tools:

- Power BI Desktop: the free Windows authoring tool. You connect to data, transform it with Power Query, build the data model (tables, relationships), write DAX measures and design report pages.

- Power Query (Get Data / Transform Data): the ETL layer inside Desktop (and dataflows) that uses the M language.

- Data model (VertiPaq/Analysis Services engine): columnar in-memory storage where tables, relationships, measures and security roles live.

- Power BI Service (app.powerbi.com): the cloud portal where reports are published to workspaces, shared, refreshed on a schedule, secured with RLS, and organized into apps and dashboards.

- Power BI Mobile: apps for viewing reports and dashboards on phones/tablets.

- On-premises data gateway: a bridge so the Service can refresh or query on-premises sources.

- Report Server (Report Server edition): on-premises hosting for organizations that cannot use the cloud.

- Other items: dataflows, paginated reports (Report Builder), deployment pipelines, and Power BI Embedded.

Typical workflow: connect and shape data in Power Query, build a star-schema model, write measures in DAX, design visuals, publish to the Service, configure gateway and scheduled refresh, set RLS, then share through an app, and monitor usage.

A192. Explain Import, DirectQuery, Live Connection and Composite models. When would you choose each?

Import: data is copied and compressed into the in-memory VertiPaq engine. Very fast visuals, full DAX and Power Query support, but data is only as fresh as the last refresh and size is limited (1 GB compressed per dataset on Pro, much larger on Premium/Fabric capacity). Default choice.

DirectQuery: only metadata is stored; each visual sends a query to the source database at run time. Data is near real time and size is not limited by the model, but performance depends on the source, there are restrictions on some DAX and Power Query transformations, and each visual issues SQL, so a well-indexed source is needed. Choose it when data volume is too large to import, when data must be real time, or when governance forbids copying the data.

Live Connection: connects to an existing Analysis Services model or a published Power BI semantic model. You cannot change the model in the report file (no new tables or relationships; report-level measures only), which supports a single governed shared model.

Composite model: mixes Import and DirectQuery tables (and other DirectQuery sources, or chained semantic models) in one model. Storage modes per table can be Import, DirectQuery or Dual. Typical use: large fact table in DirectQuery, small dimensions in Dual or Import, plus aggregations for speed.

Rule of thumb: start with Import; move to DirectQuery or composite only when freshness or volume requires it.

A193. What is a star schema and why is it the recommended modeling approach in Power BI?

A star schema has one or more central fact tables (transactions or events with numeric measures and foreign keys, for example Sales) surrounded by dimension tables (descriptive attributes such as Product, Customer, Date, Store). Dimensions connect to the fact on a one-to-many relationship, with the filter flowing from dimension to fact.

Why it is recommended:

- Performance: VertiPaq compresses narrow fact tables and low-cardinality dimension columns very well; fewer wide joins.

- Simple, predictable filter propagation: slicers on dimensions filter the fact in one direction, avoiding ambiguity.

- DAX is easier and measures behave correctly, since filters on dimension attributes work consistently across measures.

- Usability: business users see clean, labeled dimension tables.

- Reuse: conformed dimensions (one Date, one Product) are shared across several fact tables.

Contrast with a flat, denormalized table (huge, repeats text, slow, inconsistent) and a snowflake (normalized dimensions, more relationships, more complexity). In Power BI, flatten snowflakes into single dimensions in Power Query where practical.

A194. Describe relationship cardinality and cross-filter direction in Power BI. What problems does bidirectional filtering cause?

Cardinality types: one-to-many (standard; the one side has unique key values), many-to-one (same, reversed), one-to-one (both sides unique; usually better merged into one table), and many-to-many (neither side unique; allowed since 2018 but should be used sparingly).

Cross-filter direction: Single means filters flow from the one side to the many side (dimension to fact). Both (bidirectional) lets filters flow in either direction.

Only one relationship between two tables can be active; others are inactive and activated in DAX with USERELATIONSHIP.

Problems with bidirectional filtering:

- Ambiguity: when several paths exist between tables, results become unpredictable or Power BI refuses to create the relationship.

- Performance: extra filter propagation costs time, especially on large models.

- Unexpected results: a slicer on one dimension can filter another dimension, producing counts that confuse users.

Best practice: use single direction by default; use bidirectional only when really needed (for example a bridge table in a many-to-many pattern, or a slicer that should show only values related to the filtered fact), or prefer DAX approaches such as CROSSFILTER inside CALCULATE for a single measure, or apply a visual-level filter instead.

A195. What is the difference between a calculated column and a measure? When do you use each?

Calculated column: a DAX expression evaluated row by row (row context) when the model is loaded or refreshed. The result is stored in the table and consumes memory. It can be used on rows, columns, slicers, filters and in relationships.

Measure: a DAX expression evaluated at query time using the filter context of the visual (rows, columns, slicers, filters). Nothing is stored per row, it is recalculated when the report is interacted with, so it adds almost no memory.

Use a calculated column when you need a value to slice or filter by (for example Age Group, Profit Category) or as part of a relationship or sort key. Use a measure for aggregations that must respond to the user's selections (total sales, margin %, YoY growth, distinct count).

Rules of thumb: never sum a calculated column when a measure over the base columns would do; do row-level cleanup in Power Query (or the source) when possible because it compresses better and folds to the source; ratios must be measures (divide totals, not average the row ratios).

A196. Explain row context and filter context in DAX and how they interact.

Filter context is the set of filters applied to the model when a measure is evaluated. It comes from visual axes/rows/columns, slicers, page/report/visual filters, RLS, and filter arguments of CALCULATE. It determines which rows of each table are visible for an aggregation. Example: in a matrix with Year on rows, SUM(Sales[Amount]) in the 2023 row is evaluated with filter context Year = 2023.

Row context is the notion of the current row. It exists in calculated columns and inside iterator functions (SUMX, AVERAGEX, FILTER, ADDCOLUMNS, etc.). It lets you reference columns of the current row, for example Sales[Qty] * Sales[Price].

Key facts:

- Row context does not filter other tables or apply to related rows automatically; it only enables column references in that row. RELATED and RELATEDTABLE follow relationships from the current row.

- Row context does not become filter context by itself. Context transition turns row context into filter context when a measure or CALCULATE is evaluated inside a row context.

- Aggregations like SUM do not create row context; they use the existing filter context.

Interaction example: SUMX(Product, [Total Sales]) iterates over products (row context per product); the measure [Total Sales] contains an implicit CALCULATE, so each product's row context is converted to a filter on that product.

A197. What is context transition in DAX? Give an example where it happens and a pitfall.

Context transition is the conversion of the current row context into an equivalent filter context. It happens when CALCULATE (or CALCULATETABLE) is evaluated inside a row context, and also whenever a measure is called, because a measure reference is implicitly wrapped in CALCULATE.

Mechanism: for each column of the current row (of the table iterated), CALCULATE adds a filter equal to that row's value. Example, in a calculated column on Customer:

```
Customer Sales = CALCULATE ( SUM ( Sales[Amount] ) )
```

or simply = [Total Sales]. For each customer row, the row context is converted to a filter Customer[CustomerKey] (and all other columns) = current values, and the SUM is computed only for sales of that customer through the relationship.

In an iterator:

```
Avg Sales per Customer =
AVERAGEX ( Customer, [Total Sales] )
```

Pitfalls:

- If the iterated table has duplicate rows (no unique key), context transition filters on all columns of the row, and duplicates produce double counting or surprising values, because all duplicates match the same filter.

- Performance: calling a measure inside an iterator over a large table causes a context transition per row, which can be slow. Prefer iterating over a small table (for example a dimension or VALUES of a column).

- Calling measures in calculated columns over large tables is expensive and often avoidable.

A198. When should you do a transformation in Power Query versus in DAX?

Power Query (M) is for preparing data before it enters the model: cleaning, type conversion, removing columns/rows, splitting and merging columns, unpivoting, appending and merging queries, deduplicating, filtering, and creating dimension tables. It runs at refresh, can fold back to the source (so the database does the work) and reduces model size.

DAX is for analytics inside the model: measures that respond to user selections (ratios, YTD, rankings, time intelligence), dynamic calculations dependent on filter context, calculated tables for specific modeling needs, and RLS expressions.

Guidelines:

- Push work as far upstream as possible: source/SQL view, then Power Query, then DAX.

- Use Power Query for static, row-level shaping (a Full Name column); use DAX measures for anything that must respond to slicers.

- Avoid calculated columns when the same logic can be done in Power Query, because query-time columns compress better and, with folding, run on the server.

- Reshaping (unpivot, pivot) is a Power Query job; running totals and percent of total are DAX jobs.

A199. What is query folding in Power Query, how can you check whether it is happening, and what breaks it?

Query folding means Power Query translates your applied steps into a single native query (for example SQL) executed by the data source, instead of pulling all data into the mashup engine and transforming locally. Benefits: less data transferred, faster refresh, enables incremental refresh and DirectQuery-compatible transformations.

How to check: right-click a step in Applied Steps and see whether View Native Query is enabled (if greyed out, the step does not fold, and typically all later steps do not either). You can also use Query Diagnostics, or the step indicator icons in newer versions, or check the source's query log.

Common folding breakers: adding an Index column, custom M functions that cannot be translated, merging queries from different sources, Table.Buffer, some pivot/unpivot variants, certain data type changes or text operations, using Excel/CSV/JSON files (no folding as they are not query engines), referencing steps after a non-foldable step.

Best practices: put foldable steps (filters, column selection, joins, grouping, type changes) first, non-foldable steps last; use database views or native SQL with Value.NativeQuery carefully (folding of later steps is limited); keep staging queries and reference them instead of duplicating; disable load on intermediate queries.

A200. What is the difference between Merge and Append in Power Query?

Append stacks tables vertically: rows from two or more tables with (ideally) the same columns are combined into one longer table, like SQL UNION ALL. Use it to combine monthly files or regional tables. Columns are matched by name; missing columns become null.

Merge joins tables horizontally on matching key columns, like a SQL JOIN. You choose the join kind: Left Outer (default), Right Outer, Full Outer, Inner, Left Anti, Right Anti. The result adds a column holding a nested table that you expand to pick the columns you want.

Examples: Append 12 monthly sales extracts into Sales; Merge Sales with a Product lookup to bring in the Category column (though in a star schema you would usually keep the lookup as its own dimension and relate it instead of merging).

Tips: ensure join key data types match and trim/clean text keys; use anti joins to find unmatched records; merging on non-folding sources is slow on large data.

A201. Explain Row-Level Security in Power BI: static vs dynamic RLS, how roles are defined, and how they are tested and assigned.

RLS restricts the rows a user can see, by DAX filter expressions on tables in the model, defined per role.

Static RLS: a role has a hard-coded rule, for example [Region] = "West" on the Region table. You create one role per region and assign users or groups to it. Simple, but hard to maintain.

Dynamic RLS: a single role uses the signed-in identity, via USERPRINCIPALNAME() (or USERNAME()), and a security mapping table (UserEmail, Region) related to the dimension. Example rule on the security table: [UserEmail] = USERPRINCIPALNAME(). Filter propagates through relationships (which must filter in the direction toward the data; for dimension-to-fact use single direction and, if the security table is on the many side or reaches the fact via a chain, use bidirectional filtering with "Apply security filter in both directions" enabled).

Steps: In Desktop, Modeling > Manage roles, define the role and DAX filters, use View as to test with a user. After publishing, in the Service open the semantic model > Security and add users or Microsoft Entra security groups to the role.

Key points:

- RLS applies to Viewers. Workspace Admins, Members and Contributors (anyone with edit access) are not restricted. Use Viewer role or apps for end users.

- RLS works with Import and DirectQuery (in DirectQuery it can also pass identity to the source).

- Filters are applied before any measure evaluation; CALCULATE with ALL cannot remove them.

- Object-level and column-level security (OLS) hide tables/columns, configured with external tools such as Tabular Editor.

A202. What is an on-premises data gateway? Explain the types, when you need one, and common refresh problems.

A gateway is software installed on a machine inside your network that securely relays queries and refresh requests between Power BI Service (and Power Apps, Power Automate, Logic Apps) and on-premises data sources, using outbound connections via Azure Service Bus.

Types:

- Standard (enterprise) mode: shared by many users and services, supports Import refresh and DirectQuery/live connections, managed centrally with clustering for high availability.

- Personal mode: runs as the user, for Import refresh only, for one person, not shared.

- Virtual network (VNet) data gateway: a managed service for data sources inside an Azure virtual network, no installation required.

You need a gateway when the Service has to reach on-premises databases, files on network shares, or cloud sources inside a private network. Cloud sources reachable on the public internet (Azure SQL with a public endpoint, SharePoint Online) do not need one.

Common problems: gateway machine is offline or asleep, expired credentials, data source in the gateway not matching the dataset's source names exactly (server/database name differences), firewall blocking Service Bus ports, mismatched privacy levels, refresh timeouts (2 hours on shared capacity, 5 hours on Premium), source slow or locked, gateway version out of date, and different users mapped to the data source. Remedies: gateway clustering, monitoring, service account, correct data source mapping, incremental refresh.

A203. Explain incremental refresh: how it works, prerequisites and limitations.

Incremental refresh partitions a table by date so only recent partitions are refreshed, instead of reloading the entire table every time. It speeds up refresh, lowers load on the source and allows larger datasets.

Setup:

1. In Power Query create two parameters of type Date/Time named exactly RangeStart and RangeEnd.

2. Filter the date/datetime column of the fact table: >= RangeStart and < RangeEnd (one end inclusive only, to avoid duplicates).

3. Right-click the table > Incremental refresh, set "Archive data starting N years before refresh date" and "Incremental refresh data starting N days before refresh date". Optionally enable "Detect data changes" (uses a max-update-date column to refresh only changed partitions), and "Get the latest data in real time with DirectQuery" (Premium, hybrid table).

4. Publish; the Service creates partitions (for example yearly, monthly, daily) and the first refresh loads everything, later refreshes only the incremental window.

Prerequisites and limitations: the filter step should fold to the source (otherwise each refresh still reads all data, which defeats the purpose), the source must be reasonably able to filter by date; the date column should be immutable (for example OrderDate, not ModifiedDate unless detect changes is used); republishing from Desktop overwrites the semantic model and the partitions (use ALM Toolkit/XMLA for metadata-only deploys); Pro supports it for datasets up to its size limits, and XMLA endpoint features need Premium/Fabric capacity. Not usable with Live Connection.

A204. What are the main techniques to optimize the performance of a Power BI data model and report?

Data model:

- Use a star schema; avoid wide flat tables and unnecessary bidirectional relationships.

- Remove unused columns and tables; reduce cardinality (split datetime into Date and Time, round decimals, avoid GUIDs and long text keys).

- Use integer surrogate keys for relationships.

- Choose appropriate data types; disable Auto date/time and use a single Date table.

- Prefer measures over calculated columns; move row-level logic to the source or Power Query (with folding).

- Use incremental refresh and aggregation tables for big facts; Import where possible.

DAX:

- Use variables (VAR) to avoid recomputation; avoid iterators over big tables with context transition; use DIVIDE; filter columns rather than whole tables inside CALCULATE (Sales[Year] = 2023 instead of FILTER(Sales, ...)); avoid FILTER(ALL(table)) when a column filter will do.

Report design:

- Limit visuals per page (about 8 to 10); avoid too many slicers and high-cardinality tables; use "Apply" buttons for slicers and "Edit interactions" to stop unnecessary cross-filtering; use bookmarks/drill-through instead of everything on one page; avoid custom visuals that are slow; use report-level filters to cut data.

Tools: Performance Analyzer (visual timing), DAX Studio and VertiPaq Analyzer (query plans and memory), Tabular Editor Best Practice Analyzer, Power BI Desktop's Optimize ribbon.

DirectQuery: tune the source with indexes/views, reduce visuals, enable query reduction options.

A205. How do you diagnose a slow Power BI report using Performance Analyzer, DAX Studio and VertiPaq Analyzer? What do the numbers tell you?

Performance Analyzer (Desktop, View ribbon): start recording, refresh visuals or interact, and review per-visual durations split into DAX query, Visual display and Other. Copy the DAX query of a slow visual. If DAX query is high, the measure or model is the problem; if Visual display is high, the visual itself (too many data points, custom visual) is the issue; "Other" often means waiting for other visuals.

DAX Studio: paste the copied query, enable Server Timings and Query Plan, and run (clear cache first for cold-cache tests). Server Timings show Storage Engine (SE) vs Formula Engine (FE) time and number of SE queries. A high FE share suggests complex iterators or row-by-row logic; many SE queries or large "materialization" row counts suggest inefficient filtering or too much data. Callbacks (CallbackDataID) in the SE query indicate the SE asked the FE for logic and are slow.

VertiPaq Analyzer (inside DAX Studio, Advanced > View Metrics): shows table and column size, cardinality, encoding, dictionary size, relationships. Look for high-cardinality columns, unneeded columns eating memory, and huge calculated columns.

Typical fixes: remove or reduce cardinality of columns, rewrite measures with variables and column filters, avoid nested iterators, add aggregation tables, simplify the visual, fix relationships, and for DirectQuery review the SQL generated and the source indexes.

A206. What are role-playing dimensions and how do you handle a Date table related to Sales by both Order Date and Ship Date?

A role-playing dimension is one physical dimension used in several roles by a fact table. Example: one Date table used for Order Date, Ship Date and Delivery Date.

Power BI allows only one active relationship between two tables, so options are:

1. Keep one active relationship (for example Sales[OrderDate] to Date[Date]) and create the others as inactive relationships. Activate them in specific measures with USERELATIONSHIP:

```
Sales by Ship Date =
CALCULATE ( [Total Sales], USERELATIONSHIP ( Sales[ShipDate], 'Date'[Date] ) )
```

2. Create separate copies of the Date table (Order Date, Ship Date), each with its own active relationship. This lets users slice by either date independently and gives each its own field set, at the cost of extra tables (small anyway). Good for self-service reports where different slicers are needed at the same time.

Choose option 1 for a compact model with a few specific measures; option 2 when users need to freely choose which date to filter or show both side by side in visuals. Tip: mark the Date table as a date table and keep it contiguous, covering full years.

A207. How do you model a many-to-many relationship in Power BI? Describe the bridge table pattern and the native many-to-many option.

Scenario: Students and Courses, Products and Promotions, Customers and Segments: each entity relates to many of the other.

Bridge (junction) table pattern: create a table with the unique key pairs (StudentKey, CourseKey). Relate Student[Key] one-to-many to Bridge[StudentKey] and Course[Key] one-to-many to Bridge[CourseKey]. Facts relate to one of the dimensions. To let filters flow from the second dimension through the bridge to the fact,

set cross-filter direction appropriately (often both for the bridge-to-dimension relationship) or use CROSSFILTER in measures. This keeps keys unique and filter behavior explicit.

Native many-to-many relationship: since 2018 you can relate two tables on non-unique columns directly (cardinality many-to-many). It works for simple cases, but has limited semantics (no referential integrity, blank rows behavior, weak relationships that make RELATED unusable, and may produce confusing totals).

Best practice: prefer the bridge table with unique dimension keys; use native many-to-many cautiously, mainly when relating two facts at a shared grain via a conformed dimension rather than directly fact to fact. Watch totals: with many-to-many, summing allocations can double count, so decide on allocation rules (weights) if totals must reconcile.

A208. What types of filters exist in Power BI reports, and how do they differ from slicers?

Filter scopes in the Filters pane:

- Visual-level filter: affects one visual only.

- Page-level filter: affects all visuals on a page.

- Report-level filter: affects all pages.

- Drill-through filter: passes context from one page to a detail page when the user right-clicks a data point.

Filter types: basic, advanced (conditions such as contains, greater than, and/or), relative date/time (last 30 days), Top N, and "is not blank" style filters.

Slicers are visuals placed on the canvas; they are interactive for end users, visible and can be synced across pages (View > Sync slicers). Filters in the pane are less visible and are best used to define a report's baseline scope or to give advanced users controls; they can be locked or hidden.

Filters and slicers combine with AND logic across fields. In DAX, measures can override or remove report filters using CALCULATE with ALL, REMOVEFILTERS or KEEPFILTERS. Note that filters never override RLS.

Also useful: Edit interactions controls which visuals a slicer or visual cross-filters or cross-highlights.

A209. Explain drill-down, drill-through, bookmarks and tooltip pages. What does each do for dashboard usability?

Drill-down: navigating down a hierarchy inside the same visual (Year > Quarter > Month > Day) with the drill arrows. Drill-up goes back. "Expand all" shows the next level for all items.

Drill-through: right-clicking a data point (for example a product) and jumping to a dedicated detail page that is filtered by that value. Configure by placing the field in the drill-through well of the target page; a back button is added automatically. Great for summary-to-detail navigation without cluttering the summary page.

Bookmarks: saved states of a report page (filters, slicer selections, visibility of visuals, sort). Combined with buttons and the Selection pane, they create toggles (switch between chart and table, show/hide a filter panel) and guided navigation or storytelling.

Tooltip pages: a small custom report page used as a hover tooltip for visuals, showing extra context such as a trend line and KPIs for the hovered item.

Together they allow a clean top-level view with progressive disclosure of detail: overview > drill down or through > tooltip for quick context, with bookmarks for view switching. Report page navigation buttons and page navigator also help users move around.

A210. Describe Power BI Service workspaces, roles, apps and sharing options. What is the recommended way to distribute reports to a large audience...

Workspace: a collaborative container in the Service for content (semantic models, reports, dashboards, dataflows). My Workspace is personal; shared workspaces are for teams.

Workspace roles: Admin (full control, manage access), Member (edit, publish, share), Contributor (create and edit content, cannot publish apps or manage access), Viewer (view only, subject to RLS). Admin, Member and Contributor are not restricted by RLS.

Distribution options:

- Share a report/dashboard link directly (requires Pro or Premium per user licensing unless in Premium capacity): ad hoc, hard to govern.

- Publish a workspace app: a packaged, versioned collection of content with a navigation layout and audiences (different groups see different content). This is the recommended way for large audiences.

- Embed in Teams, SharePoint Online or a website (secure embed), or Power BI Embedded for custom apps.

- Subscriptions (email snapshots) and data-driven alerts on dashboard tiles.

Also: certify and endorse (promote) trusted semantic models, use sensitivity labels, and keep one model with many thin reports. With Premium per capacity or Fabric F64+ capacity, free users can view content in apps. Otherwise viewers need Pro or Premium Per User licenses.

A211. What are dataflows, and how do they differ from semantic models (datasets)? When would you use them?

A dataflow is a cloud-based Power Query (M) ETL process in the Service that stores its output as tables in Azure Data Lake (or Dataverse). It lets you define reusable transformation logic once and reuse the cleaned entities across many semantic models and workspaces. Gen2 dataflows in Microsoft Fabric add more destinations (lakehouse, warehouse) and performance improvements.

A semantic model (formerly dataset) is the analytical model: tables, relationships, measures, hierarchies, RLS, loaded into the VertiPaq engine and used by reports.

Differences: dataflows hold no relationships, measures or RLS; they are about data preparation and storage. Semantic models are what reports bind to.

Use dataflows when: several reports need the same cleaned entity (Customer, Product), you want to centralize logic and avoid duplicated refresh load on the source, you want self-service ETL for analysts, or to land staged data to a lake. Caution: avoid chaining many dataflows without need, and heavy transformations are better done in a warehouse or lakehouse for large volumes. Incremental refresh and linked/computed entities are available (Premium for some features).

A212. Why do you need a dedicated Date table, and how do you create and configure one for time intelligence?

Time intelligence functions (TOTALYTD, SAMEPERIODLASTYEAR, DATESINPERIOD, etc.) require a contiguous date column with no gaps and one row per day, covering all years in the data. A dedicated Date table also lets you add fiscal year, week, holiday flags and sort columns.

Creating it: in DAX

```
Date =
```

```
ADDCOLUMNS (
    CALENDAR ( DATE ( 2020, 1, 1 ), DATE ( 2027, 12, 31 ) ),
    "Year", YEAR ( [Date] ),
    "MonthNo", MONTH ( [Date] ),
    "Month", FORMAT ( [Date], "MMM" ),
    "Quarter", "Q" & QUARTER ( [Date] ),
    "YearMonth", FORMAT ( [Date], "YYYY-MM" )
)
```

or CALENDARAUTO(), or in Power Query / the source warehouse.

Configuration: set Modeling > Mark as date table with the Date column (this validates uniqueness and lets DAX use it correctly), relate Date[Date] one-to-many to the fact table date column (the fact column must be date-typed without time), use Sort by Column (Month sorted by MonthNo), hide the raw fact date column, and turn off the Auto date/time option to avoid hidden tables.

A213. Explain table storage modes (Import, DirectQuery, Dual) and how aggregation tables improve performance in a composite model.

Each table in a composite model has a storage mode:

- Import: cached in memory.

- DirectQuery: queried live from the source.

- Dual: can act as either; it is cached but served from the cache when queried together with Import tables, and queried in the source when queried together with DirectQuery tables of the same source, avoiding costly cross-source joins. Usually set for dimensions related to a DirectQuery fact.

Aggregation tables: a pre-summarized Import table (for example Sales by Date, Product Category and Region) mapped to a detail DirectQuery fact (billions of rows) via Manage aggregations. Each column is mapped with a summarization (Sum, Count, Min, Max, GroupBy) to the detail table column. When a visual's query can be answered from the aggregation (the grain and measure match), the engine uses the fast in-memory aggregation; otherwise it transparently falls back to DirectQuery on the detail table. User-defined aggregations are invisible to report authors (the agg table is hidden).

Design: keep dimensions Dual, detail fact DirectQuery, aggregation Import at the grain most reports use. Alternatively, Fabric's automatic aggregations can build and maintain them (Premium). Benefits: interactive speed on massive data with drill-down to detail only when needed.

A214. What are Power Query parameters and how are they used (for example for environment switching or filtering)?

A parameter is a named value (text, number, date, list, etc.) defined in Power Query (Manage Parameters) that can be referenced in queries, filters, function arguments or data source settings. It can have a list of allowed values and a current value.

Uses:

- Switch environments: ServerName and DatabaseName parameters used in the SQL source step so you can point at Dev, Test or Prod without editing queries; in the Service the values can be changed under dataset Settings > Parameters, and deployment pipelines can set rules per stage.

- Filter data volume during development: a RowLimit or StartDate parameter to reduce data in Desktop, changed to full for production.

- Incremental refresh: RangeStart and RangeEnd special parameters.

- Folder path for file sources, so the same query works from different locations.

- Custom functions: parameters as arguments, for example a function that imports one file and is invoked over a list of files.

Parameters differ from What-if parameters in Desktop, which are DAX-based and used for scenario analysis by report readers.

A215. What is the difference between a Power BI report and a dashboard?

Report: a multi-page, interactive document built in Power BI Desktop (or in the Service) on top of one semantic model. It has slicers, filters, drill-through and full visuals and supports editing.

Dashboard: a single-page canvas that exists only in the Service. It consists of tiles pinned from one or more reports (and other sources such as Excel, streaming data, images, text, video). Tiles are static snapshots that update when data refreshes; clicking a tile takes you to the report it came from. Dashboards support data alerts, natural language Q&A, and a mobile layout, and can combine content from several models, which reports cannot.

Reports are for exploration and analysis; dashboards are for at-a-glance monitoring of key metrics. A dashboard is not created in Desktop and has no slicers or page filters. In practice, many teams now put overview pages inside reports and share them via apps and use dashboards for cross-report monitoring and alerts.

A216. What are best practices for designing an effective, user-friendly Power BI dashboard or report?

Purpose and audience: start by identifying who uses it, what decisions they make and the KPIs that matter; design for 5 to 10 key metrics on the landing page.

Layout: place the most important information top-left (F or Z reading pattern), KPI cards across the top, trends in the middle, details lower; align and size visuals consistently on a grid; leave white space.

Visual choice: line charts for trends over time, bar/column for comparisons, stacked only when parts-of-whole matter, scatter for correlation, maps only when geography matters; avoid pie/donut with many slices, 3D, dual-axes clutter, and gauges unless showing target.

Color and formatting: consistent palette and theme (JSON theme), use color to convey meaning (red/green for variance, with color-blind safe choices), limit fonts, clear titles and units, format numbers (K, M, %), tooltips, and sorted bars.

Interactivity: a small set of slicers (date, region), sync slicers, drill-through for details, bookmarks, reset button, edit interactions; avoid overwhelming filters.

Context: show targets, prior period and variance with each KPI; add last refresh date and definitions of measures.

Performance and governance: limited visuals per page, star schema model, certified model, accessibility (alt text, tab order), mobile layout, naming conventions, folders for measures, documentation, and testing with real users.

A217. How do you handle errors, nulls, and inconsistent data types in Power Query?

Nulls: use Replace Values (null with 0 or a default), Fill Down/Up for hierarchical exports, or Table.SelectRows to remove them. Be careful: replacing null with 0 changes averages. In M, the null-coalescing operator is "??"; older approach: if [x] = null then 0 else [x].

Errors: use Remove Errors, Replace Errors, or Keep Errors to inspect. Errors usually come from invalid type conversions (text "N/A" to number), division by zero or missing columns. In M use try ... otherwise:

```
= Table.AddColumn(Source, "Amount", each try Number.From([AmountText]) otherwise null, type nullable number)
```

Data types: set types explicitly with correct locale (Change Type with Locale for dates like 31/12/2023), trim and clean text (Text.Trim, Text.Clean, Text.Proper), standardize case, remove duplicates, check with Column quality / distribution / profile (View > Column profile on entire dataset, not just first 1000 rows).

Robustness: avoid relying on column positions; use names; make steps tolerant of missing columns with MissingField.Ignore options; keep a staging query with a reference; document steps; and keep the "Changed Type" step near the start so downstream steps work with expected types. Finally validate row counts and totals against the source after load.

A218. Describe Power BI licensing and the differences between Free, Pro, Premium Per User and capacity (Premium/Fabric) in terms of what an analys...

Free: author in Desktop, and use My Workspace in the Service; cannot share with others or consume content shared by others (unless the content is in a Premium/Fabric capacity workspace at a sufficient SKU).

Pro: per-user license that lets you publish to shared workspaces, share, collaborate, and consume content from other Pro users. Dataset size limit 1 GB per model, 8 scheduled refreshes per day, 10 GB storage per user.

Premium Per User (PPU): per-user license that unlocks Premium features (larger models up to 100 GB, 48 refreshes per day, paginated reports, deployment pipelines, XMLA read/write endpoints, AutoML-style features, dataflow enhanced compute) but content is only consumable by other PPU users.

Capacity (Power BI Premium P-SKUs, or Microsoft Fabric F-SKUs): purchased per capacity (reserved compute); lets the organization distribute content to free users (at F64/P1 and above), larger models, more refreshes (up to 48 and via API), dedicated resources, Fabric workloads, and XMLA endpoints. Pricing and limits change over time, so verify with Microsoft documentation. Analysts should know the limits when designing models and refresh schedules.

A219. What does the VertiPaq engine do and why does cardinality matter for model size and speed?

VertiPaq is the in-memory columnar storage engine behind Import models (the Storage Engine of Analysis Services Tabular). Data is stored column by column rather than row by row, and each column is compressed independently using dictionary encoding (distinct values stored once, rows store small integer ids), value encoding (for numeric ranges) and run-length encoding (RLE) on sorted segments. Tables are split into segments of about 1 million rows for parallel scanning, and the engine can scan billions of values per second.

Cardinality (number of distinct values in a column) is the primary driver of size: low cardinality compresses extremely well; high-cardinality columns (GUIDs, timestamps with seconds, free text, decimals with many digits) have huge dictionaries and poor compression, using memory and slowing scans and relationships.

Practical steps: remove unnecessary high-cardinality columns; split datetime into date and time (or keep date only); round or convert decimals to integers (scale factor); use integer keys instead of text; sort data in the source so RLE works better; avoid calculated columns on large tables (they compress less than source columns); reduce columns in the fact table and move descriptive attributes to dimensions. Use VertiPaq Analyzer to see the dictionary size, data size and hierarchy size per column.

A220. What are the main visuals in Power BI and when would you choose a matrix over a table, a waterfall, a scatter chart or a decomposition tree?

Table: flat list of rows and columns with detail values; use it for precise, row-level lookups and exports. A matrix is like a pivot table: row and column groupings, subtotals, expand/collapse hierarchies, conditional formatting and stepped layout; use it for cross-tab analysis (product category by month) and for finance statements.

Bar/column and line charts: comparing categories and showing trends. Combo chart: a measure with different scale (revenue columns and margin % line).

Waterfall: shows how a start value changes through positive and negative contributions to an end value (profit bridge, variance between periods).

Scatter chart: relationship between two measures with an optional bubble size and play axis; spot outliers and clusters (price versus volume per product).

Decomposition tree: interactive root-cause style drill, breaking one measure down by a dimension of the user's choice at each level (sales by region, then category). Key influencers analyzes which factors drive a metric being high/low.

Others: card and multi-row card for KPIs, KPI visual (value, trend, target), gauge, treemap for part-to-whole hierarchy, funnel for stages, map/filled map/shape map for geography, slicer, and R/Python visuals. Pick the visual that answers the specific question and minimizes cognitive load.

A221. What are field parameters and what-if parameters? How do they differ and what are the use cases?

What-if parameter (Modeling > New parameter): creates a numeric range slicer and a measure holding the selected value (generated by a GENERATESERIES calculated table plus a SELECTEDVALUE measure). Use it for scenario analysis, for example a discount % slider to see projected revenue:

```
Projected Revenue = [Total Sales] * (1 + 'Growth Rate'[Growth Rate Value])
```

Field parameters (Modeling > New parameter > Fields): create a calculated table that lists fields or measures and lets users pick through a slicer which measure or dimension appears in a visual. Use cases: letting users switch a chart between Sales, Profit and Quantity, or between Category, Region and Channel on the axis, without building many visuals or bookmark combinations. Behind the scenes it is a table built with NAMEOF references.

Difference: a what-if parameter feeds a number into a calculation; a field parameter changes which field/measure is displayed. Both reduce report page clutter and give users self-service flexibility.

A222. How do you manage development, testing and production of Power BI content using deployment pipelines and version control?

Deployment pipelines (Premium/PPU/Fabric) provide Development, Test and Production stages, each linked to a workspace. You develop in Dev, deploy to Test for validation by business users, then deploy to Prod. The pipeline compares stages, copies content, and allows deployment rules that change data source connections and parameters per stage (for example Dev SQL server to Prod SQL server). Content in each stage keeps its own refresh schedule and permissions; the Prod workspace is published as an app.

Version control: save projects as Power BI Project (PBIP) format, which splits a report and semantic model into text files (TMDL and JSON) that work with Git. Fabric Git integration connects a workspace to Azure DevOps or GitHub, so changes are branched, reviewed in pull requests and merged. Tools like Tabular Editor and ALM Toolkit compare and deploy model metadata without overwriting data or partitions.

Good practices: separate workspaces per environment, thin reports connected to shared certified semantic models, naming conventions, documented measures, automated tests with DAX queries or Best Practice Analyzer rules, service principals for automation, and release notes. Avoid editing in Prod and avoid publishing PBIX directly over a Prod model with incremental refresh.

A223. Write DAX measures for Total Sales, Total Cost, Profit, and Profit Margin % given a Sales table with Quantity, UnitPrice and UnitCost column...

Because the amount depends on a product of two columns in each row, you need an iterator (row context) rather than SUM of a single column.

```
Total Sales = SUMX ( Sales, Sales[Quantity] * Sales[UnitPrice] )

Total Cost = SUMX ( Sales, Sales[Quantity] * Sales[UnitCost] )

Profit = [Total Sales] - [Total Cost]

Profit Margin % = DIVIDE ( [Profit], [Total Sales] )
```

Explanation: SUMX iterates each row of Sales, evaluates the expression with that row's values (row context), then sums the results under the current filter context. A SUM(Sales[Quantity]) * SUM(Sales[UnitPrice]) would be wrong (multiplying totals). If a SalesAmount column already exists in the source or Power Query, SUM(Sales[SalesAmount]) is faster and preferable. DIVIDE handles division by zero by returning BLANK instead of an error. Format Profit Margin % as a percentage. Margin must be computed from totals, never by averaging row-level margins.

A224. Explain CALCULATE with examples: override a filter, add a filter on a related dimension, and the difference between boolean filter arguments...

CALCULATE (<expression>, <filter1>, <filter2>, ...) evaluates the expression in a modified filter context. Each filter argument replaces existing filters on the columns it mentions (unless wrapped in KEEPFILTERS).

```
Sales 2023 = CALCULATE ( [Total Sales], 'Date'[Year] = 2023 )

Sales Bikes = CALCULATE ( [Total Sales], Product[Category] = "Bikes" )
```

```
Sales Big Orders = CALCULATE ( [Total Sales], FILTER ( Sales, Sales[Quantity] > 10 ) )

Sales Bikes West =
CALCULATE ( [Total Sales], Product[Category] = "Bikes", Region[Name] = "West" )
```

Points:

- A boolean argument like Product[Category] = "Bikes" is shorthand for FILTER(ALL(Product[Category]), Product[Category] = "Bikes"). It overrides any existing filter on Category from slicers, so a slicer choosing Accessories is ignored for this measure. To intersect with the slicer instead use KEEPFILTERS:

```
Bikes In Selection =
CALCULATE ( [Total Sales], KEEPFILTERS ( Product[Category] = "Bikes" ) )
```

- Multiple filters are combined with AND. For OR across columns use FILTER with ||, or a table filter, e.g. FILTER(ALL(Product[Category], Product[Color]), ...).

- FILTER(Sales, ...) iterates rows of the table and allows conditions involving several columns or measures; it keeps existing filters on the filtered table's columns and can be slower. Prefer column boolean filters when they are enough.

- Boolean arguments can only use a single column and constants (no measures).

- Filters on dimensions propagate to the fact via relationships.

A225. Explain ALL, ALLEXCEPT, ALLSELECTED and REMOVEFILTERS with a measure for each use case.

ALL(table or columns): returns all rows ignoring filters; as a CALCULATE modifier removes filters from the given table/columns. REMOVEFILTERS does the same but can only be used as a CALCULATE modifier and is more readable. ALLEXCEPT(table, col1, ...): removes all filters on the table except the listed columns. ALLSELECTED: removes filters from the current visual (rows/columns inside the visual) but keeps filters from outside it, such as slicers and page filters; it defines the "visual total".

```
Grand Total Sales = CALCULATE ( [Total Sales], REMOVEFILTERS ( Product ) )

Sales % of All Products = DIVIDE ( [Total Sales], [Grand Total Sales] )

Sales % of Category =
DIVIDE ( [Total Sales], CALCULATE ( [Total Sales], ALLEXCEPT ( Product, Product[Category] ) )
)

Sales % of Visual Total =
DIVIDE ( [Total Sales], CALCULATE ( [Total Sales], ALLSELECTED ( Product ) ) )
```

Use ALL for percent of grand total ignoring even slicers; ALLSELECTED when the percentage should respect the slicer selection so that the visual totals 100%; ALLEXCEPT for percent within a group. Note that ALL(Product) clears filters from every column of Product but not from related tables like Sales; filters on Date remain unless also removed.

A226. Write measures for Year-to-Date sales, Prior-Year sales and Year-over-Year growth %, using a marked Date table.

```
Sales YTD = TOTALYTD ( [Total Sales], 'Date'[Date] )

-- Equivalent with CALCULATE
Sales YTD v2 = CALCULATE ( [Total Sales], DATESYTD ( 'Date'[Date] ) )

Sales PY = CALCULATE ( [Total Sales], SAMEPERIODLASTYEAR ( 'Date'[Date] ) )

Sales YoY Growth =
VAR CurrentSales = [Total Sales]
VAR PriorSales = [Sales PY]
RETURN
    DIVIDE ( CurrentSales - PriorSales, PriorSales )
```

For a fiscal year ending 31 March: TOTALYTD([Total Sales], 'Date'[Date], "3/31").

Notes: time intelligence works only with a contiguous Date table, marked as date table and related to the fact. SAMEPERIODLASTYEAR shifts the dates in the current filter context back one year, so at month grain it returns the same month last year. DATEADD('Date'[Date], -1, YEAR) is an alternative that is more flexible (it can shift by quarter or month). DIVIDE returns blank when there is no prior-year value. Format growth as a percentage. If the visual shows a partial current year, consider limiting PY to the same elapsed period.

A227. Write a rolling (moving) 3-month average of sales and a 12-month trailing total.

Use DATESINPERIOD to build a window ending at the last visible date, and average monthly totals with AVERAGEX over months so months with no sales are treated consistently.

```
Sales L12M =
CALCULATE (
    [Total Sales],
    DATESINPERIOD ( 'Date'[Date], MAX ( 'Date'[Date] ), -12, MONTH )
)

Rolling 3M Avg Sales =
VAR LastDate = MAX ( 'Date'[Date] )
VAR Window =
    DATESINPERIOD ( 'Date'[Date], LastDate, -3, MONTH )
RETURN
    AVERAGEX (
        VALUES ( 'Date'[YearMonth] ),
        CALCULATE ( [Total Sales], Window )
    )
```

A simpler version, if your visual is at month level: divide the trailing 3-month total by 3:

```
Rolling 3M Avg = DIVIDE ( CALCULATE ( [Total Sales], DATESINPERIOD ( 'Date'[Date], MAX (
'Date'[Date] ), -3, MONTH ) ), 3 )
```

Caveat: the first two months have fewer than three months of history, so decide whether to show blank (check that the window start is not before the first data date) or average of available months. Always use the Date table column, not the fact table column, inside the time intelligence function.

A228. Write a running (cumulative) total measure that works over dates, and one that resets each year.

All-time cumulative total up to the last date in the current filter context:

```
Running Total Sales =
CALCULATE (
    [Total Sales],
    FILTER (
        ALL ( 'Date'[Date] ),
        'Date'[Date] <= MAX ( 'Date'[Date] )
    )
)
```

Same with DATESBETWEEN:

```
Running Total v2 =
CALCULATE ( [Total Sales], DATESBETWEEN ( 'Date'[Date], BLANK (), MAX ( 'Date'[Date] ) ) )
```

Resetting every year is the year-to-date:

```
Running Total YTD = TOTALYTD ( [Total Sales], 'Date'[Date] )
```

Another variant that restarts each year using the visible Year:

```
Running Total In Year =
CALCULATE (
    [Total Sales],
    FILTER (
        ALL ( 'Date' ),
        'Date'[Year] = MAX ( 'Date'[Year] ) && 'Date'[Date] <= MAX ( 'Date'[Date] )
    )
)
```

Explanation: ALL on the date column removes the current date filter so earlier days are visible, and the condition re-applies "up to now". Note that for future dates with no data you may want to hide values: wrap with IF (MAX('Date'[Date]) <= MAX(Sales[OrderDate]), ...).

A229. How do you rank products by sales and show only the Top 5 with RANKX and TOPN? Write the DAX and explain handling of ties and filter context...

```
Product Rank =
RANKX (
    ALLSELECTED ( Product[ProductName] ),
    [Total Sales],
    ,
    DESC,
    DENSE
)
```

```
Top 5 Sales =
VAR TopProducts = TOPN ( 5, ALLSELECTED ( Product[ProductName] ), [Total Sales], DESC )
RETURN
    CALCULATE ( [Total Sales], KEEPFILTERS ( TopProducts ) )

Top 5 Share =
DIVIDE ( [Top 5 Sales], CALCULATE ( [Total Sales], ALLSELECTED ( Product ) ) )
```

Explanation:

- RANKX evaluates [Total Sales] for each product in the table argument (here all products still selected by slicers, ignoring the visual's row filter) and ranks the current product's value among them. DENSE gives 1,2,2,3 for ties; SKIP gives 1,2,2,4 (default).

- Using ALL instead of ALLSELECTED would rank across the whole product list even when a slicer excludes products; ALLSELECTED respects slicers.

- To show Top N in a visual, either use the visual-level Top N filter on the product field, or filter in DAX:

```
Sales If Top 5 = IF ( [Product Rank] <= 5, [Total Sales] )
```

- TOPN returns a table; with ties it may return more than N rows. Add a tiebreaker as an additional sort expression (for example product key) if exactly N is required.

- Totals in matrix visuals: a rank is meaningless at the total row, so use IF(HASONEVALUE(Product[ProductName]), RANKX(...)).

A230. Write DAX to count new customers versus returning customers in a selected period, given a Sales table with CustomerKey and OrderDate.

Definition: a customer is "new" in the period if their first-ever order falls inside the period; "returning" if they ordered in the period but had at least one order before the period start.

```
Active Customers = DISTINCTCOUNT ( Sales[CustomerKey] )

New Customers =
VAR PeriodStart = MIN ( 'Date'[Date] )
VAR PeriodEnd = MAX ( 'Date'[Date] )
RETURN
COUNTROWS (
    FILTER (
        VALUES ( Sales[CustomerKey] ),
        VAR FirstOrder =
            CALCULATE ( MIN ( Sales[OrderDate] ), ALL ( 'Date' ) )
        RETURN FirstOrder >= PeriodStart && FirstOrder <= PeriodEnd
    )
)

Returning Customers = [Active Customers] - [New Customers]
```

Explanation: VALUES(Sales[CustomerKey]) gives customers active in the current filter context (visual month, region, etc.). For each customer, the inner CALCULATE converts the row context to filter context (that

customer) and removes date filters with ALL('Date') to find the lifetime first order date. If the first order is within the visible period then the customer is new. Because only date filters are removed, other filters (product, region) still apply, which means "new" refers to first order within that slice; if you need a true first purchase overall remove those filters too (ALL(Sales) with only the customer filter retained via ALLEXCEPT). For performance on big tables, precompute FirstOrderDate as a column on the Customer dimension and compare.

A231. Show how to use RELATED, RELATEDTABLE and SUMX to compute values across tables, for example total discounted sales where the discount rate l...

RELATED fetches a column value from the "one" side table for the current row of the "many" side table (it follows relationships and needs row context). RELATEDTABLE returns the rows of the "many" side related to the current row on the "one" side.

```
Discounted Sales =
SUMX (
    Sales,
    Sales[Quantity] * Sales[UnitPrice] * ( 1 - RELATED ( Product[DiscountRate] ) )
)
```

Calculated column on Product that counts its order lines:

```
Order Lines = COUNTROWS ( RELATEDTABLE ( Sales ) )
```

Calculated column on Sales to bring in category:

```
Category = RELATED ( Product[Category] )
```

Notes:

- RELATED only works from many to one direction; from one to many use RELATEDTABLE or aggregate with a measure and context transition.

- Do not create RELATED columns on the fact just to slice: slice directly by the dimension column; this keeps the fact table narrow.

- In a measure with SUMX over Sales, row context exists per Sales row so RELATED works; in a plain measure without iterator it would fail.

- Inactive relationships are not used by RELATED; use USERELATIONSHIP or LOOKUPVALUE as alternatives.

A232. Write a calculated column that categorizes customers by age (under 18, 18-34, 35-54, 55+) using SWITCH, and explain why SWITCH(TRUE(), ...) ...

```
Age Group =
SWITCH (
    TRUE (),
    Customer[Age] < 18, "Under 18",
    Customer[Age] <= 34, "18-34",
    Customer[Age] <= 54, "35-54",
    "55+"
)
```

SWITCH normally compares one expression to constant values. Passing TRUE() as the expression makes it test each condition in order and return the result of the first condition that evaluates to TRUE, which acts like a readable chain of IF/ELSE IF statements. The last argument is the default. Order matters because earlier conditions are evaluated first, so each later one needs only its upper bound.

Notes: add a sort-order column (or use Sort by Column) so the groups display in a sensible order instead of alphabetically; calculate age from BirthDate in Power Query or with DATEDIFF(Customer[BirthDate], TODAY(), YEAR) if needed (but remember TODAY evaluates at refresh in calculated columns, so age won't update until refresh). As a calculated column it is stored and can be used as a slicer or axis; if you need it to change with the selection (for example dynamic buckets) use a measure with a disconnected bucket table.

A233. How do you handle blanks and division by zero in DAX? Show DIVIDE, IF with ISBLANK, COALESCE, and HASONEVALUE/SELECTEDVALUE.

```
Margin % = DIVIDE ( [Profit], [Total Sales] )          -- BLANK when denominator is 0 or
blank
Margin % 0 = DIVIDE ( [Profit], [Total Sales], 0 )     -- 0 as the alternate result

Sales Display =
IF ( ISBLANK ( [Total Sales] ), 0, [Total Sales] )

Sales Display 2 = COALESCE ( [Total Sales], 0 )

Selected Region =
SELECTEDVALUE ( Region[Name], "Multiple regions" )

Only When One Product =
IF ( HASONEVALUE ( Product[ProductName] ), [Total Sales] )
```

Explanation:

- DIVIDE is safer and slightly faster than writing IF(denominator=0, ...).

- BLANK is not the same as 0 or empty string: a visual hides rows where all measures are BLANK, which keeps charts clean. Converting blanks to 0 makes every dimension member appear (for example every product with no sales), which can bloat visuals and hurt performance; do this only when needed.

- ISBLANK distinguishes blank from 0; "= 0" would also treat blank as 0.

- SELECTEDVALUE returns the single value in context or an alternate result otherwise; useful for titles and parameters.

- HASONEVALUE tests whether exactly one value is visible, to avoid meaningless totals.

A234. Write the dynamic RLS setup: tables, relationship, DAX role filter, and a variant where a manager sees all their regions, using USERPRINCIPA...

Model: Sales (fact) -> Region (dimension, one-to-many from Region to Sales). Add a table UserRegion (UserEmail, RegionKey), one row per allowed combination. Relate Region[RegionKey] and UserRegion[RegionKey]; the filter must flow from UserRegion to Region to Sales, so relate UserRegion (many) to Region (one) with cross-filter direction Both and enable "Apply security filter in both directions", or flip the design so UserRegion filters Region directly.

Role definition (Manage roles) applied on UserRegion:

```
[UserEmail] = USERPRINCIPALNAME ()
```

Region and Sales are then filtered by relationship propagation. Single-table alternative on Region if the email is in the dimension:

```
[ManagerEmail] = USERPRINCIPALNAME ()
```

Manager seeing multiple regions: the mapping table has several rows for that manager, so several regions pass the filter. Admin seeing everything: add a role with no filter, or mark an "All" row logic:

```
[UserEmail] = USERPRINCIPALNAME ()
    || LOOKUPVALUE ( UserRole[Role], UserRole[UserEmail], USERPRINCIPALNAME () ) = "Admin"
```

Test: Modeling > View as roles with "Other user" email in Desktop; in the Service use "Test as role" on the semantic model's Security page. Do not add the security table to visuals, hide it, and manage the users in the source. Remember workspace members with edit rights bypass RLS, so publish via app for end users.

A235. Write Power Query M code that loads a CSV with a header row, promotes headers, sets types, trims text, removes duplicates and filters out ro...

```
let
    Source = Csv.Document(File.Contents("C:\Data\orders.csv"), [Delimiter = ",", Encoding =
65001, QuoteStyle = QuoteStyle.Csv]),
    Promoted = Table.PromoteHeaders(Source, [PromoteAllScalars = true]),
    Typed = Table.TransformColumnTypes(Promoted, {
        {"OrderID", Int64.Type}, {"OrderDate", type date},
        {"Customer", type text}, {"Amount", type number}}),
    Trimmed = Table.TransformColumns(Typed, {{"Customer", Text.Trim, type text}}),
    NoNulls = Table.SelectRows(Trimmed, each [OrderID] <> null),
    Deduped = Table.Distinct(NoNulls, {"OrderID"})
in
    Deduped
```

Unpivot (wide to long). A table with columns Product, Jan, Feb, Mar... becomes Product, Month, Sales:

```
let
    Source = Excel.CurrentWorkbook(){[Name = "SalesWide"]}[Content],
    Unpivoted = Table.UnpivotOtherColumns(Source, {"Product"}, "Month", "Sales")
in
    Unpivoted
```

UnpivotOtherColumns is preferred over Table.Unpivot because new month columns added later are included automatically. A power-users tip: build Date table month values by converting the Month text to a date with Date.FromText("1 " & [Month] & " 2024"). After unpivoting the data is tidy: one row per product per month, which suits a star schema and DAX aggregations.

A236. Write a measure for closing inventory balance (semi-additive) where the stock snapshot table has one row per product per day, and the total ...

Inventory is semi-additive: add across products, but not across time. Use the last date that has data within the current period.

```
Closing Stock =
CALCULATE (
    SUM ( Inventory[QtyOnHand] ),
    LASTNONBLANK ( 'Date'[Date], CALCULATE ( COUNTROWS ( Inventory ) ) )
)
```

Simpler if snapshots exist for every day (or every month end):

```
Closing Stock Simple =
CALCULATE ( SUM ( Inventory[QtyOnHand] ), LASTDATE ( 'Date'[Date] ) )
```

More accurate across products having different last dates (each product's own last snapshot in the period):

```
Closing Stock Per Product =
SUMX (
    VALUES ( Product[ProductKey] ),
    CALCULATE (
        SUM ( Inventory[QtyOnHand] ),
        LASTNONBLANK ( 'Date'[Date], CALCULATE ( COUNTROWS ( Inventory ) ) )
    )
)
```

Opening balance uses the day before the first date: CALCULATE([Closing Stock], DATEADD(...)) or PREVIOUSMONTH. Average stock: AVERAGEX(VALUES('Date'[Date]), CALCULATE(SUM(Inventory[QtyOnHand]))). Power BI also has time-intelligence helpers like CLOSINGBALANCEMONTH, which wrap this pattern. Explain in the report that totals for time are balances at the end of period.

A237. Company context: A national retail chain (200 stores, 50,000 SKUs) wants a Sales Performance dashboard for executives, regional managers and...

Step 1 - Requirements: interview the three audiences. Executives: revenue, margin, YoY growth, vs target; regional managers: store ranking, category mix, stock-outs; store managers: daily sales, top products, staff or traffic metrics. Agree on definitions (net sales, margin, same-store sales).

Step 2 - Data model: star schema. Import dimensions; the 300 million-row fact is handled with Import plus incremental refresh (last 3 years history, refresh last 7 days), keeping only needed columns and integer keys; consider aggregation tables (Sales by Date, Store, Category) with the detail table in DirectQuery if the size is too big for Import. A single marked Date table (with fiscal calendar), single-direction relationships, dimension hierarchies (Region > District > Store; Category > Subcategory > Product).

Step 3 - Measures in a dedicated measures table: Total Sales, Margin %, Units, Average Basket, Sales PY, YoY %, YTD, same-store sales, vs Target with variance. Use variables and DIVIDE.

Step 4 - Report pages: (1) Executive overview: KPI cards with PY and target variance, trend line, regional map or bar, category treemap. (2) Regional analysis: store ranking matrix with conditional formatting, drill-through

to (3) Store detail page; (4) Product performance with Top N, ABC. Slicers: Date, Region, Category synced across pages. Tooltip pages for trends.

Step 5 - Security: dynamic RLS so regional and store managers see only their stores.

Step 6 - Performance: limited visuals per page, Performance Analyzer testing, user testing with actual volumes.

Step 7 - Deployment: publish to a workspace, scheduled refresh through a gateway, distribute through an app with audiences, certify the dataset, add subscriptions and data alerts, document metrics, mobile layout, and train users.

A238. Company context: A logistics company's Power BI report takes about 40 seconds to load a page, and users complain. The model is Import mode (...

Step 1 - Measure: use Performance Analyzer in Desktop, record a page load, and rank visuals by duration; split DAX query time versus visual rendering. Copy slow queries into DAX Studio and use Server Timings to see storage versus formula engine time. Use VertiPaq Analyzer to find the largest columns and cardinality.

Step 2 - Fix the model: split the flat table into a star schema (Fact plus Customer, Product, Route, Date dimensions); remove unused columns; reduce cardinality (split datetime, drop GUIDs and free text, round decimals, integer keys); disable Auto date/time and use a Date table; remove or replace bidirectional relationships with single direction; turn calculated columns into Power Query steps or source columns; apply incremental refresh to cut refresh, and consider aggregation tables for the common grain.

Step 3 - Fix the DAX: replace FILTER(Table,...) with column predicates in CALCULATE, use variables, avoid nested iterators and measures called inside large iterators, use DIVIDE, simplify DISTINCTCOUNT on huge columns where possible.

Step 4 - Fix the report page: cut to roughly 8 to 10 visuals; split into pages with drill-through or bookmarks; reduce high-cardinality tables and matrices (limit rows, use Top N); use slicer "Apply" buttons or the dropdown style; use Edit interactions to turn off needless cross filtering; avoid heavy custom visuals; use report-level filters to narrow default scope (for example last 90 days).

Step 5 - Re-test: compare Performance Analyzer before and after; test with the cold cache and in the Service with realistic concurrency; target page loads under 5 seconds. Document the changes and set up monitoring with usage metrics.

A239. Company context: A SaaS company wants an executive KPI scorecard showing Monthly Recurring Revenue (MRR), churn rate, customer acquisition c...

Step 1 - Define each KPI precisely with the business: MRR = sum of active subscriptions monthly value at month end; Churn rate = customers (or MRR) lost in month divided by customers at the start of the month; CAC = sales and marketing spend divided by new customers; NRR = (start MRR + expansion - contraction - churn) / start MRR.

Step 2 - Model: Subscription fact (snapshot per month or start/end dates), Customer, Plan and Date dimensions, a Targets table at month grain related to Date, and spend facts.

Step 3 - Measures (examples):

```
MRR = CALCULATE ( SUM ( Subs[MonthlyValue] ), LASTDATE ( 'Date'[Date] ) )
MRR PY = CALCULATE ( [MRR], SAMEPERIODLASTYEAR ( 'Date'[Date] ) )
MRR vs Target % = DIVIDE ( [MRR] - [MRR Target], [MRR Target] )
Churn Rate = DIVIDE ( [Churned Customers], [Opening Customers] )
```

```
Status =
SWITCH ( TRUE (), [MRR vs Target %] >= 0, "On track", [MRR vs Target %] >= -0.05, "Watch",
"Off track" )
```

Step 4 - Layout: top row of KPI cards (value, vs PY or target, small trend sparkline) with conditional color from the Status measure (and icons so color-blind users can still read it); middle: MRR trend with target line, MRR waterfall (new, expansion, contraction, churn); lower: churn by segment, cohort retention matrix; slicers for date, plan, region. Use the KPI visual or cards with reference labels.

Step 5 - Governance: define each KPI in a tooltip or definitions page, show last refresh date, and keep target values in a managed table rather than hard-coded. Validate with finance by reconciling to the billing system.

A240. Company context: A national insurance company has 12 regional managers, and each should only see policies and claims for their own region in...

Step 1 - Choose dynamic RLS rather than 12 static roles, since people and regions change.

Step 2 - Add a security mapping table (UserEmail, Region) maintained in the source (SQL table or SharePoint list), loaded to the model; managers covering two regions have two rows. National head has either a row per region or a flag.

Step 3 - Relationships: Region dimension one-to-many to Policy and Claims facts. Relate the security table to Region on region; ensure the filter flows from security table to Region (set cross-filter to Both on that relationship with "Apply security filter in both directions"), or filter the Region dimension directly if it contains the manager email.

Step 4 - Roles: role "RegionalManager" with the filter on the security table: [UserEmail] = USERPRINCIPALNAME(). Role "Executive" with no filter on the tables (sees all) or use a role-flag logic in one role.

Step 5 - Test with View as in Desktop for sample users (including a two-region manager and an unmapped user, who sees nothing), and with Test as role in the Service.

Step 6 - Publish; in the semantic model Security settings, add the Entra security groups (RegionalManagers, Executives) to roles. Share through a workspace app so people are Viewers; remind that workspace Admin/Member/Contributor are exempt.

Step 7 - Check measures: totals should be limited to the allowed region, and any "company total" benchmark measure that uses ALL will still respect RLS; if managers must see the national total for comparison, compute it in a separate unsecured table or an aggregated model.

Step 8 - Audit and maintain: hide the security table, document, review access quarterly.

A241. Company context: A finance team receives 36 monthly Excel files (one per month, same layout) in a SharePoint folder with inconsistent header...

Step 1 - Connect: Get Data > SharePoint Folder (or Folder). Filter the file list to the relevant extension and name pattern (exclude temp files).

Step 2 - Use the Combine Files feature (Combine & Transform): Power Query creates a sample-file query, a parameter, a custom function and a transformed table. Edit the sample file transformation (the function applies it to every file).

Step 3 - Clean in the sample transformation: remove the top title rows (Remove Top Rows), promote headers, handle inconsistent header names by renaming to a standard set (Table.RenameColumns with

MissingField.Ignore), remove blank rows/total rows, set types with locale, trim text, and unpivot month columns if wide.

Step 4 - Add the source file name as a column (month) so each row knows its origin and you can derive the period; add error handling with try ... otherwise or remove errors.

Step 5 - Load and model: keep the combined table as a fact, create dimensions (Account, Cost Centre), relate to the Date table.

Step 6 - Validate: reconcile row counts and totals per file against the manual master.

Step 7 - Schedule: publish and set a scheduled refresh (SharePoint Online needs no gateway; OneDrive/SharePoint credentials as OAuth). New monthly files dropped into the folder are picked up automatically. Use incremental refresh if the folder grows and a date can be derived. Document the expected file naming and layout, and add a data-quality check page listing files loaded and row counts.

A242. Company context: A manufacturing firm's scheduled refresh in the Power BI Service started failing overnight. The model pulls from an on-prem...

Step 1 - Read the error: open Workspace > semantic model > Refresh history and the failure details (error code and message) and the activity ID; this tells whether the problem is credentials, gateway, timeout, memory, or query.

Step 2 - Check the gateway: Manage gateways page shows the cluster status; verify the gateway machine is on, the service running, and the gateway version supported. Check the Event Viewer or gateway logs. Confirm the data source credentials in Settings > Gateway connection and that each data source is mapped (server and database names must match what is in the model exactly).

Step 3 - Credentials: expired SQL password or service account password change is the most common cause; update credentials and test with "Edit credentials". For the Excel network share, verify the gateway service account has read access to the path and the file is not locked or moved.

Step 4 - Query and source: run the same query in Desktop; check for schema changes (renamed or removed column breaks Power Query steps), source being offline for maintenance, or long-running query timeouts (shared capacity limit is 2 hours). Look at privacy level conflicts (Formula.Firewall errors) and fix by setting privacy levels or restructuring queries.

Step 5 - Capacity: for out-of-memory errors reduce model size, use incremental refresh, or move to Premium capacity; stagger heavy refreshes.

Step 6 - Fix and refresh manually to confirm, then configure failure email notifications to owners, add the gateway to a cluster for high availability, and monitor with refresh history or the Admin API. Document the incident and consider moving the Excel file to SharePoint Online to remove the dependency.

A243. Company context: A telecom operator has a call-detail fact table with 2 billion rows and needs interactive dashboards. Analysts also need dr...

Step 1 - Assess: Import of 2 billion rows exceeds practical model size and refresh windows on Pro; Premium/Fabric could handle larger, but hourly refresh of the whole table would be wasteful.

Step 2 - Choose a composite model: keep the detail call table in DirectQuery against a performant warehouse (columnstore indexes, partitioned by date, statistics maintained, or a Fabric warehouse/Direct Lake), with dimensions (Date, Customer, Plan, Tower) as Dual tables.

Step 3 - Add user-defined aggregation tables in Import mode (for example Calls by Day, Tower and Plan) with GroupBy and Sum/Count mappings to the detail table. Most summary visuals hit the aggregation at memory speed; only drill-through or very detailed queries fall to DirectQuery. Alternatively use automatic aggregations in Premium, or Fabric Direct Lake.

Step 4 - Optimize the source: pre-aggregate in warehouse views, create indexed views, push filtering with report default filters (last 7 days), avoid complex calculated columns; consider hybrid tables (incremental refresh with a real-time DirectQuery partition for the latest hour).

Step 5 - Report design: fewer visuals per page, apply query reduction settings (Apply button on slicers, disable cross-highlighting where not needed), put detailed record tables on a drill-through page with a required filter so no unfiltered query is sent.

Step 6 - Security: RLS defined in the model; for DirectQuery with SSO, pass the user identity to the source if source-level security exists.

Step 7 - Test with Performance Analyzer and DAX Studio, monitor the SQL load, and set capacity guard rails. Communicate the trade-offs between freshness, speed and cost.

A244. Company context: A CEO asks for a "real-time" operations dashboard showing orders as they happen on an e-commerce site, refreshed within sec...

Clarify "real time": often near real time (1 to 5 minutes) is enough, and the cost of true streaming is higher.

Options:

1. DirectQuery to an operational store or warehouse with automatic page refresh (Premium or fixed interval in Pro with minimum limits, or change-detection refresh on a measure). Simple, but load on the source and limited DAX/Power Query support.

2. Streaming dataset (push dataset via REST API, Azure Stream Analytics or Power Automate): data pushed into the Service shows in dashboard tiles instantly. Limited: no full DAX modeling, no historical analysis beyond a retention window, reports on push datasets have limited features. Newer approach: Microsoft Fabric Real-Time Intelligence (eventstream and KQL database) with Power BI reports connected via DirectQuery or Real-Time dashboards.

3. Hybrid tables with incremental refresh: history in Import and the latest partition in DirectQuery.

4. Direct Lake mode (Fabric): reads Delta tables with near-import speed and fresh data.

Recommendation: put events through an event stream into a KQL database or Delta table, build a model with historical Import data (or Direct Lake) plus live partition, use automatic page refresh set to a modest interval (for example 10 to 30 seconds with change detection) on a single focused page with a few simple cards and line charts, and a separate analytical page for deeper history. Test the load on the source and ensure the wall display uses a view-only account in a capacity workspace. Explain trade-offs to the CEO.

A245. Company context: A consumer goods company has sales facts at Day, Product and Store grain, but the budget is stored at Month and Product Cat...

Problem: facts at different grains cannot relate directly to each other without double counting; relating Budget to the Product table (grain Product) would repeat category budget for each product.

Step 1 - Conformed dimensions: create dimension tables at the budget grain: a Category dimension (distinct categories) and use Date's month (a Month/YearMonth key) as a month-level bridge. Dimensions: Date (day), Product (product -> category), Store/Region.

Step 2 - Relationships: Budget[CategoryKey] relates to Category dimension (one-to-many); Product also relates to Category (Product[CategoryKey] many-to-one to Category) so Category filters both Sales (via Product) and Budget. For time: create a Month table (first of month, one row per month) and relate Budget[Month] and Date[MonthStart] to it (or use the Date table with Budget on the month's first day and a measure-level fix). Slicers and axis use Category and Month dimension columns, not Product or Day.

Step 3 - Measures:

```
Actual Sales = SUM ( Sales[Amount] )
Budget = SUM ( Budget[Amount] )
Variance = [Actual Sales] - [Budget]
Variance % = DIVIDE ( [Variance], [Budget] )
```

Step 4 - Guard against invalid slicing: budget exists only at category/month/region, so when a user puts Product or Day in the visual the budget would repeat. Use:

```
Budget Safe = IF ( ISINSCOPE ( Product[ProductName] ) || ISINSCOPE ( 'Date'[Date] ), BLANK
(), [Budget] )
```

(or allocate budget down using a share-of-sales allocation if the business wants it).

Step 5 - Validate totals against the budget spreadsheet at each level, and document the grain limitation in tooltips.

A246. Company context: A subscription streaming service wants a customer retention analysis: for each monthly signup cohort, what percentage of cu...

Step 1 - Cohort definition: add a calculated column on Customer: CohortMonth = STARTOFMONTH-style date of signup:

```
CohortMonth = DATE ( YEAR ( Customer[SignupDate] ), MONTH ( Customer[SignupDate] ), 1 )
```

Step 2 - Month offset: Activity has a month column; offset = months between cohort and activity month. Add to Activity (using RELATED for cohort):

```
MonthsSinceSignup =
DATEDIFF ( RELATED ( Customer[CohortMonth] ), DATE ( YEAR ( Activity[ActivityDate] ), MONTH (
Activity[ActivityDate] ), 1 ), MONTH )
```

Step 3 - Measures:

```
Cohort Size = CALCULATE ( DISTINCTCOUNT ( Customer[CustomerKey] ), REMOVEFILTERS (
Activity[MonthsSinceSignup] ) )
Active Customers = DISTINCTCOUNT ( Activity[CustomerKey] )
Retention % = DIVIDE ( [Active Customers], [Cohort Size] )
```

Make sure Cohort Size is counted from Customers (not only from active ones) and not affected by the offset in the matrix columns.

Step 4 - Visual: matrix with CohortMonth on rows, MonthsSinceSignup on columns, Retention % as values, with a color-scale conditional format (heat map). Add a cohort-size column and an average row.

Step 5 - Blank future cells: cohorts have no data beyond today. Hide with IF(cohort month + offset > TODAY, BLANK()).

Step 6 - Validate on a small cohort manually. Mention alternative: if an Activity table is huge, precompute the cohort and offset in SQL or Power Query.

A247. Company context: Five departments (Sales, Marketing, Finance, Supply Chain, HR) each built their own Power BI reports with their own copies ...

Step 1 - Diagnose: inventory existing reports, find where definitions differ (revenue gross vs net, customer active definition), and agree on official definitions with business owners.

Step 2 - Build a governed, shared semantic model (a "gold" model): one dataset per subject area or a central enterprise model with conformed dimensions (Customer, Product, Date) and agreed measures. Source it from the warehouse or from dataflows so cleaning logic is shared. Place it in a dedicated workspace with controlled write access.

Step 3 - Certify and promote it (endorsement), and publish a data catalog description with definitions; set build permission so departments can create reports from it.

Step 4 - Convert department reports into thin reports connected live to the shared semantic model (Get Data > Power BI semantic models), keeping only report-specific visuals and a few report-level measures. Departments needing extra tables can use composite models on top of the shared model.

Step 5 - Apply security centrally (RLS in the shared model) and use sensitivity labels.

Step 6 - Governance process: ownership, change management with deployment pipelines, naming conventions, a review cycle for new measures, usage metrics to retire duplicate reports, and training so departments reuse instead of rebuilding.

Step 7 - Reconcile: run old and new side by side for a period, explain remaining differences, then retire legacy reports.

A248. Company context: A retailer wants to know which products drive 80 percent of revenue (ABC / Pareto analysis) and wants a Pareto chart in Pow...

Step 1 - Base measure: [Total Sales] = SUM(Sales[Amount]).

Step 2 - Cumulative share by rank. Rank products by sales and compute running share of the total within the visual's selection:

```
Product Sales Rank =
RANKX ( ALLSELECTED ( Product[ProductName] ), [Total Sales], , DESC, DENSE )

Cumulative Sales =
VAR CurrentRank = [Product Sales Rank]
RETURN
    SUMX (
        FILTER (
            ALLSELECTED ( Product[ProductName] ),
            RANKX ( ALLSELECTED ( Product[ProductName] ), [Total Sales], , DESC, DENSE ) <=
CurrentRank
        ),
        [Total Sales]
    )
```

```
Cumulative % =
DIVIDE ( [Cumulative Sales], CALCULATE ( [Total Sales], ALLSELECTED ( Product ) ) )

ABC Class =
SWITCH ( TRUE (), [Cumulative %] <= 0.8, "A", [Cumulative %] <= 0.95, "B", "C" )
```

Step 3 - Visual: line and clustered column chart. X axis: ProductName sorted by [Total Sales] descending; columns: [Total Sales]; line: [Cumulative %] on the secondary axis (0 to 100%). Add a constant line at 80%.

Step 4 - Slicers: because of ALLSELECTED the chart responds to date, region and category slicers.

Step 5 - Summary: another table counting products per ABC class and share of revenue.

Performance: this pattern is expensive for tens of thousands of products. Limit to category level, add a Top N filter, or precompute in the warehouse. Also note that ties in RANKX with DENSE may mix products; use a tiebreaker.

A249. Company context: In a matrix showing Margin % by product, the Total row displays 18 percent, but when analysts average the row values they g...

Step 1 - Understand: a measure is re-evaluated in the total row's own filter context (all products), it is not a sum of the rows above. For a ratio measure such as DIVIDE([Profit],[Total Sales]) the total is total profit divided by total sales, which is a weighted average, not the simple average of row percentages. That is correct: products with higher sales weigh more. The average of row percentages (24 percent) is statistically misleading.

Step 2 - Check the measure definition: if the measure uses SUM then no issue; if it uses an expression only valid for a single product (such as Sales[Price] picked with MAX or SELECTEDVALUE), the total row evaluates with many products and gives blank, an error, or the max value.

Step 3 - Fix Avg Price: compute weighted from totals: DIVIDE([Total Sales],[Total Units]), not AVERAGE of price column (simple average ignores quantity).

Step 4 - If the business wants totals to equal the sum of visible rows (for example a rounded or per-row calculation), build the measure with an iterator over the rows in the visual:

```
Sum Of Row Values =
SUMX ( VALUES ( Product[ProductName] ), [Row Level Measure] )
```

Or use ISINSCOPE to change the total row behavior:

```
Measure With Total =
IF ( ISINSCOPE ( Product[ProductName] ), [Row Calc], SUMX ( VALUES ( Product[ProductName] ),
[Row Calc] ) )
```

Step 5 - Communicate: add a tooltip or footnote that totals are computed on totals, and agree with finance on the intended definition; verify against source data.

A250. Company context: A finance department runs a recurring monthly Excel pack of profit and loss statements with pivot tables and wants to move ...

Step 1 - Discovery: collect the existing workbooks, document data sources, logic (formulas, manual adjustments), audience, and frequency; identify pain points (manual copy-paste, version chaos).

Step 2 - Prioritize: pick one high-value pack for a pilot; define success metrics (time saved, accuracy).

Step 3 - Rebuild the data layer: replace manual imports with Power Query connections to the ledger, ERP exports or warehouse; build a star schema with Account hierarchy, Cost Center, Entity and Date; capture manual adjustments in a controlled table instead of editing cells.

Step 4 - Recreate the calculations as DAX measures (Revenue, COGS, Gross Margin, Opex, EBITDA, variance vs budget and prior year) and a P&L matrix with a proper account hierarchy, sign conventions, and conditional formatting. For finance-style layouts consider the matrix with a calculation-group or custom ordering via an Account sort key.

Step 5 - Reconcile: compare totals line by line with the Excel pack for several months and document variances; fix until they tie out.

Step 6 - Keep the familiarity: offer "Analyze in Excel" (PivotTables connected live to the Power BI model) and export options so analysts can still use Excel, plus a table view.

Step 7 - Roll out: workspace app with RLS by entity, scheduled refresh before the month-end close deadlines, training sessions, a feedback loop, then retire the old pack after a parallel run of one or two cycles.

A251. Company context: A multinational shows revenue in USD, but regional managers want to see sales in their local currency and the CFO wants to ...

Step 1 - Model: Sales fact has Amount and CurrencyCode (local) and a Date key. Create a Rates table (Date, CurrencyCode, RateToUSD), and a Currency dimension (disconnected) used for the selector slicer.

Step 2 - Convert at transaction level in Power Query or the warehouse (merge rates by date and currency) to store AmountUSD. If conversion must be dynamic, use measures:

```
Selected Currency = SELECTEDVALUE ( Currency[Code], "USD" )

Sales USD =
SUMX ( Sales, Sales[Amount] * LOOKUPVALUE ( Rates[RateToUSD], Rates[Date], Sales[OrderDate],
Rates[CurrencyCode], Sales[CurrencyCode] ) )

Sales In Selected Currency =
VAR TargetRate =
    CALCULATE ( AVERAGE ( Rates[RateToUSD] ), Rates[CurrencyCode] = [Selected Currency],
LASTDATE ( 'Date'[Date] ) )
RETURN DIVIDE ( [Sales USD], TargetRate )
```

Step 3 - Scenario: create a What-if parameter "FX Shock %" from -10 to +10 percent with step 1 and a measure:

```
Sales USD Scenario = [Sales USD] * ( 1 + 'FX Shock'[FX Shock Value] )
```

Step 4 - Visuals: currency slicer (single select), a format string measure to show the right symbol (dynamic format string), a baseline versus scenario column chart.

Step 5 - Performance and accuracy: precompute rates per transaction for large data, use end-of-month versus transaction-date rates per finance policy, and document the policy. Test conversions against a few known transactions.

A252. Company context: A call center wants an operations dashboard with Average Handle Time (AHT), Service Level (percent of calls answered within...

Step 1 - Model: Calls fact (CallID, AgentKey, TeamKey, Date key, TimeKey, QueueSeconds, HandleSeconds, Answered flag, Abandoned flag), dimensions Agent, Team, Date and a Time-of-day dimension (hour, half-hour) to avoid a datetime column.

Step 2 - Measures:

```
Total Calls = COUNTROWS ( Calls )
Answered Calls = CALCULATE ( [Total Calls], Calls[Answered] = TRUE () )
Abandoned Calls = CALCULATE ( [Total Calls], Calls[Abandoned] = TRUE () )
AHT (sec) = DIVIDE ( SUM ( Calls[HandleSeconds] ), [Answered Calls] )
Answered In 20s = CALCULATE ( [Answered Calls], Calls[QueueSeconds] <= 20 )
Service Level % = DIVIDE ( [Answered In 20s], [Total Calls] )
Abandon Rate = DIVIDE ( [Abandoned Calls], [Total Calls] )
AHT (mm:ss) = FORMAT ( [AHT (sec)] / 86400, "nn:ss" )
```

Service level definitions vary (denominator answered versus offered); agree and document.

Step 3 - Conditional formatting: a status measure or rules (Service Level >= 80% green, 70 to 80 amber, below red) applied to cards and table via Format by rules or field value.

Step 4 - Layout: top KPI cards (Calls, SL %, AHT, Abandon Rate) with vs target; line chart of calls and service level by hour (column and line combo); heatmap matrix of day-of-week by hour for staffing; team ranking bar chart; table of agents with drill-through to an agent page. Slicers: date, team, queue.

Step 5 - Refresh: DirectQuery or frequent incremental refresh (every 30 minutes) with automatic page refresh for intraday monitoring. RLS: team leads see their team only.

A253. Company context: An HR analytics team tracks employees whose manager and department change over time (slowly changing dimension). Sales reps...

Problem: if the Rep dimension has only the current territory, historical sales get restated to the new territory. We need a Type 2 slowly changing dimension.

Step 1 - Dimension design: build RepTerritory history with one row per rep per assignment: RepAssignmentKey (surrogate), RepID, Territory, ValidFrom, ValidTo (open-ended as 9999-12-31), IsCurrent. Best done in the warehouse/ETL.

Step 2 - Assign the surrogate key to each sale at load time, in the ETL or Power Query: match the sale's RepID and where OrderDate between ValidFrom and ValidTo. Sales then carries RepAssignmentKey, and relates many-to-one to the dimension. Now slicing by Territory automatically uses the territory at time of sale, and a Rep-level view can use RepID to group history.

Step 3 - If you cannot change the ETL, derive in DAX with a virtual relationship:

```
Sales By Territory At Sale Date =
SUMX (
    RepAssignment,
    CALCULATE (
        [Total Sales],
        FILTER ( ALL ( 'Date'[Date] ), 'Date'[Date] >= RepAssignment[ValidFrom] &&
'Date'[Date] <= RepAssignment[ValidTo] ),
        TREATAS ( VALUES ( RepAssignment[RepID] ), Sales[RepID] )
    )
```

```
)
```

This is slower and harder to maintain; prefer the surrogate key approach.

Step 4 - Reporting: distinct count of reps uses RepID, not the surrogate key. Provide a "current territory" attribute in addition for "as is" reporting, and expose a toggle (as-was versus as-is) using a disconnected slicer if the business needs both.

Step 5 - Test using a rep who changed territory mid-year: sales before the move must be credited to the old territory, after the move to the new, and total sales must equal the unfiltered total.

A254. Company context: Executives want a single chart where they can choose which measure (Revenue, Profit, Units) and which breakdown (Region, Ca...

Option A - Field parameters (recommended, available in Desktop):

Step 1 - Modeling > New parameter > Fields. Create "Metric" parameter and add the measures [Total Sales], [Profit], [Units Sold] with display names. Create "Breakdown" parameter and add the columns Region[Name], Product[Category], Channel[Name].

Step 2 - Add both parameters as slicers (single select, tile or dropdown style).

Step 3 - Build one bar chart: put the Breakdown parameter field on the Axis and the Metric parameter field on Values. Power BI expands the selected field automatically. Titles can be dynamic with a measure:

```
Chart Title = SELECTEDVALUE ( Metric[Metric], "Revenue" ) & " by " & SELECTEDVALUE (
Breakdown[Breakdown], "Region" )
```

Step 4 - Number formats differ by metric (currency, count): use a dynamic format string on the parameter or per-measure formats.

Option B - Classic disconnected table plus SWITCH measure: create a table Metric with names, a SWITCH(SELECTEDVALUE(...)) measure returning the chosen measure; works in older versions and gives control over logic. Axis selection requires bookmarks or the field parameter feature.

Considerations: a default selection when none or multiple are chosen, ensure the slicer is single select, test with RLS and with tooltips, and keep a sensible layout. Result: simpler page, less rendering work, and user control.

A255. Company context: A fast-growing retailer has an Import-mode Power BI model of 4 GB that refreshes fully each night in 3 hours and now hits t...

Step 1 - Baseline: record refresh duration per table (refresh history, SQL Profiler or the Fabric capacity metrics app), model size, top memory consumers via VertiPaq Analyzer, and capacity utilization during peaks.

Step 2 - Reduce size: remove unused columns and tables; drop or split high-cardinality columns; replace text keys with integers; convert datetimes to date plus a Time dimension if needed; remove calculated columns by pushing logic upstream; set numeric precision lower; keep history only as long as the business needs, and move older history to aggregates.

Step 3 - Reduce refresh work: implement incremental refresh on large facts with detect-data-changes; ensure query folding of the date filter; refresh dimensions fully (small). Consider disabling "Include in report refresh" for static tables and refresh them on a separate schedule or via the enhanced refresh API for specific tables/partitions.

Step 4 - Reduce memory peaks: refresh happens by loading a shadow copy, so peak is up to about twice the model size. Avoid full refreshes during business hours, stagger large refreshes, use partition-level refresh through XMLA, and for the largest tables consider aggregations with detail in DirectQuery or Direct Lake in Fabric.

Step 5 - Optimize DAX queries that load the engine at peak times: fix the heaviest measures from the capacity metrics report.

Step 6 - Capacity: scale up temporarily (or enable autoscale) if the above is not enough, and separate heavy datasets onto different capacities.

Step 7 - Verify: re-run baselines; success is refresh time under an agreed target (for example 30 minutes), model size reduced significantly, no memory evictions or throttling in the capacity metrics app at peak, and unchanged report results confirmed by comparing totals before and after.

Excel

A256. Compare VLOOKUP, INDEX-MATCH and XLOOKUP. When would you choose each one?

VLOOKUP(lookup_value, table_array, col_index_num, [range_lookup]) searches the first column of a range and returns a value from a column to the right. Limitations: it cannot look left, the column index is a hard-coded number (so inserting or deleting a column can break or silently change results), and the default range_lookup is TRUE (approximate match), which gives wrong answers on unsorted data if you forget to type FALSE.

INDEX-MATCH combines two functions: MATCH finds the row position of the key, INDEX returns the value at that position from any column. It looks in any direction, does not depend on column numbers, and only references the two columns involved, so it is more robust and slightly faster on large sheets.

XLOOKUP(lookup_value, lookup_array, return_array, [if_not_found], [match_mode], [search_mode]) is the modern replacement (Excel 365/2021+). It defaults to exact match, looks left or right, has a built-in if_not_found argument, supports wildcard and approximate matches (next smaller / next larger), searches from last to first, can return multiple columns (spilling), and also works horizontally so it replaces HLOOKUP.

Choice: use XLOOKUP when everyone has a modern Excel version; use INDEX-MATCH when you must stay compatible with Excel 2016/2013 or share with others on older versions; use VLOOKUP only for quick, simple, right-looking lookups or legacy workbooks.

A257. What is the difference between exact match and approximate match in VLOOKUP, and when is approximate match appropriate?

With range_lookup = FALSE (or 0) VLOOKUP returns a value only when the lookup value matches exactly; otherwise it returns #N/A. With TRUE (or omitted) it assumes the first column is sorted ascending and finds the largest value that is less than or equal to the lookup value, using a binary search. If the data is not sorted the result can be wrong without any error.

Approximate match is appropriate for banded lookups: tax slabs, commission tiers, grade boundaries, shipping-weight bands. Example: a table in F2:G6 with lower bounds 0, 10000, 50000, 100000, 250000 in F and rates in G, sorted ascending.

```
=VLOOKUP(B2, $F$2:$G$6, 2, TRUE)
```

returns the rate for the highest threshold not exceeding B2. For key lookups such as employee ID or product code always use exact match:

```
=VLOOKUP(A2, Products!$A:$D, 3, FALSE)
```

The XLOOKUP equivalent for bands is =XLOOKUP(B2, $F$2:$F$6, $G$2:$G$6, , -1), where match_mode -1 means exact or next smaller.

A258. Explain relative, absolute and mixed cell references and give an example of when mixed references are useful.

A relative reference (A1) changes when a formula is copied: copy it one row down and it becomes A2. An absolute reference ($A$1) never changes, since both column and row are locked. A mixed reference locks only one part: $A1 locks the column but lets the row change; A$1 locks the row but lets the column change. F4 cycles through the four forms while editing.

Mixed references are ideal for two-way tables, where one formula is filled across and down. Example: a multiplication or pricing grid with prices in B1:F1 (row) and quantities in A2:A10 (column):

```
=$A2*B$1
```

Copy this to the whole grid: the quantity column stays fixed on A, the price row stays fixed on row 1. Without mixed references you would have to write each formula by hand. Other uses are a fixed tax-rate cell ($B$1), and sensitivity grids.

A259. What is a PivotTable, and how do you create one from raw data? What are its main areas?

A PivotTable is an interactive summary tool that groups, aggregates (sum, count, average, min, max, distinct count, etc.) and cross-tabulates a large dataset without writing formulas. It does not change the source data.

Steps: (1) make sure the data is a clean rectangular range with one header row, no blank rows or columns, no merged cells, ideally converted to an Excel Table (Ctrl+T). (2) Click any cell in the data, Insert > PivotTable, choose the source and a destination (new worksheet or the data model). (3) Drag fields into the four areas: Filters (report-level filter), Columns, Rows, and Values. (4) Change Value Field Settings to pick the aggregation and use Show Values As for % of total, running total, % difference from, rank, etc. (5) Format numbers, sort, group dates or numbers, and add slicers or a timeline.

Key points: it needs a refresh (Data > Refresh All) when the source changes unless the source is a Table and you refresh; text in a numeric column makes Excel default to Count instead of Sum; and pivot tables are fast on hundreds of thousands of rows.

A260. What is a pivot cache? Why does a PivotTable not update automatically, and how do you handle a source range that grows?

When a PivotTable is created Excel stores a copy of the source data in memory inside the workbook, called the pivot cache. The table reads from the cache, not live from the sheet, which is why it is fast but also why it does not update by itself when source cells change. You must refresh it (right-click > Refresh, Alt+F5, Data > Refresh All, VBA RefreshTable, or the option "Refresh data when opening the file").

Multiple pivots built from the same source share one cache if created by copy/paste or by choosing the existing connection, which keeps file size down. Building each pivot from scratch creates separate caches and inflates file size.

Growing data: if the source is a fixed range like A1:H5000, new rows added below are not included. Solutions: base the pivot on an Excel Table (it expands automatically), use a dynamic named range (INDEX-based, not OFFSET), or load via Power Query / the data model. Also, deleted items may linger in filter drop-downs; set PivotTable Options > Data > Number of items to retain = None.

A261. What is the difference between a calculated field and a calculated item in a PivotTable? What are their limitations?

A calculated field is a new field created with a formula that uses other fields in the pivot, for example Profit = Revenue - Cost, or Margin% = Profit / Revenue. It is computed after the underlying fields are summed, so it works on totals: a calculated Margin% is sum(Profit)/sum(Revenue), which is correct, whereas averaging row-level percentages would not be.

A calculated item is a new item within an existing field, for example in a Region field, creating "Domestic" = North + South. It works on the items of that field.

Limitations: calculated fields cannot refer to cell addresses, worksheet functions that need ranges, or other calculated fields' items; they use SUM of fields (so ratios per row are not available). Calculated items can make pivots slow, break grouping, and cannot be used with the data model or when a field is grouped. Better modern alternatives: add the column in the source data or Power Query, or create DAX measures in the data model (Power Pivot), which handle ratios, distinct counts and time intelligence properly.

A262. What are slicers and timelines, and how do they differ from regular PivotTable filters?

Slicers are visual button filters linked to a PivotTable, PivotChart or Excel Table. They show the current filter state clearly (selected buttons highlighted), allow multi-select with Ctrl, and one slicer can be connected to several pivots (Slicer > Report Connections) provided they share the same cache. Timelines are slicers designed for date fields; they let the user filter by days, months, quarters or years by dragging a bar.

Regular filters (Filters area or the row/column label drop-downs) are hidden in drop-down menus, only apply to one pivot, and don't show the active selection at a glance. Slicers and timelines are therefore preferred for dashboards because end users can see and use them without knowing pivots. They use more screen space and can slow very large workbooks when many are connected.

A263. What are dynamic array functions in Excel? Explain spilling, the # operator and name a few key functions.

Dynamic arrays (Excel 365 and 2021) let a single formula return multiple values that automatically "spill" into neighbouring cells. You write the formula in one cell and Excel fills the range, resizing it when the inputs change. The spilled range is shown with a blue border. If something blocks the spill range you get a #SPILL! error.

The spill range operator # refers to the whole spilled result, for example =SUM(A2#) sums everything the formula in A2 spills, and a data validation list can use =$E$2# so the drop-down grows automatically.

Key functions: FILTER (return rows meeting a condition), SORT and SORTBY, UNIQUE, SEQUENCE, RANDARRAY, XLOOKUP (can return multiple columns), XMATCH, TAKE, DROP, CHOOSECOLS, VSTACK, HSTACK, TOCOL, LET and LAMBDA. Example:

```
=SORT(FILTER(A2:D100, C2:C100>1000), 4, -1)
```

returns all rows with column C above 1000, sorted by column 4 descending. Benefits: no Ctrl+Shift+Enter, fewer helper columns, formulas that adjust to data size. Caveats: not supported in older versions, and implicit intersection with @ may appear in files from older Excel.

A264. What are volatile functions in Excel? Give examples and explain why they matter for performance.

A volatile function recalculates every time any cell in the workbook changes (or on every calculation), even if its own inputs did not change. Examples: NOW, TODAY, RAND, RANDBETWEEN, RANDARRAY, OFFSET, INDIRECT, and in some contexts INFO and CELL. Any formula that depends on a volatile function is recalculated too, so a chain of dependents becomes expensive.

In large workbooks, heavy use of OFFSET and INDIRECT (for example in thousands of lookup formulas or dynamic ranges) causes slow typing, long save times and sluggish scrolling. Alternatives: replace OFFSET-based dynamic ranges with INDEX (non-volatile): =SUM(A2:INDEX(A:A, COUNTA(A:A))); replace INDIRECT sheet references with direct references, CHOOSE, or Power Query; use a single cell with TODAY() and reference it rather than calling TODAY many times; convert random numbers to values once finished (Paste Special > Values). You can also set calculation to manual (F9 to recalc) while building large models.

A265. What are named ranges and Excel Tables, and what advantages do structured references give over normal cell references?

A named range gives a cell or range a meaningful name (Formulas > Define Name, or the Name Box), such as TaxRate or Sales_2024, which you can use in formulas: =Revenue*TaxRate. Names can be workbook or sheet scoped, can refer to constants or formulas (dynamic names), and are managed in the Name Manager (Ctrl+F3).

An Excel Table (Ctrl+T) is a structured range with a name (for example tblSales) and named columns. Formulas use structured references such as =SUM(tblSales[Amount]) or =[@Qty]*[@Price]. Advantages: formulas are readable; ranges expand automatically when rows are added, so charts, pivots, data validation and formulas stay current; calculated columns fill down automatically; automatic banding, filter buttons and a total row; and they are ideal sources for Power Query and PivotTables. Names are also useful for documenting assumptions in models and for easier auditing, though over-using them can make logic harder to trace if the naming is not consistent.

A266. Explain Power Query vs Power Pivot in Excel. When would you use each?

Power Query (Data > Get & Transform) is the ETL tool: Extract data from files, folders, databases, web and APIs; Transform it (remove columns, filter, split, unpivot, merge, append, change types, group, custom columns in the M language); and Load it to a sheet or the data model. Every step is recorded, so it is repeatable: when new data arrives, you click Refresh and all cleaning steps rerun. It replaces manual copy-paste cleaning and many lookup formulas (by merging queries).

Power Pivot is the data modelling and calculation engine (xVelocity/VertiPaq in-memory columnar store). You load multiple tables, define relationships (star schema), and write DAX measures such as Total Sales := SUM(Sales[Amount]) or year-over-year using time-intelligence functions. It handles millions of rows beyond the 1,048,576 row limit and supports distinct count, many-to-one relationships and PivotTables across tables without VLOOKUPs.

Typical workflow: Power Query cleans and shapes data, loads it to the data model; Power Pivot defines relationships and DAX measures; PivotTables, PivotCharts and slicers form the report. Use Power Query alone for cleaning, merging files and reshaping; add Power Pivot when you need multi-table analysis, large volumes or advanced measures.

A267. What types of conditional formatting are available in Excel, and how do formula-based rules work?

Conditional formatting (Home > Conditional Formatting) changes cell appearance automatically based on values. Built-in types: Highlight Cells rules (greater than, between, text contains, duplicate values, dates occurring), Top/Bottom rules (top 10 items, above average), Data Bars, Color Scales (2- or 3-colour gradients) and Icon Sets (arrows, traffic lights).

Formula-based rules (New Rule > Use a formula to determine which cells to format) give full control. The formula is written relative to the top-left cell of the applied range and must return TRUE or FALSE. Use mixed references to colour the whole row based on one column. Example: applying to A2:H100, highlight rows where Status in column E is "Overdue":

```
=$E2="Overdue"
```

Highlight duplicates in column A: =COUNTIF($A$2:$A$100,$A2)>1. Highlight dates older than 30 days: =$D2<TODAY()-30. Common mistakes: wrong anchor (using $E$2 colours everything the same), applying the rule to the wrong range, and conflicting rules (order in Manage Rules and Stop If True matter).

A268. What is data validation, and what kinds of restrictions can you apply? How would you create a drop-down list?

Data validation (Data > Data Validation) restricts what users can enter in a cell and so protects data quality. Allowed types: Whole number, Decimal, List, Date, Time, Text length and Custom (formula). You can also set an Input Message (a tooltip), and an Error Alert with a Stop, Warning or Information style.

To create a drop-down: select the cells, Data Validation > Allow: List > Source. Either type comma-separated items ("Yes,No,Maybe"), reference a range (=$F$2:$F$10), a named range, a Table column via a named range, or a spill range (=$F$2#) so the list grows automatically. Custom example: allow only unique IDs in A2:A500 using =COUNTIF($A$2:$A$500,A2)=1; allow only dates in the future: =A2>TODAY().

Dependent drop-downs use INDIRECT or, better, a dynamic formula (e.g., FILTER) to return items based on the first selection. Limitations: validation does not stop pasted values, and existing invalid data is not removed; use Data > Data Validation > Circle Invalid Data to find it.

A269. Describe Excel's What-If Analysis tools: Scenario Manager, Goal Seek and Data Tables. When would you use each?

Goal Seek (Data > What-If Analysis > Goal Seek) works backward on a single input: you set a formula cell to a target value by changing one input cell. Example: find the sales volume required to reach a profit of 500,000, or the interest rate that gives a required loan payment. It returns one solution and is a one-off (not dynamic).

Scenario Manager stores named sets of values for up to 32 changing cells (for example Best, Base, Worst case with different growth, cost and price assumptions), lets you switch between them and creates a Scenario Summary report. It is simple for a few discrete cases; many analysts instead use a scenario selector cell with CHOOSE or INDEX.

Data Tables (one-variable and two-variable) recalculate a formula for many input values automatically using the {=TABLE()} array. Example: show NPV or monthly payment for a grid of interest rates and loan terms. Two-variable tables need the output formula in the top-left corner cell, the row input cell and the column input cell. They are great for sensitivity analysis but can slow calculations (set Automatic except for Data Tables if needed).

For optimization with constraints, use Solver instead.

A270. What is Excel Solver and what are its three components? Which solving methods does it offer?

Solver is an add-in (File > Options > Add-ins > Manage Excel Add-ins > Solver Add-in) for optimization. Its three components are: (1) the Objective cell, a formula you want to maximize, minimize or set to a value (profit, cost, risk); (2) Variable cells (decision variables) that Solver changes; and (3) Constraints, conditions such as capacity <= limit, quantities integer or binary, or non-negative.

Methods: Simplex LP (for linear models; fast and guaranteed to find the global optimum, can provide Sensitivity and Limits reports), GRG Nonlinear (smooth nonlinear problems; finds a local optimum, so try different starting values), and Evolutionary (non-smooth or discontinuous models with IF/lookup logic; random search, not guaranteed optimal). Typical uses: product mix, staff scheduling, budget allocation, transport/assignment problems, portfolio weights.

Practice points: tick "Make Unconstrained Variables Non-Negative", build the model so the objective is a pure formula of the variable cells, and keep the Answer/Sensitivity reports for the explanation of shadow prices and binding constraints.

A271. What is a macro, and how does it relate to VBA? How do you record and run one, and what are the risks?

A macro is a stored set of instructions that automates repetitive tasks. In Excel, macros are written in VBA (Visual Basic for Applications), the programming language built into Office. You can record a macro (View > Macros > Record Macro, or Developer tab), perform the actions, stop recording, and Excel generates VBA code, visible in the VBA editor (Alt+F11). You can assign it to a shortcut key, a button or a shape, and run it via Alt+F8.

Workbooks containing macros must be saved as .xlsm (or .xlsb). Recorded code is often inefficient (uses Select and ActiveCell, absolute addresses), so a good practice is to edit it: use Range objects directly, loops, variables, and turn off Application.ScreenUpdating during execution.

Risks: macros can contain malicious code, so Excel blocks them from internet-origin files by default; only enable macros from trusted sources or use trusted locations / digital signatures. Macros are not undoable with Ctrl+Z, so test on copies. They also don't run in Excel for the web, so for cloud workflows consider Office Scripts or Power Automate.

A272. How do you choose the right chart type in Excel? Give guidance for common analytical goals.

Pick the chart based on the question you are answering. Comparison across categories: clustered column or bar (bar for long labels). Trend over time: line chart (column for few periods). Part-to-whole: stacked column or 100% stacked, or pie/doughnut only for few (up to 5) categories adding to 100%. Relationship between two numeric variables: scatter plot (XY), bubble chart for a third variable. Distribution: histogram or box and whisker. Ranking: sorted bar chart. Contribution to a total change: waterfall chart. Process/funnel stages:

funnel chart. Combining different units (e.g., revenue as columns and margin % as a line): combo chart with a secondary axis. Geographic: filled map. Compact trends in cells: sparklines.

Good practice: start axes at zero for bars, avoid 3-D effects and heavy gridlines, title with the message (insight), label directly rather than relying on legends, use consistent colours, order categories logically, and limit the number of series. Charts built from Tables or dynamic ranges update automatically.

A273. What is an array formula? Explain the difference between legacy Ctrl+Shift+Enter formulas and dynamic array formulas, with an example.

An array formula performs calculations on a set of values rather than a single value, and may return one value or many. In legacy Excel (before 365), you confirmed array formulas with Ctrl+Shift+Enter, and Excel wrapped them in braces {=...}. Multi-cell results needed you to select the output range first, and the range could not be resized easily.

Example: sum of sales for a given region without SUMIF, using a conditional array:

```
{=SUM(IF(A2:A100="East", C2:C100))}
```

or non-CSE using SUMPRODUCT: =SUMPRODUCT((A2:A100="East")*C2:C100). A classic legacy example is the unique count: {=SUM(1/COUNTIF(A2:A100,A2:A100))}.

In dynamic-array Excel, any formula is array-aware: just press Enter, =SUM(IF(A2:A100="East",C2:C100)) works, and formulas like =UNIQUE(A2:A100) spill into multiple cells. Old CSE formulas still work and appear with braces. Array formulas can be slow on big ranges, so prefer SUMIFS/COUNTIFS where possible, and avoid whole-column references inside array operations.

A274. What are the best practices for designing an Excel dashboard for business stakeholders?

Define the audience and decisions first: what 5 to 8 KPIs matter, and what comparison (target, prior period) makes each meaningful. Then structure the workbook in three layers: a Data sheet (raw data in Tables, or loaded with Power Query), a Calculations/Model sheet (pivots, helper formulas, named ranges), and a Dashboard sheet (output only, with gridlines off).

Design: place the most important KPIs top-left as big numbers with variance vs target and conditional icons; follow with trend charts and breakdowns; keep a consistent colour palette and fonts; use the right chart per question; align objects to a grid; and keep whitespace. Interactivity: slicers and timelines connected to multiple pivots, drop-downs driven by data validation, dynamic titles using formulas (="Sales for "&B2).

Robustness: base everything on Tables or the data model so it refreshes in one click (Refresh All); protect formulas and sheets; document assumptions and the data refresh date; test with edge cases (empty periods, #N/A); print setup for one page. Avoid cluttered 3-D charts, excessive colours, hard-coded numbers, and volatile or overly complex formulas that make the file slow.

A275. List the common Excel error values (#N/A, #REF!, #VALUE!, #DIV/0!, #NAME?, #NUM!, #NULL!, #SPILL!) and how you handle them.

#N/A: value not available, typically a lookup with no match. #REF!: a reference is invalid, such as a deleted row or column. #VALUE!: wrong data type, e.g., adding text to a number or a space-containing "number". #DIV/0!: division by zero or blank. #NAME?: unrecognised function or name (typo, missing add-in or named range). #NUM!: invalid numeric operation (square root of a negative, an iteration that fails to converge, number too

large). #NULL!: incorrect range operator (space between ranges that do not intersect). #SPILL!: a dynamic array's spill range is blocked or too large. ##### is not an error: the column is too narrow.

Handling: diagnose first using Formulas > Evaluate Formula, Trace Precedents and Error Checking. Wrap with IFERROR(value, alternative) for general cases, but prefer IFNA for lookups so other real errors are not hidden, and prevent division errors with =IF(B2=0,0,A2/B2). Do not blanket-mask errors in a model: unseen errors lead to wrong totals. Use XLOOKUP's if_not_found argument for missing keys.

A276. What Excel features and functions do you use for data cleaning before analysis?

Text: TRIM (removes extra spaces), CLEAN (non-printing characters), SUBSTITUTE (replace characters, e.g. non-breaking space CHAR(160)), PROPER/UPPER/LOWER, LEFT/RIGHT/MID, TEXTBEFORE/TEXTAFTER/TEXTSPLIT (365), Flash Fill (Ctrl+E) and Text to Columns. Numbers and dates stored as text: VALUE, DATEVALUE, multiply by 1, or Text to Columns > Date; check with ISTEXT/ISNUMBER. Duplicates: Data > Remove Duplicates, UNIQUE, conditional formatting for duplicates, COUNTIFS flags. Blanks and errors: Go To Special > Blanks, IFERROR, filter and fill down. Inconsistent categories: Find & Replace, lookup mapping table, or fuzzy merge in Power Query. Validation: Data Validation to prevent bad entry, and Circle Invalid Data.

For repeatable workflows, do the cleaning in Power Query (trim, clean, change type, replace values, remove duplicates, remove errors, split columns, unpivot), because the steps are recorded and rerun on refresh. Always keep a copy of the raw data untouched, document each change, and compare row counts and totals before and after cleaning.

A277. A large workbook with many formulas is slow. How do Excel's calculation modes work, and what would you do to improve performance?

Calculation modes (Formulas > Calculation Options): Automatic (recalculates dependents whenever a precedent changes), Automatic except for Data Tables, and Manual (recalculates on F9, Shift+F9 for the active sheet). Excel uses a dependency tree and recalculates only dirty cells, except volatile functions that always recalculate.

Improvement steps: (1) Identify the cause: find volatile functions (OFFSET, INDIRECT, TODAY, RAND), array formulas over huge ranges, whole-column references in SUMPRODUCT or array formulas, and lots of VLOOKUPs on unsorted data. (2) Replace repeated lookups with one helper column, or use XLOOKUP/INDEX-MATCH with exact ranges. Use approximate-match binary search on sorted data where appropriate. (3) Replace SUMPRODUCT/array formulas with SUMIFS/COUNTIFS. (4) Use Tables or exact ranges instead of full columns where a function evaluates the entire array. (5) Move heavy transformations to Power Query or the data model; use PivotTables instead of thousands of SUMIFS. (6) Convert finished calculations to values. (7) Remove unused styles, excessive conditional formatting and formatting of the whole sheet; fix the used range. (8) Save as .xlsb to reduce size, enable multi-threaded calculation, avoid links to closed workbooks. (9) Use LET to avoid recalculating the same sub-expression. During development, switch to manual calculation.

A278. What is the difference between SUMIF, SUMIFS and SUMPRODUCT? When would you prefer each?

SUMIF(range, criteria, [sum_range]) sums values that meet one condition. SUMIFS(sum_range, criteria_range1, criteria1, criteria_range2, criteria2, ...) sums values that meet all of several conditions (AND logic); note that the sum_range comes first in SUMIFS but last in SUMIF. All criteria ranges must be the same

size as sum_range. They support operators in text (">=100", "<>East") and wildcards (*, ?), and concatenating with cell references (">="&B1). They are fast and ignore errors in criteria columns.

SUMPRODUCT(array1, array2, ...) multiplies arrays element by element and sums the products. It can do conditional sums with arbitrary logic, including OR logic and calculations on arrays, for example =SUMPRODUCT((A2:A100="East")+(A2:A100="West"), C2:C100) or weighted averages =SUMPRODUCT(Q,P)/SUM(Q). It is more flexible but slower, and breaks on text in numeric arrays unless coerced.

Preference: SUMIFS/COUNTIFS for AND-based conditional aggregation; SUMPRODUCT for weighted calculations, OR logic, or conditions on computed values (such as YEAR(date)=2024) that SUMIFS cannot evaluate; or in modern Excel, SUM(FILTER(...)) or SUM with array logic.

A279. What are the benefits of converting a range into an Excel Table?

Converting a range to a Table (Ctrl+T) provides: automatic expansion when you add rows or columns, so formulas, charts, PivotTables, data validation lists and conditional formats pick up new data; calculated columns that auto-fill consistently (one formula per column); structured references with readable names (=[@Quantity]*[@Price], =SUM(tblSales[Total])); built-in filter buttons and sorting; banded formatting; a Total Row with SUBTOTAL functions; slicers for tables; stable naming (tblSales) for Power Query and VBA (ListObjects); and easier removal of duplicates. Tables improve data integrity by keeping the dataset contiguous with a single header row.

Limitations: you cannot have merged cells in tables, structured references can look unfamiliar to people used to A1 references, Tables cannot be shared in legacy shared workbooks, and the dynamic spill functions cannot be spilled inside a table.

A280. Explain LET and LAMBDA in Excel. How do they improve formulas, and what is a practical example?

LET(name1, value1, [name2, value2], ..., calculation) assigns names to intermediate results inside a single formula. Benefits: the expression is calculated once and reused (faster), formulas are more readable, and long formulas are shorter. Example: average of sales above the average:

```
=LET(s, B2:B100, m, AVERAGE(s), AVERAGE(FILTER(s, s>m)))
```

LAMBDA lets you define your own reusable function without VBA. Syntax: LAMBDA(parameter1, ..., calculation). You test it by calling it immediately: =LAMBDA(x, x*1.18)(100). To reuse it, save it in Name Manager with a name, such as AddGST, and call =AddGST(100). Lambda helper functions (MAP, REDUCE, SCAN, BYROW, BYCOL, MAKEARRAY) enable loops over arrays. Example: running total with =SCAN(0, B2:B13, LAMBDA(a,v, a+v)). Practical uses: reusable cleaning function (e.g., CleanName =LAMBDA(t, PROPER(TRIM(t)))), recursive calculations, standard KPI calculations used across a workbook. Caveats: needs Microsoft 365, and workbooks using it are not backwards compatible; document each named LAMBDA.

A281. A reference table in A1:D500 has Names in column A and Employee IDs in column D (the ID column is to the right of the Name column). Write a ...

The lookup column (ID) is in D and the return column (Name) is in A, which is to its left, so VLOOKUP cannot do it directly.

INDEX-MATCH:

```
=INDEX($A$2:$A$500, MATCH(G2, $D$2:$D$500, 0))
```

MATCH with 0 returns the row position of an exact match for G2 in column D, and INDEX returns the item in the same position from column A.

XLOOKUP:

```
=XLOOKUP(G2, $D$2:$D$500, $A$2:$A$500, "Not found")
```

Both work leftwards. Wrap INDEX-MATCH in IFNA(..., "Not found") to handle missing IDs. If IDs might be stored as text in one list and numbers in the other, convert with VALUE or TEXT, or the match will fail with #N/A.

A282. A sales sheet has Region in column A, Product in column B and Price in column C (rows 2 to 1000). Write formulas to return the price for the...

(a) INDEX-MATCH with a multi-criteria match (array logic; in pre-365 Excel confirm with Ctrl+Shift+Enter):

```
=INDEX($C$2:$C$1000, MATCH(1, ($A$2:$A$1000=G1)*($B$2:$B$1000=G2), 0))
```

The multiplication creates an array of 1s where both conditions are true; MATCH finds the first 1.

(b) XLOOKUP, where the arrays are concatenated:

```
=XLOOKUP(1, ($A$2:$A$1000=G1)*($B$2:$B$1000=G2), $C$2:$C$1000, "No match")
```

Alternative: =XLOOKUP(G1&"|"&G2, $A$2:$A$1000&"|"&$B$2:$B$1000, $C$2:$C$1000).

(c) SUMIFS returns a number when exactly one row matches:

```
=SUMIFS($C$2:$C$1000, $A$2:$A$1000, G1, $B$2:$B$1000, G2)
```

This returns 0 when there is no match and a sum if there are duplicates, so use it only if the combination is unique. Another alternative is a helper column D with =A2&"|"&B2 and an ordinary lookup on it.

A283. Write a formula that assigns a customer discount based on order value using a lookup table: 0 for under 1,000; 5% for 1,000 to 4,999; 10% fo...

Lookup table approach (preferred, easier to maintain): put thresholds in F2:F5 (0, 1000, 5000, 20000) and the discounts in G2:G5 (0, 0.05, 0.10, 0.15), sorted ascending by threshold.

```
=XLOOKUP(B2, $F$2:$F$5, $G$2:$G$5, 0, -1)
```

Here match_mode -1 means exact match or next smaller item. Legacy equivalent:

```
=VLOOKUP(B2, $F$2:$G$5, 2, TRUE)
```

or =LOOKUP(B2, $F$2:$F$5, $G$2:$G$5).

IFS approach (hard-coded; use only for fixed rules):

```
=IFS(B2>=20000, 0.15, B2>=5000, 0.10, B2>=1000, 0.05, TRUE, 0)
```

Conditions must be ordered from the highest threshold downward because IFS returns the first TRUE. A nested IF version is =IF(B2>=20000,0.15,IF(B2>=5000,0.1,IF(B2>=1000,0.05,0))). The table approach is better because business users can change rates without editing formulas, and the logic is auditable.

A284. Dates are in column A (A2:A1000) and amounts in column C. Write formulas to get total sales for March 2025 (a) using SUMIFS with date bounda...

(a) Put the first day of the month in F1 (for example 2025-03-01). Then:

```
=SUMIFS($C$2:$C$1000, $A$2:$A$1000, ">="&F1, $A$2:$A$1000, "<"&EDATE(F1,1))
```

This is robust: it works with date-times (timestamps) because the upper bound is strictly less than the first day of the next month. An alternative upper bound is "<="&EOMONTH(F1,0) but it would miss transactions later on the last day if the dates contain times.

(b) SUMPRODUCT:

```
=SUMPRODUCT((YEAR($A$2:$A$1000)=2025)*(MONTH($A$2:$A$1000)=3)*$C$2:$C$1000)
```

It is flexible, but slower on big data, and it returns #VALUE! if the date column contains text or blanks that are not dates. SUMIFS is preferred for speed. A month-total for a pivot would use Group by Months and Years.

A285. Customer IDs are in A2:A1000 with duplicates. Write formulas to (a) count distinct customers, (b) flag the first occurrence of each customer...

(a) Distinct count in Excel 365:

```
=COUNTA(UNIQUE(A2:A1000))
```

Legacy (no blanks in the range):

```
=SUMPRODUCT(1/COUNTIF(A2:A1000, A2:A1000))
```

With possible blanks: =SUMPRODUCT((A2:A1000<>"")/COUNTIF(A2:A1000, A2:A1000&"")).

(b) First-occurrence flag in B2 (fill down):

```
=IF(COUNTIF($A$2:A2, A2)=1, "First", "Repeat")
```

The expanding range $A$2:A2 counts occurrences up to the current row.

(c) Customers with more than one purchase:

```
=SUMPRODUCT(--(COUNTIF(A2:A1000, UNIQUE(A2:A1000))>1))
```

or in 365: =ROWS(FILTER(UNIQUE(A2:A1000), COUNTIF(A2:A1000, UNIQUE(A2:A1000))>1)). A PivotTable alternative: add the data to the data model and use Distinct Count in Value Field Settings.

A286. Given a sales table in A1:D500 (Rep, Region, Month, Revenue), write dynamic-array formulas to list the top 5 reps by revenue for Region "Wes...

First aggregate revenue per rep within West, then sort and take the top 5.

```
=LET(
  d, FILTER(A2:D500, B2:B500="West"),
  reps, UNIQUE(INDEX(d,,1)),
  tot, SUMIFS(INDEX(d,,4), INDEX(d,,1), reps),
  TAKE(SORTBY(HSTACK(reps, tot), tot, -1), 5)
)
```

Explanation: FILTER keeps only the West rows; UNIQUE gets the list of reps; SUMIFS (with an array of criteria) gives each rep's total; HSTACK puts the names and totals side by side; SORTBY orders them descending by total; TAKE returns the first 5 rows. The result spills into 5 rows by 2 columns.

Without LET/TAKE (older approach, helper columns): in a pivot, filter Region = West, sort Revenue descending, and apply a Top 10 filter on Rep with value 5. For a single largest value use =LARGE(range,k) with INDEX-MATCH to return the name; ties require a tiebreaker.

A287. Names are in column A like " john SMITH " with extra spaces and inconsistent case. Write formulas to clean and standardise names, and to ...

Clean spaces and fix case:

```
=PROPER(TRIM(CLEAN(SUBSTITUTE(A2, CHAR(160), " "))))
```

SUBSTITUTE converts non-breaking spaces (common in web data) to normal spaces, CLEAN removes non-printing characters, TRIM removes leading, trailing and repeated spaces (leaving single spaces), PROPER capitalises each word. Result: "John Smith". Beware of names like "McDonald" or "O'Neil" which PROPER mishandles.

Assume the cleaned name is in B2. First name:

```
=LEFT(B2, FIND(" ", B2&" ") - 1)
```

Last name:

```
=TRIM(MID(B2, FIND(" ", B2&" ") + 1, 100))
```

In Excel 365: =TEXTBEFORE(B2," ",1,,,B2) for the first name and =TEXTAFTER(B2," ") for the rest; or =TEXTSPLIT(B2," ") to split all words. Flash Fill (Ctrl+E) and Text to Columns (delimiter: space) also work. Finally, paste values and remove the helper columns.

A288. Monthly revenue is in B2:B13. Write formulas for the running (cumulative) total, the percentage share of the annual total, and the month-ove...

Running total in C2, filled down:

```
=SUM($B$2:B2)
```

The anchored first cell with a relative end cell makes an expanding range. 365 alternative: =SCAN(0, B2:B13, LAMBDA(a,v,a+v)).

Percentage of total in D2:

```
=B2/SUM($B$2:$B$13)
```

Format as percent. Or use a pivot table with Show Values As > % of Grand Total.

Month-over-month growth in E3 (first month has no prior value):

```
=IF(B2=0, "", (B3-B2)/B2)
```

or =B3/B2-1, with IFERROR/IF to avoid #DIV/0! when the prior month is zero. For year-over-year with monthly data in a table use =B14/B2-1. Be careful about formatting as percentage and about the base: growth from a very small base looks huge.

A289. Compute a sales rep's commission with tiered marginal rates: 2% on the first 50,000, 4% on the portion from 50,000 to 100,000, and 6% on any...

Marginal (progressive) commission applies each rate only to its slice, like income tax. Direct formula using MAX/MIN:

```
=0.02*MIN(B2,50000) + 0.04*MAX(0, MIN(B2,100000)-50000) + 0.06*MAX(0, B2-100000)
```

Example: revenue 120,000 gives 1,000 + 2,000 + 1,200 = 4,200.

Table-driven version that is easier to maintain: thresholds in F2:F4 (0, 50000, 100000), rates in G2:G4 (0.02, 0.04, 0.06), and the incremental rate (rate difference) in H2:H4 (0.02, 0.02, 0.02):

```
=SUMPRODUCT((B2>$F$2:$F$4)*(B2-$F$2:$F$4)*$H$2:$H$4)
```

where H is the difference in rate from the previous tier. Both give 4,200 for 120,000. The common mistake is applying the top rate to the whole revenue (cliff structure, 7,200 in this example), which is only correct if the plan says the rate applies to all revenue once a tier is reached; in that case use a plain approximate lookup: =B2*VLOOKUP(B2,$F$2:$G$4,2,TRUE).

A290. Write formulas for the following date tasks: employee tenure in years and months from a hire date in B2, the last day of the month for an in...

Tenure in complete years and months:

```
=DATEDIF(B2, TODAY(), "Y") & " years, " & DATEDIF(B2, TODAY(), "YM") & " months"
```

DATEDIF is an undocumented but working function; "Y" is complete years, "YM" is remaining months after the years.

End of month for the invoice date:

```
=EOMONTH(C2, 0)
```

(EOMONTH(C2,1) gives the end of next month.) Start of month: =EOMONTH(C2,-1)+1.

Due date 30 working days after invoice date, skipping weekends and holidays:

```
=WORKDAY(C2, 30, $H$2:$H$20)
```

If the weekend is not Saturday/Sunday, use WORKDAY.INTL(C2, 30, 11, $H$2:$H$20) (11 = Sunday only). For 30 calendar days simply =C2+30.

Working days between start in D2 and end in E2:

```
=NETWORKDAYS(D2, E2, $H$2:$H$20)
```

It counts both start and end dates inclusively. Remember that dates are serial numbers; if a date shows as a number or does not calculate it may be stored as text, so convert with DATEVALUE.

A291. Quantities are in B2:B20 and unit prices in C2:C20. Write formulas for the total revenue without a helper column, the weighted average price...

Total revenue (sum of quantity times price):

```
=SUMPRODUCT(B2:B20, C2:C20)
```

Weighted average price (price weighted by quantity):

```
=SUMPRODUCT(B2:B20, C2:C20) / SUM(B2:B20)
```

This differs from the simple average AVERAGE(C2:C20), which treats all prices equally regardless of volume.

Average price excluding zero-quantity rows (simple average of prices):

```
=AVERAGEIFS(C2:C20, B2:B20, ">0")
```

If you want the weighted average only for rows where quantity is positive, zero quantity already contributes nothing to the weighted formula, so the first weighted formula is already correct. Guard against division by zero with =IF(SUM(B2:B20)=0, 0, SUMPRODUCT(B2:B20,C2:C20)/SUM(B2:B20)).

A292. Write a conditional formatting formula to (a) highlight duplicate emails in A2:A1000, case-insensitively and ignoring trailing spaces, and (...

(a) Select A2:A1000, Conditional Formatting > New Rule > Use a formula, then enter:

```
=SUMPRODUCT(--(TRIM(LOWER($A$2:$A$1000))=TRIM(LOWER($A2))))>1
```

COUNTIF is already case-insensitive, so the simplest formula is =COUNTIF($A$2:$A$1000,$A2)>1; however COUNTIF will not ignore trailing spaces, and it treats long text strings over 255 characters incorrectly, so the SUMPRODUCT version is more rigorous. Anchor the range with $ and keep the row relative.

(b) Select A2:F500 and use:

```
=AND($E2<TODAY(), $F2<>"Paid")
```

The $ before the column letters means every cell in the row evaluates columns E and F of its own row, so the whole row is coloured. Blank due dates evaluate as 0 and therefore are "past", so add $E2<>"" when blanks are possible: =AND($E2<>"", $E2<TODAY(), $F2<>"Paid").

A293. Rank salespeople by revenue in a sheet where ties exist, producing unique ranks with ties broken by the order in the list, and a dense rank....

Standard ranking: RANK.EQ gives tied values the same rank and skips the next numbers (1,2,2,4):

```
=RANK.EQ(B2, $B$2:$B$50, 0)
```

Unique rank, breaking ties by order of appearance:

```
=RANK.EQ(B2, $B$2:$B$50, 0) + COUNTIF($B$2:B2, B2) - 1
```

The expanding COUNTIF counts how many equal values appeared up to this row, so the first tied item keeps its rank and the following ones are pushed down.

Dense rank (1,2,2,3):

```
=SUMPRODUCT(($B$2:$B$50>B2)/COUNTIF($B$2:$B$50, $B$2:$B$50)) + 1
```

It counts the number of distinct higher values, then adds 1. In 365: =XMATCH(B2, SORT(UNIQUE($B$2:$B$50),,-1)).

Second-highest seller's name (assuming a unique second value):

```
=INDEX($A$2:$A$50, MATCH(LARGE($B$2:$B$50, 2), $B$2:$B$50, 0))
```

If the top value is tied, LARGE(...,2) returns the same tied value; use the dense approach on UNIQUE values: =XLOOKUP(LARGE(UNIQUE($B$2:$B$50),2), $B$2:$B$50, $A$2:$A$50).

A294. Write a VBA macro that loops through the rows of the active sheet's used range, highlights in red every row where the value in column D is b...

Deleting rows must be done from the bottom up, because deleting a row shifts the rows below it.

```
Sub CleanAndHighlight()
    Dim ws As Worksheet, lastRow As Long, i As Long
    Set ws = ActiveSheet
    Application.ScreenUpdating = False
    Application.Calculation = xlCalculationManual
    lastRow = ws.Cells(ws.Rows.Count, "A").End(xlUp).Row
    For i = ws.UsedRange.Rows.Count To 2 Step -1
        If Application.WorksheetFunction.CountA(ws.Rows(i)) = 0 Then
            ws.Rows(i).Delete
        ElseIf IsNumeric(ws.Cells(i, "D").Value) And ws.Cells(i, "D").Value <> "" Then
            If ws.Cells(i, "D").Value < 0 Then
                ws.Rows(i).Interior.Color = RGB(255, 199, 206)
            End If
        End If
    Next i
    Application.Calculation = xlCalculationAutomatic
    Application.ScreenUpdating = True
End Sub
```

Efficiency tips: turn off ScreenUpdating, set calculation to manual and restore them (also in an error handler); avoid Select/Activate; read values into a Variant array for big loops (arr = rng.Value) and process in memory; use AutoFilter or SpecialCells(xlCellTypeBlanks) to delete many rows at once instead of looping; declare variables with Option Explicit; and handle errors with On Error GoTo.

A295. Company context: You work in the finance team of a retail chain. At month end you must reconcile the sales ledger (system export, 12,000 row...

Step 1: Prepare the data. Put each source in its own Table. Check types: invoice numbers may be text in one and numbers in the other, with spaces or prefixes. Clean with TRIM, CLEAN and a consistent format (e.g., =TEXT(A2,"0") or VALUE). Make sure dates are real dates and amounts are numbers; rounding differences should be rounded to 2 decimals with ROUND.

Step 2: Compare totals and counts: SUM and COUNTA for both tables, so you know the scale of the difference (150 fewer records plus amount differences).

Step 3: Flag unmatched items using lookups. In the ledger table:

```
=IFNA(XLOOKUP([@Invoice], tblBank[Reference], tblBank[Amount]), "Missing in bank")
```

Add a difference column: =[@Amount] - [@BankAmount]. In the bank table do the same in reverse to find payments that have no ledger invoice.

Step 4: Classify. Missing in bank (timing differences, unsettled), amount mismatches (fees, partial payments, currency), duplicates in either list (=COUNTIF([Invoice],[@Invoice])>1), and date cut-off issues (settlement after month end). Use a pivot table by category to quantify each bucket, and check that the buckets sum to the 18,420 difference.

Step 5: Document and resolve: build a reconciliation summary (Ledger total, less timing items, plus unrecorded items, equals Bank total) and unexplained difference (target zero), and list items for follow-up. For a repeatable monthly process, use Power Query: load both tables, Merge with a Full Outer join on the reference, expand columns, add custom columns for status and difference, and refresh each month.

A296. Company context: A subscription software company's CRM has 40,000 customer records with duplicates caused by multiple sign-ups ("Jon Smith, ...

Step 1: Keep the raw sheet untouched and work on a copy (or load into Power Query so the source stays unchanged).

Step 2: Standardise the key fields. Create helper columns: CleanEmail =LOWER(TRIM(CLEAN(B2))) and CleanName =PROPER(TRIM(CLEAN(A2))). Also remove non-breaking spaces with SUBSTITUTE(x, CHAR(160), " "). Without standardisation Remove Duplicates would treat "Jon.Smith@Gmail.com" and "jon.smith@gmail.com" as different in some operations (Remove Duplicates is case-insensitive, but extra spaces still create differences).

Step 3: Identify duplicates before deleting. Add a flag column: =COUNTIF($D$2:$D$40001, D2) for occurrences of the email. Add =COUNTIFS($D$2:D2, D2) to mark the first occurrence (1) vs the repeats. Filter on counts > 1 and review a sample to confirm that same email means the same person; also check same name with different emails and phone numbers using COUNTIFS on multiple fields.

Step 4: Decide which record survives: the most recent or most complete. Sort by last activity date descending before removing, so the first occurrence is the one to keep.

Step 5: Remove duplicates: Data > Remove Duplicates on the CleanEmail column (or use =UNIQUE(), or in Power Query use Remove Duplicates after sorting; add Table.Buffer if needed for order stability). Fuzzy near-duplicates (typos) can be found with Power Query Merge > Use fuzzy matching, followed by manual review.

Step 6: Validate: unique count =COUNTA(UNIQUE(D2:D40001)); compare before and after counts; keep a log of removed records; add data validation or a duplicate check at entry in future.

A297. Company context: A pharmaceutical distributor pays sales reps monthly commission. The plan: 3% of revenue for reps in Region North and South...

Step 1: Put the data in a Table (tblReps) and put plan parameters in an assumptions block, not inside formulas: base rates by region (a small lookup table: North 3%, South 3%, East 4%, West 4%), bonus rate 1%, bonus threshold 100%, cap 15,000.

Step 2: Base commission:

```
=[@Revenue] * XLOOKUP([@Region], tblRates[Region], tblRates[Rate], 0)
```

Step 3: Attainment: =[@Revenue]/[@Quota].

Step 4: Bonus (only on revenue above quota, if attained):

```
=IF([@Attainment]>=BonusThreshold, MAX(0,[@Revenue]-[@Quota])*BonusRate, 0)
```

Step 5: Apply the cap and probation rule:

```
=IF([@Status]="Probation", 0, MIN(CapAmount, [@Base]+[@Bonus]))
```

Step 6: Validate: test with edge cases (revenue exactly at quota, below quota, huge revenue to hit the cap, zero quota to avoid #DIV/0! with IF([@Quota]=0, 0, ...)). Reconcile total commission vs total revenue (effective rate should be around 3 to 5%). Create a pivot by region and a conditional format to flag capped reps. Use named cells for parameters so a change in plan only requires editing the assumption cells, and lock formula cells with sheet protection before sending the sheet to payroll.

A298. Company context: A regional bank branch network sends you 12 monthly Excel files (same columns, one sheet each) in a shared folder, plus one...

Use Power Query's folder connector.

Step 1: Data > Get Data > From File > From Folder; select the folder. Excel lists all files. Click Transform Data (rather than Combine & Load directly) so you can check things.

Step 2: Filter the file list (e.g., Extension = .xlsx, exclude temp files starting with "~$").

Step 3: Choose Combine > Combine & Transform Data. Select the sheet or table to combine. Power Query creates a sample file query, a function and a parameter, and appends all files into one table with a Source.Name column which you can use to extract the month.

Step 4: Clean in the combined query: promote headers, remove duplicate header rows if each file repeats headers (filter out rows where a column equals its header text), set data types, trim text, remove errors, add a Month column from the file name, and remove totals rows.

Step 5: Load to a Table or the data model; build PivotTables on it.

Step 6: Each month, drop the new file into the folder and click Data > Refresh All. The pipeline is repeatable and has no manual copy-paste, which removes human error. Considerations: inconsistent column names across files should be handled (rename steps, or use Table.TransformColumnNames), each file must have the same structure, and moving the folder requires editing the source path parameter. Documentation of query steps and a row-count check against file totals provide control.

A299. Company context: A coffee-shop chain is planning to open a new store. The finance lead has a profit model with price, units sold, variable c...

Step 1: Build the model with inputs in separate cells: Price (B1), Units (B2), Variable cost per unit (B3), Fixed costs (B4). Profit in B6: =B1*B2 - B3*B2 - B4.

Step 2: Two-variable sensitivity with a Data Table. Place the formula reference =B6 in the top-left corner of a grid (say D10). Put price values 3, 3.5, ..., 6 across row E10:I10, and units 2000, 3000, ..., 6000 down column D11:D15. Select D10:I15, Data > What-If Analysis > Data Table, Row input cell = B1 (Price), Column input cell = B2 (Units). Excel fills the grid with profit for each pair. Apply a colour scale to highlight negative profit.

Step 3: Break-even units at price 4.50 can be computed analytically: Fixed costs / (Price - Variable cost): =B4/(4.5-B3). To confirm with Goal Seek: Set cell B6 to value 0 by changing B2, with B1 set to 4.5.

Step 4: Scenarios: store Best/Base/Worst assumptions in Scenario Manager or a selector cell using CHOOSE, and create a summary. Tips: input cells for a data table must be on the same sheet as the table; keep the corner formula linked, never type over the table values; and set calculation to "Automatic except data tables" if it slows. Present key conclusions (e.g., minimum price at which the store makes money at 3,500 units).

A300. Company context: A furniture manufacturer makes tables, chairs and desks. Each needs wood hours, labour hours and has a profit: table 70 pro...

Step 1: Layout. Decision cells: B2:B4 (units of table, chair, desk), initial value 0 or any number. Profit per unit in C2:C4 (70, 30, 90). Resource usage matrix: wood hours in D2:D4 (4,1,5), labour hours in E2:E4 (3,2,4).

Step 2: Formulas. Total profit (objective): =SUMPRODUCT(B2:B4, C2:C4). Wood used: =SUMPRODUCT(B2:B4, D2:D4). Labour used: =SUMPRODUCT(B2:B4, E2:E4). Capacities in separate cells (400 and 300).

Step 3: Solver setup (Data > Solver). Set Objective: total profit cell, To: Max. By Changing Variable Cells: B2:B4. Constraints: wood used <= 400; labour used <= 300; B3 >= 20; B2:B4 = integer (int); B2:B4 >= 0 (or tick "Make Unconstrained Variables Non-Negative"). Solving Method: Simplex LP (the model is linear). With integer constraints it uses branch and bound; set integer optimality to a small percentage if it is slow.

Step 4: Solve and choose Keep Solver Solution; generate the Answer Report (final values, binding constraints) and, for LP without integer restrictions, the Sensitivity Report (shadow prices tell the value of one more hour of wood or labour).

Step 5: Interpret: identify which resource is binding, how much profit improves by adding capacity, and check that the solution is sensible. Save as a Scenario if testing alternatives. Mention that results are only as good as the assumptions (profit constant per unit, no demand limits), and that you should add demand constraints if the market is limited.

A301. Company context: An e-commerce company's leadership wants a one-page weekly sales dashboard from a transaction export (Order ID, Date, Categ...

Step 1: Load and prepare. Import the export with Power Query (Get Data), set data types, trim text, remove duplicate Order IDs, create Profit =Revenue-Cost and Month columns; load to a Table or the data model.

Step 2: KPIs. In the data model write measures (or use pivot calculated fields): Total Revenue = SUM(Revenue), Profit Margin % = DIVIDE(SUM(Profit), SUM(Revenue)), Orders = DISTINCTCOUNT(OrderID), AOV = DIVIDE([Total Revenue],[Orders]). Without the data model, compute with formulas: =SUM(tblSales[Revenue]), =SUM(tblSales[Profit])/SUM(tblSales[Revenue]), =COUNTA(UNIQUE(tblSales[Order ID])).

Step 3: Pivots on a hidden Calc sheet: revenue by region, by category, monthly revenue (date grouped by Months and Years).

Step 4: Charts: KPI cards at the top, a line or column chart for the monthly trend, a sorted bar chart for revenue by category, a map or bar for regions. Make the charts PivotCharts (or table-based) so they refresh.

Step 5: Interactivity: slicers (Region, Category) and a timeline for Date, connected to all pivots via Report Connections.

Step 6: Polish and test: hide gridlines, align to a grid, consistent colours, headline titles with formulas, a "last refreshed" cell, protect the sheet except slicers, and set the print area to one page. Validate totals against the source system. Refresh all with one click each week.

A302. Company context: A wholesaler tracks inventory in a sheet with Item, Current Stock, Reorder Level, Lead Time (days) and Last Restock Date. T...

Step 1: Convert the data to a Table (tblStock) so new items are picked up automatically.

Step 2: Status column with a formula:

```
=IF([@[Current Stock]]<=0, "Out of stock", IF([@[Current Stock]]<=[@[Reorder Level]], "Reorder", "OK"))
```

Step 3: Conditional formatting on the table rows (formula rules, applied to the whole table body):

```
=$C2<=$D2
```

for red fill when stock is at or below reorder level (assuming C is Current Stock and D Reorder Level), and

```
=TODAY()-$F2>90
```

for amber (stale stock) where F is the last restock/movement date. Put the out-of-stock rule first with Stop If True. Add a data bar on current stock if desired.

Step 4: Morning order list on a separate sheet with a dynamic array:

```
=SORT(FILTER(tblStock[[Item]:[Lead Time]], tblStock[Current Stock]<=tblStock[Reorder Level], "Nothing to order"), 2, 1)
```

Step 5: Suggested order quantity formula: =MAX(0, [@[Reorder Level]]*2 - [@[Current Stock]]) or based on lead-time demand.

Step 6: Data validation on the stock and date columns (whole number >= 0, valid dates) to prevent bad entries, and sheet protection to lock formulas. If the manager wants emails, use Power Automate or a VBA macro, noting security constraints.

A303. Company context: A market research firm receives a messy CSV export from a survey tool: dates stored as text ("2025/03/07", "07-Mar-25"), nu...

Step 1: Inspect: use Ctrl+Arrow, filters on each column, ISTEXT/ISNUMBER checks, and COUNTBLANK to understand issues. Keep an untouched raw copy.

Step 2: Do the cleaning in Power Query so it is repeatable: Data > From Text/CSV > Transform Data.

- Remove junk rows: filter out rows where the key column equals "Total" or is null; remove top/bottom rows if needed.

- Trim and clean text: Transform > Format > Trim, Clean.

- Standardise categories: Transform > Format > Lowercase, then Replace Values (y, yes -> Yes; n, no -> No), or merge with a mapping table.

- Numbers: Replace "$" and "," with nothing then set the type to Decimal Number; or use Change Type with Locale.

- Dates: set Data Type > Date using Locale where formats vary; for mixed formats split the logic with custom columns and try/otherwise (try Date.From([d]) otherwise null), then review errors.

- Blank-looking columns: trim then replace empty strings with null; Remove Columns when 100% null.

- Remove duplicates and rows with errors (Keep Errors to inspect them first).

Step 3: Without Power Query, use formulas or Text to Columns: =VALUE(SUBSTITUTE(SUBSTITUTE(A2,"$",""),",","")), =DATEVALUE(), and TRIM; then paste as values.

Step 4: Validate: compare row counts and totals with the source, check for min/max outliers, and keep a data dictionary and the query steps as documentation. Load the clean table and create the pivots from it.

A304. Company context: An HR department at a 3,000-person company has an attendance export with Employee ID, Date, Clock-in, Clock-out. Management...

Step 1: Hours worked. With time values (Excel stores times as fractions of a day), use:

```
=(D2-C2)*24
```

For overnight shifts where clock-out is after midnight: =MOD(D2-C2,1)*24. Round with ROUND(x,2). Check for missing punches with IF(OR(C2="",D2=""),"Missing",...).

Step 2: Overtime:

```
=MAX(0, E2-8)
```

where E is hours worked, with 8 as a named cell (StdHours) so policy can change.

Step 3: Monthly summary: add a Month column =EOMONTH([@Date],0) or use PivotTable with Employee ID in rows, Date grouped by Months, and Sum of Hours and Sum of Overtime in values. Alternatively =SUMIFS(tblAtt[Hours], tblAtt[ID], $A2, tblAtt[Date], ">="&$B$1, tblAtt[Date], "<="&EOMONTH($B$1,0)).

Step 4: Macro to prepare the report: record or write VBA that refreshes data connections and pivots (ThisWorkbook.RefreshAll), applies number formats, sorts by overtime descending, applies conditional

formatting for overtime above a limit, sets the print area, and exports to PDF (ws.ExportAsFixedFormat xlTypePDF, path). Assign it to a button. Use Application.ScreenUpdating = False for speed.

Step 5: Controls: validation for clock-in earlier than clock-out, duplicates, and missing punches; protect formulas; and test on a small sample first.

A305. Company context: A mid-sized manufacturer has a loan of 500,000 at 8% annual interest. The CFO asks: what is the monthly payment for 5 years...

Step 1: Inputs in cells: Loan (B1 = 500000), Annual rate (B2 = 8%), Years (B3 = 5).

Step 2: Monthly payment:

```
=-PMT(B2/12, B3*12, B1)
```

PMT returns a negative number (cash outflow), so a minus sign makes it positive. Result is about 10,138. Rate per period = annual/12 and number of periods = years*12.

Step 3: Total interest paid: total payments minus the principal:

```
=B5*B3*12 - B1
```

where B5 holds the payment. Result about 108,300. Alternatively use =-CUMIPMT(B2/12, B3*12, B1, 1, B3*12, 0).

Step 4: Amortization schedule: columns for Period, Opening Balance, Payment, Interest (=Opening*B2/12), Principal (=Payment - Interest), Closing Balance (=Opening - Principal). Or use IPMT and PPMT per period. The last closing balance should be 0.

Step 5: Which rate gives a payment of 9,500? Use Goal Seek: Set cell (payment) to value 9500 by changing the rate cell. Result is about 5.1% (verify with the RATE function: =RATE(60, -9500, 500000)*12). The answer shows the sensitivity: lower rate reduces monthly cost. Present a data table of payments across rates and terms.

A306. Company context: The accounts receivable team of a B2B distributor has a list of open invoices (Customer, Invoice No, Invoice Date, Due Date...

Step 1: Put the report date in a cell (for example B1) as an input: =TODAY() or a fixed month-end date, and name it AsOfDate. Put the open invoices in a Table.

Step 2: Days overdue column:

```
=AsOfDate - [@[Due Date]]
```

Step 3: Bucket column:

```
=IF([@DaysOverdue]<=0, "Current", IF([@DaysOverdue]<=30, "1-30", IF([@DaysOverdue]<=60, "31-60", IF([@DaysOverdue]<=90, "61-90", "90+"))))
```

A table-driven alternative with thresholds in a small lookup table: =XLOOKUP([@DaysOverdue], tblBuckets[From], tblBuckets[Label], , -1).

Step 4: Summary. Insert a PivotTable with Customer in rows, Bucket in columns, Sum of Amount in values, Grand totals on; sort columns in logical order (custom sort or prefix numbering). Alternatively use SUMIFS per customer and bucket:

```
=SUMIFS(tblAR[Amount], tblAR[Customer], $A5, tblAR[Bucket], B$4)
```

Step 5: Format: conditional formatting to colour the 90+ column, show percentages of total outstanding per bucket and a chart. Add a check that the sum of all buckets equals the ledger total of open invoices. Mention that a credit-note or partial payment must be netted so the report matches the ledger.

A307. Company context: A consumer-goods company has 36 months of monthly sales and wants a 6-month forecast with a confidence range, and to unders...

Step 1: Prepare data: two columns, a continuous monthly date series (no missing months, real dates) and sales. Plot a line chart to see trend, seasonality and outliers. Handle missing months (interpolate) or outliers (investigate) first.

Step 2: Quick forecast: select the data and use Data > Forecast Sheet. Choose line or column chart, forecast end date (6 months ahead), confidence interval (95%), and seasonality (auto-detect or set to 12). Excel uses FORECAST.ETS (exponential triple smoothing, AAA version) and creates a table with Forecast, Lower Confidence Bound and Upper Confidence Bound.

Step 3: Formula versions for transparency:

```
=FORECAST.ETS(A38, $B$2:$B$37, $A$2:$A$37, 12)
=FORECAST.ETS.CONFINT(A38, $B$2:$B$37, $A$2:$A$37, 0.95, 12)
=FORECAST.ETS.SEASONALITY($B$2:$B$37, $A$2:$A$37)
```

The last one tells you the detected season length; a value near 12 confirms yearly seasonality. For a non-seasonal trend use FORECAST.LINEAR or TREND, and compare with a 3-month moving average (AVERAGE of the previous 3 rows) as a baseline.

Step 4: Evaluate accuracy by holding out the last 6 months: forecast them using the first 30, compute MAPE =AVERAGE(ABS((actual-forecast)/actual)) (as an array formula), and compare models. Choose the model with the lowest error.

Step 5: Present with the actual and forecast lines, the confidence band, key assumptions, and caveats (promotions, one-off events, short history, structural change). Refresh the forecast as new data arrives by extending the data table.

A308. Company context: An insurance company's claims team needs a data-entry sheet where staff choose Country, then State/Province (only valid sta...

Step 1: Create a Lists sheet. Put the geographical hierarchy in an Excel Table (tblGeo) with columns Country, State, City, or in separate lists.

Step 2: Country drop-down: Data Validation > List with a source of =UNIQUE(tblGeo[Country]) placed in a helper cell, say Lists!H2, and the validation source =Lists!$H$2# (spill reference) so it extends automatically.

Step 3: Dependent State drop-down: in Lists!J2 write =SORT(UNIQUE(FILTER(tblGeo[State], tblGeo[Country]=Entry!B2))) and use =Lists!$J$2# as the source for the State cell. For a column of entries use named formulas or a helper per row. Older Excel approach: one named range per country with the same name as the country and validation source =INDIRECT($B2) (names cannot contain spaces).

Step 4: City drop-down similarly based on the State.

Step 5: Claim amount validation: Allow: Decimal, between 100 and 1000000, with an Error Alert (Stop) message explaining the limits. Claim ID uniqueness: Allow Custom with

```
=COUNTIF($A$2:$A$5000, A2)=1
```

Step 6: When Country changes, State/City should be cleared: this requires a small Worksheet_Change macro. Also: validation can be bypassed by pasting, so protect the sheet (unlock only input cells), use Circle Invalid Data to audit, and consider a Microsoft Form or Power Apps front end for stronger control.

A309. Company context: A telecom operator has two files: a Customers table (Customer ID, Name, Plan, Signup Date) and a Usage table (Customer ID, ...

Problem: VLOOKUP on millions of rows is slow, memory heavy, and the sheet limit is 1,048,576 rows, so 5 million rows cannot even be stored on a worksheet.

Step 1: Use Power Query to load both tables as connections only (Data > Get Data), not to worksheets. Clean types and keys (trim, same data type for Customer ID in both).

Step 2: Load both queries to the Data Model (Load To > Add this data to the Data Model). Data model compresses data in memory with a columnar engine and handles millions of rows.

Step 3: Create a relationship in Power Pivot (Diagram View): Customers[Customer ID] (one) to Usage[Customer ID] (many). Replace the lookups with the relationship so Plan is available for usage rows without a lookup column.

Step 4: Build a PivotTable from the data model: Plan in rows, Month in columns, and a measure. DAX measures: Avg Minutes per Customer := DIVIDE(SUM(Usage[Minutes]), DISTINCTCOUNT(Usage[Customer ID])), Avg Data GB := AVERAGE(Usage[Data GB]).

Step 5: Add slicers and a PivotChart; set the pivot to refresh on open. Save as .xlsx (model compression is automatic) or .xlsb. If still too large, aggregate in Power Query before loading (Group By Customer and Month) or move to a database/Power BI.

Step 6: Validate: row counts per table, compare a handful of customers manually, and check the many-to-one relationship has no orphan IDs (usage rows without a customer record) by reviewing blanks in the Plan row of the pivot.

A310. Write formulas to count (a) orders in column C with amount above 500 in region "East" (column A), (b) orders where the status in column B is...

(a) COUNTIFS with two criteria, both must be true:

```
=COUNTIFS($A$2:$A$1000, "East", $C$2:$C$1000, ">500")
```

(b) Count blank status cells (works for truly empty cells and for cells containing an empty string):

```
=COUNTIFS($B$2:$B$1000, "")
```

Alternatives are =COUNTBLANK(B2:B1000), which also counts formulas returning "", and =COUNTA(...) for non-blank cells. To count only truly empty cells use =SUMPRODUCT(--(B2:B1000="")) carefully or =ROWS(B2:B1000)-COUNTA(B2:B1000) for text/number cells.

(c) AVERAGEIFS ignores rows that do not meet the criteria but returns #DIV/0! if none match, and errors inside the average range cause an error. To ignore error values use AGGREGATE or FILTER with IFERROR:

```
=IFERROR(AVERAGE(IF(($A$2:$A$1000="East")*ISNUMBER($C$2:$C$1000), $C$2:$C$1000)), 0)
```

(array formula in older Excel, plain Enter in 365). A simple version without errors in the data is =IFERROR(AVERAGEIFS($C$2:$C$1000,$A$2:$A$1000,"East"),0).

Math and Statistics

A311. Explain mean, median and mode. When would you prefer the median over the mean?

Mean is the sum of values divided by the count. Median is the middle value of the sorted data (average of the two middle values if n is even). Mode is the most frequent value.

The mean uses every observation and is optimal for symmetric data, but it is sensitive to outliers and skew. The median is robust: changing the largest value to an extreme number does not move it. Prefer the median for skewed data or data with outliers, such as income, house prices, session duration or order value. Example: salaries 40k, 45k, 50k, 55k, 1,000k have mean 238k but median 50k, and the median describes a typical employee far better. The mode is useful for categorical data (most common product category) or multimodal numeric data. A big gap between mean and median is itself a signal of skew.

A312. What is the difference between variance and standard deviation, and why does the sample variance divide by n-1?

Variance is the average squared deviation from the mean; standard deviation is its square root, which returns to the original units and so is easier to interpret.

Population variance = sum((x - mu)^2) / N. Sample variance = sum((x - xbar)^2) / (n - 1). Dividing by n-1 (Bessel's correction) makes the sample variance an unbiased estimator of the population variance. Reason: the sample mean xbar is computed from the same data and sits closer to the points than the true mean mu does, so deviations from xbar are systematically smaller. Only n-1 deviations are free once xbar is fixed (degrees of freedom). Example: data 2, 4, 6: mean 4, squared deviations 4, 0, 4 sum to 8; sample variance 8/2 = 4, SD = 2; population version would give 8/3 = 2.67. Note that the sample standard deviation is still slightly biased as an estimator of sigma, even though the variance is unbiased.

A313. How do you detect outliers, and how should you handle them?

Detection: (1) IQR rule: flag points below Q1 - 1.5*IQR or above Q3 + 1.5*IQR, where IQR = Q3 - Q1. (2) Z-score: flag |z| > 3 (works poorly when the outlier itself inflates the mean and SD; use the robust modified z-score based on the median and MAD instead). (3) Visual: box plots, histograms, scatter plots. (4) Model based: Cook's distance and leverage in regression, isolation forest for multivariate data.

Example: Q1 = 20, Q3 = 30, IQR = 10, fences are 5 and 45; a value of 80 is an outlier.

Handling depends on cause. First investigate: data entry or instrumentation error (fix or remove), genuine rare event (keep, it may be the most important data, such as fraud), or a different population (analyse separately). Options: keep and use robust statistics (median, trimmed mean, quantile regression), winsorize (cap at the 1st/99th percentile), log transform, or use non-parametric tests. Always document the decision and report

results with and without outliers as a sensitivity check. Never delete points just because they hurt the model fit.

A314. What is the difference between correlation and causation? Give an example and explain how you would establish causation.

Correlation means two variables move together; causation means changing one produces a change in the other. Correlation can arise from (a) X causing Y, (b) Y causing X (reverse causality), (c) a confounder Z causing both, (d) selection or collider bias, or (e) pure chance (spurious correlation).

Example: ice cream sales and drowning deaths correlate because hot weather (confounder) raises both. Another: users who use a premium feature churn less, but engaged users may be both likelier to use the feature and to stay.

To establish causation: run a randomized controlled experiment (A/B test), which balances confounders on average. When you cannot randomize, use quasi-experimental methods: difference-in-differences, regression discontinuity, instrumental variables, propensity score matching or synthetic control, plus controlling for known confounders and checking the assumptions of each method. Supporting evidence also includes temporal ordering, dose-response, plausible mechanism and replication.

A315. Compare Pearson and Spearman correlation. When does each fail or mislead?

Pearson r measures the strength of a linear relationship between two continuous variables, ranges from -1 to 1, and is sensitive to outliers. It equals covariance divided by the product of standard deviations. Spearman rho is Pearson computed on the ranks, so it measures monotonic (not necessarily linear) association and is robust to outliers and to monotone transformations. Kendall tau is another rank measure, good for small samples with ties.

Failures: Pearson can be near zero for a perfect nonlinear relationship (y = x^2 with x symmetric around 0), can be driven by one extreme point, and is distorted by restricted range. Spearman captures y = exp(x) as rho = 1 whereas Pearson is below 1. Neither detects non-monotonic relationships (a U shape). Both say nothing about causation, and a correlation computed on pooled groups can reverse within groups (Simpson's paradox). Always plot the data (Anscombe's quartet shows four datasets with identical r but very different shapes).

A316. State the Central Limit Theorem and explain why it matters for data analysts.

CLT: if X1..Xn are independent, identically distributed with finite mean mu and finite variance sigma^2, then the sampling distribution of the sample mean approaches a normal distribution with mean mu and standard deviation sigma/sqrt(n) as n grows, regardless of the shape of the original distribution.

Why it matters: it justifies z and t based confidence intervals and hypothesis tests for means (and proportions, which are means of 0/1 variables) even when the raw data are not normal. It tells us the standard error shrinks as 1/sqrt(n): quadrupling the sample size halves the margin of error. Rule of thumb: n of about 30 is enough for mildly skewed data, but heavily skewed data (revenue with whales) or heavy tails (infinite variance, e.g. Cauchy) may need far more or fail altogether. CLT is about the distribution of the sample mean, not about the data becoming normal.

A317. What is the Law of Large Numbers and how does it differ from the Central Limit Theorem? Does it support the gambler's fallacy?

Law of Large Numbers (LLN): as n grows, the sample mean converges to the true mean mu (weak law: in probability; strong law: almost surely). It says where the average goes. The CLT says how the fluctuation around mu is distributed: (xbar - mu) * sqrt(n) / sigma tends to a standard normal. LLN gives consistency; CLT gives the shape and rate (sigma/sqrt(n)) needed for inference.

It does not support the gambler's fallacy. After 10 heads in a row, the next flip of a fair coin is still 50% heads because flips are independent. The proportion converges to 0.5 not because tails are due, but because the early imbalance is diluted by an ever larger number of future flips: the absolute difference between heads and tails can still grow like sqrt(n), while the proportion difference shrinks like 1/sqrt(n).

A318. What is the difference between standard deviation and standard error?

Standard deviation (SD) describes the spread of individual observations in a sample or population. Standard error (SE) describes the uncertainty of an estimate, for example the SD of the sampling distribution of the sample mean: SE = s / sqrt(n). The SD does not shrink as you collect more data (it converges to sigma), while the SE does shrink with sqrt(n). Use SD to describe variability of the data (error bars showing typical spread), and SE (or a confidence interval) to describe how precisely a mean or a difference is estimated. Example: s = 20, n = 100 gives SE = 2; with n = 400 SE = 1, while s stays about 20. For a proportion, SE = sqrt(p(1-p)/n).

A319. What is a p-value? What does it not mean?

The p-value is the probability, assuming the null hypothesis is true, of observing a test statistic at least as extreme as the one actually observed. It is P(data this extreme | H0), a measure of how surprising the data are under H0.

It is NOT: the probability that H0 is true; the probability that the result occurred by chance; the probability the finding will replicate; a measure of effect size or practical importance; or 1 minus the probability that the alternative is true. A p of 0.03 means that if there were truly no effect, only 3% of experiments would produce a result this extreme or more. We reject H0 when p < alpha (commonly 0.05), which controls the false positive rate at alpha. A tiny p-value with a huge sample may reflect a trivial effect, and a large p-value does not prove the null (it may just be low power). Always report the effect size and confidence interval alongside p.

A320. Explain Type I error, Type II error, significance level and statistical power, and how they trade off.

Type I error (false positive): rejecting H0 when it is true; probability = alpha (significance level). Type II error (false negative): failing to reject H0 when the alternative is true; probability = beta. Power = 1 - beta, the probability of detecting a real effect of a given size.

Analogy: Type I is convicting an innocent person; Type II is letting a guilty one go free.

Trade-offs: with fixed sample size and effect, lowering alpha (for example from 0.05 to 0.01) reduces Type I errors but raises beta and lowers power. Power increases with larger sample size, larger true effect, lower noise (variance), and higher alpha (or a one-sided test). Typical design targets: alpha = 0.05, power = 0.8 (or 0.9). Which error is worse depends on context: in a drug safety trial a Type I error (approving an unsafe drug) is costly; in cancer screening a Type II error (missing disease) is costlier, so you accept more false positives.

A321. How should you interpret a 95% confidence interval? What are common misinterpretations?

Correct frequentist interpretation: if we repeated the sampling and interval construction many times, about 95% of the intervals so constructed would contain the true parameter. The procedure has 95% coverage; any one computed interval either contains the parameter or does not.

Common wrong statements: "there is a 95% probability the true mean lies in this particular interval" (that is a Bayesian credible interval statement, requiring a prior), "95% of the data lie in the interval" (that would be a prediction/tolerance interval), or "a repeated experiment will land inside this interval 95% of the time" (not so; it is lower).

Structure: estimate +/- critical value * SE. Width shrinks with sqrt(n), grows with variance and with confidence level. Practical value: the interval shows the range of effect sizes compatible with the data, so you can judge both significance (does it exclude 0?) and practical relevance (is even the lower bound worth shipping?).

A322. How are confidence intervals and hypothesis tests related? If a 95% CI for a difference is [-0.5, 3.0], what can you conclude?

A two-sided test at level alpha rejects H0: theta = theta0 exactly when theta0 lies outside the (1 - alpha) confidence interval (for the same test statistic and standard error). So a 95% CI for a difference that includes 0 corresponds to p > 0.05; the interval [-0.5, 3.0] includes 0, so the result is not statistically significant at the 5% level.

But the interval is more informative than the verdict: the data are compatible with effects from a slight harm (-0.5) to a sizeable gain (+3.0). The point estimate is about 1.25 (midpoint). It is wrong to conclude "no effect"; the study is inconclusive (maybe underpowered). If a gain of 1 is practically meaningful, you may want more data. If decisions need a minimum effect, compare the interval to that threshold (equivalence and non-inferiority tests use this logic).

A323. What are the different types of t-tests and when do you use each?

One-sample t-test: compare a sample mean to a known value (is the mean delivery time 30 minutes?). Two-sample independent t-test: compare means of two unrelated groups (control vs treatment); either pooled variance (assumes equal variances) or Welch's t-test (does not assume equal variances and is the safe default). Paired t-test: compare two measurements on the same units (before/after on the same users); it analyses the differences, removing between-subject variance, and so is more powerful when pairing is real.

Assumptions: independent observations (within the design), approximately normal data or means (fine with large n by the CLT), continuous outcome. For heavy skew and small n use Mann-Whitney U (independent) or Wilcoxon signed-rank (paired), or bootstrap. Test statistic: t = (difference in means) / SE, compared with the t distribution with appropriate degrees of freedom.

A324. Explain the chi-square goodness-of-fit and chi-square test of independence. What are their assumptions?

Both compare observed counts O with expected counts E using chi2 = sum((O - E)^2 / E).

Goodness-of-fit: tests whether a single categorical variable follows a specified distribution (is a die fair? does traffic split 50/50 across variants, i.e. a sample ratio mismatch check?). df = k - 1 (minus the number of estimated parameters).

Independence/homogeneity: tests whether two categorical variables are associated in a contingency table (device type vs conversion). Expected count for a cell = (row total * column total) / grand total. df = (rows - 1) * (columns - 1).

Assumptions: independent observations, counts (not percentages), each observation in exactly one cell, and expected counts of roughly 5 or more per cell (otherwise use Fisher's exact test). It indicates association but not its strength or direction; use Cramer's V, odds ratios or standardized residuals for that. With huge samples it flags trivially small associations.

A325. What is ANOVA, what does the F-test tell you, and what do you do after a significant result?

One-way ANOVA tests H0: all group means are equal against H1: at least one differs. It splits total variance into between-group variance (explained by group membership) and within-group variance (noise). F = MS_between / MS_within, with df1 = k - 1 and df2 = N - k. A large F means group means differ more than expected from noise.

Why not many t-tests: with k groups there are k(k-1)/2 pairwise tests, inflating the family-wise error rate (5 groups give 10 tests, so about 1 - 0.95^10 = 40% chance of at least one false positive).

Assumptions: independence, approximately normal residuals, homogeneity of variance (check with Levene; use Welch ANOVA otherwise).

A significant F says only that some difference exists. Follow with post hoc comparisons that control error: Tukey HSD for all pairs, Dunnett for comparisons to a control, or Bonferroni/Holm adjusted t-tests. Report effect size (eta squared). Non-parametric alternative: Kruskal-Wallis.

A326. What is the multiple comparisons problem and how can it affect A/B testing with many metrics or segments?

When you run m independent tests each at alpha = 0.05, the probability of at least one false positive (family-wise error rate) is 1 - (1 - alpha)^m. For m = 20 it is 1 - 0.95^20 = 64%. So slicing an experiment by many segments or tracking dozens of metrics almost guarantees a "significant" finding by chance (p-hacking, data dredging, the "green jelly bean" cartoon).

Remedies: pre-register a single primary metric and a small set of guardrails; correct for multiplicity (Bonferroni: use alpha/m; Holm: step-down, uniformly better; Benjamini-Hochberg: controls false discovery rate and is preferred when screening many hypotheses); treat segment findings as hypotheses to confirm in a follow-up experiment; test for an interaction (treatment x segment) rather than comparing separate segment p-values; and report how many comparisons were made.

A327. Compare Bonferroni correction with Benjamini-Hochberg (FDR). When would you choose each?

Bonferroni controls the family-wise error rate (FWER, probability of any false positive) by testing each hypothesis at alpha/m. It is simple and valid under any dependence, but conservative and loses power as m grows.

Benjamini-Hochberg controls the false discovery rate: the expected proportion of rejected hypotheses that are false. Procedure: sort p-values p(1) <= ... <= p(m); find the largest k with p(k) <= (k/m) * q; reject hypotheses 1..k. It is valid for independent or positively dependent tests (Benjamini-Yekutieli for arbitrary dependence).

Example, m = 5, q = 0.05, p-values 0.001, 0.008, 0.02, 0.04, 0.30: thresholds are 0.01, 0.02, 0.03, 0.04, 0.05. Largest k with p(k) <= threshold is k = 4 (0.04 <= 0.04), so four are rejected. Bonferroni uses 0.01 and rejects only the first two.

Choose Bonferroni (or Holm) when any false positive is costly (confirmatory trial, few hypotheses, safety endpoints). Choose BH when screening many hypotheses (genes, hundreds of segments or metrics) where you accept a small fraction of false leads and want power.

A328. Describe the normal distribution and its key properties, including the empirical rule.

The normal (Gaussian) distribution is a symmetric bell-shaped distribution fully described by its mean mu and standard deviation sigma. Mean = median = mode. Its density is proportional to exp(-(x - mu)^2 / (2 sigma^2)). Empirical (68-95-99.7) rule: about 68.3% of values lie within 1 SD of the mean, 95.4% within 2 SD (95% within 1.96 SD), 99.7% within 3 SD. Standardizing z = (x - mu)/sigma gives the standard normal with mean 0 and SD 1. Sums and linear combinations of independent normals are normal (means add; variances add). It arises through the CLT, and many methods (linear regression errors, t-tests) assume it. Check normality with a Q-Q plot or histogram; formal tests like Shapiro-Wilk are over-sensitive with large n.

A329. When would you model data with a binomial versus a Poisson distribution? How are they related?

Binomial(n, p): the number of successes in a fixed number n of independent trials, each with success probability p (number of purchases among 100 visitors). Mean np, variance np(1-p). Poisson(lambda): the count of events in a fixed interval of time or space when events occur independently at a constant average rate (support tickets per hour, server errors per day). Mean = variance = lambda. A mean far below the variance signals overdispersion, in which case use negative binomial.

Relationship: Binomial(n, p) approaches Poisson(lambda = np) when n is large and p is small (for example 1000 users, 0.2% chance each, gives about Poisson(2)). For large n with moderate p, the binomial is approximately normal (np and n(1-p) both at least 10). Poisson events are separated by exponentially distributed waiting times.

A330. What is the exponential distribution and what does the memoryless property mean?

The exponential distribution with rate lambda models the waiting time until the next event in a Poisson process. Density f(x) = lambda * exp(-lambda x) for x >= 0; mean 1/lambda; variance 1/lambda^2; P(X > x) = exp(-lambda x).

Memoryless: P(X > s + t | X > s) = P(X > t). Having already waited s units tells you nothing about the remaining wait. Proof: exp(-lambda (s+t)) / exp(-lambda s) = exp(-lambda t). Example: if buses arrive as a Poisson process with a mean gap of 10 minutes, then after waiting 20 minutes already, the expected additional wait is still 10 minutes (the waiting-time paradox). It is the only continuous memoryless distribution (geometric is the discrete analogue). Real-world caveat: customer lifetimes and machine wear usually are not memoryless; use Weibull or other hazard models.

A331. State Bayes' theorem and explain prior, likelihood and posterior with an example.

Bayes' theorem: P(A | B) = P(B | A) * P(A) / P(B), where P(B) = P(B | A) P(A) + P(B | not A) P(not A).

Prior P(A): belief about A before seeing data. Likelihood P(B | A): how probable the observed data are if A is true. Posterior P(A | B): updated belief. Equivalent odds form: posterior odds = prior odds * likelihood ratio.

Example: a spam filter. 20% of emails are spam. The word "free" appears in 50% of spam and 5% of non-spam. P(spam | "free") = 0.5*0.2 / (0.5*0.2 + 0.05*0.8) = 0.10 / (0.10 + 0.04) = 0.714. The key lesson is that the prior (base rate) matters; ignoring it is the base-rate fallacy, and P(B | A) is not P(A | B).

A332. State the key properties of expectation and variance, including linearity and variance of a sum.

Expectation is linear regardless of dependence: E[aX + bY + c] = aE[X] + bE[Y] + c, and E[X + Y] = E[X] + E[Y] even when X and Y are dependent. Product: E[XY] = E[X]E[Y] only if uncorrelated (e.g. independent).

Variance: Var(aX + c) = a^2 Var(X). Var(X + Y) = Var(X) + Var(Y) + 2 Cov(X, Y); for independent variables the covariance term vanishes. Var(X - Y) = Var(X) + Var(Y) - 2Cov(X, Y), which is why pairing (positive correlation) reduces variance of a difference. Var(X) = E[X^2] - (E[X])^2. For the mean of n iid variables: Var(xbar) = sigma^2/n.

Linearity of expectation is the trick behind many puzzles (indicator variables): expected number of fixed points in a random permutation of n items is n * (1/n) = 1.

A333. What are common types of sampling bias and how do you mitigate them?

Selection bias: sample not representative of the population (online survey excludes offline users). Non-response bias: respondents differ from non-respondents (satisfied or angry customers answer reviews). Survivorship bias: only analysing those who survived a filter (studying successful startups). Convenience/volunteer bias: using whoever is easy to reach. Undercoverage: sampling frame misses part of the population. Measurement/response bias: leading questions, social desirability. Look-ahead and time bias: using information not available at decision time.

Mitigation: probability sampling (simple random, stratified, cluster) from a complete frame, follow-ups to increase response rate, weighting (post-stratification) to match known population margins, comparing respondents to known benchmarks, randomization in experiments, careful question design, and being explicit about the target population when you generalize.

A334. What is Simpson's paradox? Why does it happen and how do you decide whether to use the aggregated or the segmented data?

Simpson's paradox: a trend that appears in several subgroups reverses or disappears when the groups are combined. It happens when group sizes are unbalanced and the grouping variable is associated with both the treatment and the outcome (a confounder), so weighted averages of the subgroup rates differ from the pooled rate.

Example: Treatment A cures 93% of small and 73% of large kidney stones, B cures 87% and 69%. A is better in both subgroups. But A was given mostly to hard (large) cases, B mostly to easy ones, so overall A shows 78% vs B 83%.

Which to trust is a causal question, not a statistical one. If the subgroup variable is a confounder (stone size affects treatment choice and outcome), use the segmented/adjusted result. If it is a mediator (lies on the causal path from treatment to outcome), conditioning would remove part of the effect, so use the aggregate. Draw a causal diagram. In A/B tests, randomization avoids it unless allocation ratios differed across segments or traffic mix shifted over time.

A335. What is survivorship bias? Give an example of how it can distort an analysis.

Survivorship bias occurs when the analysed data include only entities that passed some selection filter, ignoring those that dropped out. Classic example: in World War II, analysts proposed armoring the parts of returning bombers with the most bullet holes; Abraham Wald pointed out that planes hit in other places never returned, so those unhit areas on survivors were the vulnerable ones.

Business examples: computing average customer lifetime value only from currently active customers overstates it; analysing only users who completed onboarding to find what drives retention; mutual fund performance averaged over funds that still exist (failed funds closed); reviewing "habits of successful founders" without failed ones.

Mitigation: define the cohort at the start (all users who signed up), track everyone including those who left, use survival analysis (Kaplan-Meier, Cox) that handles censoring, and ask what filter produced the dataset.

A336. What factors determine statistical power and sample size in a test? Give the sample size formula for comparing two means.

Power depends on: effect size (bigger true difference is easier to detect), variability sigma (less noise gives more power), sample size n, significance level alpha (larger alpha gives more power), one- vs two-sided test, and design (paired designs, stratification, covariate adjustment like CUPED reduce variance).

Sample size per group for a two-sided test comparing means with equal group sizes: n = 2 * sigma^2 * (z_(1-alpha/2) + z_(1-beta))^2 / delta^2, where delta is the minimum detectable difference. For alpha = 0.05 and power 0.8: (1.96 + 0.84)^2 = 7.85, so n = 15.7 * sigma^2 / delta^2 per group. Rule of thumb: n is proportional to 1/delta^2, so halving the detectable effect quadruples the required sample. For proportions replace 2 sigma^2 by p1(1-p1) + p2(1-p2). Example: sigma = 10, delta = 2 gives n = 15.7 * 100 / 4 = 392 per group.

A337. What is the bootstrap, when is it useful, and how would you compute a 95% bootstrap confidence interval for the median in Python?

The bootstrap estimates the sampling distribution of a statistic by resampling the observed data with replacement many times and recomputing the statistic on each resample. It is useful when no simple formula exists for the standard error (median, percentiles, ratios, correlation, AUC), when distributions are non-normal, and for complex metrics. Limitations: needs a representative sample, performs poorly for extremes (maximum) and very small samples, and assumes iid data (use block bootstrap for time series).

```
import numpy as np
rng = np.random.default_rng(0)
data = np.array(values)
boots = [np.median(rng.choice(data, size=len(data), replace=True))
         for _ in range(10000)]
lo, hi = np.percentile(boots, [2.5, 97.5])
print(lo, hi)
```

The percentile interval is the 2.5th and 97.5th percentiles of the bootstrap medians. For a two-group comparison, resample each group separately, compute the difference in the statistic, and check whether the interval excludes 0.

A338. Contrast the frequentist and Bayesian approaches to inference. What are the pros and cons of Bayesian A/B testing?

Frequentist: parameters are fixed unknowns; probability describes long-run frequency of data under repeated sampling. Outputs are p-values and confidence intervals, with error rates controlled by design. Bayesian: parameters have probability distributions; prior beliefs are updated with data via Bayes' theorem to a posterior, from which you directly compute statements such as P(B better than A) = 0.93 or the expected loss from choosing B.

Pros of Bayesian A/B testing: intuitive probability statements, natural incorporation of prior knowledge (helpful with small samples), a decision-theoretic framing (expected loss, value remaining), and more flexibility in stopping (though calibration of false positive rates still matters). Cons: results depend on prior choice, which must be justified; computation can be heavier; "peeking is fine" is only partly true because you can still ship false winners if you stop on a threshold repeatedly; stakeholders may expect frequentist guarantees. Good practice is to check the frequentist operating characteristics through simulation either way.

A339. Explain maximum likelihood estimation. Derive the MLE for the success probability of a Bernoulli sample.

MLE chooses the parameter value that makes the observed data most probable: theta_hat = argmax L(theta) = argmax product of f(x_i | theta); in practice maximize the log-likelihood, which turns products into sums. Properties: consistent, asymptotically normal, asymptotically efficient; invariant to reparametrization; can be biased in small samples (the MLE of normal variance divides by n, not n-1).

Bernoulli derivation: let x_i be 0 or 1 with k successes in n trials. L(p) = p^k (1-p)^(n-k). log L = k log p + (n-k) log(1-p). Derivative: k/p - (n-k)/(1-p) = 0, so k(1-p) = (n-k)p, so k = np, giving p_hat = k/n, the sample proportion. Second derivative is negative so it is a maximum. Example: 7 heads in 10 flips gives p_hat = 0.7. Its standard error is sqrt(p_hat(1-p_hat)/n) = sqrt(0.21/10) = 0.145. Link to ML: minimizing squared error is MLE under Gaussian noise; minimizing log loss is MLE for logistic regression.

A340. List the assumptions of linear regression and how you would diagnose violations.

(1) Linearity: the mean of y is linear in the predictors. Check residuals vs fitted plot for curvature; fix with transformations, polynomial terms or splines. (2) Independence of errors: no autocorrelation (time series, clustered data). Check Durbin-Watson, ACF of residuals; fix with clustered standard errors or time series models. (3) Homoscedasticity: constant error variance. Check residual spread vs fitted values, Breusch-Pagan test; fix with log transform, weighted least squares or robust (HC) standard errors. (4) Normality of residuals: needed for exact small-sample inference, not for unbiased coefficients. Check Q-Q plot. (5) No perfect multicollinearity: check VIF. Also: no influential outliers (Cook's distance, leverage) and exogeneity (errors uncorrelated with predictors; violated by omitted variable bias, which makes coefficients biased).

Violations of 3 or 4 mainly affect standard errors and p-values; violations of 1 or exogeneity bias the estimates themselves.

A341. What is multicollinearity, how do you detect it and what are its consequences?

Multicollinearity means predictors are strongly correlated with each other. Consequences: coefficient estimates become unstable and have inflated standard errors (signs may flip, individual predictors look

insignificant even though the model as a whole fits well); interpretation of individual coefficients ("holding others constant") becomes unreliable. Predictions on data with the same correlation structure remain fine.

Detection: correlation matrix; Variance Inflation Factor VIF_j = 1 / (1 - R_j^2), where R_j^2 comes from regressing predictor j on the others (VIF above 5 or 10 is a warning; VIF = 10 means R_j^2 = 0.9 and the standard error is sqrt(10) = 3.2 times larger); condition number of the design matrix.

Remedies: drop or combine redundant variables, use domain knowledge to choose one, PCA, ridge regression (L2 shrinkage), centering variables for polynomial/interaction terms, or collect more data. If you only care about prediction, it may be acceptable.

A342. What are eigenvalues and eigenvectors and how are they used in PCA?

For a square matrix A, an eigenvector v is a nonzero vector whose direction is unchanged by A: A v = lambda v, where the scalar lambda is the eigenvalue. They solve det(A - lambda I) = 0. Sum of eigenvalues = trace; product = determinant.

PCA: center the data and compute the covariance matrix C (symmetric, so its eigenvalues are real and eigenvectors orthogonal). The eigenvectors are the principal component directions; each eigenvalue is the variance of the data along its eigenvector. Sorting eigenvalues in descending order, the first component captures the most variance. Explained variance ratio of component i = lambda_i / sum(lambda). Project data onto the top k eigenvectors to reduce dimensionality while keeping most variance. Equivalent implementation: SVD of the centered data matrix. Standardize features first when units differ, and note PCA components are linear combinations that may be hard to interpret.

A343. What is the dot product and cosine similarity? When is cosine similarity preferred over Euclidean distance?

Dot product of vectors a and b: a . b = sum(a_i b_i) = |a||b| cos(theta). Cosine similarity = (a . b) / (|a||b|), the cosine of the angle between them, from -1 to 1 (0 to 1 for non-negative vectors). It measures direction, ignoring magnitude. Example: a = (1, 2, 3), b = (2, 4, 6): dot = 28, |a| = sqrt(14), |b| = sqrt(56) = 2 sqrt(14), cosine = 28/28 = 1 (same direction), though Euclidean distance is sqrt(14) = 3.74.

Prefer cosine for high-dimensional sparse data such as text (TF-IDF vectors), embeddings and recommenders where document length or user activity level should not matter. Euclidean distance is better when magnitude matters (spatial locations, normalized numeric features). For unit-length vectors, squared Euclidean distance = 2 - 2*cosine, so they rank the same. Cosine distance = 1 - cosine similarity.

A344. Explain gradient descent and the role of derivatives and the learning rate.

Gradient descent minimizes a differentiable loss L(w) by iteratively moving against the gradient: w_new = w - eta * grad L(w). The gradient (vector of partial derivatives) points in the direction of steepest increase, so the negative gradient is the steepest descent. The learning rate eta sets step size: too small means slow convergence; too large causes overshooting, oscillation or divergence. For a convex loss (e.g. linear regression MSE, logistic loss) gradient descent reaches the global minimum with a suitable eta; for non-convex losses (neural networks) it finds local minima or saddle points.

Variants: batch (all data per step), stochastic (one sample), mini-batch (a few hundred samples, standard), and momentum/Adam adaptive methods. Feature scaling speeds up convergence. Computing gradients through composed functions uses the chain rule (backpropagation). Example: L(w) = (w - 3)^2, gradient 2(w - 3); the minimum is at w = 3 where the gradient is zero.

A345. Derive the gradient of the mean squared error for simple linear regression and explain how it relates to the closed-form solution.

Model y_hat = w x + b, loss L = (1/n) sum (y_i - w x_i - b)^2. Partial derivatives: dL/dw = -(2/n) sum x_i (y_i - w x_i - b); dL/db = -(2/n) sum (y_i - w x_i - b). Setting both to zero gives the normal equations. From dL/db = 0: b = ybar - w xbar. Substituting gives w = sum((x_i - xbar)(y_i - ybar)) / sum((x_i - xbar)^2) = Cov(x, y) / Var(x), which equals r * (s_y / s_x).

In matrix form: L = (1/n)|y - Xw|^2, gradient = -(2/n) X^T (y - Xw); set to zero: w = (X^T X)^(-1) X^T y (requires X^T X invertible; fails with perfect multicollinearity). Gradient descent iterates w <- w - eta * gradient and converges to the same solution for convex MSE; it is preferred when the number of features is large because the matrix inverse costs O(p^3).

A346. What is the difference between covariance and correlation?

Covariance Cov(X, Y) = E[(X - E X)(Y - E Y)] measures the direction of joint variability: positive when the variables tend to be above their means together, negative when opposite. Its magnitude depends on the units of X and Y (so Cov of height in cm and weight in kg differs from height in metres), making it hard to compare. Correlation rescales it: r = Cov(X, Y) / (sigma_X sigma_Y), unitless and bounded in [-1, 1], so strength can be compared across pairs. Independence implies zero covariance, but zero covariance does not imply independence (X uniform on -1 to 1 and Y = X^2 are uncorrelated yet fully dependent). Correlation is invariant to positive linear rescaling of either variable. The covariance matrix is used in PCA and in multivariate normal models.

A347. What are non-parametric tests such as Mann-Whitney U, and when should you use them instead of a t-test?

Non-parametric tests make few distributional assumptions and typically work on ranks. Mann-Whitney U (Wilcoxon rank-sum) compares two independent groups: it tests whether a random observation from one group tends to be larger than one from the other (stochastic dominance); under equal shapes it compares medians. Wilcoxon signed-rank is the paired analogue; Kruskal-Wallis extends to more than two groups; Spearman for correlation.

Use them with small samples that are clearly non-normal, ordinal data (satisfaction ratings 1 to 5), or heavy outliers. Pros: robust, valid for ordinal data. Cons: lower power than the t-test when normality actually holds (about 95% efficiency for normal data), answers a different question than a difference in means (the mean is what drives revenue), and with ties or unequal shapes interpretation is subtle. With large samples the t-test is usually fine by the CLT; consider a bootstrap on the mean or median when you need a specific estimand.

A348. Explain the bias-variance trade-off and how it relates to estimator quality (MSE).

For an estimator theta_hat, MSE = E[(theta_hat - theta)^2] = Bias^2 + Variance. Bias is the systematic error, E[theta_hat] - theta; variance is how much the estimate fluctuates across samples. In prediction, expected test error = irreducible noise + bias^2 + variance.

Simple models (linear model on curved data) have high bias and low variance (underfit). Flexible models (deep tree, high-degree polynomial) have low bias but high variance (overfit). Tuning complexity (regularization strength, tree depth, k in k-NN) moves along the trade-off; cross-validation picks the minimum test error. Example: the sample variance with divisor n is biased low but has lower MSE than the unbiased n-1 version for

normal data, an example where accepting bias reduces total error. Ridge regression and shrinkage estimators work on the same principle. Remedies: more data reduces variance, ensembles (bagging) reduce variance, boosting reduces bias.

A349. What are the types of missing data (MCAR, MAR, MNAR) and how do they affect your handling choices?

MCAR (missing completely at random): missingness is unrelated to any data, observed or not (a sensor randomly drops readings). Complete-case analysis is unbiased though less efficient. MAR (missing at random): missingness depends on observed variables but not on the missing value itself, given those (older respondents skip the income question, and age is recorded). Multiple imputation or model-based methods using the observed covariates give valid results; complete-case analysis can be biased. MNAR (missing not at random): missingness depends on the unobserved value itself (high earners hide income). Hard: no purely data-driven fix; need sensitivity analysis, selection models or additional data, plus a missing indicator as a diagnostic.

Practical steps: quantify missingness per column, check whether it correlates with other variables or the target, prefer multiple imputation or model-based imputation over mean imputation (which shrinks variance and distorts correlations), and never impute using the test set information.

A350. What is the difference between a confidence interval, a prediction interval and a tolerance interval?

Confidence interval: a range for a population parameter (the mean) such that the procedure covers the true value 95% of the time. Width shrinks to 0 as n grows (SE = s/sqrt(n)).

Prediction interval: a range for a single future observation. Must include both the uncertainty of the estimated mean and the individual noise: approximately xbar +/- t * s * sqrt(1 + 1/n). It does not shrink to zero; it converges to roughly mu +/- 1.96 sigma.

Tolerance interval: a range covering at least a specified proportion (say 95%) of the population with a stated confidence (say 99%).

Example: n = 100, mean 50, s = 10. CI half-width = 1.98 * 10/10 = 1.98, so about [48.0, 52.0]. Prediction interval half-width = 1.98 * 10 * sqrt(1.01) = 19.9, so about [30.1, 69.9]. In regression, the confidence band is for the mean response at x, while the prediction band is wider and for a new observation at x.

A351. What is regression to the mean, and how can it mislead analysts?

Regression to the mean: when a measurement has random noise, extreme values tend to be followed by values closer to the average, simply because the extreme included a lucky or unlucky random component. It applies whenever the correlation between two measurements is below 1. Examples: students with the top test scores do slightly worse on a retest; the stores with the worst sales last quarter improve next quarter even without intervention; a hospital with an unusually bad month improves next month.

Misleading conclusions: attributing the improvement to an intervention applied after selecting extreme cases (a coaching program for the worst performers, a campaign for the lowest-engagement users), or concluding punishment works better than praise.

Defence: use a randomized control group drawn from the same extreme group, so both regress equally and the difference isolates the treatment effect; or model measurement reliability. Be suspicious of any before/after analysis where the subjects were chosen because of an extreme before value.

A352. What is the delta method and why is it needed for ratio metrics (for example revenue per session) in A/B tests?

When the randomization unit (user) differs from the analysis unit (session or pageview), the metric is a ratio of two sums, R = sum(Y_i) / sum(X_i), where each user contributes Y_i (revenue) and X_i (sessions). Observations are not independent at the session level because sessions from the same user are correlated, so a naive session-level t-test understates variance and inflates false positives.

Delta method: approximate the variance of a function of means via a first-order Taylor expansion. For R = ybar / xbar: Var(R) is approximately (1/(n * xbar^2)) * [Var(Y) + R^2 Var(X) - 2 R Cov(X, Y)], computed at the user level. This gives a correct standard error for the ratio. Alternatives: bootstrap over users, or analyse a user-level metric (revenue per user) directly. The same issue applies to click-through rate when the unit is a user.

A353. Explain the sampling distribution of the difference of two proportions and write the z-test statistic, including when you use the pooled var...

For independent groups with sample proportions p1 = x1/n1 and p2 = x2/n2, the difference p1 - p2 is approximately normal (large n, with np and n(1-p) at least 10) with mean p1_true - p2_true and variance p1(1-p1)/n1 + p2(1-p2)/n2.

Hypothesis test (H0: p1 = p2): use the pooled proportion p = (x1 + x2)/(n1 + n2) because under H0 both groups share one rate. z = (p1 - p2) / sqrt(p(1-p)(1/n1 + 1/n2)).

Confidence interval for the difference: use the unpooled SE, (p1 - p2) +/- 1.96 * sqrt(p1(1-p1)/n1 + p2(1-p2)/n2), since no assumption of equality is made. Small differences between the two SEs are common; both are standard. For small counts use Fisher's exact test. The test is equivalent to the chi-square test on the 2x2 table (z squared = chi-square with 1 df).

A354. Probability: given a disease affecting 1% of a population, a test has 95% sensitivity and 90% specificity. A random person tests positive. W...

Bayes. P(D) = 0.01, P(+ | D) = 0.95, P(+ | not D) = 1 - 0.90 = 0.10.

P(+) = 0.95*0.01 + 0.10*0.99 = 0.0095 + 0.099 = 0.1085.

P(D | +) = 0.0095 / 0.1085 = 0.0876, about 8.8%.

Intuition with 10,000 people: 100 sick, of whom 95 test positive; 9,900 healthy, of whom 990 test positive. Of 1,085 positives only 95 are truly sick (8.8%). Most positives are false positives because the disease is rare (base-rate effect). The positive predictive value rises with prevalence and specificity; a follow-up confirmatory test is typical.

A355. Two fair dice are rolled. (a) What is the probability that the sum is 7? (b) Given that at least one die shows a 6, what is the probability ...

There are 36 equally likely outcomes.

(a) Sum 7: (1,6), (2,5), (3,4), (4,3), (5,2), (6,1) are 6 outcomes, so P = 6/36 = 1/6.

(b) Outcomes with at least one 6: 36 - 25 (no six at all: 5*5) = 11. Only (6,6) has both. P = 1/11, about 9.1%. Note it is not 1/6: the conditioning event "at least one 6" is different from "the first die is a 6" (which would give 1/6). Check via formula: P(both 6 | at least one 6) = (1/36) / (11/36) = 1/11.

A356. Birthday problem: how many people are needed in a room for the probability of a shared birthday to exceed 50%? Compute the exact probability...

Assume 365 equally likely birthdays. P(no match among n) = (365/365) * (364/365) * ... * ((365 - n + 1)/365). The answer is n = 23. For n = 23 the product is about 0.4927, so P(at least one match) = 1 - 0.4927 = 0.5073 (50.7%). For n = 22 it is 47.6%. For n = 30 it is about 70.6%, and for n = 57 about 99%. The result is surprising because the number of pairs grows quadratically: 23 people have 253 pairs, each with a 1/365 chance, so the expected number of matching pairs is 253/365 = 0.69.

```
import numpy as np
rng = np.random.default_rng(1)
trials = 100000
hits = 0
for _ in range(trials):
    b = rng.integers(0, 365, size=23)
    if len(set(b)) < 23:
        hits += 1
print(hits / trials)   # about 0.507

p_none = 1.0
for k in range(23):
    p_none *= (365 - k) / 365
print(1 - p_none)      # 0.5073
```

A357. You flip a fair coin repeatedly. What is the expected number of flips until you first see two heads in a row (HH)? Is it the same as the exp...

They are different: HH takes 6 flips on average, HT takes 4.

HH via states. Let E0 = expected additional flips from no progress, E1 = after one H. E0 = 1 + 0.5*E1 + 0.5*E0. E1 = 1 + 0.5*0 + 0.5*E0 (heads finishes, tails resets). Substitute: E0 = 1 + 0.5*(1 + 0.5*E0) + 0.5*E0 = 1.5 + 0.75*E0, so 0.25*E0 = 1.5, E0 = 6.

HT: wait for the first H, expected 2 flips. After an H, wait for a T: each flip is T with probability 1/2, expected 2 flips. Total 4. (If the next is H, you are still in the "just saw H" state, so no reset is needed, which is why HT is faster than HH, where a T resets all progress.)

A358. Sample size calculation: a landing page converts at 10%. You want to detect an absolute lift to 11% with 5% two-sided significance and 80% p...

Formula: n per group = (z_(1-alpha/2) + z_(1-beta))^2 * [p1(1-p1) + p2(1-p2)] / (p2 - p1)^2.

z values: 1.96 and 0.8416, sum 2.8016, squared 7.849.

Variances: p1(1-p1) = 0.10*0.90 = 0.09; p2(1-p2) = 0.11*0.89 = 0.0979; sum 0.1879.

Delta squared: 0.01^2 = 0.0001.

n = 7.849 * 0.1879 / 0.0001 = 7.849 * 1879 = about 14,748.

So roughly 14,750 visitors per variant (about 29,500 total). If you wanted a 1% relative lift (10% to 10.1%) it would be about 100 times larger, since n scales with 1/delta^2. With 90% power, the multiplier becomes (1.96 + 1.2816)^2 = 10.51, giving about 19,750 per variant.

A359. A sample of 64 delivery times has a mean of 50 minutes and a standard deviation of 16 minutes. Construct a 95% confidence interval for the t...

SE = s / sqrt(n) = 16 / 8 = 2.

With the normal approximation: 50 +/- 1.96*2 = 50 +/- 3.92, giving [46.08, 53.92]. Using the t distribution with 63 df (critical value about 1.998): margin 3.996, giving [46.0, 54.0]. The two agree closely at n = 64.

Interpretation: the procedure used to build the interval captures the true mean delivery time in 95% of repeated samples; the data are consistent with a true average between about 46 and 54 minutes. If the company's target is 45 minutes, the interval excludes it, suggesting deliveries are slower than target. To halve the margin of error to about 2 minutes, we would need four times the sample (n = 256).

A360. Control converts 200 out of 4,000 visitors; treatment converts 240 out of 4,000. Run a two-proportion z-test and state your conclusion.

p_c = 200/4000 = 0.05; p_t = 240/4000 = 0.06; difference = 0.01.

Pooled p = (200 + 240)/8000 = 0.055. SE = sqrt(0.055 * 0.945 * (1/4000 + 1/4000)) = sqrt(0.051975 * 0.0005) = sqrt(0.0000259875) = 0.005098.

z = 0.01 / 0.005098 = 1.96. Two-sided p-value = about 0.050 (0.0498).

Unpooled 95% CI for the difference: SE = sqrt(0.05*0.95/4000 + 0.06*0.94/4000) = sqrt(0.00001188 + 0.00001410) = sqrt(0.00002598) = 0.005097; CI = 0.01 +/- 0.00999 = [0.0000, 0.0200].

Conclusion: borderline. At alpha = 0.05 the result is right at the boundary, a 20% relative lift (0.06/0.05) with a lower bound on the CI of about zero. I would not call it conclusive; with a pre-registered alpha of 0.05 it just reaches significance, but the evidence is weak, and I would consider collecting more data or considering cost and prior beliefs before shipping.

A361. Group A (n = 25) has mean 52 and SD 10; group B (n = 25) has mean 47 and SD 10. Test whether the means differ at the 5% level.

Two-sample t-test (equal n, equal SD so pooled and Welch coincide).

SE = sqrt(s1^2/n1 + s2^2/n2) = sqrt(100/25 + 100/25) = sqrt(8) = 2.828.

t = (52 - 47) / 2.828 = 1.768 with df = 25 + 25 - 2 = 48.

Critical value (two-sided, 0.05, 48 df) is about 2.011. Since 1.768 < 2.011, we fail to reject H0; the two-sided p-value is about 0.083.

95% CI for the difference: 5 +/- 2.011*2.828 = 5 +/- 5.69 = [-0.69, 10.69], which includes 0 (consistent with the test).

Effect size: Cohen's d = 5/10 = 0.5 (medium). The test is underpowered for d = 0.5 with n = 25 per group (power is about 41%); about 64 per group would be needed for 80% power. So "not significant" does not mean "no difference".

A362. A test of whether a customer converts, for two page layouts: Layout A: 30 converted, 70 did not; Layout B: 50 converted, 50 did not. Perform...

Observed table: A: 30/70; B: 50/50. Row totals 100 and 100; column totals: converted 80, not converted 120; grand total 200.

Expected under independence: A converted = 100*80/200 = 40; A not = 60; B converted = 40; B not = 60.

chi2 = (30-40)^2/40 + (70-60)^2/60 + (50-40)^2/40 + (50-60)^2/60 = 2.5 + 1.667 + 2.5 + 1.667 = 8.333.

df = (2-1)(2-1) = 1; critical value at 0.05 is 3.84, so reject H0. p-value is about 0.004.

Equivalent z-test: p_A = 0.30, p_B = 0.50, pooled 0.40, SE = sqrt(0.4*0.6*(0.02)) = sqrt(0.0048) = 0.0693, z = 0.20/0.0693 = 2.887; z^2 = 8.33, matching chi-square.

Conclusion: conversion differs significantly (30% vs 50%); the 20 point gap has a CI of roughly 0.20 +/- 1.96*sqrt(0.21/100 + 0.25/100) = 0.20 +/- 0.133 = [0.067, 0.333].

A363. A support center receives on average 3 calls per hour (Poisson). What is the probability of (a) no calls in an hour and (b) at least 2 calls...

Poisson pmf: P(X = k) = exp(-lambda) lambda^k / k!, with lambda = 3.

(a) P(0) = e^(-3) = 0.0498 (about 5%).

(b) P(1) = 3 e^(-3) = 0.1494. P(X >= 2) = 1 - P(0) - P(1) = 1 - 0.0498 - 0.1494 = 0.8009 (about 80%).

Also: for 2 hours lambda = 6, so P(0) = e^(-6) = 0.0025. Mean and variance are both 3, SD = 1.73. The time between calls is exponential with mean 20 minutes.

A364. You flip a fair coin 10 times. What is the probability of (a) exactly 7 heads and (b) at least 7 heads?

Binomial(10, 0.5): P(X = k) = C(10, k) / 2^10 = C(10, k) / 1024.

(a) C(10, 7) = 120, so P = 120/1024 = 0.1172.

(b) P(X >= 7) = (C(10,7) + C(10,8) + C(10,9) + C(10,10)) / 1024 = (120 + 45 + 10 + 1)/1024 = 176/1024 = 0.1719.

Interpretation: getting 7 or more heads in 10 flips has a 17.2% chance with a fair coin, so a one-sided p-value for observing 7 heads is 0.17 and gives no evidence of bias. (Two-sided p = 0.344, by doubling.)

A365. Exam scores are approximately normal with mean 70 and SD 10. (a) What fraction score above 85? (b) What score is needed to be in the top 10%...

(a) z = (85 - 70)/10 = 1.5. P(Z > 1.5) = 1 - 0.9332 = 0.0668, about 6.7%.

(b) Top 10% means the 90th percentile. z_0.90 = 1.2816. Score = 70 + 1.2816*10 = 82.8, so about 83.

Check using the empirical rule: 95% lie between 50 and 90, so 2.5% score above 90; 85 lying between 1 and 2 SD above the mean is consistent with a tail of about 6.7%, between 15.9% and 2.3%.

A366. Compute the expected value, variance and standard deviation of one roll of a fair six-sided die, and the variance of the sum of two independ...

E[X] = (1+2+3+4+5+6)/6 = 21/6 = 3.5.

E[X^2] = (1+4+9+16+25+36)/6 = 91/6 = 15.1667.

Var(X) = 15.1667 - 3.5^2 = 15.1667 - 12.25 = 2.9167 = 35/12. SD = sqrt(2.9167) = 1.708.

Sum of two independent rolls: expected value 7; variance adds, 2 * 35/12 = 35/6 = 5.833; SD = 2.415. (Not 2 * SD: standard deviations do not add.) Mean of two rolls has variance 35/24 = 1.458, illustrating the sigma^2/n rule.

A367. Monty Hall: you pick one of three doors; the host, who knows where the car is, opens a different door with a goat and offers you a switch. S...

Yes, switch: it wins with probability 2/3 versus 1/3 for staying.

Reasoning: your initial pick is the car with probability 1/3. If it is (1/3), switching loses. If it is a goat (2/3), the host must open the other goat door, so the remaining door has the car and switching wins. Therefore P(win | switch) = 2/3. Bayes check: pick door 1, host opens door 3. P(host opens 3 | car at 1) = 1/2, P(opens 3 | car at 2) = 1, P(opens 3 | car at 3) = 0. Posterior for car at 2 = (1/3 * 1) / (1/3 * 1/2 + 1/3 * 1 + 0) = (1/3) / (1/2) = 2/3. The key assumption is that the host knowingly opens a goat door and always offers a switch; if he opens randomly (and happened to reveal a goat), the probabilities become 1/2 each.

```
import random
def play(switch, n=100000):
    wins = 0
    for _ in range(n):
        car = random.randrange(3)
        pick = random.randrange(3)
        opened = random.choice([d for d in range(3) if d != pick and d != car])
        if switch:
            pick = next(d for d in range(3) if d != pick and d != opened)
        wins += (pick == car)
    return wins / n
print(play(False), play(True))   # about 0.333 and 0.667
```

A368. Given x = (1, 2, 3, 4, 5) and y = (2, 4, 5, 4, 5), compute the Pearson correlation and the least-squares regression line.

Means: xbar = 3, ybar = 20/5 = 4.

Deviations dx = (-2, -1, 0, 1, 2); dy = (-2, 0, 1, 0, 1).

Sxy = sum(dx*dy) = 4 + 0 + 0 + 0 + 2 = 6. Sxx = 4+1+0+1+4 = 10. Syy = 4+0+1+0+1 = 6.

r = Sxy / sqrt(Sxx * Syy) = 6 / sqrt(60) = 6/7.746 = 0.775.

Slope = Sxy/Sxx = 0.6. Intercept = ybar - slope*xbar = 4 - 1.8 = 2.2. Line: y_hat = 2.2 + 0.6x.

R^2 = r^2 = 0.6, so 60% of the variance of y is explained. Prediction at x = 6: 2.2 + 3.6 = 5.8. Residuals at x = 1..5: y - y_hat = 2-2.8, 4-3.4, 5-4.0, 4-4.6, 5-5.2 = -0.8, 0.6, 1.0, -0.6, -0.2 (sum to zero, as required).

A369. Perform two steps of gradient descent on f(w) = (w - 3)^2 starting at w = 0 with learning rate 0.1. What happens with learning rate 1.1?

Gradient f'(w) = 2(w - 3).

Step 1: gradient at 0 is -6; w = 0 - 0.1*(-6) = 0.6.

Step 2: gradient at 0.6 is 2(0.6 - 3) = -4.8; w = 0.6 + 0.48 = 1.08.

Each step multiplies the distance to the optimum by (1 - 2*0.1) = 0.8 (distance 3, then 2.4, then 1.92), so it converges geometrically to 3.

With eta = 1.1: w1 = 0 - 1.1*(-6) = 6.6 (distance to optimum 3.6); w2 = 6.6 - 1.1*(2*3.6) = 6.6 - 7.92 = -1.32 (distance 4.32). The distance is multiplied by |1 - 2.2| = 1.2 each step, so it diverges. Stability needs |1 - 2*eta| < 1, i.e. 0 < eta < 1 here. eta = 0.5 reaches w = 3 in one step.

A370. Find the eigenvalues and eigenvectors of A = [[2, 1], [1, 2]] and explain what they mean geometrically.

Characteristic equation: det(A - lambda I) = (2 - lambda)^2 - 1 = 0, so 2 - lambda = +/-1, giving lambda = 3 and lambda = 1. Check: trace 4 = 3 + 1; determinant 3 = 3 * 1.

For lambda = 3: (A - 3I)v = [[-1, 1], [1, -1]] v = 0, so v1 = v2: eigenvector (1, 1)/sqrt(2). For lambda = 1: [[1, 1], [1, 1]] v = 0, so v = (1, -1)/sqrt(2). They are orthogonal, as expected for a symmetric matrix.

Geometry: A stretches vectors along the (1,1) direction by a factor of 3 and along (1,-1) by a factor of 1, leaving those directions unchanged. If A were a covariance matrix, the data cloud would be an ellipse elongated along (1,1), with variance 3 along that axis and 1 along the other; PCA's first component would be (1,1)/sqrt(2) explaining 3/4 = 75% of the variance.

A371. What is the expected number of rolls of a fair die needed to see every face at least once? (coupon collector)

Break the process into stages. When you have seen i distinct faces, the probability the next roll gives a new face is (6 - i)/6, so the expected rolls in that stage is 6/(6 - i) (geometric).

Total = 6/6 + 6/5 + 6/4 + 6/3 + 6/2 + 6/1 = 1 + 1.2 + 1.5 + 2 + 3 + 6 = 14.7.

In general for n coupons: n * H_n, where H_n = 1 + 1/2 + ... + 1/n, about n ln n + 0.577 n. Last stages dominate: the final new face alone takes 6 rolls on average. Linearity of expectation makes this work even though stages are dependent in time.

A372. What is the probability of getting at least one six in four rolls of a fair die? Explain the complement trick.

Use the complement: P(at least one six) = 1 - P(no six in four rolls). Each roll avoids a six with probability 5/6 and rolls are independent, so P(no six) = (5/6)^4 = 625/1296 = 0.4823. Therefore P(at least one six) = 1 - 0.4823 = 0.5177, about 51.8%.

This is the classic bet that the Chevalier de Mere found profitable. For comparison, at least one double six in 24 rolls of two dice: 1 - (35/36)^24 = 1 - 0.5086 = 0.4914, which is less than half. Complement counting is easier whenever "at least one" would otherwise require summing many cases.

A373. Brain teaser: a family has two children. (a) At least one is a boy; what is the probability both are boys? (b) The older child is a boy; wha...

Assume each child is independently a boy or girl with probability 1/2. Equally likely ordered outcomes (older, younger): BB, BG, GB, GG.

(a) "At least one is a boy" leaves BB, BG, GB, so P(both boys) = 1/3.

(b) "The older is a boy" leaves BB, BG, so P(both boys) = 1/2.

They differ because the conditioning information differs: in (a) the information eliminates only GG, while in (b) it specifies which child. The younger child is independent of the older, so the answer for (b) is simply the probability the younger is a boy. A related twist: if you learn that at least one child is a boy born on a Tuesday, the answer becomes 13/27 (about 0.48), because among 14 x 14 = 196 equally likely (sex, weekday) pairs per ordered child pair, 196 - 13*13 = 27 contain a Tuesday boy, and 13 of those have two boys. The lesson: the answer depends on how the information was obtained, so always clarify the protocol.

A374. An e-commerce company redesigned its checkout page. In a two-week A/B test, the new checkout converted 5.4% of 50,000 visitors against 5.0% ...

Step 1: Sanity checks. Verify randomization and instrumentation: check for sample ratio mismatch (expected 50/50; chi-square on user counts), confirm both arms ran the same days, bots and internal traffic were excluded, and the metric definitions match the plan.

Step 2: Test. x_c = 2,500, x_t = 2,700. Pooled p = 5,200/100,000 = 0.052. SE = sqrt(0.052*0.948*(2/50000)) = sqrt(0.049296 * 0.00004) = sqrt(0.00000197) = 0.001404. Difference = 0.004. z = 0.004/0.001404 = 2.85, p = 0.0044 (two-sided).

Step 3: Interval. Unpooled SE = sqrt(0.05*0.95/50000 + 0.054*0.946/50000) = sqrt(9.5e-7 + 1.02e-6) = sqrt(1.97e-6) = 0.001404. CI: 0.004 +/- 0.00275 = [0.00125, 0.00675], so a lift of 0.1 to 0.7 percentage points, relative lift 8% (CI roughly 2.5% to 13.5%).

Step 4: Beyond the primary metric. Check guardrails (average order value, revenue per visitor, refunds, page load time, support contacts, payment failure rate), and segments (device, new vs returning) by testing interactions, not by mining many p-values. Check novelty effects by looking at the daily trend of the lift.

Step 5: Business impact. If monthly traffic is 500k visitors at AOV $60, a 0.4 point lift is 2,000 extra orders, about $120k per month, versus the cost of maintaining the redesign.

Recommendation: statistically significant and practically meaningful; launch if guardrails are clean and the lift is stable over time, ideally with a staged rollout and a small long-term holdout to confirm.

A375. A pharmaceutical company runs a drug trial. The primary endpoint (blood pressure reduction) shows p = 0.12. However, among 20 pre-specified ...

Step 1: Hierarchy of evidence. The primary endpoint, the pre-registered confirmatory test, was not significant (p = 0.12). Subgroup analyses are exploratory unless a hierarchy of testing was pre-specified.

Step 2: Multiplicity. With 20 independent tests at alpha = 0.05, the chance of at least one false positive is 1 - 0.95^20 = 64%, and the expected number of false positives under no effect is 20*0.05 = 1. A single p = 0.03 is therefore unremarkable. Bonferroni threshold is 0.05/20 = 0.0025; p = 0.03 fails. Benjamini-Hochberg with the smallest p-value 0.03 must be below 0.05/20 = 0.0025 to be rejected, so it also fails.

Step 3: Subgroup caveats. Subgroups have smaller samples, so estimates are noisy and effects exaggerated (winner's curse). The right test is a treatment-by-subgroup interaction, not "significant in one subgroup and not in another".

Step 4: Recommendation. Do not claim efficacy for women over 60. State that the result is hypothesis-generating, and design a new confirmatory trial specifically for that subgroup with a pre-registered primary endpoint and adequate power. Be transparent in reporting all 20 analyses. Check for biological plausibility, consistent results in other trials and dose-response.

A376. A retailer emailed a discount coupon to its 100,000 loyalty customers. Compared with customers who did not open the email, those who opened ...

Problem: comparing openers to non-openers is a selection-bias problem. Customers who open emails are already more engaged, more loyal and higher spenders; opening is an outcome of engagement, not randomly assigned. The 40% mixes the campaign effect with pre-existing differences. Also, the pre/post comparison is confounded by seasonality.

Better design: (1) Randomized holdout: before sending, randomly assign, for example, 90% to receive the email and 10% to a control that does not. Compare average spend of everyone assigned to treatment versus everyone assigned to control (intent-to-treat), which is unbiased. (2) Incremental effect = mean spend (treated) - mean spend (control); report with a confidence interval and incremental revenue minus the coupon cost, so ROI is computed. (3) If you want the effect on those who actually open, use the complier average causal effect: ITT effect divided by the share who open (valid under standard instrumental variable assumptions).

If randomization is impossible: difference-in-differences against a comparable unexposed group, or propensity score matching on pre-period spending, tenure and engagement. Also check cannibalization (coupon used by customers who would have purchased anyway) and pull-forward (lower spending next month).

A377. A streaming service's daily active users dropped 10% overnight on Tuesday. The CEO asks you to find out why. How do you approach it?

Step 1: Verify the drop is real. Check data pipeline health, logging changes, delayed data, timezone or dashboard definition changes, and duplicate removal. Compare to historical day-of-week variation and compute whether a 10% change exceeds normal noise (for example, more than 3 standard deviations of the daily change).

Step 2: Scope. Slice by platform (iOS, Android, web, TV), app version, country, acquisition channel, new vs returning users, and user tenure. A drop concentrated in one slice points to a cause (for instance a bug in Android release 5.2); a uniform drop suggests an external or global cause (outage, holiday, a competitor event, a measurement change).

Step 3: Decompose. DAU = new users + returning users. Examine funnel metrics: app opens, login success rate, crash rate, latency, push notification delivery, ad spend and traffic sources. Check Simpson-style mix shifts: did the share of low-activity segments change?

Step 4: Timeline. Align with deployments, experiments launched, marketing campaign changes, outages, and external events (holiday, sports, weather).

Step 5: Hypothesis testing and action. Rank hypotheses by evidence, test the leading one (roll back a release to a holdout, compare exposed and unexposed users), and quantify impact. Communicate findings with confidence, and put alerting and anomaly monitoring in place.

A378. A hospital compares two treatments. Treatment A: small stones 81 of 87 successes, large stones 192 of 263 successes. Treatment B: small ston...

Compute rates.

Treatment A: small 81/87 = 93.1%; large 192/263 = 73.0%; overall 273/350 = 78.0%.

Treatment B: small 234/270 = 86.7%; large 55/80 = 68.8%; overall 289/350 = 82.6%.

A wins in both subgroups, yet B wins overall. This is Simpson's paradox.

Why: A was given mostly to large stones (263 of 350 = 75%), which are harder to treat, while B was given mostly to small stones (270 of 350 = 77%), which are easy. The overall rate is a weighted average with different weights.

Decision: stone size is a confounder (it influences treatment assignment and outcome and is not caused by the treatment), so compare within stone size or standardize to a common mix. Standardizing to the overall mix (small 357, large 343 of 700): A = (0.931*357 + 0.730*343)/700 = 0.833; B = (0.867*357 + 0.688*343)/700 =

0.779. A is better. Better still, run a randomized trial or adjust for confounders in a regression (logistic regression with stone size). I disagree with management.

A379. An A/B test at a travel site was designed for a 50/50 traffic split. The final counts are 50,600 users in control and 49,400 in treatment. T...

This must be checked formally: a sample ratio mismatch (SRM) test.

Expected: 50,000 each (total 100,000). Chi-square = (50,600 - 50,000)^2/50,000 + (49,400 - 50,000)^2/50,000 = 360,000/50,000 * 2 = 7.2 * 2 = 14.4. With 1 df, p is about 0.00015 (z = 3.79). A discrepancy this large would happen about 1 in 6,700 times by chance if assignment were truly 50/50.

Interpretation: a 1.2% imbalance feels small but with 100,000 users the standard deviation of the count is only 158, so 600 is 3.8 SD. SRM signals a bug in the experiment pipeline: assignment, bot filtering that affects arms differently, redirects or page load time causing treatment users to drop before being logged, a bad hash, or a filtering step dependent on post-treatment behavior. Any of these can bias the effect estimate in unknown ways.

Action: do not trust the lift. Debug the root cause (check assignment logs vs analysis logs, by device, browser and country), fix, and rerun. Commonly alert on SRM at $p < 0.001$.

A380. A product manager checks the A/B test dashboard every day and plans to stop the test as soon as the p-value dips under 0.05. On day 6 of a p...

No, this is the peeking (optional stopping) problem. A fixed-horizon test controls the false positive rate at 5% only if you look once at the pre-planned sample size. Under H0 the p-value wanders as data accumulate; checking daily and stopping at the first $p < 0.05$ can inflate the actual Type I error to 20 to 30% or more with many looks.

Also, early data are often unrepresentative (novelty effect, weekday mix: day 6 may omit a weekend) and early effect estimates are exaggerated (winner's curse).

What to do: (1) Continue until the planned sample size or duration (full weekly cycles) and analyse once. (2) If early stopping is a genuine business need, use a sequential design designed for it: group sequential tests with alpha-spending (O'Brien-Fleming, Pocock), always-valid p-values (mSPRT), or Bayesian decision rules calibrated by simulation. (3) Pre-register the stopping rules and primary metric. (4) Use the dashboard for guardrail monitoring only (stop for harm), with appropriate corrections.

A381. A social app tested a new feed layout. In week 1 engagement was up 12%, but by week 4 the lift had shrunk to 2% and was no longer significan...

Likely explanations: (1) Novelty effect: users interact more with anything new, then behavior reverts. (2) Learning curve or change aversion in the opposite direction (the opposite pattern would appear as an initial dip that recovers). (3) Early sample composition: heavy users show up in the first days, and new and casual users accumulate later (survivorship in a cumulative metric). (4) External factors: seasonality, campaign, holidays. (5) Regression to the mean/winner's curse if you only looked at the best early period.

Diagnosis: plot the daily (non-cumulative) treatment-control difference with confidence bands; analyze by user cohort (new vs existing users, based on first exposure date); check the metric over days since first exposure; see whether other metrics (retention, session length) move.

Recommendation: the sustained effect is the one that matters, so base the decision on the stabilized later-weeks effect (2%, CI including 0). Run longer or run a long-term holdout (a few percent of users kept on the old experience) to estimate the steady-state effect. If the 2% is still worth the cost, ship, but do not forecast annual gains from week-1 numbers.

A382. A ride-sharing marketplace wants to test a new driver incentive. Randomizing riders or drivers to treatment and control means they share the...

The issue is interference (violation of SUTVA, the stable unit treatment value assumption). In a two-sided marketplace, treated drivers who take more rides reduce supply available to control drivers; riders compete for the same cars. So control outcomes are affected by treatment, the measured difference is biased (typically it overstates or distorts the effect that a full rollout would have), and the global effect differs from the A/B estimate. Network effects on social platforms have the same problem.

Alternatives: (1) Switchback (time-based) experiments: alternate treatment and control across the whole market in time blocks (for example 30-minute or hourly windows), randomized and balanced across times of day; need washout periods for carry-over and analysis with block-level clustering. (2) Geo experiments: randomize whole cities or regions (matched pairs), analysed with synthetic control or difference-in-differences; fewer units mean lower power, so use pre-period covariates. (3) Cluster randomization on graph clusters for social networks. (4) Budget-split or two-sided randomization designs for marketplaces.

Analyse at the level of randomization (cluster-robust errors), plan for power at that level, and monitor spillover metrics.

A383. An online store's A/B test measures revenue per user. A single user in the treatment group made a $50,000 purchase (a B2B order) and the tre...

Step 1: Investigate. Verify the transaction is real and not fraud or a test order, and check whether the buyer was exposed to the treatment in a way that could have caused the purchase. Check whether similar whales appear in control (in expectation they should, by randomization).

Step 2: Understand the issue. Revenue is heavy-tailed. The sample mean is dominated by one point; the variance is inflated and the t-test's normality assumption is badly violated. Result with the whale is not robust. The p = 0.01 is largely driven by an unrepeatable event: for 20,000 users per arm, $50,000 adds $2.50 per user, which could exceed the true effect (say $0.40).

Step 3: Robust analysis. Report with and without the point; winsorize revenue at a high percentile (99.9th) or cap at a business-sensible value; analyse conversion rate and revenue among converters separately (a two-part model); use a log transform or bootstrap; use a rank test as a sensitivity check.

Step 4: Decision. Pre-register outlier handling policy before launching tests. If the treatment effect depends on the capped metric and is still positive and significant, believe it; otherwise the result is inconclusive. Do not simply delete data after seeing the result.

A384. A marketing team wants to compare five variants of an ad creative against the control, at alpha = 0.05 each. Variant C has p = 0.03 and othe...

Five comparisons against one control create a multiple testing problem: P(at least one false positive) = 1 - 0.95^5 = 22.6% if all variants have no effect. With p = 0.03 for C, a Bonferroni correction would use 0.05/5 = 0.01, and C would not be significant (adjusted p = 0.15). Holm gives the same here since the smallest p must be below 0.01. Dunnett's test, designed for many treatments versus one control, is also a good option.

Additionally, C is the best of five, so its observed lift is inflated (winner's curse): selecting the maximum biases the estimate upwards.

Advice: report adjusted p-values, treat C as a promising candidate, and run a confirmatory follow-up test with C against control (single comparison, pre-registered), possibly with more traffic. Also consider the practical effect size and cost, not only significance. If the team has to choose now, the downside risk of picking C is small if creative costs are equal, but they shouldn't report it as a proven winner.

A385. An A/B test of a new recommendation widget returned a 1.5% relative lift in revenue with p = 0.40 and a 95% CI of [-2%, +5%]. The stakeholde...

Absence of evidence is not evidence of absence. A p-value of 0.40 means the data are compatible with zero effect, but the CI [-2%, +5%] is equally compatible with a 5% gain, which would be commercially significant. The test was likely underpowered for the effect sizes that matter.

Steps: (1) Compute the minimum detectable effect the test had; with the observed standard error SE of about (5 - (-2))/(2*1.96) = 1.79% points, an 80% power design could only detect effects of about 2.8*1.79 = 5.0%. So it could not detect anything below 5%. (2) Decide the smallest effect worth shipping, say 2%. The CI does not rule out gains or losses of that size. (3) Options: extend the test, increase traffic, reduce variance (CUPED), or accept the uncertainty and decide on cost: if the feature is cheap and risk is low (CI lower bound -2% is tolerable), ship; if expensive to maintain, stop. (4) Equivalence testing could formally show the effect is within a band.

Message: the result is inconclusive, not negative.

A386. A news app's click-through rate (CTR) fell from 4.0% to 3.6% month over month, but CTR rose within every device segment (mobile, desktop, ta...

This is Simpson's paradox via mix shift. Overall CTR is a weighted average of segment CTRs, weighted by traffic share. If traffic shifted toward a segment with lower CTR (for example from desktop at 6% to mobile at 3%), the overall rate falls even when each segment improves.

Illustrative numbers. Last month: desktop 50% of traffic at 6.0% CTR, mobile 50% at 2.0%: overall = 0.5*6 + 0.5*2 = 4.0%. This month: desktop 20% of traffic at 6.5%, mobile 80% at 2.5%: overall = 0.2*6.5 + 0.8*2.5 = 1.3 + 2.0 = 3.3%. Every segment improved but the total declined.

Analysis: decompose the change into a rate effect (within-segment changes) and a mix effect (changes in traffic composition), e.g. Oaxaca-style or Kitagawa decomposition. Hold the mix constant: compute this month's CTR at last month's mix: 0.5*6.5 + 0.5*2.5 = 4.5%, higher than 4.0%.

Report: the underlying product performance improved; the headline fell due to a shift in audience toward mobile. Investigate why mix shifted (marketing channel change, desktop traffic decline) and use mix-adjusted metrics (standardized CTR) in dashboards, plus segment-level views.

A387. A company ran a customer satisfaction survey by email. Only 5% responded, and the average Net Promoter Score (NPS) was 62. Leadership wants ...

Concerns: (1) Non-response bias: with a 5% response rate, respondents are self-selected and likely differ (very happy or very unhappy customers, or heavy users). The direction of bias is unknown. (2) Coverage: email-only excludes customers without a valid email or who opted out. (3) Timing and mode effects, such as surveys sent right after a positive experience. (4) Margin of error: with n = 1,000 respondents the sampling error is small

(about +/- 3 NPS points depending on the distribution), but this reflects only random error, not bias. A large sample size does not fix a biased sample.

Actions: compare respondents to the full customer base on observable traits (tenure, spend, region, plan) and apply post-stratification weights; send reminders and use multiple channels to raise the response rate; take a small random sample of non-respondents and follow up intensively (non-response follow-up) to estimate bias; triangulate with behavioral data (retention, repeat rate, support tickets); present results with caveats and a range (sensitivity or bounds analysis).

A388. A bank's fraud model flags transactions. Fraud occurs in 0.1% of transactions; the model catches 99% of fraud (sensitivity) and has a 1% fal...

Compute the precision (positive predictive value) with Bayes. Per 1,000,000 transactions: fraud = 1,000; non-fraud = 999,000.

True positives = 0.99 * 1,000 = 990. False positives = 0.01 * 999,000 = 9,990.

Flagged total = 10,980. Precision = 990/10,980 = 0.0902, only about 9%.

So about 91% of flagged transactions are legitimate: the base-rate fallacy. The "99%" described sensitivity and specificity, not the probability that a flag is fraud.

Implications: review capacity and customer friction costs (10,980 alerts for 990 frauds); use cost-sensitive thresholds, precision-recall curves rather than accuracy (a model that flags nothing is 99.9% accurate); layer a second-stage model or rule; track precision at fixed review capacity (precision at top-k) and expected monetary value. Re-check calibration, since prevalence may differ between training data and production.

A389. A retail chain launched a loyalty program in 100 of its 200 stores (chosen by regional managers, not randomly). Average weekly sales per sto...

Because assignment wasn't random, a post-period comparison (120 vs 108) would be confounded by differences in store type. Use difference-in-differences (DiD): effect = (change in treated) - (change in control) = (120 - 100) - (108 - 100) = 20 - 8 = 12 thousand dollars per store per week (a 12% lift on the baseline of 100). The control group's +8 captures the common time trend (seasonality, economy), which is removed.

Key assumption: parallel trends. In the absence of the program, treated stores would have moved like control stores. Check with multiple pre-period weeks (do the two groups' trends match before launch?) and run a placebo test on a fake launch date. Also check no spillover (customers shopping in both) and no anticipation effects.

Implementation: regression sales_it = a + b*treated_i + c*post_t + d*(treated_i x post_t) + e_it with store and week fixed effects, standard errors clustered by store. d is the DiD estimate. If trends differ, use synthetic control or add covariates, or matching on pre-period trends. Report CI and explore effect heterogeneity. Selection by managers (picking promising regions) threatens the assumption, so ask how stores were selected.

A390. A regression for a retailer reports: weekly sales = 50 + 3.2 * ad_spend (in thousands), R^2 = 0.64, p < 0.001 for ad_spend. The marketing VP...

Interpretation: the coefficient says that weeks with $1,000 more ad spend are associated with $3,200 more sales on average, an association, not necessarily a causal effect.

Concerns: (1) Confounding/reverse causality: the company may spend more on ads in high-season weeks (holidays) or promotions, so seasonality drives both. Or budgets may be set as a percentage of expected sales.

(2) Omitted variables (price, promotions, competitor actions); include them and time controls. (3) Diminishing returns: the true response is concave (saturation), a linear model extrapolates incorrectly outside observed spend; try log transform or marketing mix modelling with adstock/saturation. (4) Autocorrelation in time series makes standard errors too small and p-values unreliable (use HAC/Newey-West errors, check Durbin-Watson). (5) R^2 = 0.64 says 64% of variance explained, not that the effect is causal.

Better: run geo-based incrementality experiments or randomized holdouts where ad spend is varied randomly, or use instrumental variables. State the CI of the coefficient. Treat the 3.2 as an upper bound on ROI until validated experimentally.

A391. A telecom analyst finds that customers who use the mobile app have 30% lower churn than those who do not, and suggests forcing everyone to i...

The relationship is correlational and likely confounded by engagement. Customers who are already satisfied and active choose to install the app; at-risk customers rarely do. Reverse causality is possible too: customers planning to leave will not bother installing. So forcing installation need not reduce churn, and might annoy customers and raise churn.

Approach to test causality: (1) Randomized experiment: randomly encourage a subset of customers (in-app prompt, incentive) to install and compare churn between those encouraged and not (intent-to-treat), then scale by the difference in install rates to get the effect of installing for compliers (IV logic). (2) If not possible, use observational methods: propensity score matching or regression adjustment on tenure, plan, usage, support calls, demographics measured before the app adoption; include timing-aware methods (compare churn after install with matched non-adopters, avoid immortal time bias, as customers must survive until they install). (3) Sensitivity analysis (E-value) for unmeasured confounding.

Report: correlation is a hypothesis generator; evaluate the intervention through an experiment and estimate costs and benefit.

A392. A hospital uses a screening test for a condition. As a data scientist you can choose the decision threshold on the model's risk score. Expla...

Sensitivity (recall, true positive rate) = TP/(TP + FN): the share of sick patients detected. Specificity = TN/(TN + FP): the share of healthy patients correctly cleared. Precision (PPV) = TP/(TP + FP). Lowering the threshold increases sensitivity but lowers specificity (more false alarms); raising it does the reverse. The ROC curve plots sensitivity versus 1 - specificity across all thresholds; AUC is the probability that a randomly chosen positive gets a higher score than a randomly chosen negative (0.5 is random, 1 is perfect). For rare conditions, precision-recall curves are more informative since AUC-ROC can look good even when precision is poor.

Choosing the threshold: set by costs. If a missed cancer (FN) costs 50 times a false alarm (FP) leading to an unnecessary biopsy, choose a threshold that gives high sensitivity (e.g. 95%) and accept lower specificity; optimal predicted-probability threshold = C_FP / (C_FP + C_FN) = 1/51 = 2%. Consider prevalence: low prevalence makes PPV low even with high specificity, so use a two-stage screening process. Report operating points with confidence intervals, validate calibration, and choose thresholds on validation data, not the test set.

A393. A video platform tested a new recommendation algorithm. Click-through rate rose 3% (significant), but average watch time per user fell 2% (s...

Step 1: Understand the metrics hierarchy. Define the north-star metric (long-term user value, such as retention or total watch time) and guardrails. CTR is a proxy; higher CTR with lower watch time suggests more clickbait: users click more but are less satisfied and abandon videos sooner.

Step 2: Dig deeper. Decompose watch time = sessions per user * videos per session * minutes per video. Look at completion rates, short-click ("bounce") rates, search and dislike rates, diversity of content, new vs heavy users, creators' side effects (creator revenue and exposure).

Step 3: Statistical caution. Both effects are significant, but multiple metrics increase false positives; confirm the pre-registered primary/guardrail metrics. Retention at 30 days could lag, so extend or use a long-term holdout (1 to 5% of users) for several months. Test the interaction by segment, with an eye on a negative effect on a valuable segment.

Step 4: Decision framework. If watch time (a guardrail tied to revenue) fell significantly and the primary long-term metric is flat, do not ship as is. Iterate on the model (optimize for watch-time-weighted or satisfaction objectives rather than raw clicks), and re-test. Document the trade-off and quantify revenue impact (ad impressions depend on watch time).

A394. Your team plans an A/B test with a required sample of 14,750 users per arm. The site gets 3,000 eligible new visitors per day, split 50/50. ...

Total required = 2 * 14,750 = 29,500 users. At 3,000 per day: 29,500 / 3,000 = 9.8 days, so the minimum is 10 days.

Practical considerations: (1) Weekly seasonality: behavior differs between weekdays and weekends, so run full weeks. Round up to 14 days (two complete weekly cycles), which also gives more sample (42,000 total) and extra power. (2) Novelty and primacy effects: a longer window reveals whether the lift decays. (3) Conversion lag: if purchases happen days after the first visit, the last-entered users need time to convert; either extend the window or use a fixed observation window per user (for example 7-day conversion), which adds up to 7 days after enrollment ends. (4) Do not stop early when the p-value looks significant (peeking). (5) Holidays, marketing campaigns, and other overlapping experiments. (6) Ramp-up: start at 5 to 10% traffic to catch bugs, then go to 50/50, and the clock begins at full allocation.

Decision: plan 14 days of full allocation plus the conversion window, and fix the analysis plan ahead of time.

A395. A city's data show that months with higher ice cream sales also have more drowning incidents (correlation 0.85). A councillor proposes banni...

The correlation is real but not causal. The common cause is temperature/season: hot weather increases both ice cream consumption and swimming (more exposure to drowning risk). Temperature is a confounder: it affects both variables, creating a spurious association between them.

Evidence to show: control for the confounder. Within a temperature band (for example only days at 30 C), the relationship between ice cream sales and drownings should largely vanish; a regression of drownings on ice cream sales and temperature should show an ice cream coefficient close to zero. Also note no plausible mechanism, and that banning ice cream would not reduce swimming.

Communication to the councillor: two things rise together in summer; to reduce drownings, target actual drivers (lifeguards, swimming lessons, signage). A correlation of 0.85 means that about 72% (0.85^2) of the

variance in one is linearly shared with the other, yet that shared variance may entirely come from the third variable. A randomized or natural experiment would be needed to claim causality.

A396. A growth team wants to reduce variance in A/B tests and has heard about CUPED. Explain how it works with an example, and when it is helpful.

CUPED (Controlled-experiment Using Pre-Experiment Data) uses a pre-experiment covariate X (for example each user's spend in the 4 weeks before the test) that is correlated with the outcome Y but unaffected by treatment. Compute the adjusted metric Y_adj = Y - theta * (X - mean(X)), where theta = Cov(X, Y) / Var(X), estimated on pooled data. The adjusted metric has the same expected treatment effect (since X is independent of assignment because it is pre-treatment) but variance Var(Y)(1 - rho^2), where rho is the correlation between X and Y.

Example: if pre-period and in-test spend correlate at rho = 0.7, variance is reduced by rho^2 = 49%, so you need about half the sample size, or equivalently confidence intervals are sqrt(0.51) = 0.71 of the previous width (29% narrower).

When helpful: metrics that are stable per user (revenue, engagement, time spent) and populations with history. Not helpful for new users with no pre-period data (use other predictors), or metrics with weak autocorrelation. It must use only pre-treatment covariates; using post-treatment variables biases the estimate. Equivalent to regression adjustment (ANCOVA) with the covariate.

A397. A company A/B tests a new onboarding flow. 40% of users in the treatment group never complete onboarding and drop out; the analyst removes t...

No. This breaks randomization and creates selection bias. Treatment assignment is random, but completing onboarding is a post-treatment outcome; dropouts in treatment are likely different (less motivated) users. Removing them leaves a treatment group made of the most motivated users compared with an unfiltered control group. The comparison is not like for like.

Correct analyses: (1) Intent-to-treat (ITT): analyse everyone as randomized, comparing all treatment users with all control users. This is unbiased for the effect of offering the new flow, including the dropouts as a real cost of the design. (2) If a like-for-like comparison of completers is desired, apply the same filter to control (users who would complete the old flow), but that is not identifiable directly; use principal stratification or compare on pre-defined, pre-treatment segments. (3) Also analyse the funnel: completion rate in each arm, then outcomes among completers with caution.

Takeaway: a flow where 40% of users drop out might be a worse design (check the control's completion rate); ITT gives the honest business answer. Never condition on a post-randomization variable.

A398. A food-delivery company wants to know whether a new fast-delivery feature increases order frequency. They cannot randomize, because the feat...

Approach: a synthetic control or difference-in-differences design with comparison cities.

Step 1: Collect weekly orders per user (or per capita) for the treated city and many untreated candidate cities for a long pre-period (for example 26 weeks) and post-period.

Step 2: Check comparability. Pre-trends should be parallel. Use cities with similar size, seasonality, competitive conditions, demographics, and no other launches.

Step 3: Estimate. Option A (DiD): effect = (treated post - treated pre) - (control post - control pre). Option B (synthetic control): choose non-negative weights across donor cities that reproduce the treated city's pre-period series; the effect at each post week = actual - synthetic. This works well with a single treated unit.

Step 4: Inference. With one treated unit, use placebo (permutation) tests: apply the same method to each donor city as if it were treated, and compare the treated city's post/pre RMSE ratio with the placebo distribution (p = rank / number of cities). Also run pre-period placebo dates.

Step 5: Threats and robustness: concurrent events in the treated city (weather, promotions, competitor exit), anticipation, spillovers from nearby cities, and drift in pre-trends. Report effect with an interval and a plot of treated vs synthetic city series.

Randomized city-level rollout would be better next time (staggered rollout with randomized order).

A399. A survey for a mobile game finds that 30% of its 2,000 sampled players say they spend money in the game. The company wants a 95% confidence ...

Interval: p_hat = 0.30, n = 2,000. SE = sqrt(0.30*0.70/2000) = sqrt(0.0001050) = 0.01025. Margin = 1.96*0.01025 = 0.0201. CI = [0.280, 0.320], so 28.0% to 32.0%. (Conditions satisfied: np = 600 and n(1-p) = 1400 both exceed 10.)

Sample size for margin E = 0.01: n = z^2 * p(1-p) / E^2 = 1.96^2 * 0.21 / 0.0001 = 3.8416 * 0.21 / 0.0001 = 0.80674/0.0001 = 8,067, about 8,070 players. If p is unknown, use the worst case p = 0.5: n = 3.8416 * 0.25 / 0.0001 = 9,604.

Caveats: this assumes a simple random sample; self-reported spending is subject to response and social-desirability bias, which no sample size fixes; use actual transaction data if available; clustered or stratified samples need a design effect adjustment (effective n = n / deff).

A400. A fintech app tested three onboarding variations (A, B, C). Average first-week transaction counts per user are: A 4.1, B 4.4, C 4.9, with n ...

Better: omnibus one-way ANOVA first, then post hoc comparisons with multiplicity control (Tukey HSD), reporting effect sizes and CIs.

Grand mean = (4.1 + 4.4 + 4.9)/3 = 4.4667. Between-group SS = n * sum((mean_i - grand)^2) = 1000 * (0.3667^2 + 0.0667^2 + 0.4333^2) = 1000 * (0.13444 + 0.00444 + 0.18778) = 1000 * 0.32667 = 326.67. df_between = 2, MS_between = 163.33.

Within-group MS = SD^2 = 9 (pooled), df_within = 3000 - 3 = 2997.

F = 163.33 / 9 = 18.15 on (2, 2997) df; the critical value at 0.05 is about 3.00; p is about 1.4e-8. Reject H0: at least one differs.

Post hoc: SE of a pairwise difference = sqrt(2*9/1000) = 0.134. C vs A: 0.8/0.134 = 5.96; C vs B: 0.5/0.134 = 3.73; B vs A: 0.3/0.134 = 2.24. Tukey critical value for 3 groups with large df: q = 3.31, so the HSD = 3.31 * sqrt(MS_within/n) = 3.31 * sqrt(0.009) = 3.31 * 0.0949 = 0.314. Differences: C-A 0.8 and C-B 0.5 exceed 0.314 (significant); B-A 0.3 does not (not significant after multiplicity adjustment, although the unadjusted test would give p = 0.025).

Effect size: eta squared = 326.67/(326.67 + 26,973) = 0.012 (small); the practical value is C's lift of 0.8 transactions (about 20% relative to A). Check assumptions (Levene) and skew of count data (consider a Poisson/negative binomial model or Kruskal-Wallis).

Machine Learning Fundamentals (up to Linear and Logistic Regression)

A401. Walk me through the end-to-end machine learning workflow for a typical analytics project.

A practical workflow has these stages:

1. Problem framing: translate the business question into a prediction target, decide whether it is classification, regression or clustering, define the unit of prediction (customer, loan, visit), the prediction time and the success metric (business and statistical).

2. Data collection and understanding: pull data from databases or files, check grain, joins, time coverage, target definition and quality.

3. Exploratory data analysis: distributions, missing values, outliers, correlations with the target, class balance.

4. Data splitting: create train/validation/test (or cross-validation folds) BEFORE any fitting step, to avoid leakage. Use time-based splits when data is temporal.

5. Preprocessing and feature engineering: impute, encode categoricals, scale, create ratios, aggregates, date parts. Fit all transformers on training data only (use pipelines).

6. Baseline model: a naive baseline (mean, majority class) and a simple model such as linear or logistic regression.

7. Model training and tuning: try a few models, tune hyperparameters with cross-validation.

8. Evaluation: on the held-out test set with metrics matching the business cost (recall, precision, AUC, RMSE...). Also look at residuals, calibration, errors by segment.

9. Interpretation and communication: coefficients, feature importance, business impact.

10. Deployment and monitoring: serve predictions, monitor data drift, performance decay, retrain schedule.

The process is iterative; findings in evaluation often send you back to feature engineering or even problem framing.

A402. What is the difference between supervised and unsupervised learning? Give examples of each.

Supervised learning uses labelled data: every training row has inputs (features X) and a known output (target y). The model learns a mapping X to y and is evaluated by comparing predictions to known labels. Regression (predict a number such as house price) and classification (predict a category such as churn yes/no) are the two kinds. Examples: linear regression, logistic regression, KNN, Naive Bayes.

Unsupervised learning has no target. The goal is to discover structure in X: grouping similar records (clustering, e.g. customer segmentation with k-means), finding associations (market basket analysis), or summarising data. Evaluation is harder because there is no ground truth; you use internal measures (silhouette) and business usefulness.

There is also semi-supervised learning (few labels, many unlabelled rows) and reinforcement learning (learning from rewards), but data analyst interviews mostly focus on the first two. Key interview point: if you have a historical outcome column to predict, it is supervised.

A403. Why do we split data into training, validation and test sets? What is each used for?

The goal of a model is to perform well on new, unseen data, so we must estimate performance on data the model has not learned from.

Training set: used to fit model parameters (coefficients).

Validation set: used to compare models, tune hyperparameters (regularization strength, k in KNN, threshold) and make design decisions. Because we make choices based on it, it becomes slightly optimistic.

Test set: touched only once at the end to give an unbiased estimate of generalization error. If you keep tuning against the test set, it leaks into your decisions and the estimate becomes optimistic.

Typical splits are 60/20/20 or 70/15/15 or 80/20 with cross-validation replacing a fixed validation set. For classification use stratified splitting so class proportions are preserved. For time series use chronological splits (train on past, test on future). Always fit scalers, imputers and encoders on training data only.

A404. What is k-fold cross-validation and why is it preferred over a single train/test split?

In k-fold cross-validation the training data is divided into k equal folds. The model is trained k times; each time one fold is held out for validation and the remaining k-1 folds are used for training. The k validation scores are averaged (and their standard deviation reported).

Benefits: (1) every observation is used for both training and validation, so the estimate is less dependent on one lucky or unlucky split; (2) it gives a measure of variability of performance; (3) it is better with small datasets; (4) it is the standard way to tune hyperparameters (GridSearchCV).

Costs: k times more computation. Typical k is 5 or 10. Variants: stratified k-fold (keeps class ratios, use for classification), leave-one-out (k = n, high variance and expensive), repeated k-fold, group k-fold (keeps all rows of a customer in the same fold), and time-series split (forward chaining, never train on the future).

Important: preprocessing must be inside the cross-validation loop (use a Pipeline) otherwise information leaks from validation folds.

A405. When would standard k-fold cross-validation be the wrong choice, and what would you use instead?

Standard k-fold shuffles rows and assumes they are independent and identically distributed. It is wrong when:

1. Imbalanced classification: random folds may contain very few positives. Use StratifiedKFold.

2. Time-ordered data (sales forecasting, default prediction over time): shuffling lets the model train on the future and validate on the past, giving inflated scores. Use TimeSeriesSplit (expanding window) or a rolling origin.

3. Grouped data (several rows per customer, patient or store): the same entity in train and validation leaks identity information. Use GroupKFold.

4. Very large data: a single holdout is enough and cheaper.

5. Very small data: use repeated k-fold or leave-one-out.

Also, if you do feature selection or scaling on the full data before CV, the result is biased. Wrap everything in a Pipeline so it is refit inside every fold. For model selection plus honest evaluation use nested cross-validation (inner loop tunes, outer loop evaluates).

A406. Explain the bias-variance trade-off.

Expected prediction error on new data decomposes into bias squared + variance + irreducible noise.

Bias is error from wrong or too simple assumptions: the model cannot capture the true pattern (a straight line fitted to a curved relationship). High bias leads to underfitting: poor performance on both training and test data.

Variance is error from sensitivity to the particular training sample: a very flexible model (high-degree polynomial, KNN with k=1) fits noise, so predictions change a lot with different training sets. High variance leads to overfitting: excellent training performance, much worse test performance.

Irreducible noise cannot be removed by any model.

Trade-off: increasing model complexity lowers bias but raises variance, and vice versa. The best model sits at the complexity that minimises total test error.

Practical levers: to reduce variance, get more data, simplify the model, regularize, remove noisy features, use cross-validation. To reduce bias, add features or interactions/polynomial terms, use a more flexible model, reduce regularization. Learning curves (train vs validation error against training size) help diagnose which problem you have.

A407. What are overfitting and underfitting? How do you detect and fix each?

Overfitting: the model learns noise and quirks of the training set. Signs: very high training score, noticeably lower validation/test score (large gap). Fixes: more training data, fewer or simpler features, regularization (L1/L2), larger k in KNN, early stopping in gradient methods, cross-validation, removing leakage-free but noisy features.

Underfitting: the model is too simple to capture the signal. Signs: poor score on both training and validation data, residuals showing clear structure (curve) when plotted. Fixes: add relevant features, interactions or polynomial terms, transform variables, use a more flexible model, reduce regularization.

Detection tools: compare train vs validation metrics, learning curves, validation curves against a hyperparameter, residual plots. Example: a regression with train R2 = 0.98 and test R2 = 0.55 is overfit; train and test R2 both 0.30 suggests underfit or a weak signal.

A408. What is data leakage? Give examples and explain how to prevent it.

Data leakage is when information that would not be available at prediction time is used to train the model, producing unrealistically good validation scores and poor real-world performance.

Types and examples:

1. Target leakage: a feature that is a consequence of the target, e.g. 'account closure date' when predicting churn, 'collections status' when predicting loan default, 'discharge disposition' when predicting readmission at admission time.

2. Train-test contamination: fitting the scaler, imputer, encoder, PCA or feature selection on the whole dataset before splitting; target encoding computed with the full data.

3. Temporal leakage: random splits on time series, or features computed with information from after the prediction date (e.g. 12-month average that includes future months).

4. Duplicates or the same entity appearing in both train and test.

Prevention: define the prediction timestamp and only use data before it; split first, then fit transformers on training only (sklearn Pipeline); use time-based or group-based splits; review every feature with a domain expert; be suspicious of any feature with extremely high importance or a model with near-perfect AUC.

A409. Explain L1 (Lasso) and L2 (Ridge) regularization. How do they differ?

Regularization adds a penalty on coefficient size to the loss function to reduce overfitting, trading a little bias for a larger reduction in variance.

Ridge (L2): loss = RSS + lambda * sum(beta_j^2). Shrinks all coefficients toward zero smoothly but almost never exactly to zero. Handles multicollinearity well (correlated predictors share weight). Has a closed form solution.

Lasso (L1): loss = RSS + lambda * sum(|beta_j|). Can shrink some coefficients exactly to zero, performing automatic feature selection and giving sparse, interpretable models. With a group of highly correlated features it tends to pick one arbitrarily.

Elastic Net combines both: lambda * (alpha * L1 + (1-alpha) * L2).

Key points: the intercept is not penalised; features must be standardised before regularization because the penalty depends on scale; lambda (called alpha in sklearn Ridge/Lasso, and C = 1/lambda in LogisticRegression) is chosen by cross-validation; larger lambda means more bias, less variance. Use Ridge when most features are useful and correlated, Lasso when you suspect many are irrelevant.

A410. Why does L1 regularization produce sparse solutions while L2 does not? Explain intuitively.

Regularized regression can be viewed as minimising RSS subject to a constraint on the coefficients: L1 constrains sum |beta| <= t (a diamond in two dimensions), L2 constrains sum beta^2 <= t (a circle). The RSS contours are ellipses around the OLS solution; the solution is where the smallest ellipse touches the constraint region.

The L1 diamond has corners lying on the axes. Ellipses commonly first touch the region at a corner, where one coefficient is exactly zero. The L2 circle has no corners, so the contact point generally has all coefficients non-zero.

Calculus view: the L2 penalty gradient is 2*lambda*beta, which shrinks towards zero as beta gets small, so the pull vanishes near zero and the coefficient never reaches it. The L1 penalty gradient is lambda * sign(beta), a constant-magnitude push irrespective of how small beta is, so if the data gradient is smaller than lambda the coefficient is held at exactly zero (soft thresholding).

Bayesian view: L2 corresponds to a Gaussian prior on coefficients, L1 to a Laplace prior which is sharply peaked at zero.

Consequence: Lasso for feature selection, Ridge for stability with correlated predictors.

A411. How do you choose the regularization strength (lambda) and why must features be scaled before regularization?

Choosing lambda: define a grid of values on a log scale (for example 0.001 to 1000), run k-fold cross-validation for each, and pick the value with the best average validation score (or the largest lambda within one standard error of the best for a simpler model, the one-standard-error rule). sklearn provides RidgeCV, LassoCV, ElasticNetCV and LogisticRegressionCV for this. Plot a validation curve to see the U-shape: too small lambda overfits, too large underfits.

Why scale: the penalty is applied to the raw coefficient magnitudes. A feature measured in small units (metres) needs a large coefficient and would be penalised heavily, while the same feature in millimetres needs a tiny coefficient and would be barely penalised. The regularized solution would then depend on arbitrary units. Standardising (mean 0, sd 1) puts all features on equal footing. The scaler must be fit on training folds only, so use a Pipeline. The intercept is not penalised, and one-hot dummies are usually left as they are or standardised consistently.

A412. What are the assumptions of linear regression?

The classical (Gauss-Markov plus normality for inference) assumptions are:

1. Linearity: the expected value of y is a linear function of the coefficients (relationship between predictors and y is linear in parameters).

2. Independence of errors: residuals are not correlated with each other (violated in time series, giving autocorrelation; check Durbin-Watson).

3. Homoscedasticity: residuals have constant variance across fitted values (violation is heteroscedasticity).

4. Normality of residuals: needed for exact small-sample t-tests and confidence intervals; not needed for unbiased coefficients, and matters less with large samples (central limit theorem). Note it is the residuals that should be normal, not y or the predictors.

5. No perfect multicollinearity: predictors are not exact linear combinations of each other; high multicollinearity inflates standard errors.

6. Exogeneity: errors have mean zero and are uncorrelated with predictors (no omitted variable correlated with the predictors).

Diagnostics: residual vs fitted plot (linearity, heteroscedasticity), Q-Q plot (normality), VIF (multicollinearity), Durbin-Watson (independence), Cook's distance (influential points).

A413. Explain how Ordinary Least Squares (OLS) estimates the coefficients, including the normal equation.

OLS chooses the coefficients beta that minimise the residual sum of squares, RSS = sum (y_i - yhat_i)^2 = (y - X beta)'(y - X beta), where X is the design matrix (with a column of ones for the intercept).

Setting the derivative with respect to beta to zero gives the normal equation: X'X beta = X'y, so beta_hat = (X'X)^-1 X'y.

For simple regression this reduces to slope = Cov(x, y) / Var(x) and intercept = mean(y) - slope * mean(x).

Properties: under the Gauss-Markov assumptions OLS is BLUE (best linear unbiased estimator). The fitted line always passes through the point of means. Residuals sum to zero (with an intercept) and are orthogonal to the predictors.

Practical notes: X'X must be invertible, which fails with perfect multicollinearity or when features exceed observations; with near-multicollinearity the inverse is unstable and variances explode, which is why Ridge adds lambda*I to X'X. For very large data, gradient descent or QR/SVD decompositions are used instead of explicit inversion.

A414. What is R-squared and how do you interpret it? What are its limitations?

R2 = 1 - SS_res / SS_tot, the proportion of variance in the target explained by the model relative to a baseline that always predicts the mean. R2 = 0.7 means 70% of the variance in y is explained by the predictors.

Range: 0 to 1 for OLS with an intercept on training data; it can be negative on test data if the model is worse than predicting the mean.

Limitations: (1) it never decreases when you add predictors, even useless ones, so it rewards overfitting (use adjusted R2 or test-set performance); (2) it does not show whether the model is correctly specified, a high R2 can come with a curved residual pattern; (3) it says nothing about bias or whether coefficients are causal; (4) it depends on the variance of y, so it is not comparable across different datasets; (5) a low R2 is not

necessarily bad when the outcome is inherently noisy (human behaviour); (6) it is not in the units of the target, so use RMSE/MAE for practical error size. Always evaluate R2 on held-out data.

A415. What is adjusted R-squared and when does it differ materially from R-squared?

Adjusted R2 = 1 - (1 - R2) * (n - 1) / (n - p - 1), where n is the number of observations and p the number of predictors. It penalises model complexity: adding a predictor increases adjusted R2 only if the improvement in fit is larger than what would be expected by chance. Unlike R2, it can go down (and can be negative).

It differs materially when p is large relative to n or when added variables are useless. Example: n = 50, p = 20, R2 = 0.60 gives adjusted R2 = 1 - 0.4 * 49/29 = 0.32, revealing heavy overfitting that R2 hides.

Use: compare nested or non-nested models with different numbers of predictors on the same target and data; for selection, if adjusted R2 drops after adding a variable, drop it. Limitations: it is still an in-sample measure; for predictive work prefer cross-validated error, and information criteria or t-tests can also guide selection. It cannot be compared across models with different dependent variables (for instance y versus log y).

A416. What is multicollinearity, what problems does it cause and how do you handle it?

Multicollinearity is high correlation among independent variables, so one can be predicted well from the others (for example house size in square feet and number of rooms, or income and spending).

Problems: coefficients become unstable, with large standard errors, so small data changes flip signs or magnitudes; individually insignificant p-values despite a high overall R2 and significant F-test; hard to interpret individual effects. Importantly, overall predictions are usually still fine, and it does not bias the coefficient estimates in the strict sense, it inflates their variance. Perfect collinearity (dummy variable trap) makes OLS impossible.

Detection: correlation matrix/heatmap, VIF (above 5 to 10 is a concern), condition number of X, unstable coefficients across samples.

Remedies: drop or combine redundant variables (ratios, averages), use domain knowledge to keep the more meaningful one, Ridge regression or Elastic Net, PCA/dimension reduction if interpretation is not needed, collect more data, and drop one dummy category (drop_first). If the goal is only prediction, you may leave it.

A417. What is the Variance Inflation Factor (VIF)? How is it computed and what threshold is used?

For each predictor X_j, regress it on all the other predictors and obtain R2_j. Then VIF_j = 1 / (1 - R2_j). It tells by how much the variance of the coefficient of X_j is inflated compared to the case where predictors are uncorrelated. The standard error is inflated by sqrt(VIF).

Interpretation: VIF = 1 means no correlation with other predictors; 1 to 5 moderate and generally acceptable; above 5 (some use 10) indicates problematic collinearity. VIF = 10 means R2_j = 0.9.

Process: compute VIF for all features, remove (or combine) the feature with the highest VIF, recompute, repeat until all are under the threshold. Do not decide only on a threshold: keep variables that are essential to the question.

Caveats: VIF should be computed after adding a constant and, for categorical variables created with one-hot encoding, high VIF between dummy columns and the intercept is expected if you keep all categories; VIF of interaction or polynomial terms is naturally high (centre variables first). VIF only detects linear relations among predictors and ignores the target.

A418. What is heteroscedasticity? How do you detect it and what can you do about it?

Heteroscedasticity means the variance of the residuals is not constant across the range of fitted values or predictors, for example the error in predicting salary or house price grows with the size of the value (funnel shape in the residual plot).

Consequences: OLS coefficients remain unbiased and consistent, but the usual standard errors are wrong, so t-tests, p-values and confidence intervals are unreliable; OLS is no longer the most efficient estimator.

Detection: plot residuals vs fitted values (fan or cone shape), scale-location plot, formal tests: Breusch-Pagan, White test.

Remedies: (1) transform the target, commonly log(y) or sqrt(y), which often stabilises variance for right-skewed money variables; (2) use heteroscedasticity-robust standard errors (HC0-HC3, in statsmodels cov_type='HC3'); (3) weighted least squares when the variance structure is known; (4) add missing predictors or a non-linear term, since heteroscedasticity can signal model misspecification; (5) model the ratio or per-unit variable. If your purpose is pure prediction, it matters less, but prediction intervals will be miscalibrated.

A419. How do you perform residual analysis for a linear regression model and what patterns should you look for?

Residual = actual minus predicted. Plots and checks:

1. Residuals vs fitted values: should look like a random horizontal band around zero. A curve (U-shape) indicates non-linearity or a missing term; a funnel indicates heteroscedasticity.

2. Q-Q plot of (standardised) residuals: points near the 45-degree line indicate normality; heavy tails indicate outliers; skew indicates a need for a transform.

3. Histogram of residuals: roughly symmetric and centred at zero.

4. Residuals vs each predictor and vs omitted candidate variables: patterns suggest missing non-linear terms or variables.

5. Residuals vs order or time: runs or waves suggest autocorrelation; confirm with the Durbin-Watson statistic (about 2 means none).

6. Leverage and influence: Cook's distance, leverage and standardised residuals to spot points that change the fit strongly; investigate rather than automatically delete them.

Actions: add polynomial or interaction terms, transform variables (log), use robust standard errors, handle outliers (verify data errors), add lag terms for time dependence. Also check residuals by segment (region, product) to find where the model is systematically wrong.

A420. How do you interpret the coefficients, p-values and confidence intervals in a multiple linear regression output?

Coefficient: the expected change in y for a one-unit increase in that predictor, holding all other predictors constant. For a dummy variable it is the average difference versus the reference category. For log(y) models, 100*beta is approximately the percentage change in y per unit of x. For log-log models the coefficient is an elasticity.

Intercept: expected y when all predictors are zero; may not be meaningful if zero is outside the data range (centre the variables to make it meaningful).

Standard error: uncertainty of the estimate. t-statistic = coefficient / SE.

p-value: the probability of seeing a t-statistic at least this extreme if the true coefficient were zero. A small p-value (under 0.05) is evidence the variable has a non-zero linear relationship given the other predictors. It is not the probability the null is true, and it does not measure importance or effect size.

95% confidence interval: range that would contain the true coefficient in 95% of repeated samples; if it excludes zero the coefficient is significant at the 5% level.

F-test tests whether at least one predictor is useful. Remember correlation is not causation, and multicollinearity inflates standard errors.

A421. Why can't we use linear regression for a binary classification problem? What does logistic regression do differently?

Linear regression predicts an unbounded number, so for a 0/1 target it can output values below 0 or above 1 that are not valid probabilities. Its errors for a binary target are also not normal and have non-constant variance (heteroscedastic by construction), and the fit is sensitive to extreme x values which drag the line and shift the decision boundary.

Logistic regression models the probability of class 1 as p = 1 / (1 + exp(-(b0 + b1x1 + ... + bpxp))). The sigmoid squeezes the linear score into the range (0, 1). Equivalently, the log-odds ln(p / (1 - p)) is a linear function of the predictors. Coefficients are estimated by maximum likelihood (minimising log loss) rather than least squares. The output is a probability that can be converted to a class using a threshold (default 0.5). The decision boundary is linear in the feature space. Despite the name it is a classification model.

A422. Explain odds, log-odds (logit) and odds ratio. How do they relate to logistic regression coefficients?

Probability p of an event; odds = p / (1 - p). If p = 0.8, odds = 4 (or 4 to 1). If p = 0.5, odds = 1. Odds range from 0 to infinity.

Log-odds (logit) = ln(odds), which ranges over all real numbers, is 0 when p = 0.5, and is symmetric around that point.

Logistic regression assumes log-odds are linear: ln(p / (1 - p)) = b0 + b1x1 + ...

Interpretation: a one-unit increase in x1, holding others fixed, adds b1 to the log-odds, i.e. multiplies the odds by exp(b1), the odds ratio. Example: b1 = 0.69 gives an odds ratio of 2: each additional unit doubles the odds of the event. b1 = -0.35 gives 0.70: odds fall by 30%. An odds ratio of 1 (b = 0) means no effect.

Caution: an odds ratio is not a relative risk. The change in probability from one unit of x depends on the starting probability (non-linear); the effect is largest near p = 0.5. For rare events (p small) odds and probabilities are approximately equal. For a dummy variable, exp(b) is the ratio of odds for that group versus the reference group.

A423. What is the sigmoid function and what are its key properties?

sigmoid(z) = 1 / (1 + e^(-z)). It maps any real number z (the linear score b0 + b'x) to a value in (0, 1), so the output can be read as a probability.

Properties: sigmoid(0) = 0.5; as z tends to +infinity the value tends to 1 and to -infinity tends to 0; S-shaped and monotonic increasing; symmetric: sigmoid(-z) = 1 - sigmoid(z); derivative sigmoid'(z) = sigmoid(z) * (1 - sigmoid(z)), maximum 0.25 at z = 0, which gives the neat gradient for log loss: the gradient of the loss with respect to the weights is X'(p - y)/n.

It is the inverse of the logit function. The decision boundary p = 0.5 corresponds to z = 0, a linear hyperplane in feature space. For multi-class problems the generalisation is softmax.

A424. What is the cost function of logistic regression and why is mean squared error not used?

Logistic regression minimises the negative log-likelihood, also called log loss or binary cross-entropy:

J = -(1/n) * sum [y_i * ln(p_i) + (1 - y_i) * ln(1 - p_i)], where p_i = sigmoid(b'x_i).

If y = 1 the loss is -ln(p): small when p is near 1, huge when p is near 0. If y = 0 the loss is -ln(1-p). So confident wrong predictions are punished severely.

Why not MSE: when MSE is combined with the sigmoid, the resulting cost surface is non-convex with flat regions and local minima, so gradient descent may stall; gradients also vanish when the sigmoid saturates, even when the prediction is badly wrong. Log loss with a sigmoid is convex, giving one global minimum, and it is the proper maximum likelihood objective under a Bernoulli model.

There is no closed form solution; optimisation uses gradient descent, Newton-Raphson or iteratively reweighted least squares (lbfgs in sklearn). With regularization, add lambda * penalty (L2 by default in sklearn, with C = 1/lambda).

A425. What are the assumptions of logistic regression, and what happens with perfect separation?

Assumptions:

1. Binary (or properly coded) outcome and independent observations.

2. Linearity of the log-odds in the continuous predictors (not linearity of probability). Check with binned plots or add splines/polynomials.

3. No severe multicollinearity.

4. No extremely influential outliers.

5. Sufficiently large sample: a rule of thumb is at least 10 events per predictor (events being the rarer class).

Not required: normally distributed predictors, normal residuals or homoscedasticity.

Perfect (complete) separation: some predictor or combination of predictors splits the classes perfectly. The likelihood keeps increasing as coefficients grow, so maximum likelihood estimates diverge to infinity: huge coefficients and standard errors, convergence warnings, p-values meaningless. It often arises from small samples, many dummies, or target leakage.

Fixes: investigate for leakage, remove or combine the offending variable, collect more data, merge sparse categories, or use regularization (L2/L1) or Firth's penalised likelihood, which keep estimates finite.

A426. How should the classification threshold be chosen in logistic regression? Why is 0.5 not always right?

Logistic regression outputs probabilities; the threshold converts them to classes. 0.5 is only optimal when classes are balanced and false positives and false negatives cost the same. In practice neither holds.

How to choose:

1. Business costs: assign a cost to FP and FN. For churn, a wasted discount (FP) is cheaper than a lost customer (FN), so use a lower threshold. Pick the threshold that minimises expected cost, or maximises expected profit. Theoretical optimum threshold = cost_FP / (cost_FP + cost_FN) for calibrated probabilities.

2. Metric based: choose the threshold maximising F1 or F-beta on a validation set, or meeting a constraint (recall at least 90% with the best precision), or Youden's J (TPR - FPR) from the ROC curve.

3. Capacity constraints: if the team can contact only the top 1,000 customers, rank by probability and cut off there.

Lowering the threshold increases recall and lowers precision; raising it does the reverse. Tune the threshold on validation data, not on test data, and report it. Also check calibration, since class_weight or resampling shifts predicted probabilities away from true rates.

A427. Explain the confusion matrix and the metrics derived from it.

For a binary classifier the matrix cross-tabulates actual and predicted classes:

TP: actual positive, predicted positive. FP: actual negative, predicted positive (Type I error). FN: actual positive, predicted negative (Type II error). TN: actual negative, predicted negative.

Metrics:

Accuracy = (TP + TN) / total.

Precision (positive predictive value) = TP / (TP + FP).

Recall (sensitivity, true positive rate) = TP / (TP + FN).

Specificity (true negative rate) = TN / (TN + FP).

False positive rate = FP / (FP + TN) = 1 - specificity.

F1 = 2PR / (P + R).

Example: TP = 80, FP = 20, FN = 40, TN = 860 (total 1000). Accuracy = 94%, precision = 80%, recall = 66.7%, specificity = 97.7%, F1 = 72.7%. Notice the high accuracy hides that a third of positives are missed, which is why accuracy is misleading with imbalanced data. For multi-class, compute per-class and macro, micro or weighted averages.

A428. What is the difference between precision and recall? When do you prioritise one over the other?

Precision answers: of everything I flagged positive, how much was truly positive? (TP / (TP + FP)). Recall answers: of all truly positive cases, how many did I find? (TP / (TP + FN)).

Prioritise recall when missing a positive is expensive: cancer screening, fraud detection, safety defects, churn where retention offers are cheap. A false alarm only costs a follow-up check.

Prioritise precision when false alarms are costly: spam filtering of important emails, approving marketing budget for expensive outreach, recommending legal action, flagging customers for account closure.

They trade off through the threshold: lowering the threshold increases recall and decreases precision. You can have perfect recall by predicting everything positive (precision then equals the base rate), and high precision by predicting positive only when extremely sure (recall then low). When both matter, use F1 or F-beta or look at the precision-recall curve.

A429. What is the F1 score and the F-beta score? Why is the harmonic mean used and what are the limitations?

F1 = 2 * P * R / (P + R), the harmonic mean of precision and recall. F-beta = (1 + beta^2) * P * R / (beta^2 * P + R); beta > 1 weights recall more (F2 for screening), beta < 1 weights precision more (F0.5).

The harmonic mean is dominated by the smaller value, so a model cannot get a high score by excelling at one and failing the other. With precision 1.0 and recall 0.1, the arithmetic mean is 0.55 but F1 is 0.18.

Limitations: ignores true negatives, so it says nothing about performance on the negative class; depends on the threshold; treats precision and recall as equally important unless beta is changed; for multi-class you must choose macro (each class equal), micro (global counts) or weighted averaging; it is not a good metric when you need calibrated probabilities. For comparing models independent of the threshold use PR-AUC or ROC-AUC. Matthews correlation coefficient is an alternative that uses all four cells of the confusion matrix.

A430. Explain the ROC curve and AUC. How do you interpret an AUC of 0.5, 0.8 and 1.0?

The ROC curve plots true positive rate (recall) on the y-axis against false positive rate on the x-axis for every possible threshold. Lowering the threshold moves you up and to the right. AUC is the area under that curve.

Interpretation: AUC is the probability that the model assigns a higher score to a randomly chosen positive than to a randomly chosen negative. 0.5 means no discrimination (random guessing, the diagonal); 0.8 is good discrimination, a random positive outranks a random negative 80% of the time; 1.0 is perfect ranking; below 0.5 means the ranking is inverted (flip the predictions).

Strengths: threshold independent; insensitive to class prior; measures ranking quality. Weaknesses: does not reflect calibration (a model can have great AUC and badly miscalibrated probabilities); can look optimistic for very imbalanced data because FPR is divided by a large number of negatives; does not tell you the operating threshold or business cost; two models with the same AUC can behave very differently in the region you care about (use partial AUC or precision at k).

A431. When would you use a precision-recall curve (PR-AUC) rather than ROC-AUC?

Prefer the PR curve when the positive class is rare and you care mainly about the positives (fraud, rare disease, defaults at 1-2%).

Reason: ROC uses FPR = FP / (FP + TN). With 1,000,000 negatives, even 10,000 false positives give an FPR of only 1%, so the ROC curve and AUC still look excellent, while precision would be terrible (maybe under 10% when only 1,000 positives exist). The PR curve uses precision = TP / (TP + FP), which directly reflects the burden of false alarms and is sensitive to the class imbalance.

Properties: the baseline of a PR curve is the positive prevalence (a random classifier has precision equal to prevalence), not a fixed 0.5, so PR-AUC values are not comparable across datasets with different prevalence. Average precision is the usual summary (sklearn average_precision_score).

When balanced or when both classes matter equally, ROC-AUC is fine and more stable. Always report the metric along with prevalence, and select the operating point on the PR curve by the recall needed or the precision budget (for example precision at least 30%).

A432. Compare MAE, MSE and RMSE. When would you use each?

MAE = mean(|y - yhat|): average absolute error in the target's units; robust to outliers; easy to explain ('we are off by 12,000 dollars on average'); the optimum prediction is the median. Its gradient is constant, not smooth at zero.

MSE = mean((y - yhat)^2): squares errors so large errors are penalised heavily; differentiable and convex, the OLS objective; units are squared (hard to interpret); the optimum prediction is the mean. Sensitive to outliers.

RMSE = sqrt(MSE): back in the target units, still penalises large errors more; always >= MAE (equal only if all errors are the same size). A big gap between RMSE and MAE indicates a few large errors.

Choice: use MAE when all errors matter proportionally and the data has outliers or heavy tails (delivery time); use RMSE when large errors are especially costly (energy demand, house price valuations) and as the default for comparison; use MSE for optimisation. Also consider MAPE or RMSLE for relative error (watch out for zeros in MAPE) and R2 for explained variance. Whatever you choose, compare to a baseline such as predicting the mean.

A433. What is the difference between normalization (min-max scaling) and standardization? When is scaling needed?

Min-max scaling: x' = (x - min) / (max - min), maps to [0, 1]. Preserves the shape of the distribution but is sensitive to outliers (one extreme value compresses everything else).

Standardization (z-score): x' = (x - mean) / sd, giving mean 0 and standard deviation 1. Not bounded; less affected by outliers than min-max, and assumed by many regularized methods. RobustScaler uses median and IQR for heavy outliers.

Scaling is needed for: distance-based methods (KNN, k-means), gradient descent based methods (faster, more stable convergence), regularized models (Lasso, Ridge, penalised logistic regression), and when comparing coefficient magnitudes. It is not needed for plain OLS predictions or tree-based models, although it does not hurt.

Rules: fit the scaler on the training set only and apply the same parameters to validation and test data; do not scale the target (unless needed) and do not scale binary dummies unnecessarily. Log-transform skewed variables before scaling if needed.

A434. How do you encode categorical variables for linear and logistic regression? Compare the main approaches.

1. One-hot (dummy) encoding: one binary column per category. Standard for nominal variables with few levels. Drop one category (drop_first) to avoid the dummy variable trap (perfect multicollinearity with the intercept) in unregularized models; the dropped category is the reference. Problem: high cardinality blows up the dimensionality.

2. Ordinal/label encoding: map categories to integers. Only appropriate for truly ordered categories (low, medium, high) and even then assumes equal spacing. Using it on nominal categories imposes a false order and misleads linear models.

3. Target (mean) encoding: replace category with the average target in that category. Good for high cardinality, but leaks the target unless computed out-of-fold with smoothing toward the global mean.

4. Frequency/count encoding: replace with category frequency; simple, no leakage of target but can collide categories.

5. Grouping rare levels into 'Other', or hashing for very high cardinality.

6. Binary encoding for medium cardinality.

Practical notes: fit the encoder on training data and handle unseen categories (handle_unknown='ignore'); with Lasso/Ridge you may keep all levels; document the reference category for interpretation.

A435. What types of missing data exist and how do you handle missing values?

Mechanisms: MCAR (missing completely at random, unrelated to anything), MAR (missingness depends on other observed variables, such as older customers skipping an income field), MNAR (missingness depends on the unobserved value itself, such as high earners hiding income).

First understand why values are missing and how much (percentage per column and row).

Options:

1. Drop rows (listwise deletion): acceptable if few rows and MCAR; otherwise loses data and can bias results.

2. Drop columns with a very high share missing and little value.

3. Simple imputation: mean (symmetric numeric), median (skewed numeric or outliers), mode or a 'Missing' category (categoricals).

4. Add a missing indicator flag column, since missingness itself can be predictive (MNAR).

5. Model-based: KNN imputer, regression imputation, iterative imputer (MICE), multiple imputation which preserves uncertainty.

6. Domain rules, forward-fill for time series.

Pitfalls: computing imputation values on the full data (leakage; use a Pipeline with SimpleImputer fit on train only); mean imputation shrinks variance and weakens correlations; never impute the target for evaluation; missing values must be handled before scikit-learn linear models, which do not accept NaN.

A436. What problems does class imbalance create and what are the basic techniques to deal with it?

Problem: when one class is rare (1% fraud), a model can reach 99% accuracy by predicting the majority class always. The learning algorithm sees few minority examples, so it optimises mainly for the majority class and recall for the minority is poor.

Techniques:

1. Use the right metrics: precision, recall, F1, PR-AUC, ROC-AUC, confusion matrix, not accuracy.

2. Stratified splits and stratified CV so every fold has minority examples.

3. Class weights (class_weight='balanced') which increase the loss for minority errors. Simple and no change to data.

4. Resampling the training set only: random oversampling of the minority, undersampling of the majority, or SMOTE (synthesises new minority points by interpolating neighbours). Never resample before the split or the test/validation data, otherwise duplicates leak and metrics are inflated.

5. Threshold tuning on predicted probabilities, using the cost trade-off.

6. Collect more minority data, or reframe as anomaly detection when the minority is extremely rare.

Note that resampling and class weights distort predicted probabilities, so recalibrate if you need true probabilities.

A437. How does the K-Nearest Neighbours (KNN) algorithm work? What are its strengths, weaknesses and how do you choose k?

KNN is a non-parametric, instance-based method with no real training step. To predict for a new point: compute its distance (usually Euclidean) to all training points, find the k closest, and then take the majority class (classification) or average value (regression) of those neighbours; distance weighting is optional.

Choosing k: small k (1) gives a flexible, noisy boundary: low bias, high variance (overfit). Large k smooths the boundary: high bias, low variance (underfit). Choose k by cross-validation, often odd numbers for binary problems to avoid ties, with a rule of thumb around sqrt(n).

Strengths: simple, no assumptions about the data shape, captures non-linear boundaries, works for classification and regression.

Weaknesses: prediction is slow and memory-heavy since all data is stored; features must be scaled (otherwise large-range features dominate the distance); suffers from the curse of dimensionality (distances become uninformative with many features); sensitive to irrelevant features and class imbalance; no interpretable coefficients. Categorical features need suitable distance or encoding.

A438. Explain the Naive Bayes classifier. Why is it called naive and what are its pros and cons?

Naive Bayes applies Bayes theorem: P(class | features) is proportional to P(class) * P(x1 | class) * P(x2 | class) * ... * P(xn | class). It predicts the class with the highest posterior.

It is 'naive' because it assumes features are conditionally independent given the class, which greatly simplifies estimating the likelihood (each feature is modelled separately) even though the assumption is usually false.

Variants: Gaussian (continuous features assumed normal within each class), Multinomial (word counts, text), Bernoulli (binary presence features).

Pros: very fast to train and predict, works with small data and many features (text classification, spam filtering), handles multi-class naturally, a strong simple baseline. Laplace (additive) smoothing handles zero counts: without it, a word never seen in a class would zero out the entire product.

Cons: the independence assumption hurts when features are correlated; probability estimates are poorly calibrated (pushed to 0 or 1), although the ranking of classes is often still correct; Gaussian NB assumes normality; continuous features may need binning. It is a generative model, whereas logistic regression is discriminative.

A439. Explain gradient descent. What are the differences between batch, stochastic and mini-batch gradient descent, and how does the learning rate...

Gradient descent minimises a loss function J(theta) iteratively: theta := theta - eta * dJ/dtheta, where eta is the learning rate. The gradient points in the direction of steepest increase, so we step opposite to it. For linear regression with MSE the gradient is (2/n) X'(X theta - y); for logistic regression it is (1/n) X'(p - y).

Batch: uses all n rows for every update: stable, smooth convergence, but slow for large data.

Stochastic (SGD): one random row per update: very fast updates and can escape shallow local minima, but noisy path that oscillates around the minimum.

Mini-batch: a small batch (32 to 512) per update; the practical compromise, vectorised and fairly stable.

Learning rate: too small means very slow convergence; too large causes overshooting, oscillation or divergence (loss increases). Use learning-rate schedules or decay, and monitor the loss curve. Scale features first so the loss contour is round, otherwise convergence zigzags. Convex losses (linear, logistic regression) have a single global minimum so gradient descent converges to it; non-convex losses (neural networks) may find local minima. Stop when the change in loss is tiny or a maximum number of epochs is reached.

A440. What is feature engineering? Give common techniques for tabular data and explain how you decide which features to keep.

Feature engineering is creating or transforming input variables so that the model can learn the signal more easily; often it improves performance more than changing algorithm.

Techniques:

1. Transformations: log or square root for right-skewed variables (income, price), Box-Cox, polynomial terms for curvature.

2. Interactions and ratios: debt-to-income, price per square foot, utilisation = balance / limit.

3. Date/time features: day of week, month, tenure, recency, days since last purchase, holiday flag.

4. Aggregations: customer-level counts, sums, averages, trends over the last 30/90 days (RFM).

5. Binning: age groups, when relationship is non-linear (loses information if overdone).

6. Encoding categoricals, handling rare levels, missing indicators.

7. Text/geo basics: length, keyword flags, distance to city centre.

Selecting features: domain relevance and availability at prediction time (avoid leakage); remove near-constant and highly collinear features (VIF, correlation); univariate statistics (correlation, chi-square, mutual information); L1 regularization; stepwise or recursive feature elimination; check stability and cross-validated performance. Prefer parsimonious, explainable features, and document each.

A441. Write Python code using scikit-learn to fit a linear regression model with a train/test split, scale the features in a leak-free way, and re...

Use a Pipeline so the scaler is fitted on the training data only.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = pd.read_csv("houses.csv")
X = df.drop(columns=["price"])
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = Pipeline([
    ("scale", StandardScaler()),
    ("lr", LinearRegression())
])
model.fit(X_train, y_train)

pred = model.predict(X_test)
print("MAE :", mean_absolute_error(y_test, pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, pred)))
print("R2  :", r2_score(y_test, pred))
```

```
# compare to a baseline that predicts the training mean
baseline = np.full_like(y_test, y_train.mean(), dtype=float)
print("Baseline RMSE:", np.sqrt(mean_squared_error(y_test, baseline)))
```

Explanation: compare train and test scores to check overfitting, and always compare against the mean baseline. This assumes numeric features without missing values; otherwise add an imputer and encoder as in a ColumnTransformer.

A442. Fit an OLS regression with statsmodels and explain how to read the key parts of the summary output.

```
import pandas as pd
import statsmodels.api as sm

df = pd.read_csv("houses.csv")
X = df[["sqft", "bedrooms", "age"]]
X = sm.add_constant(X)          # statsmodels does not add an intercept by default
y = df["price"]

ols = sm.OLS(y, X).fit()
print(ols.summary())

# formula API alternative
import statsmodels.formula.api as smf
ols2 = smf.ols("price ~ sqft + bedrooms + age + C(city)", data=df).fit(cov_type="HC3")
```

How to read the summary:

1. R-squared and Adj. R-squared: share of variance explained, penalised for the number of predictors.

2. F-statistic and Prob (F-statistic): tests whether all slopes are jointly zero.

3. coef: estimated effect per unit change, holding other variables constant. std err: uncertainty. t and P>|t|: significance test for each coefficient. [0.025, 0.975]: 95% confidence interval.

4. Omnibus/Jarque-Bera and Skew/Kurtosis: tests of residual normality. Durbin-Watson: autocorrelation of residuals (about 2 is good). Cond. No.: a very large value warns of multicollinearity.

5. Notes at bottom warn about strong multicollinearity or a misspecified covariance.

cov_type='HC3' in the second fit gives heteroscedasticity-robust standard errors. Remember that significance is not the same as predictive performance; evaluate on held-out data.

A443. Write code to compute the VIF for every feature of a dataframe and iteratively remove features with VIF above 5.

```
import pandas as pd
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

def compute_vif(X: pd.DataFrame) -> pd.DataFrame:
    Xc = sm.add_constant(X)  # VIF is computed with an intercept in the model
    vif = pd.DataFrame({
        "feature": Xc.columns,
```

```
        "VIF": [variance_inflation_factor(Xc.values, i)
                for i in range(Xc.shape[1])]
    })
    return vif[vif.feature != "const"].sort_values("VIF", ascending=False)

def drop_high_vif(X: pd.DataFrame, threshold: float = 5.0) -> pd.DataFrame:
    X = X.copy()
    while True:
        vif = compute_vif(X)
        worst = vif.iloc[0]
        if worst.VIF <= threshold or X.shape[1] == 1:
            break
        print(f"Dropping {worst.feature} (VIF={worst.VIF:.1f})")
        X = X.drop(columns=worst.feature)
    return X

X_numeric = df.select_dtypes("number").drop(columns=["price"]).dropna()
X_reduced = drop_high_vif(X_numeric)
print(compute_vif(X_reduced))
```

Notes: drop one feature at a time because removing one changes all other VIFs; use domain knowledge when choosing between correlated features instead of blindly dropping the highest; with one-hot columns drop one category first; do this on training data only.

A444. Write code to produce residual diagnostics for a fitted statsmodels OLS model: residual vs fitted plot, Q-Q plot, and the Breusch-Pagan test...

```
import matplotlib.pyplot as plt
import statsmodels.api as sm
from statsmodels.stats.diagnostic import het_breuschpagan
from statsmodels.stats.stattools import durbin_watson

X = sm.add_constant(df[["sqft", "bedrooms", "age"]])
y = df["price"]
model = sm.OLS(y, X).fit()

fitted = model.fittedvalues
resid = model.resid

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].scatter(fitted, resid, alpha=0.4)
ax[0].axhline(0, color="red")
ax[0].set_xlabel("Fitted values"); ax[0].set_ylabel("Residuals")
ax[0].set_title("Residuals vs fitted")
sm.qqplot(resid, line="45", fit=True, ax=ax[1])
plt.show()

lm_stat, lm_p, f_stat, f_p = het_breuschpagan(resid, X)
print("Breusch-Pagan p-value:", lm_p)   # p < 0.05 suggests heteroscedasticity
print("Durbin-Watson:", durbin_watson(resid))

# if heteroscedastic: log-transform target or use robust errors
```

```
model_log = sm.OLS(np.log(y), X).fit()
model_rob = sm.OLS(y, X).fit(cov_type="HC3")
```

Interpretation: a funnel in the first plot or a small Breusch-Pagan p-value means non-constant variance; curvature means a missing non-linear term; points off the line in the Q-Q plot indicate non-normal residuals or outliers.

A445. Write code for a logistic regression classifier on an imbalanced dataset using a stratified split, class weights and a classification report...

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, average_precision_score)

X = df.drop(columns=["churn"])
y = df["churn"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

clf = Pipeline([
    ("scale", StandardScaler()),
    ("lr", LogisticRegression(class_weight="balanced", max_iter=1000))
])
clf.fit(X_train, y_train)

proba = clf.predict_proba(X_test)[:, 1]
pred = (proba >= 0.5).astype(int)

print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, digits=3))
print("ROC-AUC:", roc_auc_score(y_test, proba))
print("PR-AUC :", average_precision_score(y_test, proba))
```

Points to mention: stratify=y keeps the class ratio in train and test; class_weight='balanced' penalises minority errors more, which raises recall but inflates predicted probabilities (calibrate if needed); evaluate with ROC-AUC and PR-AUC computed from probabilities, not from hard labels; the 0.5 threshold can be tuned afterwards on validation data.

A446. Write code that evaluates a model with stratified 5-fold cross-validation while avoiding leakage from preprocessing, and reports mean and st...

```
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

```
pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
    ("clf", LogisticRegression(max_iter=1000, class_weight="balanced"))
])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scores = cross_validate(
    pipe, X, y, cv=cv,
    scoring=["roc_auc", "average_precision", "recall", "precision", "f1"],
    return_train_score=True
)

for k, v in scores.items():
    if k.startswith(("test_", "train_")):
        print(f"{k:28s} {v.mean():.3f} +/- {v.std():.3f}")
```

Why it avoids leakage: the imputer and scaler are inside the Pipeline, so in each fold they are fitted only on that fold's training part and then applied to its validation part. Comparing train_ and test_ scores reveals overfitting. For time series replace the splitter with TimeSeriesSplit; for repeated customers use GroupKFold. Keep an untouched test set for the final check after tuning.

A447. Show how to tune Ridge and Lasso regression with cross-validation in scikit-learn and inspect which features Lasso selects.

```
import numpy as np
import pandas as pd
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV, LassoCV
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

alphas = np.logspace(-3, 3, 50)

ridge = make_pipeline(StandardScaler(), RidgeCV(alphas=alphas, cv=5))
lasso = make_pipeline(StandardScaler(), LassoCV(alphas=alphas, cv=5, max_iter=10000))

for name, m in [("ridge", ridge), ("lasso", lasso)]:
    m.fit(X_train, y_train)
    rmse = np.sqrt(mean_squared_error(y_test, m.predict(X_test)))
    print(name, "best alpha:", m[-1].alpha_, "test RMSE:", rmse)

coefs = pd.Series(lasso[-1].coef_, index=X.columns)
print("Selected features:\n", coefs[coefs != 0].sort_values())
print("Dropped features:", list(coefs[coefs == 0].index))
```

Notes: scaling is essential before penalised models; alpha is lambda; the standardised coefficients are comparable in magnitude; coefficients shrunk to exactly zero by Lasso are removed from the model. Use the one-standard-error rule if you want a sparser model.

A448. Write code to plot the ROC and precision-recall curves for a classifier and select a threshold that achieves at least 80% recall with the hi...

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import (roc_curve, precision_recall_curve,
                             roc_auc_score, average_precision_score)

# proba_val: predicted probabilities for class 1 on a VALIDATION set
fpr, tpr, _ = roc_curve(y_val, proba_val)
prec, rec, thr = precision_recall_curve(y_val, proba_val)

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].plot(fpr, tpr, label=f"AUC={roc_auc_score(y_val, proba_val):.3f}")
ax[0].plot([0, 1], [0, 1], "--", color="grey")
ax[0].set_xlabel("False positive rate"); ax[0].set_ylabel("True positive rate")
ax[0].legend()
ax[1].plot(rec, prec, label=f"AP={average_precision_score(y_val, proba_val):.3f}")
ax[1].axhline(y_val.mean(), ls="--", color="grey")   # baseline = prevalence
ax[1].set_xlabel("Recall"); ax[1].set_ylabel("Precision")
ax[1].legend()
plt.show()

# precision_recall_curve returns len(thr)+1 points for prec/rec
mask = rec[:-1] >= 0.80
best = np.argmax(np.where(mask, prec[:-1], -1))
chosen = thr[best]
print(f"threshold={chosen:.3f}  precision={prec[best]:.3f}  recall={rec[best]:.3f}")

pred_test = (proba_test >= chosen).astype(int)  # apply the validation-chosen threshold to
test
```

Key point: choose the threshold on validation data and only apply it to the test set to report honest performance.

A449. Implement linear regression using batch gradient descent from scratch with NumPy and explain the key steps.

```
import numpy as np

def gradient_descent(X, y, lr=0.05, epochs=2000, tol=1e-8):
    # standardise features first for stable convergence
    mu, sd = X.mean(axis=0), X.std(axis=0)
    Xs = (X - mu) / sd
    n, p = Xs.shape
    Xb = np.hstack([np.ones((n, 1)), Xs])      # add intercept column
    theta = np.zeros(p + 1)
    losses = []
```

```
    for epoch in range(epochs):
        pred = Xb @ theta
        error = pred - y
        loss = (error ** 2).mean()
        grad = (2 / n) * Xb.T @ error          # dJ/dtheta for MSE
        theta -= lr * grad
        losses.append(loss)
        if epoch > 0 and abs(losses[-2] - losses[-1]) < tol:
            break
    return theta, mu, sd, losses

def predict(X, theta, mu, sd):
    Xs = (X - mu) / sd
    return np.hstack([np.ones((len(Xs), 1)), Xs]) @ theta

# check against the closed form
# theta_ols = np.linalg.lstsq(Xb, y, rcond=None)[0]
```

Steps: initialise theta, compute predictions, compute the error and MSE, compute the gradient (2/n) X'(X theta - y), update theta in the opposite direction scaled by the learning rate, repeat until the loss stops decreasing. Plot the loss against epochs: it should decrease smoothly; if it rises or oscillates lower the learning rate. For logistic regression only the prediction (sigmoid of the linear score) and the gradient X'(p - y)/n change.

A450. Write a scikit-learn pipeline using ColumnTransformer that imputes, scales numeric columns, one-hot encodes categorical columns, fits a logi...

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, StratifiedKFold, train_test_split

num_cols = ["age", "income", "tenure"]
cat_cols = ["region", "plan"]

preprocess = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                      ("sc", StandardScaler())]), num_cols),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                      ("oh", OneHotEncoder(handle_unknown="ignore"))]), cat_cols),
])

pipe = Pipeline([("prep", preprocess),
                 ("clf", LogisticRegression(max_iter=2000))])

param_grid = {
    "clf__C": [0.001, 0.01, 0.1, 1, 10, 100],
    "clf__penalty": ["l2"],
}
cv = StratifiedKFold(5, shuffle=True, random_state=1)
grid = GridSearchCV(pipe, param_grid, scoring="roc_auc", cv=cv, n_jobs=-1)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, test_size=0.2,
random_state=1)
grid.fit(X_train, y_train)
print(grid.best_params_, grid.best_score_)
print("Test AUC:", grid.score(X_test, y_test))
```

Notes: C is the inverse of the regularization strength (smaller C means stronger regularization); handle_unknown='ignore' prevents errors for unseen categories; all preprocessing is refit inside every fold so there is no leakage. Use solver='liblinear' or 'saga' with penalty='l1' for Lasso-style logistic regression.

A451. A retail bank wants to predict which credit card customers will churn (close their account) in the next 90 days so the retention team can ca...

1. Frame: binary classification, unit = customer at a snapshot date, target = closes account within 90 days after the snapshot. Features come only from data before the snapshot.

2. Data: demographics, tenure, products held, 3/6/12-month transaction counts and spend, trends (spend change vs previous quarter), complaints, balance, payment behaviour, app logins. Create ratios and recency features.

3. Split: use a time-based split (train on earlier snapshots, test on a later one) so it mirrors deployment; use stratified CV inside training. Keep multiple rows per customer in the same fold.

4. Preprocess: impute with median plus missing flags, one-hot encode, standardise, log-transform skewed amounts, in a Pipeline.

5. Baseline and model: logistic regression (class_weight balanced or plain) for interpretability, tune C by CV; check VIF for collinear transaction features.

6. Metrics: not accuracy (92% by predicting no churn). Use ROC-AUC and PR-AUC (baseline 0.08), recall and precision at the operating point, and lift/precision in the top decile. Choose the threshold from retention economics: call cost and discount cost versus lifetime value saved; or from capacity (call top 5,000).

7. Pitfalls: target leakage (account-closure flags, 'cancellation requested' fields, activity right before closure), resampling before splitting, ignoring that already-doomed customers cannot be saved (consider an uplift view later), calibration after class weighting, drift.

8. Communicate: odds ratios for top drivers, expected savings, and monitor monthly with retraining.

A452. A lender is building a loan default model (3% default rate). Regulators require the model to be explainable, and the business needs predicte...

1. Model choice: logistic regression (possibly with L2 or L1 regularization and binned or weight-of-evidence features) is standard in credit risk because coefficients and odds ratios are explainable, probabilities are naturally produced and reasoning is auditable.

2. Features: income, debt-to-income, utilisation, loan-to-value, delinquencies, credit history length, employment tenure; transformations such as log income, binned variables for non-linearity; handle missing values with indicators; check multicollinearity via VIF and the sign of each coefficient against business logic (higher DTI should increase risk).

3. Data and leakage: use only application-time information; exclude post-origination fields (collection status, payment history after origination). Define the default window (for example default within 12 months of

origination) and use out-of-time validation, because economic conditions change. Beware reject inference: you only observe outcomes for approved applicants.

4. Imbalance: with 3% defaults prefer stratified splits and no resampling if probabilities are needed; if class weights or SMOTE are used, recalibrate probabilities.

5. Metrics: ROC-AUC/Gini and KS for rank-ordering power, PR-AUC, and critically calibration (calibration curve, Brier score, predicted vs observed default by decile) because PD feeds pricing and provisioning. Choose the approval cutoff by expected profit: interest earned vs loss given default.

6. Validate on out-of-time and out-of-sample data, test stability (PSI) and monitor drift. Pitfalls: fairness and protected attributes, proxies for them, overfitting from too many variables, extrapolating to new segments.

A453. A real estate company wants a house price prediction model from 20,000 past sales (features: size, bedrooms, location, age, condition, sale ...

1. Explore: distribution of price (skewed, with a long tail), missing values, impossible values (0 bedrooms, size of 10 square feet), duplicates, and relationships by scatter plots.

2. Target transform: model log(price). This makes residuals more symmetric, stabilises variance (percentage errors rather than dollar errors) and prevents expensive homes dominating. Coefficients then mean approximately percentage changes. Remember to back-transform predictions (and note exp of the mean of log is a slightly biased estimate of the mean; use a smearing correction if the mean price matters).

3. Features: log of size, age or age squared (non-linear), price per area of the neighbourhood (computed from training only), bathroom/bedroom ratio, month and year of sale to capture the market trend, encode location (neighbourhood dummies with rare grouping or target encoding out-of-fold).

4. Split: random or, better, chronological (train on older sales, test on the latest months) since the market trends.

5. Fit OLS or Ridge/Lasso after standardising; check VIF (size, bedrooms and bathrooms correlate).

6. Evaluate: RMSE and MAE in dollars on the test set, MAPE or median absolute percentage error, R2 and adjusted R2; compare to a baseline (neighbourhood median price). Inspect residuals vs fitted values (heteroscedasticity, curvature) and by neighbourhood and price band.

7. Pitfalls: outliers and data errors, extrapolation outside the training range, leakage via 'list price' or 'assessed value', omitted-variable bias, stale data, and presenting causal claims from coefficients.

A454. A hospital wants to predict which patients will be readmitted within 30 days of discharge so care managers can arrange follow-up. About 15% ...

1. Framing: binary classification at the time of discharge. Each row is an index admission; target = unplanned readmission within 30 days. Exclude patients who died in hospital or were transferred (they cannot be readmitted) and planned readmissions.

2. Features available at discharge: age, diagnoses (grouped codes), comorbidity count, length of stay, number of admissions and ED visits in the last 6/12 months, medications count, lab abnormalities at discharge, discharge destination, insurance.

3. Leakage: anything recorded after discharge; fields that are only filled in for readmitted patients; billing codes finalised later.

4. Split: patient-level grouping (GroupKFold), because the same patient appears multiple times; plus a temporal holdout (latest months).

5. Model: regularized logistic regression, as clinicians want interpretable odds ratios; tune C via CV; check calibration, as risk scores guide care allocation.

6. Metrics: class imbalance, so PR-AUC and ROC-AUC; recall at the care managers' capacity (precision at top 10 percent); a cost-based threshold: the intervention is cheap compared with readmission cost, so favour recall while keeping the alert volume manageable. Report calibration plots.

7. Pitfalls: fairness across age, race or insurance groups, underdiagnosed populations, missing data that is MNAR (tests ordered only for sicker patients: use missing indicators), changes in coding practices over time, and operational feedback (a successful intervention lowers observed readmissions and distorts future training labels).

A455. A payments company has 1 million card transactions per day with 0.1% fraud. A data scientist reports 99.9% accuracy and ROC-AUC 0.97 for a l...

1. Accuracy of 99.9% is meaningless: predicting 'not fraud' for every transaction achieves it. Discard accuracy.

2. ROC-AUC of 0.97 can also mislead. With 1,000,000 negatives, an FPR of 1% means 10,000 false alarms per day against only about 1,000 fraud cases, so precision could be under 10%. Examine the PR curve, average precision (random baseline = 0.001), and precision at the recall you need.

3. Define the cost: missed fraud costs the transaction amount plus chargeback fees (FN); false alarms cost customer friction and review time (FP). Use a cost-weighted metric (dollar loss saved) and select the threshold by expected cost or by the analysts' review capacity (top 500 alerts per day).

4. Data handling: time-based split (train on earlier days, test on later), stratified CV within training; avoid random splitting that places the same card in both sets; avoid leakage features such as chargeback flags.

5. Techniques: class_weight or undersampling of the legitimate class on training data only (SMOTE has limited value for tabular fraud features), threshold tuning, features such as velocity (transactions in the last hour), amount deviation from the customer average, new merchant, location mismatch.

6. After class weighting, recalibrate probabilities if they are used downstream.

7. Monitor: fraud patterns drift quickly as fraudsters adapt; track precision, recall and alert volumes weekly and retrain often.

A456. An email service wants a quick spam filter using the text of emails. A colleague proposes Naive Bayes while another proposes logistic regres...

1. Representation: tokenise, lowercase, remove stop words (optional), build bag-of-words or TF-IDF features (CountVectorizer or TfidfVectorizer), possibly with bigrams. Fit the vectoriser on training data only.

2. Naive Bayes (Multinomial): extremely fast, works well with small data and thousands of sparse features, strong baseline for spam. Use Laplace smoothing so unseen words do not zero the probability. Weakness: the independence assumption and poorly calibrated probabilities.

3. Logistic regression with L2 (or L1 for feature selection): usually slightly more accurate with enough data since it does not assume independence, gives interpretable word weights, and produces better calibrated probabilities. Needs more tuning (C).

4. Recommendation: start with Naive Bayes as a baseline within hours, compare to regularized logistic regression using cross-validation; choose based on validation performance, latency and need for calibrated scores.

5. Metrics: spam is a minority class but the costs are asymmetric: blocking a legitimate email (false positive) is worse than letting a spam through. Prioritise precision on the spam class (or equivalently very low false positive rate), then maximise recall subject to it; examine the PR curve and set a high threshold.

6. Pitfalls: concept drift as spammers change words, duplicates between train and test, leaking sender information that identifies the dataset, and adversarial behaviour. Retrain regularly and monitor.

A457. A telecom company wants to predict subscriber churn from raw tables of customers, monthly bills, call records and support tickets. You are a...

1. Define snapshot date and a prediction window; compute features only from records before the snapshot.

2. Tenure features: months since activation, contract type and remaining contract months, days to contract end.

3. Usage features: average monthly minutes, data usage, SMS counts over 1, 3 and 6 months; trend features such as (last 3 months average) / (previous 3 months average) to detect declining usage; volatility (standard deviation).

4. Billing features: average bill, bill shock (bill / typical bill), late payments, discount expiry flag, plan price versus usage.

5. Service/support features: tickets in last 30/90 days, unresolved tickets, dropped call rate, network quality in the home area, days since last ticket.

6. Customer features: age bracket, region, device age, number of lines in the account, whether bundled products exist.

7. Transformations: log transform skewed usage, missing indicators for no-usage months, one-hot encode plan with rare levels grouped.

8. Checks: leakage (no data after snapshot; no 'port-out request' field), VIF and correlation for collinear aggregates (1, 3 and 6 month versions are highly correlated), univariate churn rate by bin (are relationships sensible?), L1 regularization or cross-validated feature importance to prune, stability of the feature distribution between train and test periods. Document definitions so they can be reproduced in production.

A458. An insurer wants to predict the cost of each claim. The claim amount is heavily right-skewed with a few very large claims; a linear regressi...

1. Inspect: histogram of claim amounts, residual vs fitted plot (funnel shape and large positive residuals), Q-Q plot (heavy right tail), Cook's distance for influential claims.

2. Verify the large claims are real (not data entry errors). Genuine large claims are exactly what the business cares about, so do not simply delete them.

3. Transform the target: log(claim) or sqrt to reduce skew and heteroscedasticity. Interpret coefficients as percentage effects; back-transform predictions with a smearing adjustment because the exp of the mean log underestimates the mean amount.

4. Transform skewed predictors (vehicle value, income) and consider non-linear terms or interactions (age x vehicle type).

5. Alternative model families within the regression toolkit: Gamma or log-link generalised linear models, or model frequency and severity separately; robust or quantile regression for median claims.

6. Evaluate with MAE and RMSE on a held-out set and compare with the baseline (mean claim); note that RMSE is dominated by a few huge claims. Check performance by claim size band and aggregate bias (sum of predicted versus sum of actual claims), since insurers care about total reserve.

7. Low R2 may just reflect inherently unpredictable claims; set expectations accordingly. Use heteroscedasticity-robust standard errors if inference is needed.

A459. A marketing team will email a discount offer to customers, and each email costs 0.10 dollars. A purchase from a responder yields 20 dollars ...

1. Economics: expected profit of emailing a customer with predicted response probability $p = 20p - 0.10$. It is positive when $p > 0.005$, so the breakeven threshold is 0.5%, far below 0.5. A threshold of 0.5 would target almost nobody.

2. This requires calibrated probabilities, so check the calibration curve and recalibrate if class weighting or sampling was used.

3. Ranking quality metrics: ROC-AUC or PR-AUC, and especially gains and lift by decile. Sort customers by predicted probability, split into 10 groups and show the response rate and cumulative share of responders captured (top 3 deciles capture 60% of responders, lift 2.0 versus random).

4. Choose cutoff by maximising cumulative expected profit on a validation set, or by budget (email top 40%).

5. Validate on a later campaign (out-of-time) and compare to the existing rule or random targeting through a holdout control group in production.

6. Pitfalls: a response model predicts who will buy, not who will buy because of the email (sure things and lost causes get targeted); consider uplift modelling and a randomised test later. Also watch for leakage (post-campaign activity), fatigue effects and selection bias if past campaigns targeted only certain customers.

A460. A health-tech startup builds a screening model for a serious disease with 1% prevalence. The logistic regression model has 98% accuracy, 40%...

1. Diagnose the metrics: with 1% prevalence, 98% accuracy is only 2 points better than doing nothing; recall of 40% means 60% of sick patients are missed, which is unacceptable for screening, where missing a case (FN) is far more costly than a follow-up test (FP).

2. Reframe the objective: for screening, aim for high recall (sensitivity), say 90%, with the highest possible precision at that recall; report sensitivity, specificity, PPV and NPV, and be explicit about the PPV impact: with 1% prevalence, even 90% specificity gives PPV of about 8%, so most positives will be false alarms confirmed by a second test.

3. Lower the threshold: choose it on validation data from the precision-recall curve to hit the recall target; quantify extra false positives and workload.

4. Handle imbalance: class_weight, stratified CV, and PR-AUC as the primary comparison metric.

5. Improve the signal: add clinically relevant features, interactions, and check for missing data patterns; use regularization.

6. Validate prospectively and on different hospitals (external validation); check calibration and subgroup performance (age, sex, ethnicity).

7. Communicate trade-offs to the director using the confusion matrix on 10,000 screened patients, which makes the numbers concrete.

A461. A retailer builds a weekly sales forecasting regression model with random 5-fold cross-validation and gets R2 = 0.95, but after launch the e...

Likely causes:

1. Temporal leakage: random folds mix past and future weeks, so the model trains on weeks adjacent to (and after) the weeks being validated; features such as lagged sales or rolling averages can include information from the future target period. Autocorrelation then makes validation artificially easy.

2. Features not available at forecast time: promotions or weather known only after the fact, actual (instead of planned) prices, the current week's footfall.

3. Preprocessing fit on all data (scaling, target encoding of store) before splitting.

4. Distribution shift: new products, seasonality and holidays not represented, pandemic or price changes.

Redo validation:

- Use chronological splits: train on weeks 1 to t, validate on t+1 to t+h; use TimeSeriesSplit with expanding windows and a gap equal to the forecasting horizon; make the final test the most recent months.

- Build lag features using only information available h weeks before the target date.

- Put preprocessing inside the pipeline.

- Evaluate with MAE, RMSE and a scaled error such as MAPE or WAPE against a naive baseline (last week's or last year's sales). If the model cannot beat the seasonal naive baseline it adds no value.

- Monitor error by store, product and horizon after launch.

A462. An e-commerce company's data scientist predicts customer lifetime value (CLV) using linear regression. A handful of whales spend 100 times t...

1. Confirm the diagnosis: compute Cook's distance and leverage, and refit with and without the whales; large coefficient changes indicate influential observations.

2. Check validity: are these genuine high-value customers, corporate buyers, resellers or duplicates and test accounts? Remove or segment only if they are a different population (for example B2B accounts get a separate model).

3. Transform: log(CLV + 1) for the target and log of skewed spend predictors; this compresses the scale and makes the relationship more linear.

4. Robust options: Huber regression, quantile (median) regression, or Winsorising the target and predictors at the 1st and 99th percentile calculated on training data.

5. Regularization such as Ridge reduces the influence of any single point on coefficients.

6. Evaluate with MAE and median absolute error besides RMSE, since RMSE is dominated by whales; also evaluate separately on the top 1% of customers because the business cares about them for revenue concentration.

7. Pitfalls: do not remove outliers only to inflate R2, ensure the same treatment in production, and report uncertainty. If the business needs to identify the whales themselves, build a separate classification model for high-value customers.

A463. A bank wants to find customers similar to its best credit-card customers with a K-Nearest Neighbours model using age, income, number of prod...

1. Scaling: balance and income are in the tens of thousands while age is 18 to 90 and number of products is 1 to 5, so the Euclidean distance is dominated by balance and income. Standardise (or min-max scale) all numeric features, fitted on training data only.

2. Categorical data: region cannot be used in Euclidean distance as an integer code. One-hot encode it (scaled sensibly), or use a distance for mixed data such as Gower, or leave it out.

3. Skewed variables: log-transform balance and income, otherwise a few very rich customers appear far from everyone.

4. Irrelevant or redundant features distort distances (curse of dimensionality): select a small meaningful set, remove highly correlated variables, or weight features by importance.

5. Choose k by cross-validation (and distance weighting); examine the validation score for k from 1 to 50.

6. Missing values must be imputed beforehand.

7. Scale and speed: prediction requires scanning all customers; use KD-tree or ball tree structures, or approximate search for millions of rows.

8. Sanity checks: look at a few examples of neighbours and their features with business users. If KNN is used for segmentation rather than prediction, clustering might fit better.

A464. An online store fits a logistic regression to predict whether a visitor will purchase. The product manager asks: 'What do the coefficients t...

1. Interpret coefficients as log-odds: e.g. page_load_seconds coefficient = -0.22 means each extra second multiplies the odds of purchase by exp(-0.22) = 0.80, a 20% reduction in the odds, holding other variables constant. Cutting one second therefore increases odds by 1/0.80 = 1.25 (25%).

2. Convert to probabilities because the effect on probability depends on the baseline: if the baseline purchase probability is 4% (odds 0.0417), new odds are 0.0521, so new probability = 0.0521/1.0521 = 4.95%. That is roughly +0.95 percentage points, about 24% relative. At a baseline of 50% the same odds ratio gives a very different change (50% to 55.6%).

3. Provide confidence intervals for the odds ratio using exp(coef +/- 1.96 SE), not just a point estimate.

4. Caveats: this is an association, not a causal effect; slow pages correlate with mobile users, poor connections and weak buyers, so confounding is likely; the relationship might not be linear in log-odds (check by binning load time); coefficients on unscaled variables are not comparable across features (standardise to compare importance).

5. To measure the real impact, run an A/B test where one group gets a faster experience. The regression gives a hypothesis and a rough estimate to size the experiment.

A465. At a delivery company, a regression model to predict parcel delivery time gives R2 = 0.99 on training data but 0.45 on test data. What is ha...

This is overfitting: the model captures noise and specifics of the training rows, so it fails to generalise. Also check for a leaky train/test setup, though in this case the gap points to variance.

Steps:

1. Check model complexity: number of features versus rows, many one-hot dummies or high-degree polynomial terms, ID-like columns (parcel ID, address) that let the model memorise.

2. Learning curve: plot train and validation error versus training set size; a persistent gap that narrows with more data means collect more data.

3. Simplify the model: remove irrelevant features, use feature selection or Lasso, drop polynomial degree.

4. Regularize: Ridge or Lasso with the penalty chosen by cross-validation.

5. Use k-fold cross-validation for a more reliable estimate and to tune hyperparameters.

6. Check for outliers and mislabeled targets that the model is memorising.

7. Confirm the test set represents the same population (no distribution shift, e.g. different city or season).

8. Re-evaluate with RMSE and MAE in minutes against a simple baseline (mean delivery time per route). The goal is a small gap between train and validation scores at an acceptable level, not the highest training R2.

A466. A real estate analyst fits a multiple regression to predict rent. R2 is 0.85, but the coefficient for 'number of bedrooms' is negative and i...

Likely cause: multicollinearity. Square footage, number of rooms and bedrooms are strongly correlated; the model cannot separate their individual effects, so coefficients become unstable, have inflated standard errors and may take counter-intuitive signs, even while the overall fit (R2 = 0.85) and predictions remain good.

Diagnosis: correlation matrix, VIF per predictor (values above 5 or 10), condition number in the statsmodels output, and coefficients that swing when you drop one variable or resample the data.

Fixes depending on the goal:

1. Interpretation goal: drop redundant variables (keep square footage and perhaps bedrooms-per-area), or create combined features such as average room size, or area per bedroom.

2. Prediction goal: multicollinearity is not harmful to predictions, so you may keep all variables, but regularize with Ridge or Elastic Net for stability.

3. Use PCA/factor reduction if explicit interpretation is not needed.

4. Collect more varied data (e.g. apartments with large area but few bedrooms).

After the fix, recheck VIFs, sign plausibility, adjusted R2 and the test error. Explain to the stakeholders that individual coefficients under collinearity cannot be read as the effect of that variable alone.

A467. A bank's loan dataset has 25% missing values in the 'annual income' column, and missingness is higher for self-employed applicants and for a...

1. Understand the mechanism: missingness depends on observed variables (employment type) and possibly on outcome-related behaviour, so it is not MCAR; it is probably MAR or even MNAR. Deleting rows would remove many self-employed applicants and bias the model.

2. Avoid dropping the column outright: income is a key credit variable.

3. Imputation: impute the median (or a model-based estimate using employment type, occupation, loan amount, region via an iterative imputer or group-wise median), fitted on training data only inside a Pipeline.

4. Add a binary 'income_missing' flag so the model can learn that missingness itself carries information (for example, self-employed with undocumented income carries higher risk).

5. Consider multiple imputation if you need valid inference.

6. Sensitivity analysis: compare model performance and coefficients under several strategies (median + flag, group median, model-based, drop rows) to see whether conclusions are stable.

7. Production: the same imputation and flag logic must be applied to new applications; monitor the share of missing income over time.

8. Pitfalls: leakage if the imputer uses the target; using a 'rejected' flag to explain missingness is leakage because the decision occurs after the application; and mean imputation understates variance.

A468. You are predicting house prices across 3,000 zip codes using linear regression. One-hot encoding the zip code creates 3,000 columns and many...

Problems with one-hot encoding: very wide sparse data, overfitting for zip codes with few observations, multicollinearity, noisy coefficients, and unseen zip codes in the test or production data.

Options:

1. Group rare zip codes into 'Other' (for example fewer than 30 sales), or group by a higher geographic level (city, county, region) using hierarchy.

2. Replace zip code with meaningful numeric attributes: median income, population density, school rating, crime rate, distance to city centre, latitude/longitude, historical median price per square foot.

3. Target encoding (mean log price per zip code) with smoothing toward the global mean: encoded = (n * mean_zip + m * global_mean) / (n + m) and, crucially, computed out-of-fold on training data to avoid target leakage. For test data use means from the full training data.

4. Regularization: keep one-hot columns but use Ridge or Lasso so small zip codes are shrunk, which is similar to a random effect.

5. Frequency encoding as a weak alternative.

6. Mixed (hierarchical) models allow partial pooling of small groups, if the course covers them.

Validate with a time- or group-aware split; test how the model does for zip codes with few sales and for entirely new zip codes. Handle unseen levels explicitly (fall back to city or global values).

A469. An HR department wants to predict employee attrition and present the main drivers to leadership. Attrition is 12%. They have 1,500 employees...

1. Small data: 1,500 rows with about 180 events (12%) is modest, so use at most about 15 to 18 predictors (rule of thumb of 10 events per variable) or use regularization.

2. Preprocess: encode categoricals (department, job level), handle missing values, standardise numeric variables, check VIF (age, tenure and years in role correlate), and cap outliers in salary (log salary).

3. Model: L1 or Elastic Net logistic regression to select features and keep coefficients interpretable. Tune C using stratified repeated cross-validation, because a single split on such small data is noisy.

4. Evaluate: ROC-AUC and PR-AUC with confidence intervals from repeated CV; recall and precision at a chosen threshold; calibrated probability if they will rank employees by risk.

5. Interpret: odds ratios with confidence intervals (for example 'working overtime multiplies the odds of leaving by 2.1') using standardised coefficients to compare drivers; stress these are associations. Be careful with stability: check that selected features are consistent across bootstrap resamples.

6. Pitfalls: leakage (exit interview data, notice-period flags), survivorship (only current employees recorded), ethics and fairness (gender, age as predictors, proxies), privacy, and the risk of using individual risk scores for

punitive action. Present group-level insights and recommend actions like workload or compensation review, ideally tested through pilots.

A470. When fitting a regression of employee salary on years of experience, education and job level, the residual plot shows a funnel shape: varian...

1. Recognise the issue: heteroscedasticity. OLS coefficients are still unbiased, but standard errors, t-tests and confidence intervals computed under the constant variance assumption are wrong (usually too narrow for the high-salary region), so the manager's interval would be misleading.

2. Confirm with a Breusch-Pagan or White test and a scale-location plot.

3. Options:

a. Log-transform salary (very common for money values): this often yields roughly constant variance and a more natural interpretation, 'each additional year of experience is associated with about a 3% higher salary'.

b. Keep salary in dollars but use heteroscedasticity-consistent (robust, HC3) standard errors: statsmodels fit(cov_type='HC3'). Coefficients are unchanged and confidence intervals become valid.

c. Weighted least squares if variance is proportional to a known variable.

d. Add missing non-linear terms such as experience squared or interactions with job level, since misspecification can look like heteroscedasticity.

4. Re-run the residual diagnostics and check normality and outliers.

5. Report the interval from the corrected model along with caveats: association rather than causation, omitted variables such as performance or negotiation, and group differences.

A471. A ride-hailing company predicts whether a rider will cancel a booking after a driver is assigned (12% cancel). Each wrongly flagged rider tr...

1. Define the cost matrix: FP = 1 dollar, FN = 4 dollars, TP and TN approximately 0 (or TP could include the incentive cost of 1 dollar if the incentive is sent and succeeds; state assumptions clearly).

2. Optimal threshold for calibrated probabilities: flag when $p * 4 > (1 - p) * 1$, i.e. $p > 1 / (1 + 4) = 0.2$. So the threshold should be about 0.2, not 0.5.

3. Verify calibration: if class_weight or resampling was used, probabilities are off; check the calibration curve and recalibrate (Platt or isotonic) on a validation set.

4. Compare variants by expected cost per booking at their best thresholds: cost = (FP * 1 + FN * 4) / N, computed on a validation set; confirm on a later test period. Also report ROC-AUC and PR-AUC for ranking quality, and log loss or Brier score for probability quality.

5. Choose the threshold by scanning thresholds from 0.01 to 0.99 and picking the cost minimum on validation data; check the cost curve is flat enough that the choice is robust.

6. Pitfalls: the assumption that the incentive works (the effect is not 100%), costs that vary by city or time of day (use segment-specific thresholds), feedback loops (incentives change cancel behaviour and future labels), threshold selection on test data, and drift. Validate through an A/B test measuring real net cost.

A472. A consumer-lending logistic regression model performed well at launch (AUC 0.78), but after 9 months its AUC is 0.66 and the approval rate h...

Possible causes:

1. Data (covariate) drift: the applicant population changed (new marketing channel, new region, economic conditions), so feature distributions differ from training.

2. Concept drift: the relationship between features and default changed (inflation, interest rates, policy changes).

3. Data pipeline issues: a source field changed definition, new missing values, unit changes, a bureau score version change.

4. Selection effects: tighter approval rules mean the model scores only a narrower population (range restriction lowers AUC) and outcomes are observed only for approved loans.

5. Outcome maturity: if defaults are not yet fully observed, recent cohorts look better or worse than they are.

Diagnosis:

- Compare feature distributions (population stability index, KS tests, summary stats) between training and recent data; check missing rates.

- Track AUC, calibration (predicted vs actual default by score band) per monthly cohort, with enough maturity.

- Examine performance by segment and channel.

- Verify upstream feature calculations.

Response: fix data issues; recalibrate the intercept if only the base rate changed; retrain on recent data with a new out-of-time validation; update feature set; consider champion-challenger testing; set up automated monitoring thresholds (PSI > 0.25 triggers review) and a retraining schedule.

A473. A genomics lab has 150 patient samples and 5,000 gene-expression features and wants to predict whether a patient responds to a drug. The fir...

Problem: p (5,000) is far greater than n (150). An unregularized model can separate the classes perfectly (perfect separation), so coefficients are not identifiable or diverge, and training accuracy tells nothing about generalisation.

Approach:

1. Use strong regularization: L1 (Lasso) or Elastic Net logistic regression, which handles p > n and selects a few genes; L2 for stability when many genes are mildly informative. Standardise features first. Tune C by cross-validation.

2. Validation design: stratified repeated k-fold or nested cross-validation to get an honest performance estimate with such small data, with all steps (scaling, feature filtering) inside the folds. A very common mistake is selecting the top genes using all samples and then cross-validating, which leaks label information and gives wildly optimistic results.

3. Pre-filtering inside folds: remove low-variance genes, optionally use univariate filters (t-test) within each training fold.

4. Metrics: AUC with confidence intervals (bootstrap), sensitivity and specificity at a clinical threshold; compare to a trivial baseline.

5. Stability: check the genes selected across folds and bootstrap samples; unstable lists mean the findings are weak.

6. Confounders: batch effects and site or sex differences that correlate with response.

7. Report uncertainty honestly and plan external validation on an independent cohort before claiming clinical use.

A474. A hospital group builds a model predicting 30-day readmission, and one feature, 'discharge summary completed', has by far the largest coeffi...

An AUC of 0.96 for a task where good models usually get 0.65 to 0.75 is a red flag for leakage.

Investigate:

1. Timing: when is 'discharge summary completed' recorded? If readmitted patients have their records updated or a summary is generated at the readmission, or the flag is populated after follow-up, it is effectively an outcome proxy that will not be available at prediction time (the moment of discharge).

2. Look at the univariate relationship: readmission rate for flag = 1 vs 0; crosstab with the target; check how it was created in the source system and with the data owner.

3. Check for other leak candidates: billing codes finalised after discharge, length-of-stay fields that include the readmission stay, 'follow-up appointment missed' flag.

4. Rebuild the dataset with a clear prediction timestamp and use only data time-stamped before it.

5. Retrain without the suspicious features and compare: performance should drop to a realistic level. If it does, the earlier model was leaking.

6. Report to stakeholders: explain why the high score was misleading, and that the deployed model would fail in production because the feature would not exist at discharge time.

Prevention: involve domain experts in feature review, create a data dictionary with availability timestamps, use time-based validation, and treat very high importance as a reason to investigate rather than celebrate.

A475. A food-delivery company predicts delivery time in minutes. Operations says late deliveries (very long times) hurt customers far more than sl...

1. Understand the cost structure: errors are asymmetric (underestimating delivery time causes late complaints) and heavy tails exist (extreme delays).

2. Metrics: report MAE (typical error in minutes, robust to rare extremes) and RMSE (penalises large errors, shows tail behaviour). A big RMSE-MAE gap indicates a few large misses. Add percentile errors (median and 90th percentile absolute error) and the share of orders delivered more than 10 minutes later than the prediction (late rate), which maps to customer experience.

3. Loss: if lateness is worse than earliness, consider a quantile (e.g. 80th percentile) prediction to show customers a conservative promise, or an asymmetric loss. In the linear regression family, quantile regression handles this directly; standard OLS predicts the mean, and MAE-type models predict the median.

4. Skewness: model log(time) or treat weather-driven extremes through weather features and flags; check residuals by hour, distance bucket and weather.

5. Baselines: compare against the restaurant's average prep plus distance-based rule.

6. Validation: time-based split because traffic patterns and menus change, and evaluate separately for rush hours.

7. Reporting: show the errors in minutes by segment, calibration of the quoted ETA (what percent of orders arrive before the quoted time), and trade-off curve between the promise level and accuracy.

Business Cases and Product Analytics

A476. You are a data analyst at Swiggy. The VP of Operations says: "Daily orders dropped 20% yesterday versus the same weekday last week. Find out...

Step 1 - Validate the drop before investigating. Check the data pipeline: did yesterday's load finish, is the row count of orders and sessions also low, is there a timezone or late-arriving data issue? Compare against the same weekday in prior 4 weeks, not just last week, to rule out a holiday, a cricket match, rain or a festival.

Step 2 - Decompose the metric. Orders = sessions x conversion (session to order) x (1 - cancellation rate). Find which factor moved:

- Sessions: 1,000,000 -> 980,000 (-2%)

- Session-to-order conversion: 8.0% -> 6.5% (-19%)

- Cancellation rate: 5% -> 5.2%

So demand is there but conversion collapsed; this points to supply or product friction rather than traffic.

Step 3 - Slice by dimensions to localize it (find the segment where the drop is concentrated). Typical cuts: city, platform (Android/iOS/web), app version, new vs returning users, restaurant tier, time of day, payment method.

```
WITH d AS (
  SELECT city, platform, DATE(created_at) AS dt, COUNT(*) AS orders
  FROM orders
  WHERE created_at >= CURRENT_DATE - 8 AND status <> 'test'
  GROUP BY 1,2,3)
SELECT city, platform,
  SUM(CASE WHEN dt = CURRENT_DATE-1 THEN orders END) AS yday,
  SUM(CASE WHEN dt = CURRENT_DATE-8 THEN orders END) AS last_wk,
  1.0*SUM(CASE WHEN dt = CURRENT_DATE-1 THEN orders END)/
      NULLIF(SUM(CASE WHEN dt = CURRENT_DATE-8 THEN orders END),0) - 1 AS pct_change
FROM d GROUP BY 1,2 ORDER BY pct_change;
```

Compute each segment's contribution to the total drop (segment delta divided by total delta) so you do not chase a segment with a big percent drop but small volume.

Step 4 - Form hypotheses by bucket: (a) technical: a bad release on app version 9.4 on Android, payment gateway failure, API latency; (b) supply: restaurants offline, delivery partner shortage, surge in 'no delivery partner' cancellations; (c) demand/external: rain, strike, competitor offer, holiday; (d) marketing: a campaign paused or push notifications failed; (e) measurement: tracking bug.

Step 5 - Test each. Example: Android 9.4 shows conversion fall 12% -> 3% starting at 7 pm, matching a release timestamp, with payment_failed events up 10x. Cross-check error logs with engineering.

Step 6 - Communicate: lead with the answer ("~85% of the drop comes from Android app version 9.4 in all cities, payment step failing since 7 pm; estimated lost orders 40k, about Rs 1.2 crore GMV"), state confidence, recommend rollback, and set up an alert on conversion by app version. Follow up with a recovery check next day.

A477. Netflix wants to define a north-star metric for its streaming product. Data available: viewing_events(user_id, title_id, device, start_ts, e...

A north-star metric should capture the value delivered to customers, be a leading indicator of revenue, and be influenceable by many teams.

Candidate: total hours watched is easy but can be inflated by autoplay or long, low-quality content. A better proposal is "weekly active viewers who complete at least X minutes of content across at least 2 distinct titles" or "quality viewing hours per member per month" where quality viewing means sessions of more than 20 minutes, not auto-played from idle. Retention is the revenue link: members who watch more hours in month 1 churn less. Validate that link with data: bucket users by first-30-day hours and plot 90-day retention; if retention rises from 55% (under 5 hours) to 90% (over 20 hours), hours is a sound proxy.

I would choose: "Average quality hours watched per paying member per week" as the north-star.

Input metrics (what teams move): new-title discovery rate (percent of sessions where user starts a title within 60 seconds), percent of members with 2+ titles started in a week, recommendation click-through, buffering ratio.

Guardrail metrics (must not degrade while chasing north-star):

- 30/60/90-day churn and cancellation rate

- Content cost per hour watched (to avoid paying for engagement)

- Customer support contacts per 1,000 members

- Video start failure rate and rebuffer rate

- Diversity of titles watched (avoid over-concentration on a few titles)

- Complaints about autoplay or accidental plays

Counter-metric thinking: if autoplay increases hours by 6% but cancellations rise 0.3 points, the guardrail blocks the launch. State measurement details: define a 'session', de-duplicate multiple devices, exclude trailers, and report both mean and median because heavy users skew the mean.

A478. Uber's rider app saw "completed trips per day" fall 15% in Bengaluru last Monday while the rest of India was flat. You have trips(trip_id, r...

Treat a marketplace as supply, demand and matching. Completed trips = requests x fulfillment rate, where fulfillment = matched x not cancelled.

1. Confirm: is it Bengaluru-wide or zone-specific? Compare with the last 4 Mondays; check if there was a public event, bandh, heavy rain or an auto-rickshaw/taxi strike.

2. Funnel: ride requests, requests with driver found, accepted, trip started, completed. Suppose requests fell 3% (small), found-driver rate fell from 92% to 78%, cancellations flat. That means supply shortage, not demand weakness.

3. Supply: drivers online per hour. If online driver-hours dropped 25% after 8 am, check whether a driver cohort went offline (strike, a new incentive scheme at a competitor, fuel price, airport restrictions, app outage for drivers).

```
SELECT date_trunc('hour', ts) AS hr, COUNT(DISTINCT driver_id) AS online_drivers
FROM driver_status WHERE online_flag = 1 AND city='Bengaluru'
  AND ts::date IN (current_date-7, current_date-14)
GROUP BY 1 ORDER BY 1;
```

4. Marketplace signals: ETA to pickup rose from 4 to 9 minutes, surge multiplier rose to 2.0, and rider conversion after seeing price dropped, which compounds the loss.

5. Zone view: map by hexagon; if drops concentrate near tech parks with a route closure, it is local.

6. Conclusion and action: example - 'a driver strike removed ~30% supply; demand intact; 70% of lost trips are unfulfilled requests'. Actions: targeted driver incentives, notify riders of longer ETAs, shift to auto/bike products, and track recovery. Also quantify opportunity cost (unfulfilled requests x average fare, e.g. 18,000 x Rs 220 = Rs 40 lakh).

A479. Amazon.in's "Add to Cart" rate on mobile web fell from 6.0% to 5.1% over two weeks, but desktop is stable. Available: events(user_id, sessio...

A gradual 2-week decline on one platform suggests either a slow-rolling change (A/B ramp, release adoption), a traffic mix shift, or a degradation that grows (page speed).

1. Check metric definition: add-to-cart rate = sessions with add_to_cart / sessions with product page view. Verify numerator and denominator separately. If the denominator rose (new low-intent traffic from a campaign) the rate falls without any product issue.

2. Mix shift test (Simpson's paradox): decompose change into within-segment rate change and mix change. If paid-social traffic share rose 10% to 25% with a 2% ATC rate, that alone explains most of the drop.

```
SELECT channel,
  COUNT(DISTINCT CASE WHEN event_name='pdp_view' THEN session_id END) AS pdp_sessions,
  COUNT(DISTINCT CASE WHEN event_name='add_to_cart' THEN session_id END) AS atc_sessions
FROM events e JOIN sessions s USING(session_id)
WHERE device='mobile_web' GROUP BY channel;
```

3. If the rate dropped within every channel, look at browser/OS (a new Safari version breaking the button), page load time (p75 LCP up from 2.5 s to 4.2 s), A/B tests ramping on mobile web, new layout pushing the button below the fold, out-of-stock or delivery-not-available messages rising.

4. Look at product-level: has the share of unavailable items or higher-priced items increased?

5. Output: say which factor explains what share of the 0.9 point drop (e.g. 0.5 mix, 0.3 latency regression, 0.1 unexplained), and propose fixes plus monitoring (alert on rate by device x channel).

A480. Flipkart is launching "Flipkart Minutes" (10-minute delivery). Define the success metrics: a primary metric, supporting metrics, and guardra...

Start with the goal: acquire frequent, profitable repeat customers for quick commerce.

Primary (north-star): weekly repeat orders per active customer, or number of customers with 3+ orders in a month (a habit metric). GMV alone can be bought with discounts.

Supporting / driver metrics:

- Delivery promise adherence: percent of orders delivered within 10 minutes (target above 90%), median and p95 delivery time

- Fill rate / item availability: items delivered divided by items ordered (target 95%+)

- Orders per dark store per day and store utilization

- Basket size, items per order, average order value

- New-customer conversion, 30-day retention of Minutes cohort

- Cross-over rate: share of Minutes users who also shop on main Flipkart

Unit economics: contribution margin per order = AOV x take rate + ad revenue - delivery cost - packaging - discounts - wastage. Track at dark-store level.

Guardrails: order cancellation rate, delivery partner safety incidents and rider attrition, wastage/expiry percent, customer complaint rate and refund rate, discount per order (so growth is not bought), cannibalization of core Flipkart orders.

Segmentation: by city, dark store age, time of day. Consider a time-lagged evaluation: dark stores take 8-12 weeks to mature, so compare by store age cohort, not calendar.

Communicate a scorecard with targets, and define the decision rule: scale a city if D30 retention is above 35%, contribution margin per order is positive by week 16, and on-time rate above 90%.

A481. A bank's credit card division reports that "average monthly spend per active card" rose 12% quarter over quarter, and leadership is celebrat...

A ratio metric can move because of numerator, denominator or mix changes.

1. Definition check: what is 'active'? If active = at least one transaction in the quarter, and low-spend cards went dormant, the denominator shrinks and the average rises (survivorship). Check number of active cards: if it fell 8% while total spend rose 3%, the 12% is mostly a denominator effect.

2. Decompose: total spend, active cards, spend per active card by segment (premium, mass, student), new vs old cards.

3. Mix effect (Simpson): if the premium segment (spend Rs 60k) grew from 20% to 28% of active cards while each segment's average was flat, the overall average rose with no behavioural change. Use a standard decomposition: change = sum(share_change x segment_avg) + sum(segment_share x avg_change).

4. Outliers: a few corporate cards with Rs 50 lakh single transactions or one-off events (tax payment, festival) can move the mean; compare median and trimmed mean.

5. Seasonality and calendar: Q4 festive quarter vs Q3; compare year over year.

6. Same-store style cohort analysis: fix the set of cards that were active in both quarters and compute their spend change (like same-store sales). Say it shows only +2%.

```
WITH both_q AS (
 SELECT card_id,
  SUM(CASE WHEN ts>=DATE '2026-07-01' THEN amount END) AS q_now,
  SUM(CASE WHEN ts<DATE '2026-07-01' THEN amount END) AS q_prev
 FROM transactions WHERE ts>=DATE '2026-04-01' GROUP BY 1)
SELECT AVG(q_now)/AVG(q_prev)-1 FROM both_q WHERE q_now IS NOT NULL AND q_prev IS NOT NULL;
```

7. Also look at revolving balance, delinquency, and attrition; spend growth with rising defaults is not healthy.

Recommendation: report spend per card on a constant cohort plus active-card rate, and pair with credit-quality guardrails.

A482. Spotify (or a music streaming app) sees DAU stay flat while MAU keeps rising for six months. Data: user_activity(user_id, date, minutes_list...

DAU/MAU is the 'stickiness' ratio: the average share of monthly users who are active on a given day. If DAU is flat and MAU is rising, DAU/MAU falls (e.g. 4.0M/10M = 40% to 4.0M/12M = 33%). It means growth is coming from lightly engaged users.

Investigation:

1. Split MAU into new users (signed up in last 30 days), resurrected, and retained. Likely a marketing push or a free-trial partnership is adding low-intent users.

2. Cohort retention curves: D1, D7, D30 for recent cohorts vs older cohorts. If D7 dropped from 38% to 24%, onboarding or acquisition quality is the problem.

3. Frequency distribution: number of active days per user in the month (L1..L30 histogram). A shift to 1-3 day users shows casual growth; the 'power user' L20+ band may still be stable.

4. Channel-level: are new users from low-cost installs with poor retention?

5. Platform/country split: growth in a market with low usage (new country launch).

6. Check for measurement changes such as passive background plays being filtered.

```
SELECT COUNT(DISTINCT user_id) FILTER (WHERE date = current_date-1) AS dau,
       COUNT(DISTINCT user_id) FILTER (WHERE date >= current_date-30) AS mau
FROM user_activity;
```

Conclusion: flat DAU with rising MAU is not necessarily bad (a funnel for future conversion), but evaluate by retained-user DAU and paid conversion. Recommend focusing on activation (first-week habit: build a playlist, follow artists), and tracking DAU/MAU by acquisition cohort rather than overall.

A483. A telecom company (Airtel/Jio style) tracks ARPU and sees it fell 5% in a quarter even though revenue grew 3%. The CFO asks you to explain. ...

ARPU = total revenue / average subscribers. Revenue +3% but ARPU -5% means subscribers grew about 8.4% (1.03/0.95 = 1.084). So growth came from adding many lower-revenue subscribers.

Decompose:

1. Mix: new subscribers by plan and circle. If 70% of additions are on Rs 149 prepaid plans or in rural circles versus a base ARPU of Rs 200, ARPU dilutes.

2. Like-for-like: ARPU of subscribers that existed in both periods. If it is flat, the entire fall is dilution; if it fell, look at downgrades, discounts, promo plans, fewer top-ups, or a tariff change.

3. Definition: are inactive/secondary SIMs counted? Dual-SIM users with low spend inflate subscriber counts; compare ARPU on active users or 'revenue per unique customer'.

4. Components: voice, data, VAS, roaming, or one-time revenue in the previous quarter.

```
SELECT p.plan, COUNT(DISTINCT b.sub_id) subs, SUM(b.revenue)/COUNT(DISTINCT b.sub_id) AS arpu
FROM billing b JOIN subscribers p USING(sub_id)
WHERE month IN ('2026-04','2026-07') GROUP BY p.plan, month;
```

5. Interpretation: lower ARPU from healthy subscriber growth may be good if the new base later upgrades; check upgrade rate and churn. Present a bridge chart: previous ARPU 200 -> mix effect -9 -> like-for-like +1 -> new ARPU 190.

Recommendations: target upsell to the low-ARPU cohort, report ARPU alongside subscriber net adds, and consider ARPU on a cohort basis.

A484. Zomato's average delivery time rose from 29 to 34 minutes over two weeks. Operations wants to know whether it is an average-versus-tail prob...

1. Look beyond the mean: compute p50, p90, p95. If p50 moved 27 -> 28 but p95 moved 45 -> 62, a minority of orders are causing the increase.

2. Break the delivery time into stages: restaurant acceptance, food preparation, rider assignment, rider arrival at restaurant (wait), travel to customer, handover.

```
SELECT date_trunc('week', placed_at) wk,
 AVG(EXTRACT(EPOCH FROM accepted-placed)/60) accept_min,
 AVG(EXTRACT(EPOCH FROM prepared-accepted)/60) prep_min,
 AVG(EXTRACT(EPOCH FROM picked_up-prepared)/60) rider_wait_min,
 AVG(EXTRACT(EPOCH FROM delivered-picked_up)/60) travel_min,
 percentile_cont(0.95) WITHIN GROUP (ORDER BY EXTRACT(EPOCH FROM delivered-placed)/60) p95
FROM orders GROUP BY 1 ORDER BY 1;
```

3. Suppose rider assignment-to-pickup grew +3.5 minutes while prep and travel are flat: it is a rider supply issue (rain, strikes, fewer riders at peak, batching changes).

4. Check confounders: distance mix (more far orders), order volume spikes at lunch/dinner (capacity), weather, restaurant mix, a change in batching algorithm.

5. Control for distance: model delivery time vs distance_km and hour to get 'excess minutes'.

6. Locate: zones and hours where excess minutes are highest; per-restaurant outliers (slow prep) and per-rider outliers.

7. Actions: rider incentives in affected zones/hours, temporarily reduce delivery radius, tighten restaurant prep SLAs, show honest ETAs. Track late-delivery rate (more than 10% over promise), cancellations and ratings as outcome measures.

A485. Instagram wants to measure the "health" of Stories. Product says time spent is up 8%, but a researcher worries about quality. Propose a metr...

Start with the product goal: help people share and connect with friends and creators, not just consume.

North-star: weekly users who both view and meaningfully interact (reply, react, share, or post) with Stories, i.e. 'meaningful story interactions per user'. Time spent is an output, not the goal.

Input metrics: stories posted per creator, story reach per post, view completion rate, tap-forward/tap-back rates, reply rate, share-to-DM rate, percent of users who post at least once a week (creator health).

Guardrails: user-reported negative experience (hide, report, 'not interested'), uninstall/deactivation rate, negative sentiment survey scores (e.g. share saying 'time well spent'), notification opt-outs, teen well-being metrics, ads load vs. engagement, latency/crash rates.

Counter-metrics: if passive consumption rises but posting falls, supply side is eroding. Track ratio of viewers to posters.

Trade-off evaluation:

- Run a long-term holdout (e.g. 1% of users with the old ranking for 8 weeks) because short-term time spent gains from novelty or addictive design may decay and harm retention.

- Look at heterogeneous effects: heavy vs light users, teens, creators.

- Use decision rules: ship if north-star up by at least 1% with no guardrail worse than -0.5% (statistically tested), else iterate.

- Triangulate quantitative with surveys: time spent up 8% but 'felt it was worthwhile' down 3 points would be a red flag.

Communication: give a one-page scorecard with confidence intervals, and describe what you did not measure.

A486. An e-commerce company (Myntra-style) has this checkout funnel for last week: Product view 1,000,000; Add to cart 120,000; Checkout start 70,...

Compute step-to-step and overall conversion:

- View -> cart: 12.0%

- Cart -> checkout: 58.3%

- Checkout -> payment: 64.3%

- Payment -> success: 80.0%

- Overall: 3.6%

The biggest relative drop is view to cart (88% leave), but that is expected browsing behaviour. Compare to benchmarks and history: if cart->checkout was 68% last month it is the anomaly. The payment->success step at 80% is low for a payment step (benchmarks 90%+), meaning 9,000 users lose at payment; that is high-intent traffic, so the revenue recovered per fix is highest there. Prioritize by potential impact: improving payment success from 80% to 85% yields +2,250 orders (+6.25%), far cheaper than raising view-to-cart by the same relative amount.

SQL (session-level, ordered funnel):

```
WITH s AS (
 SELECT session_id,
  MAX(CASE WHEN event='view' THEN 1 END) v,
  MAX(CASE WHEN event='add_to_cart' THEN 1 END) c,
  MAX(CASE WHEN event='checkout' THEN 1 END) k,
  MAX(CASE WHEN event='payment' THEN 1 END) p,
  MAX(CASE WHEN event='success' THEN 1 END) o
 FROM events WHERE ts >= current_date-7 GROUP BY 1)
SELECT SUM(v) views, SUM(c) carts, SUM(k) checkouts, SUM(p) payments, SUM(o) orders FROM s;
```

Next, segment the weak step: payment failures by method (UPI vs card vs COD), bank, device, app version, error code. Example: UPI intent failures 18% on Android. Also check user-level vs session-level (users return later), time-window for conversion, and distinguish technical failures from voluntary abandonment. Recommend experiments: retry on failure, more payment options, saved cards, price transparency (shipping fees shown late) and measure with an A/B test.

A487. You are given a table logins(user_id, login_date) for a fitness app. Product wants a weekly cohort retention table: users grouped by their s...

Definitions: cohort = week of first login (or signup); retention in week N = share of cohort members with at least one login in week N after the cohort week. Pick 'N-day' versus 'unbounded/rolling' retention explicitly; here we use classic week-N retention.

Steps:

1. Assign each user a cohort week = week of MIN(login_date).

2. For each login, compute week_number = weeks between login week and cohort week.

3. Count distinct users per (cohort, week_number); divide by cohort size.

```
WITH first AS (
  SELECT user_id, date_trunc('week', MIN(login_date)) AS cohort_wk
  FROM logins GROUP BY 1),
act AS (
  SELECT DISTINCT l.user_id, f.cohort_wk,
    (date_trunc('week', l.login_date)::date - f.cohort_wk::date)/7 AS wk_num
  FROM logins l JOIN first f USING(user_id))
SELECT cohort_wk,
  COUNT(DISTINCT CASE WHEN wk_num=0 THEN user_id END) AS cohort_size,
  ROUND(100.0*COUNT(DISTINCT CASE WHEN wk_num=1 THEN user_id END)/
        COUNT(DISTINCT CASE WHEN wk_num=0 THEN user_id END),1) AS w1,
  ROUND(100.0*COUNT(DISTINCT CASE WHEN wk_num=4 THEN user_id END)/
        COUNT(DISTINCT CASE WHEN wk_num=0 THEN user_id END),1) AS w4
FROM act GROUP BY 1 ORDER BY 1;
```

Reading the table: horizontally for the decay curve of one cohort, vertically for whether newer cohorts retain better (diagonal reading shows calendar effects like a January surge). Pitfalls: incomplete recent weeks should be blank not zero; cohort sizes differ so show N; define active (login vs workout); consider timezone. Typical insight: week-1 retention 35%, week-4 18%, flattening at 12% means a stable core; if the curve keeps falling to zero there is no product-market fit. Use findings to target activation: e.g. users who complete 3 workouts in week 1 retain 3x better, so redesign onboarding to drive that.

A488. A SaaS company's monthly customer churn rose from 2% to 3.5%. You have subscriptions(customer_id, plan, start_date, end_date, mrr, industry,...

1. Define: logo churn (customers lost / customers at start) versus revenue churn (MRR lost). 2% -> 3.5% logo churn compounds to roughly 35% annual loss. Separate voluntary (cancelled) and involuntary (failed payment) churn.

2. Locate the increase: segment by plan, tenure, industry, channel, company size, contract (monthly vs annual). Use survival (tenure) view: if churn is concentrated at month 2-3, onboarding failed; at renewal date, value or price issue.

```
SELECT acquisition_channel, plan,
 COUNT(*) FILTER (WHERE end_date BETWEEN '2026-09-01' AND '2026-09-30') AS churned,
 COUNT(*) FILTER (WHERE start_date < '2026-09-01' AND (end_date IS NULL OR end_date >= '2026-
09-01')) AS at_risk
FROM subscriptions GROUP BY 1,2;
```

3. Cohort angle: did churn rise for recent signup cohorts (acquisition quality) or all cohorts (product, price, competitor)?

4. Behavioural drivers: compare churned vs retained on usage in the 30 days before churn - logins, key feature adoption, seat usage, number of support tickets, NPS. Create leading indicators (e.g. a drop of 50% in weekly active users, no admin login for 14 days).

5. Events: price increase, outage, release removal of a feature, competitor promotion.

6. Predictive model: logistic regression or gradient boosting for 30-day churn risk with features from usage trends; evaluate with PR-AUC and lift in the top decile, not accuracy; validate on a later time period to avoid leakage.

7. Recommendations with sizing: e.g. 60% of the extra churn comes from SMB customers acquired through a discount campaign with low activation; fix: onboarding calls for customers who miss the key feature in week 1, retention offers on at-risk accounts, dunning emails for failed payments. Estimate value: saving 1 point of churn on Rs 10 crore MRR = Rs 10 lakh per month.

8. Measure the intervention through a randomized holdout.

A489. A telecom operator wants to predict which prepaid customers will churn in the next 30 days to target retention offers. Data: 24 months of us...

1. Define churn precisely: prepaid has no cancellation event, so define churn as no recharge or usage for 30 consecutive days (or 60). Define the observation window (features up to day T) and the prediction window (T to T+30). Build training snapshots monthly to avoid leakage.

2. Features (all computed using data before T): recharge frequency and recency, trend in recharge value (last 30 vs previous 90 days), data usage change, call drop rate, complaint count, days since last top-up, ratio of outgoing to incoming calls, tenure, plan type, share of calls to other networks (indicates a second SIM).

3. Model: start with logistic regression for a baseline, then gradient boosting. Time-based split (train months 1-18, validate 19-21, test 22-24). Class imbalance (churn around 5%): use stratification, class weights; evaluate with PR-AUC, recall at top k, and lift.

4. Threshold by economics, not 0.5. Expected value of targeting a customer with churn probability p: p x (uplift) x value retained - cost. If the offer saves 25% of would-be churners (uplift 0.25) and the retained customer yields Rs 400 x 6 months = Rs 2,400, then target when p x 0.25 x 2,400 > 50, i.e. p > 8.3%.

```
from sklearn.metrics import precision_recall_curve
ev = lambda p: p*0.25*2400 - 50
target = proba > 0.083
```

5. Better: uplift modelling (who is persuadable) since high-risk customers may leave regardless, and some would stay without the offer (sleeping dogs). Run a randomized pilot with 10% holdout to estimate true uplift.

6. Deployment: monthly scoring, a dashboard of lift and realized retention, drift monitoring (PSI), fairness and privacy considerations.

7. Report in business terms: top 10% scored customers contain 40% of churners (4x lift); campaign net benefit estimated at Rs X crore.

A490. A new user onboarding flow for a fintech app (UPI wallet) has these steps: download, signup, KYC upload, bank link, first transaction. Only ...

1. Build the funnel with time bounds: download 100,000 -> signup 70% -> KYC started 45% -> KYC verified 30% -> bank linked 24% -> first transaction within 7 days 18%.

2. Identify biggest absolute losses: the signup->KYC started step loses 25 points; KYC started->verified loses 15 points (people who try and fail are the most recoverable).

3. Dig into failure reasons for KYC verification: document blur, name mismatch, server timeout; segment by device model, language, state, and time of day.

4. Time-to-convert analysis: median time between steps, drop-off cliffs after 5 minutes, and re-engagement potential (users who stop at KYC and come back within 24h).

5. Qualitative: session replays, funnel surveys, customer support transcripts.

6. Compare converters vs non-converters on early behaviours (did they receive a cashback promise? did they see the first-transaction nudge?).

7. Prioritize opportunities by ICE or by estimated incremental users: improving KYC verified success from 66% (30/45) to 80% adds ~6 points to verified, ~4.3 points to first transactions (given downstream conversion). Improving every step by a small relative amount is less effective than fixing one broken step.

8. Experiments: simplified KYC with a camera guide, pre-filled bank details via account aggregator, a Rs 50 first-transaction reward with a cost cap. Measure incremental conversion and CAC, plus 30-day retention (to avoid reward hunters).

9. Guardrails: fraud rate, KYC compliance rejections, support tickets. Present expected uplift with ranges.

A491. Subscription app: you have a retention curve showing 100% in month 0, 60% in month 1, 45% in month 2, 38% in month 3, 35% in month 6, 34% in...

Reading the curve: steep early decay (40% lost in the first month), then flattening at about 34-35%. Flattening ("smile" or plateau) is the good sign: a durable core exists. The problem is early activation. Whether it is good depends on benchmarks for the category (streaming apps typically retain 30-50% at month 3; a utility app might do better) and on trend: compare newer cohorts to older ones.

LTV calculation:

Approach 1 (retention-curve): sum of survival x margin per month. Months 0-3: 1.00+0.60+0.45+0.38 = 2.43; months 4-6 roughly average 0.36 x 3 = 1.08; months 7-12 about 0.345 x 6 = 2.07; total over 12 months = about 5.58 subscriber-months. Gross margin LTV (12 months) = 5.58 x 300 x 0.70 = Rs 1,172. Beyond 12 months, the tail retains about 34% with small monthly churn (say 1.5%): additional expected months ~ 0.34/0.015 = 22.7 months (undiscounted, cap at 36 months horizon, and discount at 10-12% annually). That could add another Rs 1,500+ but with uncertainty.

Approach 2 (simple formula): LTV = ARPU x margin / churn. Using the plateau churn of 1.5%, that would be 300 x 0.7 / 0.015 = Rs 14,000, which is badly overstated because it ignores heavy early churn; use cohort curves instead.

Recommendations: report LTV by acquisition channel and compare with CAC (LTV:CAC above 3, payback under 12 months). Use a conservative 12-24 month LTV for decisions. Focus on month-0 to month-1 improvements: raising month-1 retention from 60% to 66% raises 12-month LTV by roughly 10%. Always attach discounting, margin definition and the horizon.

A492. Walmart / BigBasket wants to know why customers stop buying groceries. Define "churn" for a non-subscription retailer where users can go wee...

Non-contractual churn cannot be observed directly, so infer it from purchase gaps.

1. Look at the distribution of inter-purchase times per customer segment. If 90% of weekly shoppers reorder within 14 days, then 'no order in 30 days' is a defensible churn definition for them; for monthly shoppers the threshold would be 60-75 days. Use a customer-specific rule, such as 'inactive for more than 2.5x the customer's median purchase interval', or a probabilistic model (BG/NBD) that gives probability alive.

2. Measure with RFM: recency, frequency, monetary value.

```
SELECT customer_id,
  MAX(order_date) last_order,
  current_date - MAX(order_date) recency_days,
  COUNT(*) freq,
  PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY gap_days) median_gap
FROM (SELECT customer_id, order_date,
        order_date - LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date)
gap_days
      FROM orders) t
GROUP BY 1;
```

3. Early warning signals before the last order: widening gaps, shrinking basket size, fewer categories, more substitutions or out-of-stock items, late deliveries, coupon dependence, complaints, a drop in app opens.

4. Segment churners by cause using the last 3 orders: delivery delays, poor quality produce, price increase, competitor entry.

5. Action: trigger win-back flows at the 1.5x median gap, personalized offers on the customer's usual items, service recovery for those with bad experiences. Validate through holdout test.

6. Metrics: reactivation rate, incremental orders per contacted customer (versus holdout), cost per reactivated customer, and 90-day retention of reactivated customers.

A493. A ride-sharing company observes that 7-day retention of new riders improved from 30% to 36% after a signup-bonus launch, but 60-day retentio...

The bonus likely pulled forward usage: riders used credits within the first week, then reverted to baseline behaviour. This is a classic short-term vs long-term effect; retention measured over too short a window rewarded the bonus.

Analysis:

1. Compare cohorts before and after launch (or treatment vs control if randomised) on 7, 14, 30, 60, 90 day retention. Check that the curves converge, as the data suggest.

2. Check selection: did the bonus change who signs up (bonus hunters, multi-accounting, fraud)? Check device fingerprints, referral abuse, share of trips only paid with credit.

3. Economics: cost of bonus per acquired user (say Rs 150) vs incremental margin. If 6 points more users stay a week but no one extra stays 60 days, incremental lifetime trips are small; compute incremental lifetime margin per user: extra trips x contribution margin minus Rs 150. Probably negative.

4. Differences by segment: maybe commuters (high-need) get lasting habit, tourists do not.

5. Alternatives: structure the bonus to build habit (e.g. 'complete 5 rides in 3 weeks for Rs 200'), reduce the unconditional bonus, test different amounts, add second-trip rewards.

6. Measurement lesson: choose success metrics aligned with the business goal (lifetime value or D60 retention, number of rides by day 30) and use long-run holdouts. Avoid declaring success from the metric that moved the fastest.

Recommendation: run a follow-up A/B test with a habit-forming structure and evaluate at 60 days with CAC payback.

A494. YouTube (or an edtech app) wants to understand which early user behaviours predict long-term retention to design onboarding. You have event ...

1. Build a user-level table: features from days 0-7 (number of videos watched, subscriptions, searches, playlist adds, distinct days active, sessions) and label = active at day 60 (compute label only for users with a full 60-day window).

2. Explore: for each action threshold compute retention. Example: users who subscribe to 3+ channels within 7 days have 72% D60 retention versus 28% for others. Plot retention against count of the behaviour to find where the curve saturates (the 'magic number').

```
SELECT subs_first7 >= 3 AS hit, COUNT(*) n, AVG(active_d60::int) ret
FROM user_features GROUP BY 1;
```

3. Use a model (logistic regression/trees) to rank features while controlling for each other and for acquisition channel and device. Look at partial dependence.

4. Causality caution: engaged users do more of everything, so subscribing may simply signal intent. Ways to move towards causality: (a) randomized experiment that nudges users to subscribe to 3 channels (treatment) and compare retention among all assigned (intention-to-treat), (b) natural experiments such as UI changes that varied exposure, (c) matching or propensity scores on early engagement, (d) instrumental variables if available.

5. Choose the metric for onboarding: percent of new users hitting the 'aha' action in 7 days. Set up the nudge test; success = ITT lift in D60 retention, with guardrails on satisfaction.

6. Communicate caveats: observational result is a hypothesis, not proof; give the effect with confidence intervals.

A495. A bank's mobile app shows 4.5 million MAU but only 1.2 million log in more than once a week. The head of digital asks you to define engageme...

1. Frame the goal: engagement matters if it leads to digital adoption (lower branch cost), product cross-sell and retention of deposits.

2. Define tiers by behaviour using login frequency and transaction depth:

- Dormant: no login in 90 days

- Casual: 1-3 logins a month, balance checks only

- Regular: weekly logins plus payments or transfers

- Power: 3+ sessions weekly with investments, bill pay, cards

Set thresholds by looking at distributions and breakpoints (percentiles, k-means on frequency, recency, features used) and keep them interpretable.

3. Link tiers to value: compute per-tier average balance, products held, revenue per customer, 12-month attrition, service-call rate and cost to serve.

```
SELECT tier, COUNT(*) customers, AVG(balance) avg_bal, AVG(num_products) prod,
AVG(attrited_12m::int) attr
FROM customer_tiers GROUP BY tier;
```

Example finding: Regular users hold 1.8x balance and attrite 4% versus 11% for casual; each customer moved from casual to regular is worth Rs 1,200 per year.

4. Identify movers: which first-month actions predict moving up (set up UPI autopay, add a payee)? Design nudges for those.

5. Quantify opportunity: 3.3 million casual users x 10% conversion x Rs 1,200 = Rs 39.6 crore annually.

6. Governance: tiers recalculated monthly, a migration matrix (who moved up, down), and guardrails like complaints and fraud.

A496. A retailer (Reliance Retail / Big Basket style) has 2 million customers with transaction data: customer_id, order_date, order_value, categor...

Goal: actionable groups, not just statistical clusters.

RFM approach:

1. Compute for each customer as of a snapshot date: Recency (days since last order), Frequency (orders in 12 months), Monetary (total spend).

2. Score each dimension 1-5 by quintile (recency reversed).

```
import pandas as pd
snap = df.order_date.max() + pd.Timedelta(days=1)
rfm = df.groupby('customer_id').agg(
  recency=('order_date', lambda x: (snap - x.max()).days),
  frequency=('order_date','count'),
  monetary=('order_value','sum'))
for c,asc in [('recency',False),('frequency',True),('monetary',True)]:
    rfm[c[0]] = pd.qcut(rfm[c].rank(method='first'), 5, labels=[1,2,3,4,5] if asc else
[5,4,3,2,1])
```

3. Map score combinations to named segments: Champions (R5 F5 M5), Loyal, Potential loyalists, At risk (high F/M but low R), Hibernating, New customers. Each gets an action: Champions - early access; At risk - win-back with personalized offers; New - second purchase nudge.

Clustering alternative: k-means on log-transformed and standardized R, F, M plus behavioural features (category mix, discount sensitivity, channel share). Choose k via silhouette and elbow, but ultimately via business interpretability (4-7 segments). Check stability by re-running on different samples. Profile clusters with means and size; name them.

Pitfalls: skewed monetary values (use log), outliers (B2B buyers), segments that change monthly (track migration), and confirming that segments differ in response to treatment, which requires testing campaigns by segment. Evaluate: uplift in conversion and margin versus a random holdout.

A497. An airline wants to test raising ancillary fees (checked baggage) from Rs 600 to Rs 750. Data: bookings with fare class, baggage purchased (...

1. Define the outcome: baggage attach rate (share of bookings adding baggage) and revenue per booking = attach x price. Also watch downstream effects: overall booking conversion, customer satisfaction, check-in-desk purchases (often pricier) and gate-checking delays.

2. Elasticity from data: E = %change in quantity / %change in price. Ideal: randomised price test by route or user. If using historical changes, use difference-in-differences: routes with a price change versus similar routes without, before and after, adjusting for seasonality.

3. Randomised A/B test: randomly show Rs 750 to 50% of eligible bookings; primary metric is revenue per booking (not attach rate alone); guardrails are booking conversion and complaints. Power: if baseline attach is 40% and we want to detect a 1.5 percentage-point change, we need about 17k bookings per arm (alpha 0.05, power 0.8).

4. Example result: attach 40% -> 37% at Rs 750. Revenue per booking: 0.40 x 600 = 240 vs 0.37 x 750 = 277.5 (+15.6%). Elasticity = (-7.5%)/(+25%) = -0.30 (inelastic), so raising price increases revenue. Break-even attach at Rs 750 is 32%.

5. Segment: business travellers are less price sensitive; leisure and short-haul are more; set differentiated prices rather than a flat rise.

6. Long-run risks: brand perception, customers shifting to carry-on only (which affects operations and boarding), competitor reaction. Use a holdout for 8+ weeks.

7. Recommendation: roll out on inelastic segments first, monitor, and revisit quarterly.

A498. Ola/Uber is considering dynamic "surge" pricing changes. How would you evaluate whether a new surge algorithm improves the marketplace witho...

Marketplace objective: balance supply and demand so more riders get rides quickly and drivers earn more, while keeping long-term trust.

Metrics:

- Primary: completed trips per hour in the zone (marketplace liquidity) or gross bookings per rider-session, plus driver earnings per online hour.

- Supporting: request fulfilment rate, pickup ETA, cancellation rate, rider conversion after price display, utilization of drivers (percent time on trip).

- Guardrails: rider retention, number of surge complaints, share of users seeing surge above 2x, price fairness measures across neighbourhoods, regulatory caps.

Experiment design: standard user-level A/B testing is flawed because riders and drivers interact (interference, spillovers): treated riders consume supply that control riders could have used. Use switchback experiments (alternate algorithms by time blocks within a city, e.g. 30-minute windows randomised and balanced across hour of day) or geo-based tests (matched zones/cities), with analysis that accounts for autocorrelation by clustering standard errors at the block level. Allow a washout period at block edges.

Analysis: compare gross bookings, fulfilment and earnings per hour between algorithm blocks; use difference in means with block-level bootstrap; check heterogeneity by peak time and zone. Look for long-term effects through a few cities held on the old algorithm for 6-8 weeks.

Interpretation: if prices rose 8% but completed trips rose 3% and ETAs fell by 1 minute, the algorithm improved matching. If trips fall and revenue rises only through price, caution about future retention.

A499. A D2C brand spends Rs 5 crore a month across Google Ads, Meta, influencers, and email. Last-click attribution in the dashboard says Google g...

Last-click gives all credit to the final touch, over-crediting bottom-funnel channels such as branded search (people already decided) and under-crediting awareness channels.

Options in increasing rigor:

1. Compare rule-based models: first touch, linear, time decay, position-based. Compute channel credit under each; channels whose credit changes a lot are the ones where the model matters (e.g. Meta rises from 15% to 30%).

2. Data-driven attribution (Markov chain removal effect or Shapley): estimate each channel's contribution as the drop in conversion probability if it were removed from paths.

3. Incrementality experiments (gold standard):

- Geo lift tests: pause or cut Meta spend in matched regions for 4 weeks; measure difference-in-differences in sales.

- Conversion lift (platform holdout): randomize a 10% user holdout who are not shown ads.

- Brand search holdout: turn off branded keywords in some regions to measure cannibalization (often only 10-30% of branded search conversions are truly incremental).

4. Marketing mix modelling (MMM): regression of weekly sales on adstock-transformed spend by channel with seasonality, price, promotions; use Bayesian priors calibrated with experiments. Gives marginal ROI/saturation curves to reallocate budget.

```
sales_t = a + sum_c beta_c * saturate(adstock(spend_c,t)) + seasonality + promos + error
```

5. Reconciliation: use experiments to calibrate MMM and multi-touch attribution; report incremental ROAS (iROAS) with confidence intervals.

Decision: shift budget toward channels with highest marginal iROAS until equalized; keep checking for saturation. Note limitations: privacy constraints on tracking, offline conversions, long consideration cycles.

A500. A marketing team ran a 20% discount email campaign to 500,000 customers; 8% of recipients purchased in the following week, generating Rs 4 c...

Key issue: the 8% purchase rate includes customers who would have bought anyway (baseline). Without a holdout, incrementality must be estimated.

1. Establish baseline: use historical purchase rate of similar customers in comparable weeks (e.g. 5.5% purchased in non-promo weeks). Incremental conversion estimate = 8.0% - 5.5% = 2.5 points, i.e. 12,500 incremental buyers (not 40,000).

2. Better baselines: build a matched control from customers not emailed (unsubscribed, or those on a different cadence) using propensity matching on recency, frequency, past spend; or use difference-in-differences relative to a similar customer group over the same weeks; or synthetic control.

3. Cost: discount is given to all buyers, including those who would buy anyway. If average order is Rs 1,000, margin 40% before discount: baseline buyer's margin drops from Rs 400 to Rs 200 (20% off). Compute:

- Baseline buyers 27,500 x discount cost Rs 200 = Rs 55 lakh of subsidy

- Incremental buyers 12,500 x margin Rs 200 = Rs 25 lakh

- Net = -Rs 30 lakh before email costs, ignoring future value.

4. Also account for pull-forward (do sales dip the following weeks?), cannibalization, and effect on regular price expectations. Look at 4-week and 8-week windows.

5. Conclusion: the campaign appears unprofitable in the short run even though revenue looks big. Recommend future campaigns always keep a 10% random holdout, target by predicted uplift, test smaller discounts (10%), and report incremental margin, not gross revenue.

A501. Your company's CAC rose from Rs 800 to Rs 1,300 within two quarters while LTV is Rs 3,000. Marketing asks for more budget; finance is worrie...

1. Define CAC consistently: total sales and marketing spend attributable to acquisition / new paying customers in the period. Distinguish blended CAC (all new customers including organic) from paid CAC (only paid-attributed). Blended CAC can rise because organic share falls even when paid efficiency is flat.

2. LTV:CAC dropped from 3.75 to 2.3; payback period lengthened. Benchmarks: LTV:CAC above 3 and payback under 12 months.

3. Decompose the rise: channel mix (more spend on costly channels), within-channel cost inflation (CPMs/CPCs up due to competition or ad fatigue), conversion rate changes (landing page, app store ratings), audience saturation (marginal CAC much higher than average CAC), attribution changes (iOS privacy making reported conversions fall).

```
SELECT channel, SUM(spend) spend, SUM(new_customers) cust, SUM(spend)/SUM(new_customers) cac,
 SUM(ltv_90d)/SUM(new_customers) ltv90
FROM acquisition GROUP BY 1;
```

4. Marginal analysis: average CAC hides diminishing returns. Estimate response curves per channel; the next Rs 1 lakh may deliver CAC of Rs 2,200 versus an average of 1,300.

5. Cohort quality: is LTV (e.g. 90-day revenue) of new cohorts constant? If lower-quality customers were acquired, CAC compared to LTV is worse than it looks.

6. Recommendations: reallocate toward channels where marginal LTV:CAC > 3; cap spend on saturated channels; invest in conversion (CRO), referrals and retention (raising LTV is another route); run incrementality tests on the two largest channels; set a CAC guardrail by channel and a payback target. Do not approve more budget blindly; approve a staged budget increase tied to marginal CAC.

A502. A bank wants to decide cross-sell targeting for personal loans. Data: 3 million customers with demographics, balances, transactions, product...

Problem: the past campaign targeted customers by some rule (probably high-income, high-balance), so naive conversion models learn who was selected as much as who responds. Also, response is not incrementality.

1. Understand historical targeting: examine the selection rule; compare contacted vs non-contacted distributions. Check overlap (positivity): if no low-income customers were contacted, you cannot learn their response.

2. Propensity-to-buy model: logistic regression or gradient boosting on features using data before the campaign, evaluated with a time-split. Report lift in the top deciles. This ranks customers by likelihood of conversion given contact among those previously targeted.

3. Selection correction: inverse propensity weighting using the probability of being contacted, or model on a random sample.

4. Incrementality: run a new randomised pilot: pick a random 100k sample from the full base, contact 80% and hold out 20%. Measure uplift; fit an uplift model (two-model, causal forest) to estimate individual treatment effects, choose customers where expected incremental profit is positive.

5. Profit framework: expected profit = P(convert|contact) x net interest margin over loan life x (1 - default probability x loss given default) - contact cost. Integrate credit risk and eligibility; exclude customers with high risk or recent delinquency.

6. Constraints: regulation (consent, do-not-call), fairness (avoid discriminating by protected traits), channel capacity.

7. Monitor: campaign response, default rates after 6-12 months (the true outcome), and cannibalization of existing products.

A503. A hospital group wants to reduce readmissions within 30 days. Data: admissions(patient_id, admit_date, discharge_date, diagnosis, ward, doct...

1. Define the metric: 30-day readmission rate = discharges followed by an unplanned admission within 30 days / total eligible discharges (exclude planned readmissions, deaths in hospital, transfers). Use consistent diagnosis groups (heart failure, COPD, etc.).

2. Baseline and variation: overall rate 14%; by diagnosis, ward, doctor, discharge day (weekend discharge), length of stay, insurance and age. Use risk adjustment so that doctors treating sicker patients are not unfairly flagged: compare observed vs expected readmissions from a model that includes age, comorbidities, prior admissions. Report observed/expected ratio with confidence intervals; small volumes require funnel plots.

```
SELECT diagnosis, COUNT(*) discharges, AVG(readmit_30d::int) rate
FROM admissions WHERE discharge_date BETWEEN '2025-10-01' AND '2026-09-30'
GROUP BY 1 HAVING COUNT(*)>100 ORDER BY rate DESC;
```

3. Drivers: short length of stay, no follow-up appointment within 7 days, polypharmacy, discharge on a Friday, living alone, missed follow-up call.

4. Intervention: build a risk score, allocate follow-up calls/home visits to the highest-risk 20%. Evaluate with a randomised or stepped rollout comparing readmission rates; because of capacity limits use number needed to treat.

5. Ethical and data issues: privacy and consent, bias in risk models across groups, careful communication (clinicians own decisions).

6. KPIs: readmission rate (risk adjusted), follow-up completion within 7 days, cost per avoided readmission, patient-reported outcomes; guardrails: mortality, length of stay, emergency visits after discharge.

A504. An insurer notices rising claims cost per policy in motor insurance (Rs 11,000 to Rs 13,500 over a year). Data: policies(policy_id, vehicle_...

Claims cost per policy = claim frequency x average severity. Decompose first.

- Frequency (claims per 100 policies): 12 -> 13

- Severity (average claim): Rs 91,667 -> Rs 103,846

Combined: 1.083 x 1.133 = 1.227, i.e. +22.7% (matches 11,000 -> 13,500).

Drill into severity: repair cost inflation (parts prices), more total-loss claims, higher share of expensive vehicles, bodily injury claims, fraud, litigation. Drill into frequency: more urban policies, new drivers, weather events (flood claims in one city), fewer lapses leaving riskier pool, change in underwriting.

Mix versus rate: use a decomposition across segments (city x vehicle age x age band). A shift of the portfolio toward Mumbai and newer cars raises cost per policy even if segment-level costs are flat.

```
SELECT city, COUNT(DISTINCT p.policy_id) pols, COUNT(c.claim_id) claims,
 1.0*COUNT(c.claim_id)/COUNT(DISTINCT p.policy_id) freq, AVG(c.amount) severity
FROM policies p LEFT JOIN claims c USING(policy_id) GROUP BY 1;
```

Also check one-offs (a flood event) and development lag: recent claims are not fully reported (IBNR), so compare on a developed basis.

Actions: reprice segments with loss ratio above target (loss ratio = claims / earned premium), tighten fraud detection on high-severity claims, negotiate with garages/parts, introduce deductibles, telematics discounts. Metrics: loss ratio, combined ratio, frequency, severity, retention after price change (price-elasticity guardrail).

A505. Zomato Gold-style membership: members order 2.5x more than non-members. The product manager says "membership drives orders, let's push it ev...

This is a selection bias problem: heavy users are more likely to buy a membership, so the 2.5x gap overstates the causal effect.

Checks:

1. Pre-period behaviour: compare orders in the 3 months before joining. If members already ordered 2x more than non-members before joining, only a fraction of the gap is causal. Use difference-in-differences: members' change in orders after joining vs matched non-members' change over the same period.

2. Matching: propensity score matching on pre-join order frequency, spend, city, tenure, device; check covariate balance.

3. Event study: plot orders per week relative to join week for members and matched controls; a spike just before joining indicates users joined when they were already planning more orders (e.g. moving to a new city).

4. Experiment: randomise who sees the membership offer (promotion exposure as an instrument); estimate the effect among compliers.

5. Economics: incremental orders x contribution margin - discounts and free delivery costs - membership revenue. Example: members order 6 vs matched 5.0 (+1.0 incremental); incremental contribution Rs 30 per order x 1 = Rs 30 per month vs cost of free delivery Rs 40 per month on 6 orders... membership might break even only through fees (Rs 100/quarter).

6. Retention effect: do members retain better (6-month)? That could be the real value.

Conclusion: causal lift likely 20-30% of the naive number; push selectively to users where predicted uplift and retention benefits exceed costs.

A506. Amazon wants to test a new checkout button color/design on the mobile app. Baseline checkout conversion is 10%. The team wants to detect a 3...

Hypothesis: H0: conversion(new) = conversion(old); H1: conversion(new) is higher (two-sided test at alpha 0.05 is the safer default).

Unit: user (not session) to prevent the same person seeing both versions; assignment by hashing user_id with an experiment salt, 50/50. Analysis unit should equal randomization unit (if analysing sessions, cluster standard errors by user).

Sample size: baseline p1 = 0.10, MDE relative 3% -> p2 = 0.103, delta = 0.003.

n per arm = 2 x (z_alpha/2 + z_beta)^2 x p(1-p) / delta^2 with z = 1.96 and 0.84 (power 80%):

n = 2 x 7.85 x 0.0901 / 0.000009 ~ 157,000 per arm (roughly 314,000 users total; p(1-p) uses average p = 0.1015).

```
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize
es = proportion_effectsize(0.103, 0.10)
n = NormalIndPower().solve_power(es, power=0.8, alpha=0.05, ratio=1)
```

Duration: if 200,000 sessions/day correspond to roughly 120,000 unique users a day, 314,000 users arrive within about 3 days, but run at least 1-2 full weeks to cover weekday/weekend cycles and novelty effects.

Pre-checks: A/A test or sample ratio mismatch (SRM) check using chi-square on counts (expect 50/50); verify logging.

Metrics: primary checkout conversion; secondary revenue per user, AOV; guardrails: page load time, payment errors, refund rate.

Decision: ship if the primary lift is positive and statistically significant with the confidence interval excluding zero, no guardrail breached; if inconclusive, do not extend ad hoc (no peeking) unless using sequential methods. Avoid multiple comparisons without correction (Bonferroni/Benjamini-Hochberg for segment cuts).

A507. After a 14-day A/B test of a new recommendation carousel on Flipkart, results are: control conversion 5.00% (n = 400,000), treatment 5.12% (...

1. Validate quality: SRM check (counts near 50/50), no tracking bugs, bot filtering, no overlap with other experiments, stable lift across the two weeks (no novelty decay).

2. Quantify: absolute lift 0.12 points, relative +2.4%. Standard error for difference = sqrt(p(1-p)(2/n)) ~ sqrt(0.0475 x 5e-6) = 0.000487, z = 0.0012/0.000487 = 2.46 (p ~ 0.014 for two-sided; the p=0.04 reported suggests a more conservative method or clustered error). 95% CI roughly +0.005 to +0.23 points: the lift might be tiny. Report the interval, not only the p-value.

3. Practical vs statistical significance: is +0.12 points worth the engineering and latency cost? At 10 million monthly sessions and AOV Rs 1,500, 0.12 points = 12,000 extra orders = Rs 1.8 crore monthly GMV.

4. Revenue per user: positive but not significant; conversion up without revenue lift may mean cheaper items are being sold. Check AOV and margin.

5. Guardrail: +80 ms latency can reduce conversion over time. Check if the lift is despite the latency; consider optimisation before launch.

6. Multiple metrics: with many metrics checked, one p-value of 0.04 may be chance. Pre-registered primary metric matters.

7. Segments: look for heterogeneity (new vs returning users), but treat as exploratory.

Recommendation: launch to 50% for confirmation (or ship with a 5% long-term holdout), fix the latency issue, and monitor revenue per user and returns. Do not claim the 2.4% as certain.

A508. Your manager checks the A/B test dashboard every day and wants to stop the test as soon as p < 0.05. On day 4 of a planned 14-day test, p = ...

Problem: peeking (optional stopping) inflates the false-positive rate. With a fixed-horizon test, the p-value is only valid at the pre-planned sample size. Checking daily for 14 days and stopping at the first p < 0.05 can raise the type I error from 5% to about 20-30%. Early results also suffer from winner's curse (effect sizes overestimated) and weekday composition bias (day 4 may miss the weekend).

Options:

1. Fixed-horizon discipline: pre-register sample size and duration; only evaluate at the end. Dashboards can show metrics with a 'not yet valid' flag.

2. Group sequential tests: pre-specify interim looks (e.g. 3 looks) with alpha spending functions (O'Brien-Fleming: very strict early boundary, such as z ~ 3.7 at 25% information, and ~2.0 at the end; Pocock constant boundary).

3. Always-valid inference: mSPRT or confidence sequences allow continuous monitoring with valid error control, at the price of lower power.

4. Bayesian approach with a decision rule on expected loss; still needs care because peeking can bias decisions unless the stopping rule is accounted for.

5. Safety stopping: monitor guardrails continuously and stop for harm (futility or negative impact) using conservative bounds.

Simulation to convince stakeholders:

```
import numpy as np
np.random.seed(1); fp = 0
for _ in range(2000):
    a = np.random.binomial(1, .1, (14, 5000)); b = np.random.binomial(1, .1, (14, 5000))
    for d in range(1, 15):
        x, y = a[:d].sum(), b[:d].sum(); n = 5000*d
        p = (x+y)/(2*n); z = (y-x)/n/np.sqrt(2*p*(1-p)/n)
        if abs(z) > 1.96: fp += 1; break
print(fp/2000)   # ~ 0.2-0.3 vs 0.05
```

Recommendation: continue to day 14, report with confidence interval, and adopt sequential testing for future experiments.

A509. You cannot randomise users for a feature because the pricing change applies to an entire city (for example, a new delivery fee in Hyderabad)...

Use quasi-experimental methods with a control group of unaffected cities.

1. Difference-in-differences (DiD): compare change in orders per capita in Hyderabad (treated) before/after the fee to the change in comparable cities (Pune, Chennai) over the same window. Estimate: (Hyd_after - Hyd_before) - (Ctrl_after - Ctrl_before). Key assumption: parallel trends. Check by plotting 8-12 weeks of pre-period orders (indexed to 100) and running an event-study regression with leads that should be near zero.

```
import statsmodels.formula.api as smf
m = smf.ols('log_orders ~ treated*post + C(city) + C(week)', data=df).fit(cov_type='cluster',
cov_kwds={'groups': df['city']})
```

With few cities, clustered standard errors are unreliable: use wild cluster bootstrap or randomization inference.

2. Synthetic control: build a weighted combination of untreated cities that matches Hyderabad's pre-period trajectory; the post-period gap is the effect. Placebo tests: apply the method to untreated cities to see how unusual the Hyderabad effect is.

3. Confounders to rule out: concurrent promotions, weather, festivals, competitor moves in Hyderabad only. Choose a treatment date away from major events.

4. Alternative: time-based switchback (fee on/off in alternating weeks) if allowed; or randomise at sub-city level (zones) to get more units, though spillovers exist.

5. Report: effect on orders (-6%), AOV (+4%), revenue and margin, retention for sensitive segments; with confidence intervals. Say 'orders fell 6% (95% CI -9% to -3%), net margin per order up Rs 8, net positive'.

6. Limitations: external validity, short horizon.

A510. You run an A/B test on a social network feature where users are connected (for example a new sharing button). Treatment users are sharing mo...

Issue: network interference (violates SUTVA, the assumption that one unit's outcome does not depend on others' treatment). Control users receive spillover from treated friends, so the control group is partially 'contaminated', biasing the measured difference toward zero (underestimating the total effect). Alternatively, if the feature competes for attention, spillover can bias in the opposite direction.

Remedies:

1. Cluster randomisation: detect communities (graph clustering) and randomise entire clusters so most friends share the same assignment. Cost: fewer effective units, less power, and the design effect increases the variance; need larger sample.

2. Ego-network randomisation / two-sided designs: assign treatment to ego network and measure the effect on direct and indirect exposure.

3. Geographic or market-level experiments, e.g. whole cities, for marketplace features.

4. Measure exposure: define each user's exposure level (fraction of friends treated) and analyse outcome versus exposure in a randomised-saturation design (clusters assigned 0%, 25%, 50%, 100% treated) to estimate direct and spillover effects.

5. Time-based switchbacks for marketplace-type systems.

6. If cluster randomisation is not possible, model interference or bound the bias with sensitivity analysis.

Interpretation: report two numbers: the direct effect on treated users and the total network effect, which is what matters for a full rollout. Expect to need 2-5x more users with cluster randomisation; check balance on cluster size and pre-treatment metrics (CUPED can help).

A511. Your A/B test on a ride-hailing app shows overall treatment effect on revenue per rider is +1% (not significant), but the data science team ...

Subgroup caution: slicing by many dimensions finds spurious effects (multiple comparisons: with 20 independent subgroups, one is expected to be 'significant' at 5% by chance). Treat as exploratory unless the subgroup was pre-registered with a hypothesis (new riders respond to the incentive because they have no price anchor). Check:

1. Was 'new vs veteran' pre-specified? If not, apply multiple-testing correction (Bonferroni/Holm, or False Discovery Rate) across all slices examined.

2. Test the interaction formally: revenue ~ treatment * is_new; the interaction coefficient's p-value tells if the difference between groups is real, rather than testing each subgroup separately.

3. Power: subgroups have smaller n, so wide confidence intervals; a significant subgroup effect from a small sample overstates magnitude.

4. Replicate: run a follow-up targeted experiment on new riders.

5. Practical use: if confirmed, roll out only to new riders (personalised rollout), potentially a +6% on 25% of revenue.

CUPED (Controlled-experiment Using Pre-Experiment Data): reduces variance by adjusting the outcome with a pre-experiment covariate correlated with it (e.g. each rider's revenue in the 4 weeks before).

Y_adj = Y - theta x (X - mean(X)), theta = Cov(Y, X)/Var(X).

```
theta = np.cov(y, x)[0,1] / np.var(x)
y_adj = y - theta * (x - x.mean())
```

Variance falls by a factor (1 - rho^2); with correlation 0.7, variance falls about 50%, equivalent to halving required sample size or a shorter test. It keeps the estimate unbiased because X is independent of treatment assignment.

A512. Test results for a new onboarding flow: the treatment shows 12% sign-up completion vs 11.5% control, but you find that treatment has 52.5% o...

This is a Sample Ratio Mismatch (SRM): the observed allocation differs from the designed allocation more than chance allows. With n = 1,000,000, 52.5/47.5 is wildly improbable under a true 50/50 split:

chi-square = sum((O-E)^2/E) = (525,000-500,000)^2/500,000 x 2 = 2,500 x ... = 2 x (625,000,000/500,000) = 2,500; p-value essentially zero.

```
from scipy.stats import chisquare
chisquare([525000, 475000], f_exp=[500000, 500000])
```

Why it matters: an SRM means the groups are not comparable and results cannot be trusted - the lift of 0.5 points could be an artifact.

Common causes:

- Treatment page loads slower/faster, so more/less users are recorded (differential logging or bot filtering, redirect failures)

- Bugs in assignment (hash collisions, wrong eligibility filters applied after assignment)

- Crashes in a variant leading to lost events

- Overlap with another experiment, caching, or users re-bucketed

- Filtering on a post-treatment variable (e.g. only count users who reached step 2)

- Data pipeline joins that drop users in one arm

Steps: do not report results; investigate by segment (device, browser, country) to see where the mismatch concentrates; check assignment logs versus exposure logs; fix and re-run. Add an automated SRM check ($p < 0.001$) to the experiment platform that flags results as invalid. Explain to stakeholders that a faulty experiment is worse than none because it gives false confidence.

A513. Swiggy wants to run an experiment on a new "free delivery above Rs 199" threshold. Design the experiment end-to-end including metrics, unit,...

Objective: increase basket value/profit without hurting order frequency.

Hypothesis: lowering the free-delivery threshold from Rs 299 to Rs 199 raises order frequency and conversion but reduces AOV; net contribution margin per user may rise or fall. The experiment quantifies that.

Design:

- Unit: user, stratified by city and tenure; 3 arms: control Rs 299, T1 Rs 199, T2 Rs 249. Run at least 3-4 weeks (to cover repeat behaviour and weekly cycles).

- Exposure: ensure users see only their arm's threshold across devices; log exposures.

- Primary metric: contribution margin per user per week (orders x (AOV x take rate + delivery fee income - delivery cost - discounts)). This avoids optimising GMV that destroys profit.

- Secondary: orders per user, conversion rate, AOV, share of orders near the threshold (bunching), 4-week retention.

- Guardrails: delivery time, cancellation, rider cost per order, restaurant satisfaction, customer complaints.

- Sample size: for contribution margin per user (high variance), use pilot data: SD = Rs 120, detect Rs 3 difference: n = 2 x (2.8)^2 x 120^2/9 ~ 25,000 per arm (use alpha 0.05, power 0.8 factor 7.85: 2 x 7.85 x 14,400/9 = 25,100). Use CUPED to reduce.

- Risks: users learn the rule and delay orders (long-term behaviour shift); novelty; riders' capacity; city-level interference minimal in user-level test.

Decision: pick the arm with highest incremental margin per user whose lower confidence bound exceeds zero and satisfies guardrails; maintain a 5% holdout for 8 weeks to check lasting effects.

A514. A streaming company tests a new homepage ranking. Short-term engagement (hours watched in week 1) is up 4%, but the data science lead suspec...

Challenge: the metric that responds quickly (engagement) may not align with the long-term objective (retention, satisfaction).

Approaches:

1. Long-running holdout: keep 2-5% of users on the old ranking for 3-6 months; compare churn, hours, and satisfaction. Report that decisions are staged: ship at 95% while measuring.

2. Surrogate index: build a model that predicts long-term churn from short-term metrics (week 1-4 engagement variety, session counts, search/browse mix, completion rates, rating behaviour). Validate on historical experiments with known outcomes: the surrogate's treatment effect should predict the long-term effect. Use it for early reads but flag uncertainty.

3. Better short-term proxies: instead of hours, track completed titles, return visits across days, distinct titles started and finished, thumbs up and 'save to list' counts, survey 'would recommend'. A ranking that raises 'start but abandon within 5 minutes' rates is suspicious.

4. Segmented analysis: new members vs established; those at low engagement may benefit.

5. Guardrails: complaint volume, abandonment rate, diversity of consumption, cancel-flow visits.

6. Decision framework: define a utility with weights (e.g. +1 for completed viewing hour, -penalty for early abandon), or require long-run holdout data before full launch.

Communicate: "Week-1 hours +4% (CI +3.1 to +4.9). Surrogate predicts churn effect of +0.1 pp (CI -0.1 to +0.3); we ship to 90% and track the 5% holdout for 12 weeks."

A515. An analyst at a food delivery company reports that a test improved average order value from Rs 420 to Rs 440 and p = 0.001. A skeptical stak...

1. Practical significance: +Rs 20 is +4.8%. Multiply by volume: 1 million monthly orders x Rs 20 = Rs 2 crore additional GMV; with 20% take rate = Rs 40 lakh revenue. Compare with cost and risks. A tiny p-value with huge sample can come from a small effect, so report effect size and confidence interval, not only p.

2. Distribution: AOV is right-skewed (a few Rs 5,000 party orders). Means are sensitive to outliers; but the mean matters for revenue. Tools:

- Check results with winsorised AOV (cap at the 99th percentile) and the median.
- Bootstrap the difference in means for confidence intervals (no normality assumption).
- Mann-Whitney test as a robustness check, remembering it tests stochastic dominance rather than the mean.
- Log-transform for a relative effect.
- Central limit theorem justifies t-test with large n, but check heavy tails.

```
import numpy as np
diffs = [np.random.choice(t, len(t)).mean() - np.random.choice(c, len(c)).mean() for _ in
range(5000)]
np.percentile(diffs, [2.5, 97.5])
```

3. Mix check: AOV can increase because fewer small orders occur (conversion fell). Look at orders per user and revenue per user; AOV is a ratio metric whose denominator may be affected by treatment, so analyze revenue per user or use the delta method.

4. Conclusion: effect is probably real (+Rs 14 to +Rs 26 interval), meaningful if conversion holds, and robust to outliers.

A516. A bank wants to know if a new email nudge increased the number of customers setting up automatic bill pay. They sent the email to 100,000 cu...

Not demonstrably. Customers were not randomly assigned: the 'most engaged' were selected, and they would likely adopt more even without the email. The comparison confounds treatment with customer engagement (selection bias).

How to get a credible answer:

1. Ideal: a randomised controlled trial. Take the eligible engaged population, randomly assign 50,000 to the email and 50,000 to holdout. Measure adoption in 30 days. Difference in rates = causal effect.

2. If the campaign is already done: look for a pre-period baseline: what was the adoption rate of these 100k in the prior 30 days or in the same month last year? If baseline was 7%, uplift may be 2 points. Use difference-in-differences relative to the similar engaged customers not emailed (if some exist at the margin).

3. Regression discontinuity: if selection was based on an engagement score cutoff (e.g. score >= 80), compare customers just above and just below 80 who are very similar. The jump in adoption at the cutoff estimates the effect locally.

4. Propensity-score matching with pre-treatment variables, with caution that unobserved factors remain.

```
from sklearn.linear_model import LogisticRegression
ps = LogisticRegression().fit(X, treated).predict_proba(X)[:,1]
# match nearest neighbours on ps, compare adoption
```

5. Statistical and business reporting: report the estimated uplift with confidence interval, cost per incremental adoption (email cost Rs 0.5 x 100,000 / incremental adopters).

Recommendation: next time keep a random holdout of 10%; present current result as correlation.

A517. The COO of a logistics company (Delhivery-style) asks you to build an executive dashboard for daily operations. Data: shipments(awb, origin_...

Start by asking the COO what decisions the dashboard supports (daily intervention on delays, capacity, customer promise) and who else uses it. Executives need a one-screen summary with drill-downs, not 40 charts.

KPI hierarchy:

1. Outcome KPIs (top row, with trend vs target and vs last week):

- On-time delivery rate (delivered by promised date / due shipments), target 96%
- First-attempt delivery success (percent delivered on first attempt), target 85%
- RTO (return to origin) rate
- Average and p90 transit time
- Cost per shipment

2. Flow/health KPIs: shipments in transit by age bucket (0-1, 2-3, 4+ days), backlog at hubs (inventory over capacity), pickup completion rate, damaged/lost per 10,000.

3. Capacity and productivity: rider attendance, shipments per rider per day, hub utilization (volume / capacity), vehicle fill rate.

4. Quality guardrails: customer complaints per 1,000, NPS/CSAT, claims.

Design principles: one definition per metric (documented), data freshness stamp, red/amber/green thresholds based on targets, ranking of worst hubs, filters for region and customer, small multiples over time, annotations for known events. Alerts for threshold breaches (email/Slack) rather than relying on viewing.

Example SQL for on-time:

```
SELECT dest_hub, DATE(delivered_ts) d,
 AVG(CASE WHEN delivered_ts <= promised_ts THEN 1.0 ELSE 0 END) AS on_time
FROM shipments WHERE status='DELIVERED' GROUP BY 1,2;
```

Validate with ops managers, test with real incidents, track usage, and iterate. Avoid vanity metrics (total shipments alone).

A518. Your company's two dashboards show different monthly active users: the product dashboard says 8.2M and the finance dashboard says 7.4M. Exec...

Treat as a metric governance and reconciliation problem.

1. Gather definitions from both: What is 'active' (any event, login, transaction)? Unique by user_id, device_id, or email? Time zone (UTC vs IST)? Calendar month vs rolling 30 days? Are internal/test users, bots, and employees excluded? Are deleted/merged accounts handled? Do they use different source tables (event stream vs billing system)?

2. Reconcile quantitatively with a bridge: start from 8.2M, subtract bots (0.15M), logged-out anonymous users counted via device (0.45M), test accounts (0.02M), users whose only event was a push open (0.18M) to get 7.4M. Do it in SQL by tagging each user with the reasons they fall in one but not the other.

```
SELECT a.user_id, (b.user_id IS NULL) AS only_in_product
FROM product_active a LEFT JOIN finance_active b USING(user_id);
```

3. Decide on the canonical metric with stakeholders: pick the definition aligned with business purpose (e.g. 'active = at least one qualifying engagement event by a registered, non-test user in the calendar month, IST'). Allow variants with explicit names ('MAU (any app open)' vs 'Paying MAU').

4. Implement a single source of truth: a governed metrics layer (dbt metrics/semantic layer), versioned definitions, a data dictionary, tests, and ownership.

5. Communicate: a short note on the difference, the bridge chart, and a date after which only the canonical metric is displayed. Prevent recurrence by adding a metric review step before publishing new dashboards.

A519. While preparing a weekly report you notice that yesterday's revenue is double the usual amount in the orders table. What do you do before se...

Do not send blindly; investigate whether it is real or a data problem.

Checks in order:

1. Duplicates: count distinct order_ids vs rows. A duplicated load or a join fan-out (joining orders to payments with multiple rows per order) doubles revenue.

```
SELECT COUNT(*) rows_, COUNT(DISTINCT order_id) orders_, SUM(amount) rev FROM orders WHERE dt
= current_date-1;
SELECT order_id, COUNT(*) FROM orders WHERE dt = current_date-1 GROUP BY 1 HAVING COUNT(*)>1
LIMIT 20;
```

2. Pipeline: was the ETL run twice, did a backfill overlap? Check load logs and row counts per load batch.

3. Unusual values: outliers (one order of Rs 50 lakh, wrong currency, amount in paise rather than rupees, test transactions), missing filters on cancelled/refunded orders.

4. Definition changes: a new column or status code added (e.g. 'authorised' counted along with 'captured').

5. Business reality: marketing campaign, flash sale, B2B bulk order? Cross-check against another source (payment gateway settlements, finance ledger, order count in application DB, traffic).

6. Compare components: orders count, AOV, and revenue; if orders doubled with the same AOV and traffic is flat, suspect duplicates; if AOV doubled, suspect values.

Action: if data issue, fix or exclude with documentation, notify data engineering, and add a data quality test (row-count anomaly, unique key). If real, annotate the report with the cause. In the report mention the caveat

and time of verification. Communicate early to stakeholders that a number is under review rather than letting them see a surprising figure without context.

A520. You get a customer dataset for a churn analysis with: 12% missing income values, duplicate customer records, dates stored in mixed formats, ...

Step 1 - Profile: row counts, null percentages, distinct counts, min/max, value distributions per column, key uniqueness. Compare to source system counts.

Step 2 - Duplicates: define the business key (customer_id, or email + phone). Identify exact duplicates and near-duplicates (case, whitespace, typos). Keep the most recent or most complete record and log the rule.

```
df = df.sort_values('updated_at').drop_duplicates('customer_id', keep='last')
```

Step 3 - Dates: parse with explicit formats, flag ambiguous ones (01/02/2026 can be day-first or month-first, check against the source's locale).

```
df['signup_dt'] = pd.to_datetime(df['signup_dt'], errors='coerce', dayfirst=True)
df['signup_dt'].isna().mean()
```

Step 4 - Invalid values: age of 250 is clearly invalid (and maybe 0 or negative); set to missing, check if one data source/system causes it (e.g. year of birth entered). Apply business rules (age 18-100, signup date not in future, churn date after signup date).

Step 5 - Missing income: investigate mechanism (MCAR, MAR, MNAR). If missingness correlates with churn, the missing flag itself is a feature. Options: median imputation by segment, model-based imputation (e.g. iterative imputer), or a 'missing' category; avoid dropping 12% rows blindly. Check imputation does not change distributions significantly. Always add an is_missing indicator.

Step 6 - Validate: re-run counts and reconcile, add automated tests (unique keys, accepted ranges, not-null), and document each rule and its row impact in a data quality log.

Stakeholders: share what share of data was affected (e.g. 3% duplicates removed, 0.4% invalid ages), the assumptions, and the sensitivity of conclusions to the imputation approach. Ask the data owners to fix the upstream source.

A521. An analyst discovers that the tracking event "purchase" fires twice for iOS users after a recent app update, inflating conversion rate and r...

Treat as a data incident with technical, analytical and communication parts.

1. Confirm and quantify: compare the client-side events with the server-side orders (source of truth): count purchase events per order_id, find ratio by platform and app version, find the release date it started.

```
SELECT app_version, COUNT(*) events, COUNT(DISTINCT order_id) orders
FROM events WHERE event_name='purchase' AND platform='iOS' GROUP BY 1;
```

2. Scope: which metrics and dashboards use that event (conversion, revenue per session, funnel, experiment results, marketing attribution, ROAS)? Which decisions might have been affected (A/B tests during this window, campaign budgets)?

3. Fix forward: engineering fixes duplicate firing (idempotent event with order_id and dedupe key). Add dedupe at the pipeline level now: keep first event per order_id.

4. Backfill: recompute historical metrics deduped by order_id; mark corrected data. Re-check affected experiments: if both arms are affected equally, conclusions probably hold, otherwise re-analyze.

5. Communicate quickly and transparently: tell the data owner, head of analytics, and the report consumers: what happened, impact (iOS conversion overstated by ~8%, overall by ~3%), corrected numbers, what was done. Send corrected board figures with a short note, not hidden. Own the miss and describe the prevention: reconcile event data to order tables daily, add anomaly alerts, and release checklists that include analytics QA.

6. Prevention: contract tests on tracking plan, and use server-side events for money metrics.

A522. A telecom's billing data warehouse has customers, accounts, and invoices tables. A report of "revenue by customer segment" gives a total tha...

A total 15% higher than the ledger points to row duplication (fan-out) or including items finance excludes.

1. Check join cardinalities. Is cust_id unique in customers? If customers has multiple rows per cust_id (SCD type 2 history with start/end dates), every invoice is repeated for each historical row.

```
SELECT cust_id, COUNT(*) FROM customers GROUP BY 1 HAVING COUNT(*)>1;
SELECT account_id, COUNT(*) FROM accounts GROUP BY 1 HAVING COUNT(*)>1;
```

2. Test totals at each step: SUM(amount) from invoices alone; after joining accounts; after joining customers. The step at which the total jumps is the culprit.

3. Fix: join to the current row only (WHERE c.is_current = 1) or to the version valid on the invoice date (c.valid_from <= i.invoice_date < c.valid_to). Alternatively pre-aggregate invoices by account before joining.

```
SELECT c.segment, SUM(i.amount)
FROM invoices i
JOIN accounts a ON a.account_id = i.account_id
JOIN customers c ON c.cust_id = a.cust_id AND i.invoice_date >= c.valid_from AND
i.invoice_date < c.valid_to
GROUP BY c.segment;
```

4. Other differences: invoices including tax (GST) while finance reports net revenue, credit notes and reversals, cancelled invoices, currency, timing (invoice date vs recognised revenue date), test accounts.

5. Reconcile: produce a bridge from report total to finance total with each item, and assert in a test that SUM at the end equals SUM at the base grain.

6. Lessons: always know the grain of each table, compare to a control total, and document revenue definition.

A523. You are building a KPI tree for a consumer lending app (for example loans via a mobile app). The CEO says "grow profitable disbursals." Cons...

Start with the top-line objective: profitable disbursal growth = volume x margin x quality. Decompose:

Profit = disbursed amount x (net interest margin + fees) - credit losses - cost of funds - operating cost.

Level 1 branches:

1. Volume: Disbursals = Applications x Approval rate x Acceptance/disbursal rate x Average ticket size.

- Applications = Traffic (installs, leads) x Application start rate x Completion rate. Drivers: CAC by channel, onboarding drop-off, KYC pass rate.

- Approval rate: risk policy, bureau hit rate, model cutoffs.

- Acceptance rate: offer attractiveness (rate, tenor), time to disbursal.

2. Margin: yield (APR + fees), cost of funds, discounts.

3. Credit quality: first payment default (FPD), 30+/60+/90+ days past due, roll rates, loss given default, collection efficiency.

4. Cost: CAC, servicing cost per loan, collections cost per rupee collected.

5. Retention/repeat: repeat borrower share, top-up rate.

Trade-offs and guardrails: higher approval rate increases volume but raises default; lower price improves acceptance but reduces margin. Use risk-adjusted metrics: expected loss rate by score band, risk-adjusted margin per loan, vintage curves (cumulative default by months-on-book for each origination month). Monitor vintage performance since defaults appear with lag; judge growth with early indicators (FPD, 30+ dpd at month 3).

Other: compliance (fair-practice, fair lending, digital lending guidelines), customer complaints, fraud rate.

Targets example: approve rate 38%, FPD under 3%, CAC under Rs 900, risk-adjusted ROA above 4%.

A524. The head of a hospital asks for a dashboard on emergency room (ER) performance. List metrics, show how you would compute wait-time metrics i...

Metrics:

- Door-to-triage time, door-to-doctor time (median and 90th percentile), total length of stay (LOS)

- Left without being seen (LWBS) rate, left against medical advice

- Patient volume by hour and acuity level, bed occupancy

- Return visits within 72 hours

- Staff-to-patient ratio, boarding time (time waiting for an inpatient bed)

- Patient satisfaction

SQL:

```
SELECT date_trunc('day', arrival_ts) AS d,
 COUNT(*) AS visits,
 PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY EXTRACT(EPOCH FROM doctor_seen_ts -
arrival_ts)/60) AS median_door_to_doc_min,
 PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY EXTRACT(EPOCH FROM doctor_seen_ts -
arrival_ts)/60) AS p90_door_to_doc_min,
 AVG(CASE WHEN doctor_seen_ts IS NULL THEN 1.0 ELSE 0 END) AS lwbs_rate
FROM er_visits GROUP BY 1 ORDER BY 1;
```

Pitfalls:

- Use medians and percentiles not means (long right tail); show distributions.

- Segment by acuity: critical patients are seen immediately, so averaging with low-acuity patients hides waiting problems.

- Patients who leave without being seen have no doctor time, so excluding them flatters wait times (survivorship bias); show LWBS beside wait.

- Timestamps entered manually may be inaccurate; look for impossible sequences (doctor before arrival).

- Seasonal/hourly peaks; show heatmaps by weekday x hour for staffing decisions.

- Privacy: de-identify, aggregate small cells.

Design: top KPI tiles with targets, hourly demand vs staffing heatmap, drill-down by acuity; clinicians should validate definitions.

A525. A retail chain wants a "store performance" scorecard to compare 400 stores. Stores differ in size, location, and age. How do you create a fa...

Goal: identify stores to fix, replicate or close; the comparison must be like-for-like.

1. Core metrics: sales per square foot, same-store sales growth (SSSG; stores open at least 12 months, comparing identical periods), conversion (footfall to bills), average bill value, items per bill, gross margin, shrinkage, inventory turns, labour cost percent, customer NPS.

2. Normalise: compare sales per sq ft and per employee. Group stores into peer clusters via clustering on store format, city tier, size, catchment demographics, and compare within the cluster. Use a regression/benchmark approach: predict expected sales from catchment population, income, competitor presence, size and age; performance = actual / expected (residual). The rank by residual shows which stores outperform what their location alone would predict.

```
import statsmodels.formula.api as smf
m = smf.ols('np.log(sales) ~ np.log(area)+pop_5km+income_idx+C(tier)+age_yrs',
data=stores).fit()
stores['perf_index'] = stores.sales / np.exp(m.fittedvalues)
```

3. Handle new stores via maturity curves (a store in month 4 vs expected ramp), and remove one-offs like refurbishment closures.

4. Statistical noise: use confidence intervals or smoothing (12-month trailing) so small stores are not misclassified.

5. Avoid pitfalls: Simpson's paradox on category mix, comparing different formats, ignoring seasonality (compare to same month last year), and gaming (managers pushing sales via heavy discounts); include margin.

6. Output: a scorecard with percentile rank within peer group, traffic-light status, and recommended actions; review with regional managers to verify context.

A526. Design a metrics and monitoring plan for a customer support operation (chat + call) at an e-commerce company. Leadership wants lower cost bu...

Balance efficiency, effectiveness and experience, plus the link to commerce outcomes.

Efficiency: tickets per 1,000 orders (contact rate), cost per ticket, average handle time (AHT), agent occupancy and utilisation, first response time (FRT), automation/deflection rate (share resolved by self-serve or bot).

Effectiveness: first contact resolution (FCR = tickets not reopened or followed by another contact on the same issue in 7 days), reopen rate, resolution time (median/p90), escalation rate.

Experience: CSAT (post-ticket survey), NPS, effort score, abandonment rate in chat/call queue.

Business impact: repeat purchase rate of customers who contacted support vs matched non-contacting customers; refund and replacement costs; contact drivers by category (delivery delays, damaged item).

Guardrail logic: pushing AHT down can reduce FCR and CSAT; deflection can hide unresolved issues. Always show pairs: AHT with FCR, deflection with repeat contacts.

Key analysis:

```
SELECT category, COUNT(*) tickets,
```

```
 PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY EXTRACT(EPOCH FROM first_response_ts-
created_ts)/60) frt_med,
 AVG(reopened_flag::int) reopen, AVG(csat) csat
FROM tickets GROUP BY category ORDER BY tickets DESC;
```

Pareto the top contact reasons (80% of volume typically from 4-5 categories), then drive upstream fixes (better delivery ETA to cut 'where is my order' tickets). Staff with forecasting (Erlang C by interval).

Caveats: CSAT response bias (only extremes respond) - weigh with response rate and use sampling; avoid agent ranking on raw CSAT without adjusting for ticket difficulty.

A527. A data platform team keeps getting complaints that the "conversion rate" dashboard is wrong. You are asked to put data quality monitoring in...

Data quality dimensions: completeness, uniqueness, validity, consistency, timeliness, accuracy.

1. Identify critical data assets (events, orders, sessions) and the metrics depending on them; assign owners and SLAs (e.g. orders table updated by 7 am).

2. Automated tests at ingestion and transformation (dbt tests, Great Expectations, Soda):

- Freshness: max(event_ts) within 2 hours
- Volume: daily row count within +/- 3 standard deviations of a weekday-adjusted forecast
- Uniqueness: order_id unique; event_id unique
- Not null: user_id, timestamp
- Accepted values/ranges: platform in (android, ios, web); amount > 0; timestamp not in future
- Referential integrity: every order has a user and a product
- Distribution drift: share of null in fields, proportion by platform/app version

3. Reconciliation checks: orders table vs payments vs finance totals, event purchases vs backend orders (ratio near 1.0).

```
-- dbt schema test style
SELECT COUNT(*) FROM (SELECT order_id FROM orders GROUP BY 1 HAVING COUNT(*)>1);
```

4. Anomaly detection on the metrics themselves: z-score or seasonal model alerts on conversion by platform/country, tuned to avoid alert fatigue (severity levels, routing to owners).

5. Lineage and documentation: metric definitions in a semantic layer, change log, and release process for tracking-plan changes.

6. Incident process: severity classification, who to inform, a status banner on dashboards ('data delayed'), root-cause review, and metrics for DQ itself (number of incidents, time to detect, time to resolve).

Prioritise tests on the most business-critical tables and build iteratively rather than testing everything.

A528. A senior product manager at a marketplace company asks you to "just confirm" that the new feature caused the 5% GMV rise last month. Your an...

Principles: lead with the answer, be honest about uncertainty, show evidence, and offer a path forward. Do not simply say "no".

1. Prepare: re-check your analysis for errors; make the comparison fair (compare feature-exposed vs unexposed users within the same period, or use difference-in-differences vs last year's festival uplift). Quantify: the 5% rise decomposes to festival seasonality +3.5, marketing +1.2, feature +0.2 (CI -0.8 to +1.2).

2. Meet 1:1 first to preview findings so the PM is not surprised in a group meeting, and ask for any context you may be missing.

3. Message structure (BLUF): "GMV rose 5%; the data suggests most of that came from festival demand and the campaign. The feature's contribution is small, between -0.8% and +1.2%, which our current data cannot distinguish from zero. That doesn't mean the feature is bad; the effect may be small or our measurement lacks power."

4. Show one clear chart (waterfall bridge) and the method in an appendix.

5. Offer next steps: run a randomised holdout of 10% for 3 weeks (sample size needed), measure engagement metrics closer to the feature (usage, task completion), segment where it might matter.

6. Handle pushback: separate facts from opinions, avoid blaming, acknowledge the feature's non-GMV benefits (usability) if supported.

7. Document the analysis and decision in a shared note.

Outcome: the PM keeps trust in you because you are accurate, and decisions are better. Never tweak the analysis to produce the desired answer.

A529. You must present a quarterly churn analysis to a non-technical leadership audience (CEO, CMO, CFO) in 10 minutes. The analysis includes a gr...

Audience-first: executives want the decision, size of the opportunity, and risks, not model internals.

Structure (about 6-8 slides):

1. Executive summary (slide 1): the answer. "Quarterly churn rose from 4.1% to 5.3%; 70% of the increase comes from customers acquired via discount campaigns in the last 2 quarters. A targeted retention programme could save about Rs 6 crore annually. We recommend approving a Rs 1 crore pilot."

2. The problem in numbers: one trend chart with the change highlighted.

3. Where it is concentrated: segment/cohort chart (highlight the key segment; grey out the rest).

4. Why: top 3 drivers in plain language (e.g. no key feature use in first 14 days, 2+ support tickets, price-increase notice). Mention SHAP only as 'the model ranks these as the strongest early signals'.

5. What we propose: actions, owner, cost, expected benefit with a range (low/base/high), how success will be measured, timeline.

6. Risks and assumptions: e.g. correlation not causation, model precision; how the pilot will test causality.

7. Ask/next steps: decision needed today.

Appendix: model performance, methodology, data definitions.

Practices: one message per slide (title states the takeaway), minimal text, annotate charts, avoid jargon (say 'at-risk customers' not 'predicted probability'), prepare answers for likely questions (How accurate? What is the cost? Why now?), rehearse timing, and pre-brief key stakeholders.

A530. Three stakeholders each ask for an urgent analysis in the same week: Marketing wants campaign ROI, Finance wants a revenue forecast for the ...

Framework: clarify, prioritise using impact and urgency, negotiate scope, communicate.

1. Clarify each request: what decision depends on it, who is the decision-maker, deadline, level of precision needed, and consequence of delay. Often 'urgent' hides flexible timing.

2. Score: impact (money/decision size), urgency (hard deadlines), effort, and dependency. Example: Finance board forecast: high impact, fixed date (board), cannot slip -> first priority. Product sprint planning: decision-fixed date; a quick funnel cut (top 3 drop-offs) will do; effort low -> second with a reduced scope. Marketing ROI: important, but decision (next-month budget) is two weeks away -> schedule for the following week or deliver a quick directional version.

3. Negotiate: tell each stakeholder what you can deliver and when: "For sprint planning I'll deliver the funnel by Wednesday with the main drop-off points; the full segmentation can follow Friday. Campaign ROI I can provide a first version Monday." Offer self-serve dashboards or SQL templates to unblock.

4. Escalate transparently if all three are truly critical: ask your manager to arbitrate priorities rather than silently working overtime or dropping one.

5. Protect quality: share assumptions and caveats in rushed work.

6. Follow-up: add recurring needs to a roadmap and intake form to avoid repeated fire drills.

This shows judgment, business alignment, and communication.

A531. Tell me about a time you used data to influence a major business decision.

Use the STAR structure (Situation, Task, Action, Result) and quantify. A strong answer follows this template; replace details with your real experience.

Situation: In my previous role at an e-commerce company, leadership planned to expand free delivery to all orders to improve conversion; the cost was estimated at Rs 3 crore a quarter.

Task: I was asked to estimate the likely impact on orders and profitability before approval.

Action: I pulled 12 months of order data and found that the conversion lift from free delivery was concentrated among orders between Rs 300 and Rs 500. I designed a 3-week A/B test in two cities, defining contribution margin per user as the primary metric with delivery cost and AOV as secondary and cancellations as guardrails. I used CUPED to reduce the sample needed and checked for sample ratio mismatch. I discovered that free delivery at all values raised orders by 5% but reduced margin per user by 7%, while a Rs 299 threshold increased orders 3% with margin up 2%. I built a one-page summary with a chart of margin impact and the sensitivity to rider cost.

Result: Leadership adopted the Rs 299 threshold, avoiding roughly Rs 2.4 crore per quarter in subsidy while improving orders 3%. The method became the standard for pricing tests in the team.

Why it works: it shows structured thinking, business impact, rigor, and stakeholder influence. Emphasise your specific role ('I'), the numbers, what you learned (e.g. test design before opinion), and how you handled disagreement.

A532. Describe a time your analysis was wrong or you found an error after sharing results. What did you do?

What the interviewer is testing: integrity, ownership, speed of correction, learning.

Template using STAR:

Situation: I shared a weekly retention report showing a 6-point improvement in week-4 retention after an onboarding change, and the product team celebrated.

Task: Two days later, while re-running the query for a deeper cut, I noticed the improvement disappeared for iOS users and numbers did not reconcile with the dashboard.

Action: I immediately checked my logic and found that I had used an inner join to the events table, which excluded users with no events after day 1 (the churned ones), biasing retention upward in the later cohort where logging had changed. I corrected the query using a left join on the full cohort denominator, recomputed (the real lift was 1.5 points, not significant at the 95% level), and contacted my manager and the PM the same day with a short note: what was wrong, the corrected numbers, the decision impact, and the fix. I also added a unit test comparing cohort size to signups and asked a colleague to peer-review key queries.

Result: The PM paused the planned rollout and ran a proper A/B test. The team appreciated the transparency; we introduced a peer review checklist, and no similar error recurred.

Key points: take responsibility without excuses, correct quickly, communicate proactively, and describe the process improvement. Avoid claiming you never make mistakes or blaming a teammate.

A533. Tell me about a time you had to work with messy or incomplete data and still deliver a result on a deadline.

The interviewer wants to see pragmatism, judgment about acceptable quality, and clear communication about limitations.

Sample STAR answer:

Situation: Finance needed a customer profitability analysis for a pricing decision in five days, but customer cost data came from three systems with different IDs and 20% of records missing region and product tags.

Task: Deliver a reliable-enough ranking of customer segments by profitability.

Action: I first agreed with the stakeholder on the decision and the needed precision (segment-level ranking, not exact numbers). I profiled the data and built a mapping table between IDs using email and phone with fuzzy matching, achieving an 88% match. For unmatched records I tested sensitivity: I ran the analysis with them excluded, assigned proportionally, and assigned worst-case to a low-profit segment; the ranking of the top 3 and bottom 2 segments did not change under any scenario. I documented assumptions, flagged in the deliverable that the middle segments were not reliably ordered, and raised a ticket for the data owners to fix tagging.

Result: I delivered on day 4; the decision (raise prices for the two least profitable segments) went ahead, resulting in about Rs 80 lakh annual margin improvement. Later, when the data was fixed, the ranking confirmed.

Takeaways: scope to the decision, quantify uncertainty with sensitivity analysis, be transparent, and drive the long-term fix. Avoid saying you waited for perfect data.

A534. Tell me about a time you disagreed with a stakeholder or manager about an analytical approach or conclusion. How was it resolved?

The interviewer evaluates collaboration, evidence-based persuasion, and humility.

Sample STAR answer:

Situation: The marketing head wanted to report campaign ROI using last-click attribution, which showed 5x ROAS on a branded search campaign and asked me to build the dashboard that way.

Task: I believed the metric overstated impact because branded search captures customers who were already going to buy.

Action: I did not refuse or go around them. I asked what decision the dashboard would inform (budget allocation) and agreed that the metric should reflect incremental revenue. I proposed a quick test: pause

branded ads in two matched regions for three weeks. Meanwhile I built the dashboard they asked for but added a column marked 'estimated incrementality' with a range. I showed a small pre-read of the logic with a simple example. The test showed only 22% of branded conversions were incremental; the effective ROAS was 1.1x.

Result: The head of marketing reallocated 30% of the branded budget to prospecting channels and agreed to make incrementality testing standard. Our relationship improved because I brought data and options, not just objections.

Lessons: seek the shared goal, test disagreement empirically when possible, keep respect, and be willing to be wrong. If the result had gone the other way I would have updated my view and said so.

A535. Tell me about a time you explained a complex analysis to a non-technical audience.

What they look for: audience empathy, simplification without distortion, and actionable outcomes.

Sample STAR:

Situation: I built a customer lifetime value model using BG/NBD and Gamma-Gamma for a retail team that needed to decide which customers get high-cost loyalty offers.

Task: Explain the model and get store managers to trust and use it.

Action: I avoided the model equations. I used an analogy: "The model looks at how often and how recently a customer has shopped, like judging if a regular at a cafe is still coming by." I translated outputs into four tiers (Protect, Grow, Maintain, Let go) and shared three real customers as examples. I validated credibility by showing a backtest: customers the model labelled top-tier a year ago spent 4.1x more in the following year. I answered the 'what if the model is wrong' question by showing the accuracy by tier and defining a manual-override process. I created a one-page guide and a 20-minute live session with Q and A.

Result: 80% of managers used the tiers in the following quarter, and loyalty offer ROI improved 35%.

Tips: start with the 'so what', use visual aids, relate to their world, check understanding by asking them to explain it back, and be ready to give the technical appendix to the curious.

A536. Describe a project where you took initiative beyond your assigned tasks and created measurable impact.

Interviewers look for ownership, proactivity, and results.

Sample STAR:

Situation: Our team spent about 10 hours weekly manually compiling operations reports in spreadsheets, and numbers frequently differed between versions.

Task: Not formally assigned; I noticed the inefficiency while doing the reports and believed we could automate it.

Action: I discussed it with my manager, estimated the time saved (500 hours a year) and risks, and got agreement to spend two hours a week on it. I mapped data sources, wrote SQL views for the core metrics, and built a scheduled dashboard in Power BI/Tableau with documented definitions. I ran it in parallel with the manual process for two weeks to reconcile differences and found two logic mistakes in the old spreadsheets (cancelled orders had been included). I trained users and created a short FAQ.

Result: Reporting time fell from 10 hours to 30 minutes a week, errors dropped to zero in the next quarter, and the dashboard was adopted by two other teams. The corrected definition removed an overstatement of about 3% in sales.

Key points: identify a real problem, quantify impact, secure buy-in rather than going rogue, deliver incrementally, validate, and share credit. Quantify with hours saved, error reduction, revenue influenced.

A537. Tell me about a time when the data did not support what leadership wanted to hear, or when you had to deliver bad news.

Tests courage, tact, and objectivity.

Sample STAR:

Situation: The CEO had publicly committed to a new loyalty programme and asked for a post-launch review expecting positive results after 3 months.

Task: Evaluate whether the programme increased purchase frequency and profit.

Action: My matched-control analysis (members vs similar non-members, with difference-in-differences) showed only a 1.2% lift in frequency, and the programme cost exceeded incremental margin by Rs 40 lakh. I double-checked with a second method (propensity matching and a regression with controls) and had a colleague review. I prepared the message: first, what worked (higher app engagement, strong sign-up among high-value customers), then the finding on profitability with confidence intervals, and finally options: restructure tiers, remove free-shipping benefit, target high-potential segments. I previewed the findings with the head of loyalty, then the CEO, framing it as an opportunity to improve ROI rather than a failure.

Result: The company redesigned the programme with tiered benefits; six months later incremental margin turned positive (+Rs 25 lakh).

Lessons: verify twice, separate facts from opinion, present options and recommendations, avoid surprising people in public, stay calm and factual, and maintain trust by being consistent whatever the news.

A538. Why do you want to be a data analyst, and tell me about the project you are proudest of.

Structure: motivation (1 minute), project (3 minutes with STAR), link to the role (30 seconds).

Motivation: highlight curiosity about how decisions get made with evidence, enjoyment of turning messy data into clear recommendations, and a link to business impact. Avoid generic phrases like 'I love numbers'; give an example, such as noticing a pattern in college data or an internship insight that changed a decision.

Project example (adapt to your own):

Situation: During an internship at a retailer, store managers complained about stockouts but head office saw high inventory.

Task: Understand the discrepancy and reduce stockouts.

Action: Joined sales, inventory, and replenishment data in SQL; computed stockout days by store-SKU; discovered 60% of stockouts were in 15% of SKUs with long lead times and misaligned reorder points. Built a Python script using demand variability and lead time to propose reorder points, and a dashboard showing risk SKUs. Piloted with 10 stores and tracked stockouts against 10 control stores.

Result: Stockouts down 28% in pilot stores with inventory value up only 3%; rollout to 120 stores.

Link: "I enjoy projects where analysis leads directly to a change in operations, and your role at <company> involves exactly that: ..." Prepare to discuss the tools (SQL, Python, Tableau), challenges, and what you would do differently. Be honest about your contributions.

A539. A stakeholder sends this request: "Can you pull the number of active users?" It is ambiguous. Walk through how you clarify a vague analytics...

Never start querying a vague request; first agree on the question, decision, and definition.

Clarifying questions:

1. Purpose: "What decision will this inform?" (board report, marketing budget, capacity planning). The purpose determines the definition and accuracy needed.

2. Definition of 'active': placed an order, opened the app, or made a payment? In which window (daily, 28-day, calendar month)? Registered users or including guests? Include cancelled orders? Which time zone?

3. Scope: which geographies, platforms, user types (new, returning, corporate)?

4. Granularity and comparison: total or by city/week; compare to last month or target?

5. Format and deadline: one number in an email, a chart, or a recurring dashboard? Who is the audience?

6. Existing figures: is there a number already reported that this should match?

Example resolution: "Active users = unique customers with at least one delivered order in the calendar month in IST, excluding test and internal accounts, by city." Write the definition down and confirm via a short message.

```
SELECT date_trunc('month', delivered_at) m, city, COUNT(DISTINCT user_id) AS
monthly_ordering_users
FROM orders WHERE status='delivered' AND is_test=false GROUP BY 1,2;
```

Offer a mock-up or sample output, stating assumptions. Provide the number with a definition footnote, and propose also providing 'app-active users' to avoid confusion. Save the definition in the shared metrics glossary.

A540. Estimation: Swiggy is considering launching in a new Tier-2 city with a population of 2 million. Estimate the number of daily food delivery ...

Use a top-down funnel with explicit assumptions, then sanity-check.

Population 2,000,000 -> households (4.5 people each) = 444,000.

Addressable: households with smartphone, UPI and meaningful income for eating out/ordering: assume 35% = 155,000 households (about 700,000 people).

Order frequency among adopters: ordering platforms typically see 40% of addressable households ordering at least monthly; active households = 62,000. A monthly active household orders about 3 times per month in Tier-2 (lower than metros at 5-6) = 186,000 orders a month = about 6,200 orders per day.

Mature-state cross-check using a benchmark: metros show roughly 1 order per day per 400-500 people; in a Tier-2 city, 1 per 300-350 people would be higher than expected, so use 1 per 350 (2M / 350 = 5,700). Both land at about 6,000 orders per day.

Delivery partners: peak hours (lunch and dinner) carry about 60% of orders in 4 hours = 3,600 orders in 4 hours = 900 per hour. If a rider completes 2.5 deliveries/hour at peak, we need 360 concurrent riders at peak. With part-time utilisation (average rider is active 5 hours per day; peak overlap), total registered active riders about 600-800.

Validate: orders per rider per day = 6,000 / 700 = 8.6, plausible (typical 12-18 for full-timers, lower for part-timers). Restaurants needed: assume 1 order/day per partner restaurant average = 1,500-2,500 listed restaurants.

State sensitivities: if adoption is 25% instead of 35%, orders fall to 4,300. Recommend phased pilots and metrics to update the estimate (week-8 orders per 1,000 population).

A541. Estimation: Estimate the total number of UPI-style digital payments processed per day by a mid-sized fintech app with 20 million registered ...

Step 1 - Active base: of 20M registered users, assume 40% monthly active = 8M MAU. Daily active share of MAU = 25% = 2M DAU.

Step 2 - Frequency: each DAU makes about 2.5 transactions per day (mix of merchant, P2P, bills) = 5M transactions/day.

Step 3 - Peak load: traffic is concentrated; assume peak hour carries 10% of daily volume = 500,000 per hour = about 139 TPS average in the peak hour. Peak minute may be 2-3x the hourly average (salary day, festival sales): 350-420 TPS. Add a safety factor of 2 for spikes and retries: design for approximately 800 TPS.

Sanity check: national UPI handles about 500-600 million txns/day, around 6,000-7,000 TPS average; a share of 5M/day (<1%) implies about 60 TPS average, consistent with 139 at peak hour.

Capacity: a single application server may handle about 100-200 TPS depending on processing; so 6-8 servers with autoscaling plus database throughput planning. Storage: 5M transactions x 1 KB = 5 GB per day, about 1.8 TB per year of raw transaction data (plus indexes, logs).

Things to state: assumptions, how to refine with real data (peak-to-average ratio from last 90 days), and where bottlenecks lie (bank-side latency, third-party APIs). As an analyst, I would use the historical logs to compute actual p99 TPS and growth forecast.

A542. Forecasting: A retailer wants a demand forecast for the next 8 weeks for 500 SKUs to plan inventory. Data: 3 years of weekly sales per SKU, ...

1. Understand: decision is reorder quantity per SKU; costs of stockout vs overstock differ; lead time 2 weeks. Segment SKUs by demand pattern: smooth, intermittent (many zero weeks), seasonal, new. Use ABC-XYZ classification (volume value x variability).

2. Baselines first: naive (last week), seasonal naive (same week last year), moving average. A model must beat these.

3. Models: for smooth/seasonal SKUs use exponential smoothing (ETS) or SARIMA; for many SKUs with covariates use gradient boosting (LightGBM) with features: lags (1, 2, 4, 52), rolling means, price, promotion flag, holiday flags, day-of-year, product attributes; global model across SKUs. Intermittent demand: Croston/SBA. New SKUs: analogue products.

```
import lightgbm as lgb
X = feats[feats.week < cutoff]; y = X.pop('units')
model = lgb.LGBMRegressor(n_estimators=500, learning_rate=0.05).fit(X, y)
```

4. Validation: time-series cross-validation (rolling origin), never random splits. Metrics: WAPE = sum|error| / sum(actual) (robust to zeros), bias (mean error), MASE vs naive; evaluate by segment; avoid MAPE with zeros.

5. Probabilistic forecasts: produce prediction intervals (quantile regression) to set safety stock: Reorder level = demand over lead time + z x sigma x sqrt(lead time). Choose z by service level (95% -> 1.65).

6. Use: convert forecasts into order quantities, consider promo uplift, supplier MOQ; monitor forecast accuracy and bias weekly; override process with planners; retrain monthly.

Example outcome: WAPE 22% vs naive 31%; stockout rate reduced from 8% to 5% with inventory flat.

A543. Paradox: A hospital reports that overall treatment B has a higher success rate than treatment A (A 78%, B 83%). But within both mild and sev...

This is Simpson's paradox: the aggregate trend reverses when the data are split by a confounding variable (severity).

Example numbers:

- Mild: A treated 87 patients, 81 successes (93%); B treated 270, 234 successes (87%)

- Severe: A treated 263, 192 successes (73%); B treated 80, 55 successes (69%)

- Overall: A = (81+192)/(87+263) = 273/350 = 78%; B = (234+55)/(270+80) = 289/350 = 83%.

Why it happens: treatment A was given mostly to severe cases (75% of A patients) while B was mostly given to mild cases (77% of B patients). Severity affects success strongly, so the group mix dominates the aggregate.

Which to recommend: depends on the causal structure. If severity is a confounder (it affects both treatment assignment and outcome), the stratified (within-severity) comparison is the valid one: A is better in both groups, so recommend A. Aggregate result is misleading because assignment was not random. Conversely, if the splitting variable is a consequence of treatment (a mediator or collider), conditioning would be wrong; so reason with a causal diagram, not a rule of thumb.

Steps: compute a standardised (adjusted) success rate by weighting each stratum by the overall severity mix: with a mix of 51% mild and 49% severe, A = 0.51x93 + 0.49x73 = 83.2% versus B = 0.51x87 + 0.49x69 = 78.2%. Report with confidence intervals and recommend a randomised trial or regression adjustment (logistic regression with severity, age). The same logic applies to business metrics (conversion by channel mix, average spend by segment).

A544. A video game company (or any freemium app) has 3% of users paying, with a heavy-tailed revenue distribution: top 1% of payers generate 50% o...

Issues:

1. ARPU is dominated by a few 'whales': a large noisy variable. Experiments on ARPU have low power; one whale in an arm can swing results.

2. A single ARPU hides the funnel: conversion to paying (3%) x average revenue per payer (ARPPU) x retention.

3. Increasing ARPU by pushing whales may harm long-term player base health, and 97% free users create network value (matchmaking, community).

Analysis:

- Decompose: ARPU = payer conversion x ARPPU. Report both, plus median spend of payers, and Pareto/Lorenz curve of spend.

- Segment: free, minnow, dolphin, whale tiers based on cumulative spend; track migration between tiers.

- LTV by cohort; D7, D30 retention by tier.

```
SELECT tier, COUNT(*) users, SUM(spend) rev, AVG(spend) arppu
```

```
FROM (SELECT user_id, spend, NTILE(100) OVER (ORDER BY spend DESC) pct,
      CASE WHEN spend=0 THEN 'free' ELSE 'payer' END tier FROM user_spend) t GROUP BY 1;
```

Experimentation:

- Use more stable metrics for primary decisions (payer conversion, 7-day revenue winsorised at the 99.5th percentile), and report raw ARPU as secondary.

- Variance reduction: CUPED with pre-period spend; stratified randomisation by past spend tier; larger samples; longer duration.

- Bootstrap CIs for revenue; treat outliers explicitly (cap, log transform) and pre-register the rule.

- Guardrails: retention, session length, complaints, spending-limit alerts (responsible gaming).

Recommendation: set decisions on conversion and retention for the base, and test premium offers separately for high-spend segments with caution.

A545. A credit card company launched a fraud detection rule that blocks 2% of transactions; fraud losses dropped from 0.12% to 0.07% of spend, but...

Frame as a cost-benefit problem with a classification trade-off.

1. Metrics: confusion matrix on blocked transactions. Suppose the rule blocks 2% of 10 million daily transactions = 200,000. If true fraud transactions are 12,000 per day originally (0.12% of spend roughly) and now 7,000 get through, the rule catches 5,000 frauds. Then false positives = 195,000: precision 2.5%, recall 5,000/12,000 = 42%. That seems poor on precision.

2. Economic evaluation:

- Benefit: fraud avoided = 5,000 x Rs 8,000 average fraud = Rs 4 crore/day

- Cost: false positives x (customer friction cost + lost interchange + support cost + attrition risk). Suppose a declined legitimate transaction has 20% chance of being abandoned (lost interchange Rs 20 on Rs 2,000 spend = Rs 4... plus churn risk). Calculate with real numbers; include longer-term costs: card inactivity after decline (look at spend 30 days later for affected vs matched unaffected cards).

3. Measure the causal cost: compare the behaviour of cards with declined genuine transactions to matched cards (propensity) or use a randomised hold-out where the rule is applied to 90% of traffic.

4. Tuning: use a model score with a threshold; plot precision-recall and cost curve; choose threshold minimising expected cost = FN x loss + FP x friction cost. Use tiered actions: low-risk allow; medium-risk step-up authentication (OTP) rather than block; high-risk block. This reduces false-positive costs.

5. Segment: apply stricter rules for high-risk merchants and card-not-present; relax for long-standing customers.

6. Monitor: fraud rate, false positive rate, approval rate, complaints, OTP success rate, customer attrition; retrain regularly because fraudsters adapt.

A546. A ride-hailing company sees that the average rating of drivers has stayed at 4.8 for two years, yet rider complaints are rising. Data: trips...

A flat average of 4.8 on a 5-point scale is a ceiling-effect, low-information metric: most riders give 5 stars by default, so the mean barely moves even when experience worsens.

1. Check the distribution, not the mean: share of 5-star, 4-star and 1-3 star ratings. If 1-3 star trips rose from 2.0% to 3.1% while 5-star share fell slightly, the mean hides a growing tail.

2. Response bias: what share of trips are rated (say 35%)? Unhappy riders may complain instead of rating, or skip rating entirely. Compare complaints per 1,000 trips for rated and unrated trips.

3. Better metrics: percent of trips below 4 stars, complaint rate per 1,000 trips by category (safety, route, cancellation, behaviour), repeat-rider rate after a low-rated trip, driver cancellation rate, ETA accuracy, and a rolling bottom-decile driver score.

```
SELECT date_trunc('month', t.trip_date) m, COUNT(*) trips,
 AVG(CASE WHEN rating <= 3 THEN 1.0 ELSE 0 END) low_rating_rate,
 COUNT(c.ticket_id)*1000.0/COUNT(*) complaints_per_1000
FROM trips t LEFT JOIN complaints c USING(trip_id) GROUP BY 1 ORDER BY 1;
```

4. Driver-level: use Bayesian shrinkage or a minimum trip count so drivers with few ratings are not misjudged; look at the share of drivers with complaint rates above a threshold.

5. Actions: coach or deactivate the bottom tail, tag-based feedback (clean car, safe driving), surveys with a neutral scale, and track riders' next-30-day retention after bad trips.

Conclusion: keep the rating as a hygiene metric but manage with tail and complaint metrics.